CODERHOUSE

MATERIAL DE LECTURA

Data Science II

Machine Learning para la Ciencia de Datos · Guía de lectura

Contenido

MÓDULO 1 · Retomando el impulso 7

Repaso de Python para análisis de datos 7

Sintaxis SQL fundamental para consultar tablas 10

Cómo definir problemas de datos con claridad 14

MÓDULO 2 · Fundamentos de Bases de Datos 18

Arquitectura de bases de datos relacionales: componentes y uso en análisis de datos 18

Fundamentos de bases de datos 21

DML aplicado a análisis de datos 22

DCL y Seguridad de datos 34

Herramientas distribuidas en Python 36

Modelado de esquemas y relaciones: de ER a tablas 38

Bases de datos modernas y ML avanzado 41

Introducción a Machine Learning 43

Tipos de aprendizaje 45

Preparación de datos para ML 47

Integración ML + pipelines de datos 48

Normalización básica: evitar redundancias y dependencias incorrectas 50

PRE-ENTREGA **Pre-entrega: Normalización y modelado relacional** 53

MÓDULO 3 · Data Acquisition de Datos 55

Fundamentos y Tipos de Datos 55

Formatos tabulares: CSV vs Excel vs Parquet 56

Data Acquisition: El Puente entre la Realidad y el Código 58

Conceptos de APIs: endpoints, autenticación, paginación 62

Consumir APIs con requests y manejar credenciales (.env) 64

Normalización de JSON anidado a tablas relacionales 68

DML: crear, consultar, actualizar y eliminar datos 72

TCL: asegurar cambios con transacciones y ACID 76

Big Data: por qué escalar cambia la forma de trabajar 79

Herramientas distribuidas y frameworks para escalar el procesamiento 82

PRE-ENTREGA **Pre-entrega: Adquisición y Normalización con Pandas** 87

MÓDULO 4 · Introducción a Machine Learning y Data Acquisition I 89

El ciclo de vida del Machine Learning: de la pregunta al modelo 89

Aprendizaje supervisado y no supervisado: cuándo usar cada uno 92

Fuentes de datos y lectura de archivos en Python 95

Preparación inicial de datasets: limpieza, tipos y división 96

PRE-ENTREGA **Pre-entrega: Ciclo de vida de un proyecto de Machine Learning** 98

MÓDULO 5 · Data Acquisition II (APIs) 100

Fundamentos de APIs para la Ciencia de Datos 100

Autenticación, tokens y variables de entorno 104

Peticiones HTTP con Requests en Python 108

Resiliencia: manejo de errores, rate limits y reintentos 110

Estructuras JSON: mapas, listas y patrones anidados 114

De JSON a tablas: estrategias y patrones 116

Buenas prácticas: reproducibilidad, documentación y control de versiones 118

Normalización de JSON para análisis de datos 121

Estrategias de reintento y manejo de errores 122

PRE-ENTREGA **Pre-entrega: Extracción y Almacenamiento Seguro de Datos vía API** 124

MÓDULO 6 · Pipelines y Optimización de Datos 126

Arquitectura de pipelines de datos y tipos de flujo 126

Procesamiento vectorial para datos de ML 130

Optimización en memoria con Polars 133

Asistentes de IA para escribir y revisar código de datos 134

PRE-ENTREGA **Pre-entrega: Construcción de un Pipeline de Ingesta y Limpieza** 137

MÓDULO 7 · SQL Básico y Motores Modernos 139

Introducción a SQL para ciencia de datos 139

SQL aplicado al análisis exploratorio 143

Conectar DuckDB con Python y consultar datos 147

Ejecución eficiente de queries analíticas 149

Manejar archivos grandes con DuckDB 152

DuckDB frente a Pandas en análisis de datos 154

PRE-ENTREGA **Pre-entrega: Análisis de Datos Meteorológicos con DuckDB e SQL** 156

MÓDULO 8 · SQL Analítico Avanzado 158

Funciones de ventana para analizar cada fila sin perder contexto 158
Ranking, acumulados y diferencias entre filas 162
JSON en SQL: almacenar, validar y extraer atributos 163
Agregaciones multi-tabla con JOIN y GROUP BY 164
Domina los conceptos críticos de SQL Analítico 167
PRE-ENTREGA **Pre-entrega: Ingeniería de Features con SQL Analítico Avanzado** **170**

MÓDULO 9 · Automatización y Patrones de Datos **172**
Mapeo ORM con SQLAlchemy 2.0 desde cero 172
Patrones seguros para ETL y validación de esquemas 176
Automatización y Patrones de Datos en Entornos de Ingeniería 180
Sesiones, motores y operaciones seguras con ORM 183
Hooks para automatizar validaciones y tareas de datos 184
PRE-ENTREGA **Pre-entrega: Persistencia y la integración de scripts ETL** **187**

MÓDULO 10 · ML Lab I: Preprocesamiento y Pipelines **189**
EDA profundo: calidad, distribución y relaciones 189
Detección y tratamiento de valores faltantes 192
Profiling automático: pandas-profiling y Sweetviz 195
Limpieza de datos y tratamiento de nulos 197
Multivariado: pairplots, heatmaps y correlaciones 199
Dimension reduction (PCA) y visualización 201
Detección visual de anomalías y outliers 204
Feature engineering y transformadores personalizados 208
Diseño de pipelines con Scikit-Learn 212
PRE-ENTREGA **Pre-entrega: Construcción de un Pipeline de Preprocesamiento Robusto** **214**

MÓDULO 11 · ML Lab II: Modelado y Métricas **217**
Regresión lineal y logística: intuición, variables y predicción 217
Cómo aprende un modelo: línea recta, probabilidad y función de pérdida 219
Ensembles para clasificación: bagging, boosting y stacking 223
Métricas para clasificación: matriz de confusión, F1 y AUC-ROC 227
6. Errores Comunes y Mejores Prácticas 229
Validación cruzada: estimar desempeño con múltiples particiones 230
Ajuste de hiperparámetros con GridSearchCV 233
Tarjetas de repaso: decisiones clave en modelado y evaluación 234
PRE-ENTREGA **Pre-entrega: Modelado y Evaluación de Clasificación** **237**

MÓDULO 12 · Visualización de Datos 239

Fundamentos y Estrategias de Visualización de Datos 239

Componentes de una visualización efectiva 243

Percepción visual y diseño (Gestalt, Tufte) 245

El Arte y la Ciencia de la Visualización de Datos: Consolidación Final 247

PRE-ENTREGA **Pre-entrega: Dashboard de Análisis de Ventas Retail** 250

MÓDULO 13 · Storytelling 253

La importancia del Storytelling en Ciencia de Datos 253

Storytelling en Ciencia de Datos 254

Principios de Usabilidad y Visualización de Datos 256

Análisis Espacial en Ciencia de Datos 258

Mapas y Visualizaciones en Python 260

Storytelling: La Consolidación del Mensaje 262

PRE-ENTREGA **Pre-entrega: Transformando datos complejos en una narrativa para la toma de decisiones** 266

MÓDULO 14 · Datos Espaciales y Visualizaciones Animadas 268

Modelos de datos espaciales: Vector vs Ráster 268

Datos Espaciales y Visualizaciones Animadas 270

Sistemas de referencia y proyecciones (WGS84, proyectadas) 273

Accesibilidad, estética y comunicación ejecutiva 275

PRE-ENTREGA **Pre-entrega: Análisis Espacial Comparativo** 278

MÓDULO 15 · Visualización de variables categóricas 280

Introducción a la distribución y medidas resumen 280

Resumen y visualización de variables categóricas 282

Detección de outliers: teoría y criterios 284

Transformaciones univariadas para visualización 286

MÓDULO · Transformación Logarítmica 287

Fundamentos de tests: hipótesis, p-valor y supuestos 288

Pruebas no paramétricas y alternativas 290

Teoría de correlación y asociación 292

PRE-ENTREGA **Pre-entrega: Análisis Visual de Distribuciones Categóricas y Outliers** 295

MÓDULO 16 · Teoría de correlación y asociación 297

Teoría de correlación y asociación 297
Introducción a multicolinealidad y sus efectos 299
PRE-ENTREGA **Pre-entrega: Análisis de Correlación, comportamiento del Consumidor** **302**

MÓDULO 17 · Proyecto Final: Pipeline Completo de Ciencia de Datos **305**
Descripción del entregable final 305
ENTREGA FINAL **Entrega final: repositorio consolidado del pipeline de ciencia de datos** **306**
Material complementario 308

MÓDULO 1

Retomando el impulso

Repasar los fundamentos de Python y SQL necesarios para el flujo de trabajo de Ciencia de Datos.

Repaso de Python para análisis de datos

Bienvenido a la primera unidad de **Machine Learning para la Ciencia de Datos**. Si estás aquí, es porque ya comprendes que los datos son el activo más valioso de la era moderna. Sin embargo, los datos por sí solos son solo ruido. Para transformarlos en señales, predicciones y decisiones, necesitamos una herramienta poderosa, versátil y legible. En el ecosistema de la ciencia de datos, esa herramienta es **Python**.

Antes de sumergirnos en algoritmos complejos de Machine Learning o arquitecturas de datos distribuidas con DuckDB y Polars, debemos asegurarnos de que los cimientos sean sólidos. Esta unidad no es solo un repaso sintáctico; es una reconexión con el lenguaje que servirá como el "pegamento" de todo nuestro flujo de trabajo.

1. Repaso de Python para Data Science: ¿Por qué Python?

Imagínate que trabajas en el equipo de analítica de una empresa de retail como **Walmart**. Tienes millones de registros de transacciones diarias. Si intentas procesar esto manualmente o con herramientas de oficina tradicionales, te enfrentarás a límites de memoria y falta de reproducibilidad.

Python se ha convertido en el estándar de la industria no solo por ser un lenguaje de "propósito general", sino por su **ecosistema**. La mayoría de las innovaciones en IA y ML (como Scikit-Learn, que usaremos en este curso) se desarrollan primero para Python. En esta unidad, retomaremos el impulso revisando los componentes esenciales que utilizaremos para manipular esos millones de registros.

2. Variables y tipos de datos básicos

En Python, una variable es un nombre que apunta a un valor almacenado en la memoria. A diferencia de lenguajes como C++ o Java, Python es de **tipado dinámico**, lo que significa que no necesitas declarar explícitamente si una variable es un número o un texto; el intérprete lo deduce por ti.

Los tipos fundamentales

Para el análisis de datos, nos enfocamos principalmente en cuatro tipos:

1. **Enteros (`int`):** Números sin decimales. Ejemplo: El conteo de productos vendidos (`stock = 500`).
2. **Flotantes (`float`):** Números con decimales. Cruciales para métricas y probabilidades. Ejemplo: El precio de un producto (`precio = 19.99`) o la precisión de un modelo.
3. **Cadenas de texto (`str`):** Texto encerrado en comillas. Ejemplo: Nombres de categorías o IDs de clientes (`categoria = "Electrónica"`).

4. **Booleanos (`bool`):** Valores de verdad: `True` o `False` . Fundamentales para filtrar datos. Ejemplo: `es_cliente_premium = True` .

Ejemplo en contexto: Si estuvieras analizando datos de una plataforma de streaming, podrías tener:

CÓDIGO

```
usuario_id = 4502 # int
nombre_plan = "Premium" # str
costo_mensual = 12.5 # float
cuenta_activa = True # bool
```

3. Listas, diccionarios, tuplas y sets: Organizando la información

Como analista, rara vez trabajarás con un solo dato. Necesitas contenedores. Python ofrece estructuras nativas poderosas que son la base de lo que luego veremos en librerías más avanzadas.

Listas: Colecciones ordenadas y mutables

Las listas son ideales para almacenar secuencias de elementos del mismo tipo, como una serie de precios de cierre de una acción durante una semana.

CÓDIGO

```
precios = [150.2, 151.0, 149.5, 152.3, 153.0]
```

- **Dato clave:** Las listas se indexan desde 0. `precios[0]` te daría `150.2` .

Diccionarios: El formato de los datos estructurados

Un diccionario almacena pares de **clave-valor**. Son la base de los archivos JSON y son fundamentales para representar registros individuales de una base de datos.

CÓDIGO

```
cliente = {
 "id": 101,
 "nombre": "Ana García",
 "compras": [15.5, 20.0, 45.3]
}
```

- **Aplicación real:** Cuando consultes una base de datos con `SQLAlchemy` (que veremos más adelante), a menudo recibirás los resultados con una estructura similar a un diccionario.

Tuplas: Inmutabilidad por seguridad

Las tuplas son como las listas, pero **no se pueden cambiar** una vez creadas. Se usan mucho para coordenadas geográficas o dimensiones de una matriz que no deben alterarse accidentalmente. `posicion = (10.450, -66.900)` .

Sets (Conjuntos): Unicidad ante todo

Un set almacena elementos únicos. Si tienes una lista de categorías con duplicados y quieres saber cuáles son las categorías únicas, un set es tu mejor amigo.

CÓDIGO

```
categorias_sucias = ["Ropa", "Hogar", "Ropa", "Juguetes", "Hogar"]
categorias_unicas = set(categorias_sucias)
<h1 class="module" data-eyebrow="MÓDULO">Resultado: {&#x27;Ropa&#x27;, &#x27;Hogar&#x27;, &#x
<hr class="module-underline"><hr class="module-hairline">
```

4. Condicionales, bucles y funciones: El flujo del análisis

Aquí es donde pasamos de "almacenar" a "procesar".

Condicionales (`if`, `elif`, `else`)

Los usamos para tomar decisiones lógicas. En Data Science, esto se traduce en **etiquetar datos** o **filtrar anomalías**.

CÓDIGO

```
score_metrica = 0.85

if score_metrica > 0.9:
 print("Modelo excelente")
elif score_metrica > 0.7:
 print("Modelo aceptable")
else:
 print("Reentrenar modelo")
```

Bucles (`for`)

El bucle `for` es nuestro caballo de batalla para iterar sobre colecciones. Imagina que quieres aplicar un descuento a una lista de precios.

CÓDIGO

```
precios = [100, 200, 300]
precios_con_descuento = []

for p in precios:
 nuevo_precio = p * 0.9 # 10% de descuento
 precios_con_descuento.append(nuevo_precio)
```

Funciones: Reutilización y modularidad

Una función es un bloque de código diseñado para realizar una tarea específica. En el flujo de ML, creamos funciones para limpiar datos, entrenar modelos o generar reportes.

Error común de principiante: Intentar escribir todo el análisis en un solo bloque gigante de código. Las funciones permiten que tu código sea "testeable" y fácil de leer.

CÓDIGO

```
def calcular_promedio(lista_numeros):
 """Calcula el promedio de una lista de números."""
 if not lista_numeros:
 return 0
 return sum(lista_numeros) / len(lista_numeros)

ventas_mensuales = [1200, 1500, 1100, 1900]
print(f"El promedio de ventas es: {calcular_promedio(ventas_mensuales)}")
```

5. Errores comunes y Mejores Prácticas

A medida que retomas el ritmo con Python, evita estas trampas frecuentes:

1. **Confundir asignación con comparación:** * `x = 10` (asigna el valor 10 a x). * `x == 10` (pregunta si x es igual a 10). Usar `=` dentro de un `if` es un error de sintaxis clásico.
2. **No usar nombres descriptivos:** Evita variables como `x`, `y`, `df1`. Prefiere `ingresos_anuales`, `clientes_activos` o `modelo_regresión`. El código se lee muchas más veces de las que se escribe.
3. **Intentar "reinventar la rueda" con bucles:** En unidades posteriores veremos que librerías como **Polars** o **Pandas** permiten hacer operaciones sobre listas enteras sin necesidad de escribir un bucle `for` manual. A esto se le llama **vectorización**. Sin embargo, entender el bucle `for` es vital para tareas de automatización donde no hay estructuras tabulares.

Síntesis y Conexión

En esta unidad hemos recordado cómo Python maneja la información básica y el flujo lógico. Estos elementos son los átomos de cualquier proyecto de Ciencia de Datos.

- Las **variables** guardan nuestros datos.
- Las **estructuras (listas, dicts)** los organizan.
- Los **bucles y funciones** nos permiten transformar esos datos de forma sistemática.

En la siguiente unidad, daremos un paso hacia la persistencia de datos: **SQL**. Aprenderemos cómo extraer estos datos de bases de datos relacionales para luego procesarlos con el Python que acabamos de repasar. ¡Prepárate para conectar los puntos!

Sintaxis SQL fundamental para consultar tablas

Imagina que trabajas en el equipo de datos de un gigante del streaming. Tienes una base de datos con millones de registros de usuarios, películas y reproducciones. Tu jefe te hace una pregunta sencilla: "¿Cuáles son las 5 películas de ciencia ficción más vistas en el último mes por usuarios en México?".

Para responder esto, no puedes abrir un archivo Excel y filtrar manualmente; el volumen de datos colapsaría tu computadora. Aquí es donde entra **SQL (Structured Query Language)**. SQL no es solo un lenguaje de programación; es el estándar universal para comunicarnos con las bases de datos. En el flujo de Machine Learning, SQL es a menudo el primer paso: antes de entrenar un modelo, debes saber extraer, filtrar y limpiar los datos directamente desde su origen.

En esta unidad, aprenderás a construir las piezas fundamentales de una consulta SQL, desde la selección básica hasta el resumen avanzado de información.

1. El Modelo Relacional: Tu Mapa de Datos

Antes de escribir código, debemos entender dónde viven los datos. El **Modelo Relacional** organiza la información en **tablas**.

Piénsalo como una hoja de cálculo superpotente, pero con reglas estrictas: * **Tabla (Entidad):** Representa un objeto o concepto (ej. `usuarios`, `ventas`). * **Fila (Registro):** Una instancia única de ese objeto (ej. el usuario con ID 452). * **Columna (Atributo):** Una característica específica (ej. `nombre`, `email`, `fecha_registro`).

El concepto de NULL

Un error común de principiante es confundir un valor vacío con un `NULL`. En SQL, **NULL** significa "desconocido" o "dato faltante". No es lo mismo que un cero o un espacio en blanco. Si un usuario no proporcionó su número de teléfono, ese campo será `NULL`. Aprender a manejar estos "huecos" es vital para que tus modelos de ML no reciban basura.

2. La Estructura Básica: SELECT y FROM

Toda consulta SQL comienza con una intención: "Quiero **seleccionar** [esto] **desde** [aquí]".

SELECT: ¿Qué columnas quieres?

Si quieres ver todas las columnas de una tabla, usas el asterisco (`*`):

CÓDIGO

```
SELECT * FROM usuarios;
```

Sin embargo, en Data Science profesional, esto se considera una mala práctica si la tabla tiene 200 columnas. Es más eficiente y claro nombrar solo las que necesitas:

CÓDIGO

```
SELECT nombre, email, pais FROM usuarios;
```

FROM: ¿De dónde vienen los datos?

Simplemente indica el nombre de la tabla. En entornos reales como **DuckDB** o **SQLite** (que usaremos en este curso), podrías estar consultando incluso archivos CSV o Parquet directamente como si fueran tablas.

3. Filtrando la Realidad: La cláusula WHERE

Rara vez queremos *todos* los datos. El `WHERE` actúa como un portero que solo deja pasar las filas que cumplen una condición.

CÓDIGO

```
SELECT nombre, edad
FROM usuarios
WHERE pais = 'México';
```

Operadores lógicos

Puedes combinar condiciones para ser más específico: * **AND:** Se deben cumplir ambas (ej. `pais = 'México' AND edad >= 18`). * **OR:** Se debe cumplir al menos una (ej. `pais = 'México' OR pais = 'Colombia'`). * **IN:** Para buscar en una lista (ej. `pais IN ('México', 'Chile', 'España')`).

Trampa común: Para filtrar valores nulos, no uses `= NULL`. SQL usa una sintaxis especial: `WHERE telefono IS NULL` o `WHERE telefono IS NOT NULL`.

4. Ordenando el Caos: ORDER BY

Los datos en una base de datos no tienen un orden garantizado. Si quieres ver los resultados de forma organizada (por ejemplo, de la venta más reciente a la más antigua), usamos `ORDER BY`.

CÓDIGO

```
SELECT nombre_producto, precio
FROM productos
ORDER BY precio DESC; -- DESC para descendente (mayor a menor), ASC para ascendente
```

Dato técnico: Puedes ordenar por múltiples columnas. Por ejemplo, `ORDER BY pais ASC, apellido ASC` ordenará primero por país y, dentro de cada país, alfabéticamente por apellido.

5. Resumiendo Información: GROUP BY y funciones de agregación

Aquí es donde SQL se vuelve realmente poderoso para el análisis de datos. Supón que no quieres ver cada venta individual, sino **cuánto se vendió en total por categoría**.

Para esto usamos funciones de agregación: * `COUNT()`: Cuenta registros. * `SUM()`: Suma valores numéricos. * `AVG()`: Calcula el promedio. * `MAX()` / `MIN()`: Encuentra los valores extremos.

El poder de GROUP BY

Cuando usas una función de agregación y quieres ver el desglose por una columna, **debes** usar `GROUP BY`.

CÓDIGO

```
SELECT categoria, COUNT(*) AS total_productos, AVG(precio) AS precio_promedio
FROM productos
GROUP BY categoria;
```

Si incluyes `categoria` en el SELECT pero no en el GROUP BY, la mayoría de los motores de SQL te darán un error.

6. El Filtro de los Grupos: HAVING

Este es el concepto que más confunde a los estudiantes. Ya sabemos que `WHERE` filtra filas. Pero, ¿qué pasa si quieres filtrar los **resultados de una agregación**?

Por ejemplo: "Muéstrame solo las categorías que tienen más de 50 productos". No puedes usar `WHERE COUNT(*) > 50` porque el `WHERE` ocurre **antes** de que SQL sepa cuánto es el conteo.

Aquí entra `HAVING`:

CÓDIGO

```
SELECT categoria, COUNT(*)
FROM productos
GROUP BY categoria
HAVING COUNT(*) > 50;
```

Diferencia clave: * `WHERE`: Filtra datos crudos (filas individuales). * `HAVING`: Filtra datos agregados (grupos creados por GROUP BY).

7. El Orden de Ejecución (La "Trampa" de SQL)

Aunque escribimos SQL en un orden, la base de datos lo ejecuta en otro. Entender esto te ahorrará horas de frustración:

1. **FROM**: Primero localiza la tabla.
2. **WHERE**: Filtra las filas individuales.
3. **GROUP BY**: Agrupa las filas restantes.
4. **HAVING**: Filtra los grupos resultantes.
5. **SELECT**: Elige qué columnas mostrar (y aplica alias con `AS`).
6. **ORDER BY**: Ordena el resultado final.
7. **LIMIT**: Recorta la cantidad de filas devueltas.

¿Por qué importa? Por esto no puedes usar un alias creado en el `SELECT` dentro del `WHERE`. El `WHERE` se ejecuta mucho antes de que el `SELECT` asigne el nombre.

Síntesis y Mejores Prácticas

Hemos cubierto el esqueleto de cualquier consulta de datos. Al dominar `SELECT`, `WHERE`, `ORDER BY` y `GROUP BY`, ya puedes realizar el 80% de las tareas de extracción de datos en un flujo de Ciencia de Datos.

Consejos de experto: 1. **Escribe palabras clave en MAYÚSCULAS**: Aunque SQL no distingue entre mayúsculas y minúsculas, escribir `SELECT` en lugar de `select` hace que tu código sea mucho más legible para otros profesionales. 2. **Usa comentarios**: Usa `--` para comentarios de una línea. Explicar *por qué* filtraste ciertos datos es vital para la reproducibilidad de tus experimentos de ML. 3. **No abuses de SELECT** `*`: En producción, extrae solo lo que necesites. Esto reduce el uso de memoria y mejora la velocidad de tus procesos de Python posteriores.

En la siguiente unidad, daremos el paso hacia la calidad de los datos, aprendiendo a identificar problemas comunes que pueden arruinar un modelo de Machine Learning antes de empezar.

Cómo definir problemas de datos con claridad

Imagina que eres un chef preparándose para cocinar el platillo más importante de su carrera. Tienes la receta perfecta (el algoritmo de Machine Learning) y los mejores utensilios (Python y Scikit-Learn). Sin embargo, al abrir la despensa, descubres que la leche está cortada, la harina tiene grumos y las especias perdieron su olor hace años. Por más talento que tengas, el resultado será un desastre.

En el mundo de la Ciencia de Datos, este fenómeno se conoce como **GIGO: *Garbage In, Garbage Out*** (Basura entra, basura sale).

No importa qué tan avanzado sea tu modelo de Inteligencia Artificial; si los datos de entrada son deficientes, las predicciones serán erróneas, sesgadas o incluso peligrosas.

1. ¿Qué es realmente un problema de datos?

Un **problema de datos** es cualquier defecto, anomalía o limitación en el conjunto de información que reduce su fiabilidad o su valor para resolver una pregunta de negocio o entrenar un modelo de Machine Learning.

A menudo, los principiantes cometen el error de pensar que un "problema" es solo un error de programación (como un `SyntaxError`). En Data Science, los problemas son más sutiles: los datos están ahí, el código corre, pero la información **no representa fielmente la realidad**.

El impacto en el flujo de Machine Learning

¿Por qué nos obsesionamos con esto? Porque los modelos de ML "aprenden" patrones de los datos. Si tus datos sugieren que todos los clientes que compran pañales también compran cerveza (un caso

clásico de análisis de canasta), pero esto se debe a que un sistema duplicó las transacciones de un solo viernes por la noche, tu modelo aprenderá una regla falsa.

Un problema de datos no detectado a tiempo causa:

1. **Sesgos en las predicciones:** El modelo favorece incorrectamente a ciertos grupos.
2. **Métricas de rendimiento engañosas:** Puedes creer que tu modelo tiene un 99% de precisión, cuando en realidad solo está "memorizando" datos duplicados.
3. **Fallas en producción:** Un modelo que espera números y recibe fechas en texto se detendrá por completo.

2. Los pilares de la Calidad de Datos

Para diagnosticar un problema, primero debemos saber qué aspecto tiene la "buena salud" de los datos. La industria reconoce cuatro dimensiones críticas:

A. Exactitud (Accuracy)

Representa qué tan cerca están los datos de la realidad.

- **Ejemplo:** Si un sensor registra una temperatura de 100°C en una habitación de oficina, el dato es técnicamente un número, pero es **inexacto** porque no refleja la realidad física del lugar.

B. Completitud (Completeness)

Se refiere a si tenemos toda la información necesaria.

- **Ejemplo:** En una base de datos de clientes, si el 40% de los registros no tienen "Correo Electrónico", los datos están **incompletos** para una campaña de marketing digital.

C. Coherencia (Consistency)

Implica que los datos no se contradigan entre sí y sigan un formato uniforme.

- **Ejemplo:** Si en una tabla la fecha es `01/12/2023` (DD/MM/YYYY) y en otra es `12/01/2023` (MM/DD/YYYY), tenemos una **incoherencia** de formato que causará errores al unirlas.

D. Oportunidad (Timeliness)

Indica si los datos están actualizados para el uso que se les quiere dar.

- **Ejemplo:** Usar una lista de precios de propiedades del año 2010 para predecir precios hoy es un problema de **oportunidad**; los datos son obsoletos ("deteriorados").

3. Tipología de problemas comunes

Como analista o científico de datos, te enfrentarás constantemente a estos cuatro "villanos":

1. Valores Nulos (Missing Values)

Es la ausencia de un dato en una celda.

- **Ojo con la confusión:** Un valor nulo (`NULL` o `NaN`) no siempre es un error. Si un cliente no tiene segundo nombre, el campo *debe* ser nulo. El problema surge cuando el dato es esencial para el modelo (como la "Edad" en un seguro de vida) y no está.
- **Detección en Python:** `df.isnull().sum()` .

2. Duplicados

Registros idénticos que aparecen más de una vez.

- **Impacto:** Los duplicados inflan artificialmente las estadísticas. Si calculas el ingreso promedio y tienes facturas duplicadas, tu resultado será mayor al real.
- **Caso Real:** En publicidad digital (AdTech), si integras datos de Facebook y Google sin normalizar, un mismo usuario puede aparecer duplicado, distorsionando el Retorno de Inversión (ROI).

3. Incoherencias y Datos No Válidos

Datos que desafían la lógica o el formato esperado.

- **Ejemplo:** Una columna "Edad" con el valor `-5` o `250` . O una columna "Género" que contiene `M` , `Masculino` , `man` y `1` para referirse a lo mismo.
- **Solución:** Requiere procesos de limpieza (Data Cleaning) y normalización.

4. Datos Obsoletos

Información que ha perdido relevancia por el paso del tiempo.

- **El peligro:** El comportamiento humano cambia. Un modelo de recomendación de películas entrenado con datos de hace 15 años probablemente te recomiende rentar un DVD, lo cual es inútil en la era del streaming.

4. Cómo definir el problema con claridad

Cuando encuentres un error, no digas simplemente "los datos están mal". Para comunicarte de forma efectiva con ingenieros o interesados de negocio, usa una **Definición de Problema de Datos**.

Una definición útil debe responder a:

1. **¿Qué está pasando?** (Descripción técnica del síntoma).
2. **¿Cuál es la causa probable?** (Error humano, falla de sistema, integración).
3. **¿Cómo afecta al objetivo?** (Impacto en el modelo o análisis).

Ejemplo de una mala definición:

> "Hay errores en la tabla de ventas".

Ejemplo de una definición clara y profesional:

> "Se detectó una **incoherencia de formato** en la columna `transaction_date` (mezcla de formatos USA y EU). Esto impide el cálculo correcto de la estacionalidad mensual y causa que el modelo de predicción de demanda falle al procesar el 15% de los registros".

5. Resumen de la Unidad

- **Calidad sobre cantidad:** El éxito de cualquier proyecto de ML depende de la calidad de los datos de entrada (Exactitud, Completitud, Coherencia y Oportunidad).
- **Identificación temprana:** Detectar nulos, duplicados e incoherencias ahorra semanas de trabajo en etapas posteriores.
- **Lenguaje de negocio:** Traducir un error técnico ("NaN en la columna B") a impacto de negocio ("Falta de datos de ingresos impide segmentar clientes premium") es lo que diferencia a un analista junior de un especialista.

Con esta unidad, cerramos el primer módulo. Ya tienes los fundamentos de Python, SQL y la mentalidad crítica para evaluar datos. ¡Estás listo para empezar a construir flujos de trabajo más complejos!

MÓDULO 2

Fundamentos de Bases de Datos

Comprender el rol de las bases de datos y modelar esquemas relacionales.

Arquitectura de bases de datos relacionales: componentes y uso en análisis de datos

Imagina que eres un científico de datos trabajando para una plataforma de streaming como Netflix. Tienes millones de usuarios, miles de películas y un flujo constante de visualizaciones cada segundo. Si guardaras toda esta información en un solo archivo de Excel gigante, el sistema colapsaría en minutos. Las búsquedas serían lentas, los datos se duplicarían y sería imposible mantener la consistencia (¿qué pasa si un usuario cambia su correo pero solo se actualiza en una de las miles de filas donde aparece?).

Aquí es donde entran las **Bases de Datos Relacionales (RDBMS)**. No son solo contenedores de datos; son sistemas sofisticados diseñados para organizar, proteger y servir información de manera eficiente. En esta unidad, exploraremos cómo se estructuran y por qué son el cimiento indispensable para cualquier flujo de Machine Learning y Análisis de Datos.

1. ¿Qué es una Base de Datos Relacional?

Antes de entrar en tecnicismos, definamos el concepto central. Una base de datos relacional es una colección de elementos de datos con relaciones predefinidas entre ellos. Estos elementos se organizan como un conjunto de **tablas** con columnas y filas.

El porqué del modelo relacional

A diferencia de otros métodos de almacenamiento, el modelo relacional separa la **lógica** (cómo se conectan los datos) de la **implementación física** (cómo se guardan en el disco duro). Esto permite que un analista de datos pueda consultar información compleja sin preocuparse por el hardware subyacente.

El valor principal para un Data Scientist es la **Integridad de los Datos**. Las bases de datos relacionales garantizan que, si defines que un "ID de Usuario" es un número, nadie pueda insertar accidentalmente un texto ahí. Esto reduce drásticamente el tiempo de limpieza de datos más adelante en el pipeline de ML.

2. Componentes de la Arquitectura de una Base de Datos

Para entender cómo funciona una base de datos, debemos distinguir entre el software que la gestiona y los datos en sí.

El Motor de Base de Datos (SGBD)

El **Sistema de Gestión de Bases de Datos (SGBD)** o DBMS (*Database Management System*) es el software que interactúa con los usuarios, las aplicaciones y la propia base de datos para capturar y analizar datos.

Ejemplos comunes en la industria: * **PostgreSQL:** Muy querido en Data Science por su robustez y soporte para tipos de datos complejos. * **MySQL:** Extremadamente popular en aplicaciones web (como WordPress o plataformas de e-commerce). * **SQLite:** Una base de datos ligera que vive en un solo archivo, ideal para prototipos y análisis rápidos en Python (la usaremos mucho en este curso).

La Estructura Interna: El Diccionario de Datos

Dentro de la arquitectura, existe un "catálogo" o diccionario de datos que contiene los **metadatos**. Estos son los "datos sobre los datos": nombres de tablas, tipos de columnas, quién tiene permiso para ver qué, etc. Sin este componente, el motor no sabría cómo interpretar los bits guardados en el disco.

3. El Modelo Relacional: Entidades, Atributos y Relaciones

Este es el lenguaje que usarás para diseñar cómo se guardan tus datos. Si entiendes estos tres conceptos, puedes modelar casi cualquier problema de negocio.

A. Entidades (Las Tablas)

Una entidad es un objeto o concepto del mundo real sobre el que queremos guardar información. En el modelo relacional, cada entidad se convierte en una **Tabla**. * *Ejemplo:* En una tienda, las entidades serían `Cliente`, `Producto`, `Pedido`.

B. Atributos (Las Columnas)

Son las características o propiedades que describen a una entidad. Cada atributo se convierte en una **Columna** en nuestra tabla. * *Ejemplo:* Para la entidad `Cliente`, los atributos serían `nombre`, `email`, `fecha_registro`, `id_cliente`.

C. Relaciones (Las Conexiones)

Es el vínculo que existe entre las entidades. Una base de datos es "relacional" precisamente porque permite conectar estas tablas de forma lógica. * *Ejemplo:* Un `Cliente` realiza un `Pedido`. Aquí, "realiza" es la relación que conecta ambas tablas.

Ejemplo Detallado: Sistema de Biblioteca

Imagina que diseñamos una base de datos para una biblioteca:

1. **Entidad `Libro`**: Atributos: `ISBN` (Identificador único), `Título`, `Autor`, `Año_Publicación`.
2. **Entidad `Usuario`**: Atributos: `ID_Usuario`, `Nombre`, `Dirección`, `Email`.
3. **Relación `Préstamo`**: Conecta a un `Usuario` con un `Libro` específico en una fecha determinada.

4. ¿Por qué es vital para el Análisis de Datos y ML?

Como analista o científico de datos, pasarás el 80% de tu tiempo preparando datos. Las bases de datos relacionales te facilitan la vida de tres maneras:

1. **Escalabilidad:** Puedes consultar tablas con millones de filas en segundos usando SQL, algo que haría colapsar a Pandas si intentas cargar todo en la memoria RAM de tu laptop.
2. **Fuente Única de Verdad (Single Source of Truth):** En una base de datos bien diseñada (normalizada), el precio de un producto se guarda en **un solo lugar**. Si el precio cambia, se actualiza en esa tabla y automáticamente todos tus análisis de ventas reflejarán el cambio. No hay riesgo de tener versiones contradictorias de la realidad.
3. **Consistencia de Tipos:** Para entrenar un modelo de Machine Learning, necesitas que la columna `edad` sea numérica. El SGBD impide que alguien escriba "veinte" en lugar de `20`, ahorrándote errores de ejecución en tu código Python.

5. Errores Comunes y Mejores Prácticas

Error: Confundir una Hoja de Cálculo con una Base de Datos Relacional

Un error frecuente es intentar usar una tabla de base de datos como si fuera un Excel, dejando celdas vacías o mezclando tipos de datos. * **La forma correcta:** Cada columna debe tener un tipo de dato estrictamente definido (entero, texto, fecha) y cada fila debe ser única (identificada por una clave primaria).

Error: "La Gran Tabla Plana"

Muchos principiantes intentan meter todos los datos de un negocio en una sola tabla gigante con 50 columnas. Esto causa redundancia (repetir el nombre del cliente en cada fila de venta) e inconsistencias. * **La forma correcta:** Separar los datos en entidades lógicas (Clientes, Ventas, Sucursales) y relacionarlas mediante IDs. Esto se llama **Normalización**.

Consejo de experto: Los datos "vivos" vs. los datos "para análisis"

En la industria, las bases de datos transaccionales (donde ocurren las ventas) están optimizadas para la velocidad de escritura. Sin embargo, para Data Science, solemos trabajar con copias de estas bases o con "Data Warehouses" donde las relaciones pueden ser un poco más relajadas para favorecer la velocidad de lectura. Aun así, entender el modelo relacional original es crucial para saber qué significan los datos que estás extrayendo.

6. Síntesis y Conexión

Hemos aprendido que una base de datos relacional es mucho más que tablas; es una arquitectura robusta que utiliza **entidades**, **atributos** y **relaciones** para organizar la información de forma íntegra.

- Las **entidades** son los sujetos (tablas).
- Los **atributos** son las características (columnas).
- Las **relaciones** son los hilos que los unen.

En la siguiente unidad, daremos el salto de la teoría a la práctica y aprenderemos **DDL (Data Definition Language)**, el lenguaje que nos permite dar órdenes al SGBD para crear estas estructuras desde cero usando código. ¡Prepárate para construir tus primeros contenedores de datos!

Fundamentos de bases de datos

El Rol Estratégico de las Bases de Datos en Data Science

Las bases de datos son sistemas diseñados para **almacenar, organizar y gestionar datos** de manera eficiente. En proyectos de Data Science y Machine Learning, su función va más allá del simple almacenamiento: son el soporte fundamental para la **persistencia**, **calidad**, **accesibilidad** y **escalabilidad** de los datos.

Persistencia y Accesibilidad

- La **persistencia** garantiza que los datos se mantengan disponibles a lo largo del tiempo, permitiendo reproducir análisis y entrenamientos.
- La **accesibilidad** facilita que diferentes etapas del pipeline (extracción, transformación, modelado) puedan consultar y manipular datos de forma rápida y segura.

Calidad de Datos

- Las bases de datos permiten implementar controles para asegurar la **integridad** y **consistencia** de los datos.
- Evitan problemas como el **data leakage**, que puede invalidar modelos de Machine Learning, asegurando que los datos de entrenamiento y prueba estén correctamente separados.

Escalabilidad

- A medida que los volúmenes de datos crecen, las bases de datos deben soportar cargas mayores sin perder rendimiento.
- Permiten manejar grandes conjuntos de datos, facilitando la actualización y consulta eficiente, lo que es crítico para pipelines en producción.

> **Nota:** Según la documentación de scikit-learn, la preparación adecuada de datos es clave para la validez y desempeño de los modelos, y las bases de datos juegan un papel central en este proceso.

Ejemplo Práctico

Imagina un proyecto de predicción de ventas donde los datos provienen de múltiples fuentes: transacciones, inventarios y campañas de marketing. Una base de datos bien diseñada permite integrar estas fuentes, mantener la calidad y ofrecer acceso rápido para análisis exploratorios y entrenamiento de modelos, asegurando que los datos estén sincronizados y actualizados.

Contexto

En la práctica, entender el rol de las bases de datos te permitirá diseñar pipelines de datos más robustos y reproducibles. Por ejemplo, al trabajar con Python, podrás conectar tus scripts y notebooks a bases de datos para extraer datos limpios y actualizados, evitando la manipulación manual y errores.

Además, conocer cómo las bases de datos soportan la calidad y escalabilidad te ayudará a anticipar problemas comunes en proyectos reales, como datos inconsistentes o cuellos de botella en la consulta, y a implementar soluciones efectivas.

Esta base te prepara para las siguientes unidades, donde profundizaremos en tipos específicos de bases de datos y su integración práctica con Python, facilitando la construcción de pipelines completos y eficientes.

DML aplicado a análisis de datos

En esta unidad, nos enfocaremos en el Lenguaje de Manipulación de Datos (DML) aplicado específicamente a análisis y preparación de datos para ML. Aprenderás a realizar inserciones, actualizaciones y consultas optimizadas que te permitirán extraer features de manera rápida y confiable, integrando estas operaciones en tus pipelines de datos con Python y SQL.

Este conocimiento es fundamental para transformar datos crudos en información lista para modelar, mejorando la eficiencia y calidad de tus proyectos.

Lenguaje de Manipulación de Datos (DML) aplicado a análisis de datos

El **Lenguaje de Manipulación de Datos (DML)** es un subconjunto del SQL que permite modificar y consultar datos dentro de una base de datos. Las sentencias principales son:

- `INSERT` : Añade nuevos registros a una tabla.
- `UPDATE` : Modifica registros existentes.
- `DELETE` : Elimina registros.
- `SELECT` : Consulta y extrae datos.

En el contexto de **Data Science y Machine Learning**, estas operaciones se orientan a preparar y transformar datos para su análisis y modelado.

INSERT: Agregar datos para enriquecer datasets

La sentencia `INSERT` permite incorporar nuevos datos, por ejemplo, al integrar resultados de experimentos o datos externos.

CÓDIGO

```
INSERT INTO features (id, feature1, feature2) VALUES (1, 0.5, 1.2);
```

En Python, usando `sqlite3` o `SQLAlchemy`, puedes automatizar inserciones para alimentar tu base de datos desde pipelines.

UPDATE: Modificar datos para corregir o transformar

`UPDATE` es útil para limpiar o transformar datos existentes, por ejemplo, normalizar valores o corregir errores.

CÓDIGO

```
UPDATE features SET feature1 = feature1 / 100 WHERE feature1 > 100;
```

Esta operación es clave para mantener la calidad de los datos antes del entrenamiento.

DELETE: Eliminar datos irrelevantes o erróneos

Eliminar registros que no aportan valor o que pueden sesgar el modelo.

CÓDIGO

```
DELETE FROM features WHERE feature2 IS NULL;
```

Esto ayuda a evitar problemas en el entrenamiento por datos faltantes o inconsistentes.

SELECT: Consultas eficientes para extracción de features

La sentencia `SELECT` es la base para extraer datos que alimentarán los modelos. Optimizar estas consultas reduce tiempos y recursos.

Ejemplo básico:

CÓDIGO

```
SELECT feature1, feature2 FROM features WHERE label = 'positive';
```

Para optimizar:

- Usa índices en columnas filtradas frecuentemente.
- Selecciona solo las columnas necesarias.
- Aplica filtros para reducir el volumen de datos.
- Evita subconsultas innecesarias.

Integración con Python para reproducibilidad

Utiliza librerías como `pandas` para ejecutar consultas y manipular resultados:

CÓDIGO

```
import pandas as pd
import sqlite3

conn = sqlite3.connect('data.db')
query = "SELECT feature1, feature2 FROM features WHERE label = 'positive'"
df = pd.read_sql_query(query, conn)
```

Esto permite integrar SQL en pipelines reproducibles y versionables.

> **Nota**
> La manipulación eficiente de datos con DML es fundamental para preparar datasets limpios y relevantes, base para cualquier proyecto de Machine Learning exitoso.

Contexto

En proyectos reales de Data Science, la calidad y rapidez en la manipulación de datos impactan directamente en la efectividad del modelo. Por ejemplo, en una empresa de retail que quiere predecir la demanda, extraer rápidamente las variables correctas de grandes bases de datos es crucial para iterar modelos y ajustar estrategias.

Este módulo te prepara para diseñar consultas y operaciones DML que se integren en pipelines reproducibles, facilitando la preparación de datasets listos para entrenamiento. Además, al optimizar estas operaciones, reduces el tiempo de espera y recursos computacionales, acelerando el ciclo de desarrollo.

En la práctica, combinarás sentencias SQL con Python para automatizar la manipulación y extracción de datos, asegurando que cada paso sea transparente y replicable, un requisito indispensable en entornos profesionales y colaborativos.

Te pasamos un esquema para que puedas reproducirlo en tu práctica.

Preparación (instalaciones, imports, settings)

CÓDIGO

```
<h1 class="module" data-eyebrow="MÓDULO">Preparación</h1>
<hr class="module-underline"><hr class="module-hairline">
!pip install python-docx --quiet

import sqlite3
import pandas as pd
import numpy as np
import os
import time
from datetime import datetime, timedelta
from sqlalchemy import create_engine, text
from docx import Document
from docx.shared import Pt

pd.set_option('display.max_columns', None)
np.random.seed(42)

DB_NAME = 'dml_ml.db'
DB_PATH = os.path.join('/content', DB_NAME)
print("Ruta DB:", DB_PATH)
```

Crear la base y la tabla `features`; insertar datos simulados (bulk inserts)

CÓDIGO

```
<h1 class="module" data-eyebrow="MÓDULO">Crear DB y tabla features con datos simulados</h1>
<hr class="module-underline"><hr class="module-hairline">
<h1 class="module" data-eyebrow="MÓDULO">Si existe DB previo, lo removemos para reproducibili
<hr class="module-underline"><hr class="module-hairline">
if os.path.exists(DB_PATH):
 os.remove(DB_PATH)
 print("DB previa eliminada")

conn = sqlite3.connect(DB_PATH)
cur = conn.cursor()

<h1 class="module" data-eyebrow="MÓDULO">Crear tabla features: id, user_id, feature1, feature
<hr class="module-underline"><hr class="module-hairline">
cur.execute("""
CREATE TABLE features (
 id INTEGER PRIMARY KEY AUTOINCREMENT,
 user_id INTEGER NOT NULL,
 feature1 REAL, -- por ejemplo: monto, variable con outliers
 feature2 REAL, -- por ejemplo: tiempo en segundos
 category TEXT, -- categoría/canal
 label INTEGER, -- 0/1 target
 created_at TEXT
);
""")
conn.commit()
print("Tabla features creada.")

<h1 class="module" data-eyebrow="MÓDULO">Generar datos simulados</h1>
<hr class="module-underline"><hr class="module-hairline">
n = 30000 # tamaño razonable para medir tiempos
user_ids = np.random.randint(1000, 2000, size=n)
<h1 class="module" data-eyebrow="MÓDULO">feature1: skewed distribution with outliers</h1>
<hr class="module-underline"><hr class="module-hairline">
feature1 = np.random.exponential(scale=100.0, size=n) # media ~100
<h1 class="module" data-eyebrow="MÓDULO">add some large outliers</h1>
<hr class="module-underline"><hr class="module-hairline">
outlier_idx = np.random.choice(n, size=int(n*0.005), replace=False)
feature1[outlier_idx] *= 30
<h1 class="module" data-eyebrow="MÓDULO">feature2: normal distribution with some NaNs</h1>
<hr class="module-underline"><hr class="module-hairline">
feature2 = np.random.normal(loc=50, scale=15, size=n)
nan_idx = np.random.choice(n, size=int(n*0.02), replace=False)
for i in nan_idx:
 feature2[i] = np.nan
<h1 class="module" data-eyebrow="MÓDULO">category: a few categories</h1>
<hr class="module-underline"><hr class="module-hairline">
cats = ['online','store','partner','mobile']
category = np.random.choice(cats, size=n, p=[0.5,0.25,0.15,0.10])
```

```
<h1 class="module" data-eyebrow="MÓDULO">label: binary with class imbalance</h1>
<hr class="module-underline"><hr class="module-hairline">
label = np.random.choice([0,1], size=n, p=[0.85,0.15])
<h1 class="module" data-eyebrow="MÓDULO">created_at: spread over last 90 days</h1>
<hr class="module-underline"><hr class="module-hairline">
start = datetime.now() - timedelta(days=90)
created_at = [(start + timedelta(seconds=int(np.random.rand()*90*24*3600))).isoformat() for _

<h1 class="module" data-eyebrow="MÓDULO">Prepare rows with occasional NULLs in feature1 (data
<hr class="module-underline"><hr class="module-hairline">
rows = []
for i in range(n):
 f1 = float(feature1[i])
 if np.random.rand() < 0.005: # few missing feature1
 f1 = None
 f2 = None if np.isnan(feature2[i]) else float(feature2[i])
 rows.append((int(user_ids[i]), f1, f2, category[i], int(label[i]), created_at[i]))

<h1 class="module" data-eyebrow="MÓDULO">Bulk insert in batches for performance</h1>
<hr class="module-underline"><hr class="module-hairline">
batch_size = 5000
insert_sql = "INSERT INTO features (user_id, feature1, feature2, category, label, created_at)
start_time = time.perf_counter()
for i in range(0, n, batch_size):
 cur.executemany(insert_sql, rows[i: i+batch_size])
 conn.commit()
end_time = time.perf_counter()
print(f"Insertadas {n} filas en {end_time - start_time:.2f} s")

<h1 class="module" data-eyebrow="MÓDULO">Quick sanity check</h1>
<hr class="module-underline"><hr class="module-hairline">
df_head = pd.read_sql_query("SELECT COUNT(*) as cnt, SUM(CASE WHEN feature1 IS NULL THEN 1 ELS
display(df_head)

conn.close()
```

Consultas exploratorias con pandas (SELECT) y generación de dataset para ML

CÓDIGO

```
<h1 class="module" data-eyebrow="MÓDULO">SELECT con pandas: muestreo y extracción para ML</h1>
<hr class="module-underline"><hr class="module-hairline">
conn = sqlite3.connect(DB_PATH)

<h1 class="module" data-eyebrow="MÓDULO">Total rows</h1>
<hr class="module-underline"><hr class="module-hairline">
total = pd.read_sql_query("SELECT COUNT(*) AS total FROM features;", conn)['total'][0]
print("Total filas:", total)

<h1 class="module" data-eyebrow="MÓDULO">Muestra de 10 filas</h1>
<hr class="module-underline"><hr class="module-hairline">
df_sample = pd.read_sql_query("SELECT * FROM features LIMIT 10;", conn)
display(df_sample)

<h1 class="module" data-eyebrow="MÓDULO">Extraer features para modelado: seleccionar sólo fila
<hr class="module-underline"><hr class="module-hairline">
query_clean = """
SELECT id, user_id, feature1, feature2, category, label
FROM features
WHERE feature1 IS NOT NULL AND feature2 IS NOT NULL AND label IS NOT NULL;
"""
df_clean = pd.read_sql_query(query_clean, conn)
print("Filas limpias para ML:", len(df_clean))
display(df_clean.describe())

<h1 class="module" data-eyebrow="MÓDULO">Guarda un CSV de ejemplo (opcional)</h1>
<hr class="module-underline"><hr class="module-hairline">
df_clean.sample(5).to_csv('features_clean_sample.csv', index=False)
print("CSV de muestra guardado: features_clean_sample.csv")

conn.close()
```

UPDATE: crear columna escalada `feature1_scaled` y normalizar por MAX; y capear outliers en feature1

CÓDIGO

```
<h1 class="module" data-eyebrow="MÓDULO">UPDATE: crear columna calculada persistente y normali
<hr class="module-underline"><hr class="module-hairline">
conn = sqlite3.connect(DB_PATH)
cur = conn.cursor()

<h1 class="module" data-eyebrow="MÓDULO">1) Añadir columna feature1_scaled (si no existe)</h1>
<hr class="module-underline"><hr class="module-hairline">
cols = pd.read_sql_query("PRAGMA table_info(features);", conn)
if 'feature1_scaled' not in cols['name'].values:
 cur.execute("ALTER TABLE features ADD COLUMN feature1_scaled REAL;")
 conn.commit()
 print("Columna feature1_scaled agregada.")

<h1 class="module" data-eyebrow="MÓDULO">2) Calcular max non-null feature1</h1>
<hr class="module-underline"><hr class="module-hairline">
res = cur.execute("SELECT MAX(feature1) FROM features WHERE feature1 IS NOT NULL;").fetchone(
max_f1 = res[0] if res else None
print("Max feature1 (no NULL):", max_f1)

<h1 class="module" data-eyebrow="MÓDULO">3) Actualizar feature1_scaled = feature1 / max_featu
<hr class="module-underline"><hr class="module-hairline">
if max_f1 and max_f1 > 0:
 start = time.perf_counter()
 cur.execute("UPDATE features SET feature1_scaled = feature1 / ? WHERE feature1 IS NOT NULL;"
 conn.commit()
 elapsed = time.perf_counter() - start
 print(f"feature1_scaled actualizada en {elapsed:.3f} s")
else:
 print("No hay max válidos para normalizar.")

<h1 class="module" data-eyebrow="MÓDULO">4) Capear outliers: si feature1 &gt; 3*IQR + Q3 (simp
<hr class="module-underline"><hr class="module-hairline">
<h1 class="module" data-eyebrow="MÓDULO">Calcular quantiles</h1>
<hr class="module-underline"><hr class="module-hairline">
q = pd.read_sql_query("SELECT feature1 FROM features WHERE feature1 IS NOT NULL;", conn)
q1 = q['feature1'].quantile(0.25)
q3 = q['feature1'].quantile(0.75)
iqr = q3 - q1
cap = q3 + 1.5 * iqr
print(f"Outlier cap threshold: {cap:.2f}")

<h1 class="module" data-eyebrow="MÓDULO">Ejecutar UPDATE para capear (en este ejemplo, creamos
<hr class="module-underline"><hr class="module-hairline">
cols = pd.read_sql_query("PRAGMA table_info(features);", conn)
if 'feature1_capped' not in cols['name'].values:
 cur.execute("ALTER TABLE features ADD COLUMN feature1_capped REAL;")
 conn.commit()
```

```
start = time.perf_counter()
cur.execute("UPDATE features SET feature1_capped = CASE WHEN feature1 IS NULL THEN NULL WHEN
conn.commit()
elapsed = time.perf_counter() - start
print(f"feature1_capped calculada en {elapsed:.3f} s")

<h1 class="module" data-eyebrow="MÓDULO">Ver cambios (muestra)</h1>
<hr class="module-underline"><hr class="module-hairline">
display(pd.read_sql_query("SELECT id, feature1, feature1_scaled, feature1_capped FROM features

conn.close()
```

DELETE: eliminar registros con datos faltantes irreparables (ej. label NULL o feature1 NULL y feature2 NULL)

CÓDIGO

```
<h1 class="module" data-eyebrow="MÓDULO">DELETE registros inválidos (ejemplo)</h1>
<hr class="module-underline"><hr class="module-hairline">
conn = sqlite3.connect(DB_PATH)
cur = conn.cursor()

<h1 class="module" data-eyebrow="MÓDULO">Conteo antes</h1>
<hr class="module-underline"><hr class="module-hairline">
before = cur.execute("SELECT COUNT(*) FROM features;").fetchone()[0]
print("Filas antes:", before)

<h1 class="module" data-eyebrow="MÓDULO">Borramos filas donde label IS NULL OR (feature1 IS NU
<hr class="module-underline"><hr class="module-hairline">
start = time.perf_counter()
cur.execute("DELETE FROM features WHERE label IS NULL OR (feature1 IS NULL AND feature2 IS NUL
deleted = cur.rowcount
conn.commit()
elapsed = time.perf_counter() - start

after = cur.execute("SELECT COUNT(*) FROM features;").fetchone()[0]
print(f"Deleted rows (approx - SQLite rowcount): {deleted}; tiempo: {elapsed:.3f} s")
print("Filas después:", after)

conn.close()
```

SELECT optimizado: medir tiempos de agregación sin índice y con índice

CÓDIGO

```
<h1 class="module" data-eyebrow="MÓDULO">Medir tiempo de consulta agregada antes y después de
<hr class="module-underline"><hr class="module-hairline">
conn = sqlite3.connect(DB_PATH)

def time_query(sql, runs=3):
 times = []
 for _ in range(runs):
 t0 = time.perf_counter()
 pd.read_sql_query(sql, conn)
 times.append(time.perf_counter() - t0)
 return np.mean(times), times

agg_sql = """
SELECT category, label, COUNT(*) AS cnt,
 AVG(feature1_capped) AS avg_f1, AVG(feature2) AS avg_f2
FROM features
GROUP BY category, label;
"""

print("Ejecutando agregación (sin índices)...")
mean_noidx, times_noidx = time_query(agg_sql, runs=3)
print(f"Tiempo medio sin índice: {mean_noidx:.4f} s; repeticiones: {times_noidx}")

<h1 class="module" data-eyebrow="MÓDULO">Crear índice sobre columns usadas en GROUP BY / WHERE
<hr class="module-underline"><hr class="module-hairline">
cur = conn.cursor()
print("Creando índice idx_features_category_label...")
start = time.perf_counter()
cur.execute("CREATE INDEX IF NOT EXISTS idx_features_category_label ON features(category, labe
conn.commit()
idx_time = time.perf_counter() - start
print(f"Índice creado en {idx_time:.4f} s")

print("Ejecutando agregación (CON índice)...")
mean_idx, times_idx = time_query(agg_sql, runs=3)
print(f"Tiempo medio con índice: {mean_idx:.4f} s; repeticiones: {times_idx}")

<h1 class="module" data-eyebrow="MÓDULO">Mostrar resultados de la agregación</h1>
<hr class="module-underline"><hr class="module-hairline">
df_agg = pd.read_sql_query(agg_sql, conn)
display(df_agg)

conn.close()
```

Uso de SQLAlchemy para insertar un pequeño batch y demostración de to_sql desde pandas

CÓDIGO

```
<h1 class="module" data-eyebrow="MÓDULO">SQLAlchemy demo: insertar un pequeño batch con engine
<hr class="module-underline"><hr class="module-hairline">
engine = create_engine(f"sqlite:///{DB_PATH}", echo=False)

<h1 class="module" data-eyebrow="MÓDULO">Crear un pequeño DataFrame nuevo y agregarlo vía to_s
<hr class="module-underline"><hr class="module-hairline">
df_new = pd.DataFrame({
 'user_id': [9999, 10000],
 'feature1': [123.45, 67.89],
 'feature2': [12.3, 45.6],
 'category': ['online','store'],
 'label': [0,1],
 'created_at': [datetime.now().isoformat(), datetime.now().isoformat()],
 'feature1_scaled': [None, None],
 'feature1_capped': [None, None]
})

<h1 class="module" data-eyebrow="MÓDULO">to_sql requiere que las columnas existan; nos asegura
<hr class="module-underline"><hr class="module-hairline">
with engine.begin() as conn:
 df_new.to_sql('features', conn, if_exists='append', index=False)
print("Batch insert con SQLAlchemy realizado. Revisar últimas filas:")

with engine.connect() as conn:
 display(pd.read_sql_query("SELECT * FROM features ORDER BY id DESC LIMIT 5;", conn))
```

DCL y Seguridad de datos

Exploraremos los fundamentos del Control de Lenguaje de Datos (DCL) y su papel crucial en la seguridad y gobernanza de datos. Comprenderás cómo los permisos, roles y políticas de acceso protegen la integridad y privacidad de los datos, un conocimiento esencial para cualquier profesional que trabaje con datos en entornos reales. Además, aprenderás a identificar riesgos comunes de exposición y aplicar buenas prácticas para mitigarlos, asegurando que tus proyectos de análisis y Machine Learning se desarrollen en un marco seguro y confiable.

Fundamentos de DCL y Seguridad de Datos

El **Control de Lenguaje de Datos (DCL)** es un conjunto de comandos y mecanismos dentro de los sistemas de gestión de bases de datos (DBMS) que permiten administrar los **permisos de acceso** y garantizar la **seguridad** y **gobernanza** de los datos.

Conceptos Clave

- **Permisos (Privileges):** Derechos específicos otorgados a usuarios o roles para realizar acciones como `SELECT`, `INSERT`, `UPDATE`, `DELETE`.
- **Roles:** Conjuntos de permisos agrupados para facilitar la gestión de accesos.
- **Control de Acceso:** Mecanismos que regulan quién puede ver o modificar datos.
- **Gobernanza de Datos:** Conjunto de políticas, procesos y controles que aseguran la calidad, seguridad y uso adecuado de los datos.

> **Nota**
> La gobernanza no solo protege los datos, sino que también asegura que se usen de manera ética y conforme a regulaciones.

Importancia en Data Science

En proyectos de Data Science, el acceso indiscriminado a datos sensibles puede generar riesgos legales y de privacidad. Por ello, implementar controles DCL es fundamental para:

- Proteger datos personales y confidenciales.
- Mantener la integridad de los datasets.
- Cumplir con normativas como GDPR o leyes locales.

Ejemplo Práctico en SQL

CÓDIGO

```
-- Crear un rol con permisos de solo lectura
CREATE ROLE lector_datos;
GRANT SELECT ON tabla_clientes TO lector_datos;

-- Asignar rol a un usuario
GRANT lector_datos TO usuario_analista;
```

Este ejemplo muestra cómo se puede limitar el acceso para que un analista solo pueda consultar datos sin modificarlos.

Riesgos Comunes

- **Exposición accidental:** Usuarios con permisos excesivos pueden filtrar datos.
- **Modificaciones no autorizadas:** Cambios que afectan la calidad del dataset.
- **Falta de auditoría:** Imposibilidad de rastrear accesos o cambios.

> **Nota**
> **Reflexión.** ¿Cómo podrías aplicar estos controles en tu entorno actual para minimizar riesgos?

Contexto

En entornos reales de Data Science, los datos provienen de múltiples fuentes y pueden contener información sensible, como datos personales o financieros. Por ejemplo, un equipo que desarrolla modelos predictivos para una entidad financiera debe asegurarse de que solo usuarios autorizados accedan a datos de clientes.

La implementación de DCL permite definir claramente quién puede acceder a qué datos y qué operaciones puede realizar, facilitando la colaboración segura entre equipos multidisciplinarios.

Además, la gobernanza de datos establece políticas que regulan el ciclo de vida de los datos, desde su creación hasta su eliminación, asegurando cumplimiento normativo y calidad.

Este conocimiento es clave para diseñar pipelines reproducibles y seguros, integrando Python con sistemas de almacenamiento que respeten estos controles, preparando datasets confiables para entrenamiento de modelos.

Herramientas distribuidas en Python

¿Por qué necesitamos herramientas distribuidas en Data Science?

Imagina que tienes un conjunto de datos que no cabe en la memoria de tu computadora, o que las transformaciones que aplicas tardan horas en completarse. ¿Cómo escalarías tu procesamiento para manejar estos volúmenes y tiempos?

En este contexto, las herramientas distribuidas como Apache Spark y Dask se vuelven esenciales para llevar tus proyectos de Data Science al siguiente nivel.

En esta unidad, exploraremos los conceptos fundamentales de Spark y Dask, dos motores distribuidos populares en el ecosistema Python.

Introducción a Apache Spark

Apache Spark es un motor de procesamiento distribuido diseñado para manejar grandes volúmenes de datos a través de clusters de computadoras. Spark permite ejecutar tareas de procesamiento en paralelo, acelerando transformaciones y análisis que serían lentos o imposibles en una sola máquina.

- **Arquitectura:** Spark utiliza un modelo de ejecución basado en RDDs (Resilient Distributed Datasets) y DataFrames, que abstraen la distribución de datos.
- **Lenguajes soportados:** Python (a través de PySpark), Scala, Java, R.
- **Casos de uso:** procesamiento batch, streaming, machine learning a gran escala.

> **Ejemplo**
>
> Empresas como Netflix y Uber usan Spark para procesar grandes volúmenes de datos en tiempo real y alimentar sus modelos predictivos.

Introducción a Dask

Dask es una biblioteca de Python que extiende las capacidades de procesamiento paralelo y distribuido, integrándose de forma nativa con herramientas como NumPy, Pandas y scikit-learn.

- **Arquitectura:** Dask construye grafos de tareas que se ejecutan en paralelo, ya sea en una sola máquina con múltiples núcleos o en clusters.
- **Ventaja:** Permite escalar código Python existente con mínima reescritura.
- **Casos de uso:** procesamiento de datos que exceden la memoria local, paralelización de cálculos científicos.

> **Ejemplo**
> Un analista que usa Pandas puede migrar a Dask para manejar datasets más grandes sin cambiar radicalmente su código.

Comparación entre Spark y Dask

Característica	Apache Spark	Dask
Lenguaje principal	Scala, PySpark (Python)	Python nativo
Integración Python	A través de PySpark (API menos "pythónica")	Nativa, compatible con Pandas, NumPy
Facilidad de uso	Curva de aprendizaje más pronunciada	Más sencillo para usuarios de Python
Escalabilidad	Altamente escalable en clusters grandes	Escala bien en clusters pequeños y medianos
Casos de uso típicos	Big Data, procesamiento batch y streaming	Procesamiento paralelo y distribuido en Python

¿Cuándo migrar a un motor distribuido?

Migrar tus transformaciones a Spark o Dask es recomendable cuando:

- Tus datos no caben en la memoria de una sola máquina.
- El tiempo de procesamiento es excesivo para tareas críticas.
- Necesitas paralelizar cálculos para acelerar experimentos.
- Quieres integrar procesamiento distribuido en pipelines reproducibles.

> **Nota**
> No siempre es necesario migrar; para datasets pequeños o medianos, optimizar código local puede ser suficiente.

Con este panorama, estás listo para explorar cómo estas herramientas pueden integrarse en tus proyectos y cuándo es estratégico adoptarlas.

Contexto

En proyectos reales de Data Science, la escalabilidad es un desafío constante. Por ejemplo, una empresa que analiza logs de usuarios puede enfrentar volúmenes de datos que crecen exponencialmente. Usar Pandas para procesar estos datos puede volverse inviable.

Aquí es donde Spark y Dask entran en juego, permitiendo distribuir el procesamiento en múltiples nodos o núcleos, reduciendo tiempos y permitiendo análisis más complejos.

Además, la elección entre Spark y Dask dependerá del entorno tecnológico, el tamaño del cluster disponible, y la familiaridad del equipo con Python o ecosistemas JVM.

Modelado de esquemas y relaciones: de ER a tablas

Imagina que estás trabajando como Data Scientist para una empresa similar a Amazon. Tienes millones de registros de usuarios, productos, envíos y reseñas. Si guardaras toda esta información en una sola y gigantesca hoja de cálculo de Excel, pronto te encontrarías con un caos: si un cliente cambia su dirección, tendrías que buscar y actualizar miles de filas donde ese cliente realizó una compra. ¿Qué pasa si olvidas una? Los datos pierden coherencia.

Este es el problema que resuelve el **modelado de datos**. No se trata solo de crear tablas; se trata de diseñar un mapa lógico que garantice que la información sea veraz, no esté duplicada y sea fácil de consultar.

Para un profesional de Machine Learning, un buen modelo es la diferencia entre un dataset limpio y listo para entrenar, o semanas de limpieza de datos por inconsistencias estructurales.

En esta unidad, aprenderás a transformar un concepto de negocio en un **Diagrama Entidad-Relación (ER)** y, finalmente, en un esquema de tablas relacionales.

1. El Proceso de Modelado: De la idea al diseño

El modelado de datos no ocurre directamente en la base de datos. Pasa por tres etapas fundamentales que van de lo abstracto a lo concreto:

1. **Modelo Conceptual:** Es el nivel más alto. Aquí identificamos qué "cosas" (entidades) queremos rastrear. Ejemplo: "Necesitamos guardar Clientes y sus Pedidos".
2. **Modelo Lógico (Diagrama ER):** Aquí definimos cómo se conectan esas entidades y qué datos (atributos) tiene cada una. Usamos una notación visual.
3. **Modelo Físico (Tablas):** Es la implementación real en SQL. Aquí definimos tipos de datos (INTEGER, VARCHAR), claves primarias y foráneas.

2. Diagrama Entidad-Relación (ER): Los bloques de construcción

El Diagrama ER es el lenguaje universal para diseñar bases de datos. Sus componentes principales son:

A. Entidades (¿Sobre qué guardamos datos?)

Una entidad es un objeto o concepto del mundo real que es distinguible de otros. Se representan con **rectángulos**.

- **Ejemplos:** `Cliente`, `Producto`, `Vuelo`, `Curso`.

B. Atributos (¿Qué características nos interesan?)

Son las propiedades que describen a una entidad. Se representan con **óvalos** conectados a la entidad.

- **Ejemplos para un `Cliente`:** `Nombre`, `Email`, `Fecha_Registro`.

C. Relaciones (¿Cómo se conectan?)

Representan la asociación entre dos o más entidades. Se dibujan con un **rombo** que conecta las entidades.

- **Ejemplo:** Un `Cliente` **realiza** un `Pedido`.

> **Ejemplo**
>
> **Ejemplo del mundo real.** En una plataforma como **Netflix**, `Usuario` y `Película` son entidades. La relación es `Visualización`. Un atributo de `Visualización` podría ser la `Fecha` en la que se vio la película.

3. Claves Primarias y Foráneas: El pegamento de las tablas

Una vez que tenemos nuestro diagrama, debemos llevarlo al mundo de las tablas. Para que esto funcione, necesitamos un sistema de identificación y conexión.

Clave Primaria (Primary Key - PK)

Es un atributo (o conjunto de ellos) que identifica de forma **única e irrepetible** a cada fila de una tabla.

- **Regla de oro:** No puede haber dos filas con la misma PK y nunca puede ser nula (vacía).
- **Ejemplo:** En una tabla de `Usuarios`, el `ID_Usuario` o el `DNI` funcionan como PK.

Clave Foránea (Foreign Key - FK)

Es un atributo en una tabla que "apunta" a la clave primaria de otra tabla. Es el mecanismo que crea el vínculo físico entre los datos.

- **Ejemplo:** Si un cliente con `ID_Usuario = 5` hace un pedido, la tabla `Pedidos` tendrá una columna llamada `ID_Usuario` con el valor `5`. Ese campo es una FK que nos permite saber exactamente quién hizo la compra sin tener que repetir el nombre y el email del usuario en la tabla de pedidos.

4. Cardinalidad: Entendiendo las proporciones

La cardinalidad define cuántas instancias de una entidad se relacionan con cuántas instancias de otra. Es vital para decidir cómo estructurar las tablas.

1. Relación Uno a Uno (1:1)

Un registro en la Tabla A se relaciona con exactamente un registro en la Tabla B.

- **Ejemplo:** `Ciudadano` y `Pasaporte`. Un ciudadano tiene un solo pasaporte oficial, y un pasaporte pertenece a un solo ciudadano.
- **En tablas:** Normalmente se añade el ID de una tabla como FK en la otra, o se fusionan en una sola tabla si la relación es muy estrecha.

2. Relación Uno a Muchos (1:N)

Un registro en la Tabla A puede estar relacionado con muchos en la Tabla B, pero un registro en B solo se relaciona con uno en A. **Esta es la relación más común.**

- **Ejemplo:** `Cliente` y `Pedidos`. Un cliente puede hacer muchos pedidos en un año, pero cada pedido individual pertenece a un solo cliente.
- **En tablas:** La PK del lado "1" (Cliente) viaja a la tabla del lado "N" (Pedidos) como FK.

3. Relación Muchos a Muchos (N: M)

Muchos registros en la Tabla A se relacionan con muchos en la B.

- **Ejemplo:** `Estudiantes` y `Cursos`. Un estudiante se inscribe en muchas materias, y una materia tiene muchos estudiantes inscritos.
- **El desafío técnico:** Las bases de datos relacionales **no pueden implementar una relación N: M directamente** entre dos tablas.
- **La solución:** Se crea una **tabla intermedia** (también llamada tabla de unión o *junction table*). Esta tabla contiene las PKs de ambas entidades como FKs. En el caso anterior, crearíamos una tabla llamada `Inscripciones` que contenga `ID_Estudiante` e `ID_Curso`.

5. De la Teoría a la Práctica: Transformando el ER a Tablas

Sigamos el flujo completo con un ejemplo de un sistema de **Biblioteca**:

1. **Modelo Conceptual:** Queremos gestionar Socios, Libros y los Préstamos que se realizan.
2. **Identificar Relaciones:** * Un `Socio` puede pedir muchos `Libros` a lo largo del tiempo. * Un `Libro` físico puede ser pedido por muchos `Socios` (en diferentes fechas). * Estamos ante una relación **Muchos a Muchos (N: M)** entre `Socio` y `Libro`.
3. **Diseño de Tablas:** * **Tabla Socios:** `ID_Socio` (PK), `Nombre`, `Direccion`. * **Tabla Libros:** `ID_Libro` (PK), `Titulo`, `Autor`, `ISBN`. * **Tabla Préstamos (Intermedia):** `ID_Prestamo` (PK), `ID_Socio` (FK), `ID_Libro` (FK), `Fecha_Salida`, `Fecha_Devolucion`.

Observa que la tabla `Préstamos` no solo resuelve la relación N: M, sino que además guarda atributos propios de esa interacción (las fechas). Sin esta estructura, sería imposible saber quién tiene qué libro de forma organizada.

6. Errores comunes que debes evitar

Como principiante en Data Science, es normal cometer estos errores en tus primeros modelos:

- **Error 1: "Atributos Multivaluados".** Intentar meter varios datos en una sola celda (ej. en la columna `Telefonos` poner "555-123, 555-456").
- *Por qué es un problema:* SQL no puede "entender" fácilmente que hay dos números. Hace que filtrar o buscar un número específico sea una pesadilla de procesamiento.
- *Solución:* Crea una tabla separada para teléfonos o usa una relación 1:N.
- **Error 2: "Redundancia innecesaria".** Guardar el nombre del autor en la tabla de libros cada vez que aparece el autor.
- *Por qué es un problema:* Si el autor cambia su nombre artístico, tendrías que actualizarlo en 50 libros diferentes.
- *Solución:* Una tabla de `Autores` y un `ID_Autor` (FK) en la tabla `Libros`.
- **Error 3: Ignorar la Clave Primaria.** Pensar que no necesitas un ID porque "el nombre ya es único".
- *Por qué es un problema:* Los nombres cambian y se repiten (hay muchos "Juan Pérez"). Un ID numérico generado por el sistema es siempre la opción más segura y rápida para las consultas.

Resumen de la Unidad

- El **modelado** permite estructurar la información antes de escribir una sola línea de código SQL.
- El **Diagrama Entidad-Relación** utiliza rectángulos (entidades), óvalos (atributos) y rombos (relaciones).
- Las **Claves Primarias (PK)** identifican de forma única, mientras que las **Claves Foráneas (FK)** conectan tablas.
- La **cardinalidad** (1:1, 1:N, N: M) dicta cómo se diseñan las tablas y dónde se colocan las FK.
- Las relaciones **Muchos a Muchos (N: M)** siempre requieren una tabla intermedia para funcionar correctamente en el modelo relacional.

Comprender este flujo es esencial para cualquier analista. En el Machine Learning, los datos que alimentan tus modelos provienen de estas estructuras; si el modelo de base de datos está roto, tus datos estarán sesgados o incompletos.

Bases de datos modernas y ML avanzado

Bases de datos vectoriales y embeddings: Introducción

Las **bases de datos vectoriales** son sistemas diseñados para almacenar y consultar datos en forma de vectores numéricos, que representan características o *features* de objetos complejos, como texto, imá-

genes o audio. A diferencia de las bases de datos tradicionales que manejan datos estructurados (tablas, filas), las vector DB permiten realizar búsquedas basadas en similitud semántica mediante cálculos de distancia en espacios vectoriales.

¿Qué son los embeddings?

Los **embeddings** son representaciones numéricas densas de datos no estructurados, como palabras, frases o documentos, en espacios vectoriales de alta dimensión. Por ejemplo, un embedding de una palabra captura su significado contextual en un vector de números reales, facilitando comparaciones y operaciones matemáticas.

> **Ejemplo**
> En procesamiento de lenguaje natural (NLP), modelos como Word2Vec o BERT generan embeddings que permiten identificar palabras con significados similares aunque no coincidan textualmente.

Funcionamiento básico de una base de datos vectorial

- **Almacenamiento:** Cada objeto (texto, imagen) se convierte en un vector mediante un modelo de embeddings y se almacena.
- **Consulta:** Se transforma la consulta en un vector y se busca en la base los vectores más cercanos según una métrica (e.g., distancia coseno, Euclidiana).
- **Resultado:** Se recuperan los objetos con mayor similitud semántica.

Aplicaciones en la industria

- Búsqueda semántica en motores de búsqueda y asistentes virtuales.
- Recomendaciones personalizadas basadas en similitud de contenido.
- Detección de fraude y análisis de imágenes médicas.

Retrieval-Augmented Generation (RAG)

RAG es una técnica que combina la recuperación de información con modelos generativos de lenguaje para mejorar la calidad y precisión de las respuestas generadas.

- Primero, se recuperan documentos relevantes desde una base de datos (usualmente vectorial).
- Luego, un modelo generativo (como GPT) utiliza esta información para generar respuestas contextuales y fundamentadas.

> **Ejemplo**
> En chatbots avanzados, RAG permite responder preguntas complejas usando información actualizada y específica, superando limitaciones de modelos entrenados solo con datos estáticos.

Relación con pipelines de ML

Integrar bases de datos vectoriales y RAG en pipelines permite:

- Mejorar la etapa de consulta y recuperación de datos relevantes.
- Facilitar la generación de insights y respuestas basadas en datos actualizados.
- Optimizar procesos de entrenamiento y evaluación con datos enriquecidos.

Conoce los términos clave:

Término	Definición breve
Embedding	Vector numérico que representa datos complejos.
Base de datos vectorial	Sistema para almacenar y consultar vectores.
RAG	Técnica que combina recuperación y generación.

Contexto

En proyectos reales de Data Science y Machine Learning, la capacidad de buscar y recuperar información relevante de manera eficiente es crucial. Por ejemplo, en sistemas de recomendación, no basta con buscar coincidencias exactas; se requiere entender la similitud semántica entre productos o contenidos.

Las bases de datos vectoriales permiten almacenar representaciones numéricas (embeddings) que capturan esta similitud, facilitando búsquedas rápidas y precisas en grandes volúmenes de datos no estructurados.

Por otro lado, RAG representa un avance significativo en la interacción con modelos de lenguaje, al integrar recuperación de datos con generación de texto, lo que mejora la relevancia y precisión de las respuestas en aplicaciones como asistentes virtuales, chatbots y sistemas de soporte.

En esta unidad, sentamos las bases conceptuales para que puedas diseñar pipelines que integren estas tecnologías, preparándote para el taller integrador donde aplicarás estos conceptos en un proyecto completo.

Introducción a Machine Learning

¿Qué es Machine Learning?

Machine Learning es una rama de la inteligencia artificial que permite a los sistemas aprender y mejorar automáticamente a partir de datos, sin ser explícitamente programados para cada tarea específica.

En lugar de reglas fijas, ML utiliza algoritmos que identifican patrones y hacen predicciones o decisiones basadas en datos.

Objetivos principales de ML

- **Predicción:** Estimar valores futuros o desconocidos (por ejemplo, predecir ventas o detectar fraudes).
- **Clasificación:** Asignar etiquetas o categorías a datos (como identificar correos spam).
- **Descubrimiento de patrones:** Encontrar estructuras ocultas en datos complejos.

Métricas básicas para evaluar modelos

Para saber si un modelo ML es bueno, usamos métricas que cuantifican su desempeño:

Métrica	Descripción	Ejemplo de uso
Precisión	Proporción de predicciones correctas	Clasificación de emails
Recall	Proporción de casos positivos detectados	Diagnóstico médico
Error cuadrático medio (MSE)	Promedio de errores al cuadrado en predicciones continuas	Predicción de precios

> **Nota**
> La elección de la métrica depende del problema y contexto.

Overfitting: un riesgo común

El *overfitting* ocurre cuando un modelo aprende demasiado bien los datos de entrenamiento, incluyendo ruido o detalles irrelevantes, perdiendo capacidad de generalizar a datos nuevos. Esto resulta en un rendimiento excelente en entrenamiento pero pobre en producción.

> **Ejemplo**
> Un modelo que memoriza respuestas en lugar de aprender patrones.

Pipeline típico de Machine Learning

Un pipeline ML es una secuencia estructurada de pasos para transformar datos crudos en predicciones útiles:

1. **Recolección de datos**
2. **Preprocesamiento y limpieza**
3. **Selección y extracción de características**
4. **Entrenamiento del modelo**
5. **Evaluación y validación**
6. **Despliegue y monitoreo**

Este flujo asegura reproducibilidad y facilita iteraciones eficientes.

Este contenido introduce los conceptos clave de ML, preparando el terreno para profundizar en tipos de aprendizaje y preparación de datos en las siguientes unidades.

Contexto

En proyectos reales de Data Science, Machine Learning es la etapa donde los datos procesados se convierten en modelos predictivos que aportan valor. Por ejemplo, en una empresa de retail, después de limpiar y transformar datos de ventas y clientes, se puede entrenar un modelo para predecir la demanda futura o segmentar clientes para campañas personalizadas.

Entender el pipeline de ML y los riesgos como el overfitting es fundamental para diseñar soluciones robustas y evitar errores costosos. Además, conocer las métricas básicas permite evaluar objetivamente el desempeño y tomar decisiones informadas.

Esta unidad conecta con lo aprendido sobre bases de datos y pipelines de datos, mostrando cómo los datos almacenados y transformados alimentan modelos ML. También prepara para las siguientes unidades donde se explorarán tipos de aprendizaje y técnicas prácticas de preparación y modelado.

Tipos de aprendizaje

En Machine Learning, el **tipo de aprendizaje** define cómo un modelo aprende a partir de los datos.

Los tres principales paradigmas son:

1. Aprendizaje Supervisado

El aprendizaje supervisado consiste en entrenar un modelo con un conjunto de datos etiquetados, donde cada entrada tiene una salida conocida. El objetivo es que el modelo aprenda a predecir la salida correcta para nuevas entradas.

- **Ejemplo:** Clasificación de correos electrónicos como "spam" o "no spam".
- **Casos de uso en la industria:** Detección de fraude en transacciones, predicción de demanda, diagnóstico médico.

Técnicas comunes:

- Regresión lineal y logística
- Árboles de decisión
- Máquinas de vectores de soporte (SVM)
- Redes neuronales básicas

2. Aprendizaje No Supervisado

Aquí, el modelo recibe datos sin etiquetas y debe encontrar patrones o estructuras subyacentes.

- **Ejemplo:** Agrupar clientes según comportamiento de compra (clustering).

- **Casos de uso en la industria:** Segmentación de mercado, detección de anomalías, reducción de dimensionalidad.

Técnicas comunes:

- Clustering (K-means, DBSCAN)
- Análisis de componentes principales (PCA)
- Mapas autoorganizados

3. Aprendizaje por Refuerzo

Este paradigma se basa en que un agente aprende a tomar decisiones mediante interacciones con un entorno, recibiendo recompensas o penalizaciones.

- **Ejemplo:** Un robot que aprende a navegar un laberinto.
- **Casos de uso en la industria:** Optimización de rutas, juegos, sistemas de recomendación adaptativos.

Características clave:

- No requiere datos etiquetados
- Aprende a través de prueba y error
- Utiliza políticas y funciones de recompensa

> **Nota**
> En este módulo, nos enfocaremos en comprender estos tipos y sus aplicaciones prácticas, sin profundizar en algoritmos avanzados de refuerzo, manteniendo la complejidad adecuada para el nivel intermedio.

Aplicación práctica de los tipos de aprendizaje

En proyectos reales de Data Science, elegir el tipo de aprendizaje correcto es crucial para el éxito del modelo y la eficiencia del pipeline.

- **Aprendizaje supervisado** es ideal cuando se dispone de datos históricos con resultados conocidos, por ejemplo, para predecir ventas futuras o clasificar imágenes.
- **Aprendizaje no supervisado** se utiliza cuando no hay etiquetas disponibles, pero se desea descubrir patrones, como segmentar clientes para campañas de marketing personalizadas.
- **Aprendizaje por refuerzo** es útil en entornos dinámicos donde las decisiones afectan el estado futuro, como en sistemas de recomendación que se adaptan al comportamiento del usuario.

En la práctica, integrar estos tipos en pipelines reproducibles implica preparar los datos adecuadamente, seleccionar y parametrizar modelos, y validar resultados con métricas específicas.

Esta unidad te prepara para identificar el enfoque correcto y conectar con la preparación de datos y construcción de pipelines que veremos en las siguientes unidades.

Preparación de datos para ML

La preparación de datos es un paso crítico que impacta directamente en el desempeño y la confiabilidad de los modelos de ML. Incluye varias etapas clave:

Limpieza y manejo de valores faltantes

- **Detección de valores faltantes:** Identificar `NaN`, valores nulos o inconsistentes.
- **Estrategias de imputación:** Reemplazo con media, mediana, moda, o técnicas avanzadas como imputación basada en modelos.
- **Eliminación de registros o variables:** Cuando la imputación no es viable o los datos son irrelevantes.

> **Ejemplo:** En un dataset de clientes, si la edad está ausente en pocos registros, imputar con la mediana puede ser adecuado.

Feature Engineering

- **Creación de nuevas variables:** Derivar características que capturen mejor la información relevante (e.g., transformar fechas en días de la semana).
- **Transformaciones:** Escalado, normalización, codificación de variables categóricas (one-hot, label encoding).
- **Selección de características:** Identificar y conservar las variables más relevantes para el modelo.

> **Ejemplo**
> Convertir una variable categórica 'tipo de cliente' en variables dummy para modelos que no manejan categorías.

División de datasets

- **Split reproducible:** Separar datos en conjuntos de entrenamiento, validación y prueba usando semillas (`random_state`) para reproducibilidad.
- **Proporciones comunes:** 70% entrenamiento, 15% validación, 15% prueba.
- **Estratificación:** Mantener la distribución de clases en cada subset para problemas de clasificación.

Prevención de Data Leakage

- **¿Qué es?** Cuando información del conjunto de prueba se filtra al entrenamiento, generando modelos con rendimiento artificialmente alto.

- **Causas comunes:** Aplicar transformaciones antes de dividir, usar variables derivadas del futuro, o incluir datos no disponibles en producción.
- **Buenas prácticas:** Encadenar transformaciones dentro de pipelines, aplicar fit solo al conjunto de entrenamiento.

Visualización y análisis exploratorio (REINFORCE)

- Uso de **Matplotlib**, **Seaborn** y **Plotly** para:
- Detectar outliers y patrones
- Visualizar distribuciones y correlaciones
- Comunicar insights a stakeholders

> **Nota**
> La visualización es clave para validar hipótesis y detectar problemas antes de modelar.

Contexto

En proyectos reales de Data Science, la calidad y estructura de los datos determinan el éxito del modelo. Por ejemplo, en la industria financiera, un mal manejo de valores faltantes o un data leakage puede llevar a decisiones erróneas con impacto económico.

Por ello, integrar la preparación de datos dentro de pipelines reproducibles es esencial para mantener la integridad y trazabilidad.

Además, la visualización exploratoria permite a los equipos técnicos y de negocio entender el comportamiento de los datos, facilitando la toma de decisiones informadas y la detección temprana de problemas.

Esta unidad conecta directamente con los conceptos previos de integración de bases de datos y transformación de datos en Python, y prepara el terreno para la construcción de pipelines completos de ML en la siguiente unidad.

Integración ML + pipelines de datos

Flujos end-to-end en Machine Learning

Un **pipeline end-to-end** en ML es un proceso automatizado que conecta la extracción de datos, su transformación y preparación, el entrenamiento del modelo y, finalmente, su despliegue o evaluación.

Este flujo garantiza que cada paso sea reproducible y escalable, facilitando la colaboración y mantenimiento.

Componentes clave del pipeline

- **Extracción desde bases de datos:** Acceso eficiente y reproducible a los datos originales, usando consultas SQL o APIs.

- **Transformaciones reproducibles:** Limpieza, normalización y feature engineering aplicados de forma consistente.
- **Entrenamiento del modelo:** Uso de frameworks como scikit-learn para ajustar modelos con los datos preparados.
- **Despliegue básico:** Guardar modelos entrenados y preparar para inferencia o evaluación.

Importancia de la reproducibilidad

La reproducibilidad implica que cualquier persona o sistema pueda ejecutar el pipeline y obtener los mismos resultados, lo que es crítico en entornos productivos y colaborativos.

> **Nota**
> En Python, librerías como `pandas` para manipulación de datos y `scikit-learn` para ML son estándar para construir estos pipelines.

Ejemplo conceptual de pipeline

CÓDIGO

```
import pandas as pd
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

<h1 class="module" data-eyebrow="MÓDULO">Extracción (simulada)</h1>
<hr class="module-underline"><hr class="module-hairline">
data = pd.read_sql('SELECT * FROM tabla', con=conexion)

<h1 class="module" data-eyebrow="MÓDULO">Pipeline de transformación y entrenamiento</h1>
<hr class="module-underline"><hr class="module-hairline">
pipeline = Pipeline([
 ('scaler', StandardScaler()),
 ('model', LogisticRegression())
])

pipeline.fit(data[features], data[target])
```

Este ejemplo ilustra cómo integrar transformación y modelo, pero en un flujo real se debe incluir la extracción y almacenamiento reproducible de datos.

Contexto

En proyectos reales de Data Science, los datos suelen residir en bases de datos relacionales o NoSQL. La extracción directa y reproducible de estos datos es el primer paso para construir modelos confiables. Por ejemplo, un analista puede automatizar consultas SQL para obtener datasets actualizados periódicamente.

Luego, las transformaciones deben ser aplicadas de forma consistente para evitar sesgos o errores. Esto se logra mediante scripts o pipelines que documentan cada paso.

Finalmente, el entrenamiento y despliegue del modelo deben integrarse en el mismo flujo para facilitar actualizaciones y mantenimiento.

En la industria, esta integración permite responder rápidamente a cambios en los datos o requerimientos.

Este enfoque es especialmente relevante en entornos donde la trazabilidad y la calidad del modelo son críticas, como finanzas, salud o marketing digital.

> **Ejemplo**
> Una empresa de e-commerce puede extraer datos de ventas desde su base de datos, transformar variables relevantes, entrenar un modelo de predicción de demanda y desplegarlo para optimizar inventarios, todo en un pipeline reproducible y automatizado.

Normalización básica: evitar redundancias y dependencias incorrectas

Normalización: El Arte de Organizar Datos para Analíticos

Imagina que estás intentando entrenar un modelo de Machine Learning para predecir ventas, pero tus datos están en un caos: una columna llamada "Productos" contiene una lista de 5 artículos distintos separados por comas, y la dirección del cliente aparece repetida y escrita de forma diferente en cada transacción. ¿Cómo sabría tu modelo cuál es el precio individual de un artículo o cuántas veces realmente ha comprado ese cliente?

Aquí es donde entra la **Normalización**. En el mundo de Data Science, normalizar una base de datos no es solo una tarea de administración de sistemas; es el primer paso crítico para asegurar la **integridad de los datos**. Si tus bases de datos tienen redundancias (datos repetidos innecesariamente) o dependencias incorrectas, tus análisis serán lentos, propensos a errores y difíciles de escalar cada vez que necesites actualizar la información.

01 ¿Qué es la Normalización y cuál es su principal objetivo?

Es el proceso de organizar los datos en una base de datos relacional para **reducir la redundancia** y mejorar la integridad. Su objetivo es asegurar que cada dato se guarde en un solo lugar, facilitando actualizaciones y evitando inconsistencias.

02 ¿Qué exige la Primera Forma Normal (1NF)?

Establece que cada columna debe contener valores **atómicos** (indivisibles) y no debe haber grupos repetidos. *Ejemplo:* En lugar de tener una celda con "Laptop, Mouse", debes tener dos filas separadas, una para "Laptop" y otra para "Mouse".

03 ¿A qué se refiere el concepto de "Atomicidad" en bases de datos?

Un valor es atómico cuando no se puede dividir en partes más pequeñas sin perder su significado para el sistema. En 1NF, evitamos guardar listas o arrays dentro de una sola celda.

04 ¿Cuál es el requisito clave de la Segunda Forma Normal (2NF)?

Para estar en 2NF, la tabla ya debe estar en 1NF y **todos los atributos que no son clave deben depender de la clave primaria completa**, no solo de una parte de ella. Esto aplica principalmente a tablas con claves compuestas.

05 ¿Qué es una Dependencia Parcial y cómo se soluciona en 2NF?

Ocurre cuando un atributo depende solo de una parte de una clave primaria compuesta. *Solución:* Mover esos atributos y la parte de la clave de la que dependen a una nueva tabla.

06 ¿Qué establece la Tercera Forma Normal (3NF)?

La tabla debe estar en 2NF y no debe haber **dependencias transitivas**. Esto significa que los atributos que no son clave no deben depender de otros atributos que tampoco son clave. Todo debe depender "solo de la clave".

07 ¿Qué es una Dependencia Transitiva?

Es cuando una columna A depende de B, y B depende de la clave primaria (PK). *Ejemplo:* En una tabla de pedidos, si `Nombre_Ciudad` depende del `Codigo_Postal` y este depende del `ID_Pedido`, hay una dependencia transitiva de ciudad hacia la clave primaria.

08 ¿Por qué la redundancia es peligrosa para un analista de datos?

Es la repetición innecesaria de datos. Causa **Anomalías de Actualización**: si cambias el nombre de un proveedor en una fila pero olvidas hacerlo en las otras 100 donde aparece, tus datos pierden coherencia y tu modelo de ML aprenderá información contradictoria.

09 ¿Qué papel juega la Clave Primaria (PK) en la normalización?

Es la que identifica de forma única cada fila. La normalización asegura que todos los demás datos en la tabla describan directamente el objeto identificado por esa clave, evitando mezclar conceptos en la misma tabla.

10 ¿Cómo ayudan las Claves Foráneas (FK) a normalizar un esquema?

Permiten dividir tablas grandes en tablas pequeñas relacionadas. En lugar de repetir toda la info de un "Cliente" en cada "Pedido", usamos un `ID_Cliente` (FK) para conectar ambas tablas, cumpliendo con 2NF y 3NF.

PRE-ENTREGA DEL PROYECTO

Actividad evaluable · parte del proyecto integrador

Pre-entrega: Normalización y modelado relacional

El objetivo de esta actividad es aplicar los principios de normalización y modelado relacional para estructurar una base de datos que resuelva una problemática de negocio real. Como científico de datos, entender la estructura de origen es vital para garantizar la integridad de los análisis posteriores.

Cómo resolver esta entrega

Caso de estudio: Sistema de Gestión de una Biblioteca Universitaria

La biblioteca necesita migrar sus registros de hojas de cálculo a una base de datos relacional. Actualmente, tienen problemas de redundancia (los datos del autor se repiten por cada libro) e inconsistencia (hay libros con categorías mal escritas).

Pasos sugeridos

1. **Identificación de Entidades**: Identifica al menos 4 entidades principales (ej. Libros, Autores, Usuarios, Préstamos).
2. **Definición de Atributos y Claves**: Establece las Primary Keys (PK) para cada tabla y define tipos de datos adecuados (VARCHAR, INT, DATE).
3. **Establecimiento de Relaciones**: Define las Foreign Keys (FK) y la cardinalidad (1:1, 1:N o N: M). En caso de existir relaciones muchos a muchos, crea la tabla intermedia necesaria.
4. **Normalización**: Asegúrate de que el diagrama cumpla con la Tercera Forma Normal (3FN) para evitar redundancias.

Formato de entrega

Utiliza una herramienta de diagramación (como dbdiagram.io, Lucidchart, Draw.io o incluso un diagrama hecho a mano con claridad). Debes exportar el resultado final exclusivamente en formato de imagen (PNG o JPG).

Objetivos de aprendizaje

- Identificar entidades y atributos clave en un escenario de negocio para su modelado relacional.
- Implementar claves primarias y foráneas para establecer integridad referencial.
- Aplicar reglas de normalización para reducir la redundancia de datos.
- Diseñar esquemas de bases de datos que faciliten la persistencia de datos estructurados.

Criterios de aceptación

- El diagrama incluye al menos 4 entidades relacionadas coherentemente.
- Cada tabla tiene definida una Clave Primaria (PK) única.
- Las relaciones entre tablas están representadas con la cardinalidad correcta (ej. 1 a muchos).
- No existen grupos repetidos ni dependencias parciales (aplicación de normalización).
- La imagen es legible, con nombres de campos y tablas claramente identificables.

Rúbrica de evaluación: Diagrama Entidad-Relación para sistema de gestión de biblioteca universitaria

Esta rúbrica evalúa la capacidad de modelar una base de datos relacional a partir de un caso real, identificando entidades, atributos, claves y relaciones, y aplicando principios de normalización hasta 3FN. El producto final debe ser una imagen clara (PNG o JPG) con un diagrama ERD completo y bien organizado.

Criterio	Descripción	Peso
Identificación de entidades y alcance del modelo	Evalúa si el diagrama incluye las entidades principales necesarias para resolver el caso de la biblioteca y si el alcance del modelo cubre adecuadamente el problema planteado.	30%
Atributos, tipos de datos y claves primarias	Evalúa si cada tabla contiene atributos pertinentes, si las Primary Keys están correctamente definidas y si los tipos de datos elegidos son razonables para cada campo.	20%
Relaciones, Foreign Keys y cardinalidad	Evalúa si el diagrama representa correctamente las relaciones entre entidades, incluyendo Foreign Keys, cardinalidad y tablas intermedias cuando existan relaciones muchos a muchos.	25%
Normalización y ausencia de redundancia	Evalúa si el esquema evita duplicidad innecesaria de datos y si está organizado de forma compatible con la Tercera Forma Normal (3FN).	15%
Claridad visual y calidad del entregable en imagen	Evalúa la legibilidad del diagrama, el orden visual, el uso de etiquetas claras y el cumplimiento del formato de entrega solicitado (imagen PNG o JPG).	10%

Total: 100 pts | Aprobación: 70 pts

MÓDULO 3

Data Acquisition de Datos

Profundiza en la adquisición de datos desde archivos planos y hojas de cálculo, construyendo sobre tus conocimientos previos de SQL y Python. Aprenderás a diferenciar formatos como CSV, Excel y Parquet, y a aplicar técnicas prácticas de limpieza y normalización usando pandas y openpyxl.

Fundamentos y Tipos de Datos

¿Qué es la adquisición de datos?

La **adquisición de datos** es el proceso de obtener datos desde diversas fuentes —como APIs, archivos o bases de datos— y prepararlos para su uso en análisis y toma de decisiones.

Es la etapa inicial y crítica dentro del ciclo de vida analítico, pues sin datos adecuados y bien obtenidos, cualquier análisis carece de fundamento.

****Ciclo de vida analítico:** comprende la definición del problema, adquisición de datos, limpieza y preparación, análisis, modelado y comunicación de resultados.

Tipos de datos

Los datos se clasifican según su estructura y formato, lo que influye en cómo se almacenan, procesan y analizan:

Tipo de dato	Descripción	Ejemplos comunes
Estructurados	Datos organizados en tablas con esquema fijo.	Bases SQL, hojas de cálculo CSV
Semi-estructura-dos	Datos con estructura flexible, etiquetas o claves.	JSON, XML, logs de aplicaciones
No estructurados	Datos sin formato predefinido.	Texto libre, imágenes, audio, video
Vectoriales	Representación numérica de datos para ML y búsquedas.	Embeddings de texto, vectores de imagen

APIs REST y autenticación

Las **APIs REST** permiten solicitar y recibir datos estructurados (generalmente en JSON o CSV) mediante protocolos HTTP. Son una fuente clave para la adquisición de datos en tiempo real o actualizados.

Métodos comunes: GET (obtener datos), POST (enviar datos)

- **Autenticación:** tokens, claves API, OAuth
- **Manejo de paginación:** para obtener grandes volúmenes de datos en partes

Repositorios y recursos públicos

Existen múltiples repositorios donde encontrar datasets para experimentación y análisis:

- **Kaggle:** competiciones y datasets variados
- **UCI Machine Learning Repository:** datasets clásicos para ML
- **World Bank Data:** indicadores económicos y sociales
- **Data.gov:** datos abiertos del gobierno de EE.UU.

Arquitecturas de almacenamiento: Data Lakes, Lakehouses y Data Contracts

- **Data Lakes:** almacenamiento masivo de datos en su forma nativa, sin esquema rígido.
- **Lakehouses:** combinan características de Data Lakes y Data Warehouses, permitiendo análisis estructurados y flexibles.
- **Data Contracts:** acuerdos formales que definen esquemas y calidad entre productores y consumidores de datos, asegurando integridad y compatibilidad.

Para profundizar en Data Contracts, puedes consultar Martin Fowler - Data Contracts.

Contexto

En la práctica, la adquisición de datos implica seleccionar las fuentes adecuadas según el problema de negocio, entender el tipo de datos que se necesitan y cómo acceder a ellos. Por ejemplo, un analista que trabaja con datos de ventas puede obtener información estructurada desde una base SQL, mientras que para análisis de sentimiento podría usar datos no estructurados de redes sociales mediante APIs.

Además, la elección entre usar un Data Lake o un Lakehouse dependerá de la necesidad de flexibilidad versus estructura en el almacenamiento y análisis.

Los Data Contracts garantizan que los equipos que producen y consumen datos mantengan un entendimiento común sobre la calidad y formato, evitando errores en los pipelines.

Este conocimiento es esencial para preparar los datos correctamente antes de manipularlos con herramientas como Python y SQL, que veremos en la siguiente unidad.

Formatos tabulares: CSV vs Excel vs Parquet

En el mundo de la ciencia de datos, elegir el formato correcto para tus archivos puede marcar la diferencia entre un proceso eficiente y uno lento o propenso a errores.

Exploraremos los formatos CSV, Excel y Parquet, sus características, ventajas y limitaciones, para que puedas tomar decisiones informadas en tus proyectos de adquisición y almacenamiento de datos.

Al finalizar, serás capaz de diferenciar estos formatos y seleccionar el más adecuado según el contexto y necesidades específicas.

1. CSV (Comma-Separated Values)

El formato `CSV` es uno de los más simples y ampliamente usados para almacenar datos tabulares. Consiste en un archivo de texto plano donde cada línea representa un registro y los campos están separados por un delimitador, comúnmente una coma.

Características:

- **Simplicidad:** Fácil de leer y escribir con cualquier editor de texto.
- **Portabilidad:** Compatible con casi todas las herramientas y lenguajes.
- **Limitaciones:** No soporta tipos de datos complejos ni formatos enriquecidos; problemas con caracteres especiales y encoding.

Encoding y delimitadores:

- El encoding más común es UTF-8, pero puede variar y causar problemas si no se especifica.
- Delimitadores alternativos incluyen punto y coma (`;`) o tabulaciones (`\t`), especialmente en regiones donde la coma es separador decimal.

2. Excel (.xlsx)

El formato `Excel` es un archivo binario o XML comprimido que permite almacenar datos tabulares con múltiples hojas, formatos, fórmulas y metadatos.

Características:

- **Riqueza de formato:** Soporta estilos, fórmulas, gráficos y múltiples hojas.
- **Popularidad:** Muy usado en entornos empresariales y análisis exploratorio.
- **Limitaciones:** Menor portabilidad, mayor tamaño, y procesamiento más lento comparado con CSV.

Uso en Python: Librerías como `openpyxl` permiten leer y escribir archivos Excel, facilitando la manipulación avanzada.

3. Parquet

`Parquet` es un formato columnar optimizado para almacenamiento y procesamiento eficiente de grandes volúmenes de datos.

Características:

- **Compresión y eficiencia:** Reduce significativamente el tamaño y mejora la velocidad de lectura/escritura.
- **Soporte para tipos complejos:** Maneja datos anidados y tipos ricos.
- **Integración:** Compatible con ecosistemas Big Data y herramientas como Apache Spark.

Uso en Python: Librerías como `pyarrow` y `pandas` permiten trabajar con Parquet fácilmente.

Comparativa técnica resumida

Característica	CSV	Excel (.xlsx)	Parquet
Formato	Texto plano	Binario/XML comprimido	Columnar binario
Soporte de tipos	Texto simple	Tipos básicos + fórmulas	Tipos complejos
Tamaño archivo	Grande (sin compresión)	Mediano	Pequeño (compresión)
Velocidad lectura	Lenta	Media	Rápida
Portabilidad	Muy alta	Media	Alta (Big Data)
Uso típico	Intercambio simple	Análisis y reportes	Almacenamiento y análisis a gran escala

---

> **Nota**
> La elección del formato impacta directamente en la eficiencia de tus pipelines de datos y en la calidad del análisis posterior.

Contexto

En proyectos de ciencia de datos, la adquisición y almacenamiento de datos tabulares es una tarea recurrente. Por ejemplo, un analista puede recibir datos de ventas en formato CSV, mientras que un equipo financiero puede trabajar con reportes en Excel que incluyen múltiples hojas y fórmulas.

Por otro lado, en entornos de Big Data, los datos se almacenan en formatos columnar como Parquet para optimizar consultas y almacenamiento.

Comprender las características de cada formato te permitirá diseñar pipelines de ingestión más eficientes, evitar problemas comunes como errores de encoding o delimitadores incorrectos, y preparar los datos para análisis posteriores con mayor calidad.

Además, la integración con herramientas de Python como `pandas`, `openpyxl` y `pyarrow` facilita la manipulación y conversión entre formatos, lo que es clave para la reproducibilidad y escalabilidad de tus procesos de adquisición de datos.

Data Acquisition: El Puente entre la Realidad y el Código

Imagina por un momento que estás en una sala de servidores de una empresa como **Tesla** o en una planta de ensamblaje de **BMW**. A tu alrededor, hay miles de procesos ocurriendo simultáneamente: motores girando, baterías calentándose, brazos robóticos moviéndose con precisión milimétrica. Como

científico de datos, tu trabajo no es solo analizar lo que ya está en una base de datos, sino entender cómo esos eventos del mundo físico se transforman en los números que eventualmente limpiarás con `pandas`.

La **Adquisición de Datos (DAQ)** es, en esencia, el arte y la ciencia de capturar fenómenos del mundo real y traducirlos a un lenguaje que una computadora pueda procesar. Sin este proceso, no habría Machine Learning, porque no habría datos que aprender. En esta unidad, profundizaremos en qué es realmente la adquisición de datos, por qué es el primer paso crítico en cualquier flujo de trabajo de Ciencia de Datos y cómo se diferencia de simplemente "leer un archivo".

1. ¿Qué es realmente la Adquisición de Datos (DAQ)?

En las unidades anteriores, aprendiste sobre formatos como CSV, Excel y Parquet. Esos son contenedores de datos. Pero, ¿de dónde vienen esos datos originalmente?

La **Adquisición de Datos** es el proceso de medir fenómenos físicos (como temperatura, presión, sonido o luz) y convertirlos en valores digitales.

El "Porqué" antes del "Cómo"

Si trabajaras en **Fermilab** (el laboratorio de física de partículas), no podrías simplemente "descargar" un dataset de colisiones de protones de internet si nadie lo ha capturado primero. Necesitas un sistema que "escuche" los sensores del acelerador de partículas en tiempo real.

Si el proceso de adquisición falla o es mediocre, todo el análisis posterior será erróneo. En Ciencia de Datos existe una máxima: **"Garbage In, Garbage Out"** (Si entra basura, sale basura). Una mala adquisición de datos produce modelos de Machine Learning que toman decisiones peligrosas.

2. La Cadena de Adquisición: El Camino del Fenómeno al Bit

Para entender la adquisición, debemos visualizarla como una cadena de suministro. Si un eslabón falla, el dato final no es confiable.

A. El Fenómeno Físico y el Sensor

Todo comienza con algo que sucede en la realidad. Un sensor detecta este cambio.

- **Ejemplo:** Un termistor en una batería de un coche eléctrico detecta que la temperatura está subiendo.

B. El Transductor (El Traductor)

A menudo, el sensor y el transductor se confunden, pero tienen roles distintos. El transductor toma la magnitud física (calor) y la convierte en una señal eléctrica (voltaje).

- **Concepto clave:** El mundo real es **analógico** (continuo e infinito). El voltaje que sale del transductor sube y baja suavemente como una onda de música.

C. El ADC (Analog-to-Digital Converter): El Corazón del Sistema

Este es el componente más crítico para un científico de datos. El ADC es el "Puente" o el "Traductor Literario". Toma esa onda de voltaje analógica y la "fotografía" miles de veces por segundo para convertirla en números (0 y 1).

D. Software y Almacenamiento

Una vez que el ADC entrega números, un software (como el estándar de la industria **NI-DAQmx** de National Instruments) organiza esos números en estructuras que conocemos, como archivos CSV o tablas de SQL.

3. Conceptos Fundamentales: Lo que todo Científico de Datos debe saber

Para dominar la adquisición de datos, debes entender tres pilares que definen la calidad de tu "materia prima".

1. Frecuencia de Muestreo (Sampling Rate)

Es la velocidad a la que el sistema "toma fotos" del mundo real. Se mide en Hertz (Hz).

- **Ejemplo Real:** Imagina que quieres monitorizar las vibraciones de un motor de aviación en **Boeing**. Si el motor vibra 1,000 veces por segundo pero tu sistema de adquisición solo toma 500 muestras por segundo, perderás la mitad de la información. Esto crea un error llamado **Aliasing**, donde el dato digital parece algo totalmente distinto a la realidad.

2. Resolución (Bit Depth)

Si la frecuencia de muestreo es la "velocidad", la resolución es el "detalle". Se mide en bits.

- **8 bits:** Puede dividir el rango de medida en 256 niveles.
- **16 bits:** Puede dividirlo en 65,536 niveles.
- **Impacto en ML:** Si intentas predecir una anomalía médica muy sutil (como un pequeño cambio en un electrocardiograma) usando un sistema de 8 bits, el dato será demasiado "pixelado" para que tu modelo de Machine Learning detecte el patrón.

3. Rango Dinámico

Es la diferencia entre el valor más pequeño y el más grande que el sistema puede capturar. Si intentas medir un sonido de despeje de un cohete de **SpaceX** con un micrófono diseñado para susurros, el sensor se "saturará" y los datos serán inútiles.

4. Aplicaciones en la Industria Real

La adquisición de datos no es solo para laboratorios; es la columna vertebral de la industria moderna.

Caso 1: Mantenimiento Predictivo en Siemens

En las fábricas inteligentes de **Siemens**, los sistemas de adquisición monitorean constantemente la presión de las prensas hidráulicas. Si el sistema detecta una micro-variación en la presión (gracias a

una alta resolución del ADC), un modelo de Machine Learning puede predecir que una pieza fallará en las próximas 48 horas, evitando una parada de fábrica costosa.

Caso 2: Tesla y la Flota como Sistema DAQ

Cada coche **Tesla** en la calle es una unidad de adquisición de datos (DAU) andante. Captura datos de cámaras, radares, temperatura de batería y ángulo de giro. Estos datos se adquieren localmente en el hardware del coche y luego los más relevantes se suben a la nube para re-entrenar los modelos de conducción autónoma.

Caso 3: Salud y Dispositivos Wearables

Un Apple Watch utiliza fotopletismografía (sensores de luz) para adquirir datos de tu pulso cardíaco. El ADC interno convierte los reflejos de luz en tu muñeca en una serie temporal de datos que luego se analizan para detectar arritmias.

5. Errores Comunes y Malentendidos

A medida que avances en tu carrera como científico de datos, te encontrarás con estos errores conceptuales. Aprender a identificarlos te ahorrará horas de limpieza de datos innecesaria.

Confusión 1: "DAQ es solo poner un sensor"

Error: Pensar que con comprar un sensor de temperatura barato ya tienes datos de calidad. **Realidad:** El sensor es solo el inicio. Si el ADC es de mala calidad o el cable de conexión tiene interferencia eléctrica (ruido), tus datos tendrán "picos" aleatorios que tu modelo de ML interpretará erróneamente como eventos reales.

Confusión 2: Pensar que Digital es "Perfecto"

Error: Asumir que una vez que el dato es digital, es una representación exacta de la realidad. **Realidad:** El proceso de digitalización *siempre* pierde información. Al convertir una onda continua en "pasos" (cuantización), introducimos un pequeño error. Como científico de datos, debes preguntarte: "¿Es este error de adquisición aceptable para mi modelo?".

Confusión 3: Ignorar el "Timestamp" (Marca de Tiempo)

Error: Adquirir datos sin una referencia temporal precisa. **Realidad:** En la adquisición de datos para series temporales (como predecir el precio de acciones o fallos mecánicos), el momento exacto en que se tomó el dato es tan importante como el valor del dato mismo. Un desfase de milisegundos en la adquisición puede arruinar la capacidad predictiva de un modelo.

6. Síntesis y Conexión

Hemos pasado de ver los datos como simples archivos en tu computadora a entender que son el resultado de un proceso físico y electrónico complejo.

Resumen de puntos clave:

1. **Adquisición de Datos (DAQ)** es el puente entre el mundo analógico y el digital.

2. El sistema se compone de: **Sensor Transductor ADC Computadora**.
3. La calidad de tus datos depende de la **frecuencia de muestreo** y la **resolución**.
4. Empresas líderes como **Tesla, Boeing y Siemens** basan su ventaja competitiva en una adquisición de datos superior.

En las próximas unidades, dejaremos de lado la teoría del hardware para ensuciarnos las manos. Ahora que sabes de dónde vienen los datos y qué errores esperar desde la fuente, aprenderás a usar **pandas** para leer esos archivos (CSV, Excel) y realizar las primeras tareas de limpieza y normalización, asegurándote de que cualquier "ruido" que haya sobrevivido al proceso de adquisición sea manejado adecuadamente antes de llegar a tus modelos de Machine Learning.

¿Estás listo para tomar estos bits y transformarlos en conocimiento?

Conceptos de APIs: endpoints, autenticación, paginación

¿Qué es una API REST?

Una API (Application Programming Interface) REST (Representational State Transfer) es un conjunto de reglas que permite la comunicación entre sistemas a través de HTTP. Las APIs REST son ampliamente usadas para acceder a datos y servicios en la web.

Autenticación en APIs REST

Para proteger los datos, muchas APIs requieren autenticación, que es el proceso de verificar la identidad del consumidor de la API. Los mecanismos más comunes incluyen:

- **Tokens de acceso**: Cadenas alfanuméricas que identifican y autorizan al usuario o aplicación. Se envían generalmente en el encabezado HTTP `Authorization`.
- **OAuth 2.0**: Protocolo estándar que permite a las aplicaciones obtener acceso limitado a cuentas de usuario en un servicio HTTP, delegando la autenticación a un proveedor de identidad.

> **Ejemplo**
>
> **Ejemplo industrial.** La API de Twitter utiliza OAuth 2.0 para que las aplicaciones accedan a datos de usuarios sin compartir contraseñas.

Técnicas de Paginación

Cuando una API devuelve grandes conjuntos de datos, es común que limite la cantidad de registros por respuesta para optimizar el rendimiento. La paginación permite obtener los datos en partes.

Paginación por Offset

- Se especifica un número de página o un desplazamiento (`offset`) y un límite (`limit`).
- Ejemplo: `GET /items?offset=20&limit=10` devuelve 10 ítems empezando desde el ítem 21.

Paginación por Cursor

- Se utiliza un cursor (un identificador único o timestamp) para marcar la posición en la lista.
- Más eficiente para datos dinámicos o en tiempo real.
- Ejemplo: `GET /items?cursor=abc123&limit=10` devuelve los siguientes 10 ítems después del cursor `abc123`.

Gestión de Rate Limits

Las APIs suelen imponer límites en la cantidad de solicitudes permitidas en un período para evitar sobrecarga.

- **Rate limit**: número máximo de solicitudes permitidas.
- Cuando se alcanza el límite, la API responde con un código HTTP 429 (Too Many Requests).

Estrategias para manejar rate limits:

- **Retries con backoff exponencial:** esperar tiempos crecientes antes de reintentar.
- **Monitoreo de headers:** muchas APIs indican en los encabezados HTTP cuántas solicitudes quedan.

> **Nota**
> Implementar estas estrategias es clave para mantener la estabilidad y evitar bloqueos en pipelines de datos.

Conocimiento clave: Entender estos conceptos es fundamental para diseñar pipelines que interactúen con APIs de forma segura, eficiente y escalable.

Aplicación Práctica en Ciencia de Datos

En proyectos reales, los datos que necesitas suelen estar distribuidos en múltiples páginas de una API protegida por autenticación y con límites de tasa estrictos. Por ejemplo, al extraer datos de ventas de una plataforma de e-commerce o información financiera de un servicio web.

Para integrar estos datos en tu pipeline de ciencia de datos, debes:

- Configurar correctamente la autenticación para acceder a la API.
- Implementar paginación para recorrer todos los registros sin perder información.
- Gestionar los rate limits para evitar bloqueos y garantizar la continuidad del proceso.

Este conocimiento te permitirá construir pipelines reproducibles y robustos, que son la base para análisis confiables y actualizados.

En las siguientes unidades, aplicaremos estos conceptos para construir código que implemente estas técnicas, integrándolas con pandas y bases de datos locales para un flujo completo de adquisición y procesamiento de datos.

Consumir APIs con requests y manejar credenciales (.env)

En esta unidad aprenderás a realizar llamadas a APIs REST usando la librería `requests` de Python, una herramienta estándar para hacer solicitudes HTTP.

Además, verás cómo manejar paginación para obtener datos distribuidos en múltiples páginas, implementar reintentos exponenciales para robustecer tus llamadas ante errores temporales y cómo almacenar credenciales de forma segura usando archivos `.env`.

> **Nota**
> Las credenciales son datos sensibles como tokens o claves API que no deben estar expuestos en el código fuente.

1. Uso básico de requests

CÓDIGO

```
import requests

response = requests.get('https://api.example.com/data')
if response.status_code == 200:
 data = response.json()
 print(data)
else:
 print(f'Error: {response.status_code}')
```

- `response.status_code` indica el código HTTP.
- `response.json()` convierte la respuesta JSON en un diccionario Python.

2. Implementar paginación

Muchas APIs devuelven datos paginados. Por ejemplo, con parámetros `page` y `per_page`:

CÓDIGO

```
def fetch_all_pages(base_url, headers):
 page = 1
 results = []
 while True:
 params = {'page': page, 'per_page': 100}
 response = requests.get(base_url, headers=headers, params=params)
 if response.status_code != 200:
 break
 data = response.json()
 if not data['items']:
 break
 results.extend(data['items'])
 page += 1
 return results
```

3. Reintentos exponenciales

Para manejar errores temporales (e.g., 429 Too Many Requests), implementamos reintentos con backoff exponencial:

CÓDIGO

```
import time

def get_with_retries(url, headers, max_retries=5):
 delay = 1
 for attempt in range(max_retries):
 response = requests.get(url, headers=headers)
 if response.status_code == 200:
 return response
 elif response.status_code in [429, 500, 502, 503, 504]:
 time.sleep(delay)
 delay *= 2 # Exponencial
 else:
 response.raise_for_status()
 raise Exception('Max retries exceeded')
```

4. Manejo de credenciales con .env

Para no exponer claves en el código, usamos un archivo `.env` y la librería `python-dotenv`:

CÓDIGO

```
from dotenv import load_dotenv
import os

load_dotenv() # Carga variables de entorno desde .env
API_KEY = os.getenv('API_KEY')

headers = {'Authorization': f'Bearer {API_KEY}'}
```

Archivo `.env`:

CÓDIGO

```
API_KEY=tu_api_key_aqui
```

> **Importante**
>
> Nunca subir el archivo `.env` a repositorios públicos.

5. Ejemplo práctico

En el notebook `Data Acquisition.ipynb` encontrarás un ejemplo completo que implementa paginación, reintentos y uso de `.env` para consumir una API pública.

Te pasamos un código para que veas como funciona request en Python con una API real:

CÓDIGO

```
import requests
import time
import pandas as pd

<h1 class="module" data-eyebrow="MÓDULO">Coordenadas de Buenos Aires</h1>
<hr class="module-underline"><hr class="module-hairline">
LAT = -34.61
LON = -58.38

BASE_URL = "https://api.open-meteo.com/v1/forecast"

def get_with_retries(url, params, max_retries=5):
 """
 Petición GET con reintentos y backoff exponencial
 """
 delay = 1
 for attempt in range(1, max_retries + 1):
 response = requests.get(url, params=params)

 if response.status_code == 200:
 return response

 elif response.status_code in [429, 500, 502, 503, 504]:
 print(f"Intento {attempt} fallido ({response.status_code}). Reintentando en {delay}s...")
 time.sleep(delay)
 delay *= 2
 else:
 response.raise_for_status()

 raise Exception("Máximo número de reintentos alcanzado")

def fetch_weather_buenos_aires():
 """
 Obtiene clima actual y pronóstico horario
 """
 params = {
 "latitude": LAT,
 "longitude": LON,
 "current_weather": True,
 "hourly": "temperature_2m,precipitation,wind_speed_10m",
 "timezone": "America/Argentina/Buenos_Aires"
 }

 print("Consultando API del clima para Buenos Aires...")
 response = get_with_retries(BASE_URL, params)
 return response.json()

<h1 class="module" data-eyebrow="MÓDULO">----------------------------</h1>
```

```
<hr class="module-underline"><hr class="module-hairline">
<h1 class="module" data-eyebrow="MÓDULO">Ejecución principal</h1>
<hr class="module-underline"><hr class="module-hairline">
<h1 class="module" data-eyebrow="MÓDULO">-----------------------------</h1>
<hr class="module-underline"><hr class="module-hairline">
if __name__ == "__main__":
 weather_data = fetch_weather_buenos_aires()

 # Clima actual
 current = weather_data["current_weather"]
 print("\n Clima actual en Buenos Aires")
 print(f"Temperatura: {current['temperature']} °C")
 print(f"Viento: {current['windspeed']} km/h")
 print(f"Hora: {current['time']}")

 # Pronóstico horario (convertimos a DataFrame)
 hourly = weather_data["hourly"]
 df_hourly = pd.DataFrame({
 "hora": hourly["time"],
 "temperatura_c": hourly["temperature_2m"],
 "precipitacion_mm": hourly["precipitation"],
 "viento_kmh": hourly["wind_speed_10m"]
 })

 print("\n Pronóstico horario (primeras filas):")
 print(df_hourly.head())
```

Resumen

- Usamos `requests` para llamadas HTTP.
- Implementamos paginación para obtener todos los datos.
- Añadimos reintentos exponenciales para robustez.
- Guardamos credenciales en `.env` para seguridad.

Normalización de JSON anidado a tablas relacionales

En esta unidad aprenderás a realizar llamadas a APIs REST usando la librería `requests` de Python, una herramienta estándar para hacer solicitudes HTTP.

Además, verás cómo manejar paginación para obtener datos distribuidos en múltiples páginas, implementar reintentos exponenciales para robustecer tus llamadas ante errores temporales y cómo almacenar credenciales de forma segura usando archivos `.env`.

> **Nota**
> Las credenciales son datos sensibles como tokens o claves API que no deben estar expuestos en el código fuente.

1. Uso básico de requests

CÓDIGO

```
import requests

response = requests.get('https://api.example.com/data')
if response.status_code == 200:
 data = response.json()
 print(data)
else:
 print(f'Error: {response.status_code}')
```

- `response.status_code` indica el código HTTP.
- `response.json()` convierte la respuesta JSON en un diccionario Python.

2. Implementar paginación

Muchas APIs devuelven datos paginados. Por ejemplo, con parámetros `page` y `per_page`:

CÓDIGO

```
def fetch_all_pages(base_url, headers):
 page = 1
 results = []
 while True:
 params = {'page': page, 'per_page': 100}
 response = requests.get(base_url, headers=headers, params=params)
 if response.status_code != 200:
 break
 data = response.json()
 if not data['items']:
 break
 results.extend(data['items'])
 page += 1
 return results
```

3. Reintentos exponenciales

Para manejar errores temporales (e.g., 429 Too Many Requests), implementamos reintentos con backoff exponencial:

CÓDIGO

```
import time

def get_with_retries(url, headers, max_retries=5):
 delay = 1
 for attempt in range(max_retries):
 response = requests.get(url, headers=headers)
 if response.status_code == 200:
 return response
 elif response.status_code in [429, 500, 502, 503, 504]:
 time.sleep(delay)
 delay *= 2 # Exponencial
 else:
 response.raise_for_status()
 raise Exception('Max retries exceeded')
```

4. Manejo de credenciales con .env

Para no exponer claves en el código, usamos un archivo `.env` y la librería `python-dotenv`:

CÓDIGO

```
from dotenv import load_dotenv
import os

load_dotenv() # Carga variables de entorno desde .env
API_KEY = os.getenv('API_KEY')

headers = {'Authorization': f'Bearer {API_KEY}'}
```

Archivo `.env`:

CÓDIGO

```
API_KEY=tu_api_key_aqui
```

> **Importante**
> Nunca subir el archivo `.env` a repositorios públicos.

5. Ejemplo práctico

En el notebook `Data Acquisition.ipynb` encontrarás un ejemplo completo que implementa paginación, reintentos y uso de `.env` para consumir una API pública.

Te pasamos un código para que veas como funciona request en Python con una API real:

CÓDIGO

```
import requests
import time
import pandas as pd

<h1 class="module" data-eyebrow="MÓDULO">Coordenadas de Buenos Aires</h1>
<hr class="module-underline"><hr class="module-hairline">
LAT = -34.61
LON = -58.38

BASE_URL = "https://api.open-meteo.com/v1/forecast"

def get_with_retries(url, params, max_retries=5):
 """
 Petición GET con reintentos y backoff exponencial
 """
 delay = 1
 for attempt in range(1, max_retries + 1):
 response = requests.get(url, params=params)

 if response.status_code == 200:
 return response

 elif response.status_code in [429, 500, 502, 503, 504]:
 print(f"Intento {attempt} fallido ({response.status_code}). Reintentando en {delay}s...")
 time.sleep(delay)
 delay *= 2
 else:
 response.raise_for_status()

 raise Exception("Máximo número de reintentos alcanzado")

def fetch_weather_buenos_aires():
 """
 Obtiene clima actual y pronóstico horario
 """
 params = {
 "latitude": LAT,
 "longitude": LON,
 "current_weather": True,
 "hourly": "temperature_2m,precipitation,wind_speed_10m",
 "timezone": "America/Argentina/Buenos_Aires"
 }

 print("Consultando API del clima para Buenos Aires...")
 response = get_with_retries(BASE_URL, params)
 return response.json()

<h1 class="module" data-eyebrow="MÓDULO">----------------------------</h1>
```

```
<hr class="module-underline"><hr class="module-hairline">
<h1 class="module" data-eyebrow="MÓDULO">Ejecución principal</h1>
<hr class="module-underline"><hr class="module-hairline">
<h1 class="module" data-eyebrow="MÓDULO">-----------------------------</h1>
<hr class="module-underline"><hr class="module-hairline">
if __name__ == "__main__":
 weather_data = fetch_weather_buenos_aires()

 # Clima actual
 current = weather_data["current_weather"]
 print("\n Clima actual en Buenos Aires")
 print(f"Temperatura: {current['temperature']} °C")
 print(f"Viento: {current['windspeed']} km/h")
 print(f"Hora: {current['time']}")

 # Pronóstico horario (convertimos a DataFrame)
 hourly = weather_data["hourly"]
 df_hourly = pd.DataFrame({
 "hora": hourly["time"],
 "temperatura_c": hourly["temperature_2m"],
 "precipitacion_mm": hourly["precipitation"],
 "viento_kmh": hourly["wind_speed_10m"]
 })

 print("\n Pronóstico horario (primeras filas):")
 print(df_hourly.head())
```

Resumen

- Usamos `requests` para llamadas HTTP.
- Implementamos paginación para obtener todos los datos.
- Añadimos reintentos exponenciales para robustez.
- Guardamos credenciales en `.env` para seguridad.

Próximos pasos

En la siguiente unidad normalizaremos JSON anidado para convertirlo en tablas relacionales, facilitando el análisis y almacenamiento.

DML: crear, consultar, actualizar y eliminar datos

Bienvenido a la primera unidad del Módulo 2. En el módulo anterior, establecimos los cimientos de las bases de datos: aprendimos a diseñar estructuras (DDL) y entendimos cómo se organizan las tablas. Sin embargo, una base de datos con una estructura perfecta pero vacía es como una biblioteca con estanterías hermosas pero sin libros: no tiene valor práctico.

En esta unidad, nos sumergiremos en el **DML (Data Manipulation Language)**. Aquí es donde los datos cobran vida. Como analista de datos o futuro científico de datos (Data Scientist), pasarás el 80% de tu tiempo interactuando con DML. Ya sea para extraer un conjunto de entrenamiento para un modelo de Machine Learning, corregir etiquetas erróneas en un dataset o insertar nuevos registros provenientes de una API, el dominio de estas operaciones es tu herramienta de trabajo diaria más crítica.

1. ¿Qué es DML y por qué es el motor del Data Science?

El **Data Manipulation Language (DML)** es el subconjunto de SQL que se encarga de interactuar con los datos existentes dentro de las estructuras que ya creamos. Mientras que el DDL (Data Definition Language) se encarga de "la caja" (la tabla), el DML se encarga de "lo que hay dentro" (las filas).

El papel de DML en el flujo de Machine Learning

Imagina que estás construyendo un modelo para predecir el abandono de clientes en una empresa de telecomunicaciones. 1. **SELECT**: Necesitas extraer el historial de consumo de los últimos 6 meses. 2. **INSERT**: Nuevos clientes se registran cada hora; sus datos deben entrar al sistema. 3. **UPDATE**: Un cliente cambia su plan de prepago a pospago; debes reflejar este cambio para que tu modelo no use datos obsoletos. 4. **DELETE**: Por normativas de privacidad (como el GDPR), si un cliente pide ser olvidado, debes eliminar sus registros.

Sin DML, el flujo de datos se detiene. No podrías alimentar tus algoritmos ni mantener la calidad de la información.

2. SELECT: El arte de preguntar a los datos

Aunque ya viste los fundamentos de `SELECT` en el repaso, es vital profundizar en su rol como herramienta de **proyección** (elegir columnas) y **selección** (filtrar filas).

Anatomía de una consulta profesional

Cuando trabajas en Data Science, rara vez harás un `SELECT *`. En datasets con cientos de columnas (como datos genómicos o logs de servidores), esto es ineficiente y lento.

CÓDIGO

```
SELECT
 customer_id,
 total_spend,
 last_login_date
FROM users
WHERE status = 'active'
 AND country = 'Mexico'
ORDER BY total_spend DESC;
```

Análisis del porqué: * **Proyección específica:** Solo traemos las columnas necesarias para nuestro análisis, ahorrando memoria RAM en nuestro entorno de Python. * **Predicados (`WHERE`):** Filtramos el

ruido. Si solo nos interesan los usuarios activos en un país, el modelo de ML no debe "contaminarse" con datos de otros segmentos inicialmente no relevantes.

3. INSERT: Alimentando el sistema

El comando `INSERT` permite añadir nuevas filas a una tabla. En el mundo del procesamiento de datos moderno, esto suele ocurrir de dos formas: inserciones individuales (un usuario se registra) o inserciones masivas (un proceso ETL carga millones de registros a la vez).

Sintaxis básica y mejores prácticas

CÓDIGO

```
INSERT INTO productos (nombre, precio, stock, categoria_id)
VALUES ('Sensor de Humedad IoT', 25.50, 100, 5);
```

Punto de atención para principiantes: Es una excelente práctica listar las columnas explícitamente después del nombre de la tabla. ¿Por qué? Si en el futuro alguien altera la estructura de la tabla (añadiendo una columna con `ALTER TABLE`), tu código de `INSERT` no se romperá porque SQL sabrá exactamente a qué columnas te refieres, ignorando las nuevas si tienen valores por defecto.

4. UPDATE: La corrección y el mantenimiento

El comando `UPDATE` modifica valores en registros que ya existen. En Data Science, lo usamos frecuentemente para la limpieza de datos (Data Cleaning).

El peligro del "UPDATE sin WHERE"

Este es, quizás, el error más temido en el desarrollo de software y análisis de datos.

Escenario de desastre: Imagina que quieres actualizar el salario de un empleado específico cuyo `id` es 50.

CÓDIGO

```
-- ¡ERROR FATAL!
UPDATE empleados SET salario = 5000;
```

Si ejecutas esto, **todos** los empleados de la empresa ahora ganan 5000. No hay marcha atrás a menos que tengas un respaldo.

La forma correcta:

CÓDIGO

```
UPDATE empleados
SET salario = 5000
WHERE id = 50; -- El filtro es tu seguro de vida
```

Consejo de experto: Antes de ejecutar un `UPDATE`, ejecuta un `SELECT` con el mismo `WHERE`. Si el `SELECT` te devuelve exactamente las filas que quieres cambiar, entonces es seguro convertirlo en un `UPDATE`.

5. DELETE: Eliminación selectiva

`DELETE` elimina filas de una tabla. Al igual que con `UPDATE`, la cláusula `WHERE` es obligatoria en la práctica profesional.

Diferencia crítica: DELETE vs. DROP vs. TRUNCATE

Es común que los principiantes confundan estos términos. Aquí la distinción clara: * **`DELETE` (DML):** Borra *filas*. La tabla y su estructura permanecen. Es como borrar el contenido de una celda en una hoja de cálculo. * **`DROP` (DDL):** Borra *toda la tabla*. La estructura desaparece del mapa. Es como eliminar el archivo de la hoja de cálculo. * **`TRUNCATE` (DDL/DML):** Borra *todas* las filas de golpe de forma muy rápida, pero mantiene la estructura.

Ejemplo de uso de DELETE:

CÓDIGO

```
DELETE FROM logs_servidor
WHERE fecha_evento < '2023-01-01';
```

Aquí estamos haciendo una limpieza de datos históricos obsoletos para optimizar el rendimiento de nuestras consultas.

6. Errores Comunes y Falsas Concepciones

Confundir NULL con vacío

Al insertar o actualizar datos (`INSERT` / `UPDATE`), un error frecuente es tratar el valor `NULL` como si fuera el número 0 o un texto vacío. * **Realidad:** `NULL` significa "desconocido" o "ausencia de valor". En ML, los `NULL` son un problema serio que requiere técnicas de imputación. Al usar DML, debes ser consciente de si tu tabla permite `NULL` o si tiene restricciones `NOT NULL`.

Pensar que el orden de las filas es garantizado

Muchos principiantes asumen que el `SELECT` devolverá los datos en el orden en que fueron insertados (`INSERT`). * **Realidad:** En SQL, las tablas son conjuntos matemáticos. No tienen un orden inherente. Si necesitas que tus datos estén ordenados (por ejemplo, para una serie temporal en ML), **siempre** debes usar `ORDER BY`.

7. Síntesis y Conexión

Hemos aprendido que: 1. **DML** es el lenguaje de manipulación: maneja el contenido. 2. **SELECT** es para extraer información valiosa. 3. **INSERT** agrega nuevos datos al flujo. 4. **UPDATE** y **DELETE** modifican o eliminan registros, siempre bajo la tutela estricta de un **WHERE**.

Dominar estas operaciones es el primer paso para convertirte en un arquitecto de datos eficiente. Pero, ¿qué pasa si el sistema falla mientras estamos haciendo un `UPDATE` masivo? ¿Cómo evitamos que los datos queden correlacionados o incompletos? Esa es la pregunta que resolveremos en la siguiente unidad cuando hablemos de **TCL (Transaction Control Language)** y las propiedades **ACID**.

Por ahora, es momento de poner estas instrucciones en práctica en un entorno real.

TCL: asegurar cambios con transacciones y ACID

Imagina que estás trabajando en una institución financiera desarrollando un modelo de detección de fraudes. Recibes un flujo de datos que debe registrar una transferencia de dinero: 100 dólares salen de la **Cuenta A** y entran en la **Cuenta B**.

¿Qué pasaría si, justo después de restar el dinero de la Cuenta A, el servidor se apaga o el código falla? Si no tienes un sistema de control, esos 100 dólares habrían "desaparecido" del sistema: no están en la cuenta de origen ni en la de destino. Para un analista de datos, esto es una pesadilla: **tus datos ya no reflejan la realidad**. Los modelos de Machine Learning entrenados con datos inconsistentes producirán predicciones erróneas o, peor aún, decisiones catastróficas.

Aquí es donde entra **TCL (Transaction Control Language)**. TCL no es solo un conjunto de comandos; es el "seguro de vida" de la integridad de tus datos.

1. ¿Qué es TCL (Transaction Control Language)?

TCL es el subconjunto del lenguaje SQL que se encarga de gestionar **transacciones**. Una transacción no es una sola operación, sino un **bloque de operaciones** que el motor de la base de datos trata como una única unidad lógica de trabajo.

En unidades anteriores aprendiste **DML (Data Manipulation Language)** para insertar o actualizar filas. TCL es la capa que envuelve a ese DML para decidir si esos cambios deben volverse permanentes o ser descartados por completo ante un error.

La regla de oro: El "Todo o Nada"

El principio fundamental de una transacción es la **atomicidad**. Al igual que un átomo se consideraba la unidad indivisible de la materia, una transacción es la unidad indivisible de procesamiento de datos. O se ejecutan todos los pasos del bloque, o no se ejecuta ninguno.

2. Los pilares de la confianza: El acrónimo ACID

Para entender por qué usamos TCL, debemos conocer las cuatro propiedades que garantizan que una base de datos sea confiable, conocidas como **ACID**. Si vas a trabajar en arquitectura de datos para ML, este concepto es tu biblia:

1. **A - Atomicidad (Atomicity):** Como mencionamos, la transacción es un "todo o nada". Si el script de ingesta de datos falla a la mitad, la base de datos vuelve al estado inicial antes de empezar.

2. **C - Consistencia (Consistency):** Una transacción lleva a la base de datos de un estado válido a otro estado válido. Se aseguran de que todas las reglas (como que el saldo no sea negativo) se cumplan antes de finalizar.
3. **I - Aislamiento (Isolation):** Si dos científicos de datos están actualizando la misma tabla al mismo tiempo, sus transacciones no se mezclan. Cada una ocurre como si fuera la única, evitando que una lea datos "a medio cocinar" de la otra.
4. **D - Durabilidad (Durability):** Una vez que la base de datos te confirma que el cambio se guardó, ese cambio es permanente. Incluso si hay un corte de luz un segundo después, los datos estarán ahí cuando el sistema reinicie.

3. Los comandos de la "Trinidad" de TCL

Para controlar este flujo de "todo o nada", SQL nos proporciona tres comandos principales. Vamos a verlos en acción con ejemplos prácticos.

A. BEGIN TRANSACTION (El inicio del monitoreo)

Este comando le dice a la base de datos: "Atención, todo lo que viene a continuación debe ser tratado como una sola unidad. No guardes nada de forma definitiva todavía".

En SQLite, puedes usar simplemente `BEGIN` o `BEGIN TRANSACTION`.

B. COMMIT (El botón de "Guardar")

Es el comando que confirma que todo salió bien. Cuando ejecutas `COMMIT`, todos los `INSERT`, `UPDATE` o `DELETE` que hiciste desde el `BEGIN` se escriben permanentemente en el disco duro. A partir de este momento, ya no hay vuelta atrás.

C. ROLLBACK (El botón de "Pánico" o Deshacer)

Si algo falla —un error en el código Python, una restricción de clave foránea violada o simplemente te diste cuenta de que el archivo de datos estaba corrupto— usas `ROLLBACK`. Este comando deshace automáticamente todos los cambios realizados desde el último `BEGIN`, dejando la base de datos exactamente como estaba antes de empezar.

4. Ejemplos detallados y casos de uso en Data Science

Escenario 1: Limpieza de datos en un Pipeline de ML

Supongamos que estás limpiando un dataset de clientes. Quieres eliminar a los usuarios sin correo electrónico y, al mismo tiempo, actualizar la fecha de la última limpieza en una tabla de auditoría.

CÓDIGO

```
BEGIN TRANSACTION; -- Iniciamos la unidad de trabajo

-- Paso 1: Eliminar registros incompletos
DELETE FROM clientes WHERE email IS NULL;

-- Paso 2: Registrar la actividad en auditoría
INSERT INTO log_limpieza (fecha, registros_eliminados)
VALUES (CURRENT_TIMESTAMP, 150);

-- Todo parece correcto...
COMMIT;
```

¿Por qué es importante aquí? Si el `DELETE` funciona pero el `INSERT` en la auditoría falla (quizás porque la tabla de log está llena), el `COMMIT` no se ejecutaría (si lo controlamos por código) y haríamos `ROLLBACK`. Así evitamos tener una base de datos donde se borraron datos pero no quedó registro de quién o cuándo se hizo.

Escenario 2: Ingesta de archivos CSV masivos (Pandas + SQL)

Imagina que usas Python para cargar 1 millón de filas a una base de datos SQLite para entrenar un modelo. A mitad del proceso, la fila 500,001 tiene un texto donde debería haber un número, lo que causa un error de sistema.

- **Sin TCL:** Tendrías 500,000 filas cargadas y el proceso detenido. Ahora tienes "basura" o datos incompletos en tu tabla de entrenamiento. Tendrías que borrar manualmente antes de reintentar.
- **Con TCL:** Al fallar, se ejecuta un `ROLLBACK`. La tabla queda vacía (o como estaba antes). Corriges el CSV y vuelves a intentar una carga limpia.

5. Errores comunes y trampas conceptuales

Para un principiante en Data Science, es fácil confundirse con estos puntos:

1. **Pensar que `SELECT` necesita transacciones:** Un error frecuente es intentar usar `COMMIT` después de un `SELECT`. Las consultas de lectura no modifican los datos, por lo que no afectan la integridad. Solo usamos TCL con instrucciones que alteran los datos (`INSERT`, `UPDATE`, `DELETE`).
2. **Confiar en el "Autocommit":** Muchas herramientas (como DBeaver o el propio SQLite en modo consola) tienen activado por defecto el "Autocommit". Esto significa que cada línea que escribes se guarda inmediatamente. Para practicar TCL, debes asegurarte de abrir explícitamente una transacción con `BEGIN`.
3. **Olvidar cerrar la transacción:** Si haces un `BEGIN` y luego un `UPDATE`, pero olvidas hacer `COMMIT` o `ROLLBACK`, esa transacción queda "abierta". Esto puede bloquear la tabla para otros usuarios o procesos, causando que tus scripts de Python se queden esperando infinitamente (un problema conocido como *deadlock* o bloqueo).

4. **La confusión entre "Error de SQL" y "Transacción":** Es vital entender que un error de sintaxis en SQL no hace `ROLLBACK` automáticamente en todos los sistemas. Como analista, tu código (en Python o en el script SQL) debe detectar el error y ordenar el `ROLLBACK`.

6. Síntesis de aprendizaje

TCL es el pegamento que mantiene la **consistencia** en tus flujos de datos. Como futuro científico de datos:

- Usa **`BEGIN`** antes de cualquier cambio masivo o crítico.
- Usa **`COMMIT`** solo cuando estés 100% seguro de que toda la lógica del bloque se cumplió.
- Usa **`ROLLBACK`** como tu mecanismo de defensa ante datos corruptos o errores de ejecución.
- Recuerda **ACID**: La Atomicidad garantiza que si no se guarda todo, no se guarda nada; mientras que la Durabilidad asegura que tus hallazgos persistan en el tiempo.

Sin transacciones, el Machine Learning es solo "basura entra, basura sale" (*Garbage In, Garbage Out*). Con TCL, controlas la calidad de la entrada.

Big Data: por qué escalar cambia la forma de trabajar

Imagina que eres un analista de datos para una pequeña tienda de café local. Tienes una base de datos en **SQLite** (como vimos en unidades anteriores) donde registras cada venta. Con 50 transacciones al día, tu computadora procesa las consultas en milisegundos. Todo es sencillo: insertas una fila, haces un `SELECT` y obtienes un reporte.

Ahora, imagina que esa pequeña tienda se convierte en **Starbucks**. Ahora tienes miles de tiendas en todo el mundo, cada una generando cientos de transacciones por hora, además de datos de aplicaciones móviles, sensores de inventario en tiempo real, comentarios en redes sociales y registros de conexiones Wi-Fi en las tiendas. De repente, tu confiable archivo `.db` local se corrompe al intentar guardar gigabytes de datos por minuto, y una consulta simple de ventas mensuales tarda horas en completarse.

Aquí es donde nace el Big Data. No es solo "tener muchos datos"; es el punto en el que las herramientas y métodos que aprendiste para manejar datos locales dejan de funcionar y necesitas un cambio de paradigma.

1. ¿Qué es realmente el Big Data?

El término **Big Data** se refiere a conjuntos de datos tan grandes, rápidos o complejos que es difícil o imposible procesarlos con métodos tradicionales. Para un Data Scientist, el Big Data no es el objetivo final, sino el **combustible** para modelos de Machine Learning más precisos.

La diferencia fundamental: Local vs. Distribuido

Hasta ahora, hemos trabajado bajo el modelo de **procesamiento local**: - **Almacenamiento:** Un disco duro. - **Procesamiento:** Una CPU. - **Memoria:** Unos cuantos GB de RAM.

En Big Data, pasamos al **procesamiento distribuido**: el problema se divide en trozos pequeños y se reparte entre decenas o cientos de computadoras (llamadas **nodos**) que trabajan de forma coordinada (un **cluster**).

> **Importante**
>
> **Punto clave.** Un error frecuente es pensar que Big Data es simplemente "SQL en una computadora más potente". Escalar verticalmente (comprar una computadora más cara) tiene un límite físico y económico. El Big Data se trata de **escalar horizontalmente** (añadir más computadoras baratas al proceso).

2. Las características del Big Data: El modelo de las 5V

Para entender si un problema requiere arquitecturas de Big Data, los profesionales de la industria utilizamos el marco de las "V". Aunque originalmente eran tres, hoy consideramos cinco como esenciales:

1. Volumen (La cantidad)

Es la característica más obvia. Pasamos de manejar Megabytes o Gigabytes a **Terabytes** (1,000 GB) y **Petabytes** (1,000,000 GB). * **Ejemplo:** Walmart procesa más de 2.5 petabytes de datos de transacciones cada hora. Una base de datos tradicional colapsaría intentando indexar ese volumen.

2. Velocidad (El ritmo)

No se trata solo de cuánto dato hay, sino de qué tan rápido llega y qué tan pronto debe ser analizado. * **Ejemplo:** Uber recibe señales de GPS de millones de conductores y pasajeros cada segundo. Para calcular el "Surge Pricing" (precio dinámico), el sistema no puede esperar a un proceso nocturno; debe procesar la velocidad del flujo en tiempo real.

3. Variedad (La forma)

En tus primeras lecciones de SQL, aprendiste sobre datos **estructurados** (tablas con filas y columnas). El Big Data incluye: * **Estructurados:** Tablas de ventas. * **Semi-estructurados:** Archivos JSON de una API, XML. * **No estructurados:** Imágenes de cámaras de seguridad, audios de atención al cliente, archivos de texto de tweets.

4. Veracidad (La confianza)

Con grandes volúmenes, llega mucha "basura". La veracidad se refiere a la calidad y exactitud de los datos. Como futuro Data Scientist, este es tu mayor reto: si entrenas un modelo de ML con datos masivos pero ruidosos o incorrectos, el modelo será inútil. "Basura entra, basura sale".

5. Valor (El propósito)

Tener datos por tenerlos es un costo, no un beneficio. El valor es la capacidad de transformar esos datos en decisiones. El Big Data solo importa si ayuda a predecir la fuga de clientes, optimizar una ruta de entrega o detectar un fraude bancario.

3. Fuentes de datos: ¿De dónde viene el tsunami?

Para alimentar un flujo de Machine Learning a escala, los datos provienen de diversos orígenes:

1. **Internet de las Cosas (IoT):** Sensores en fábricas, medidores inteligentes de energía, o tu propio Apple Watch. Generan datos constantes y de pequeño tamaño, pero en volúmenes masivos de dispositivos.
2. **Huella Digital (Web/Social):** Logs de servidores (qué botones presionó un usuario), comentarios en Instagram, búsquedas en Google.
3. **Transacciones biomédicas:** Secuenciación genómica, donde un solo genoma humano puede ocupar cientos de gigabytes.
4. **Sistemas Transaccionales (ERP/CRM):** Los datos clásicos de ventas y clientes que ahora se integran con fuentes externas.

4. El Ciclo de Vida en Big Data: De la fuente al modelo

Trabajar con Big Data requiere un flujo de trabajo (Pipeline) más robusto que el simple "import pandas as pd".

A. Ingesta (Ingestion)

Es el proceso de obtener los datos de las fuentes y moverlos al sistema de almacenamiento. * **Batch (Lotes):** Mover grandes bloques de datos en intervalos (ej. cada noche). * **Streaming:** Mover datos en tiempo real conforme se generan.

B. Almacenamiento (Storage)

Aquí es donde abandonamos el concepto de "un solo archivo". * **Data Lakes (Lagos de datos):** Repositorios que almacenan datos en su formato original (bruto), ya sean fotos, JSON o CSV. No requieren una estructura previa (Schema-on-read). * **Data Warehouses:** Sistemas optimizados para datos estructurados y consultas rápidas.

C. Procesamiento (Processing)

Es la etapa donde transformamos los datos. Aquí es donde aplicamos técnicas de **limpieza y transformación (DML)** que aprendiste en la unidad anterior, pero distribuidas en muchos nodos. * *¿Por qué es difícil?* Imagina que quieres contar cuántas veces aparece la palabra "oferta" en 10 billones de correos electrónicos. Ninguna RAM puede cargar eso. El procesamiento distribuido divide los 10 billones en paquetes de 1 millón, los reparte en 1,000 computadoras, y luego suma los resultados parciales.

D. Análisis y ML

Finalmente, el Data Scientist toma los datos procesados (ahora limpios y estructurados) para entrenar modelos. La escala del Big Data permite que los modelos de Deep Learning "aprendan" patrones mucho más complejos que no serían visibles en una muestra pequeña de datos.

5. Errores comunes y mejores prácticas

El mito de "Más datos = Mejor modelo"

Un error de principiante es creer que 1 Terabyte de datos siempre dará un mejor modelo de ML que 10 Gigabytes. * **La realidad:** Si esos datos están sesgados o incompletos, solo estás enseñando a tu modelo a equivocarse más rápido. Siempre prioriza la **Veracidad**.

Confundir Big Data con una Base de Datos grande

Si tu base de datos de 500GB corre perfectamente en un servidor SQL potente, **no tienes un problema de Big Data**, tienes una base de datos grande. Solo necesitas Big Data cuando el escalado vertical deja de ser viable económicamente o técnicamente.

Ignorar el costo de procesamiento

En la nube (AWS, Azure, Google Cloud), cada segundo de procesamiento en un cluster de Big Data cuesta dinero. Como Data Scientist, debes ser eficiente con tus consultas. Un `SELECT *` innecesario en una tabla de un Petabyte puede costar cientos de dólares en segundos.

Resumen de la Unidad

- **Big Data** surge cuando el volumen, la velocidad y la variedad superan la capacidad de los sistemas tradicionales.
- El cambio fundamental es pasar del **procesamiento local** al **procesamiento distribuido** (Clusters).
- El modelo de las **5V** (Volumen, Velocidad, Variedad, Veracidad y Valor) nos ayuda a diagnosticar cuándo necesitamos estas arquitecturas.
- El ciclo de vida (Ingesta -> Almacenamiento -> Procesamiento -> Análisis) es la columna vertebral de cualquier proyecto de ML moderno.
- Para un Data Scientist de nivel inicial, entender estos conceptos es vital para saber **dónde** buscar los datos y **cómo** se preparan antes de llegar a sus algoritmos de Machine Learning.

En la siguiente unidad, veremos herramientas específicas que nos permiten manejar estos datos desde Python sin que nuestra computadora explote en el intento.

Herramientas distribuidas y frameworks para escalar el procesamiento

Imagina que eres el dueño de una pequeña biblioteca. Cuando un cliente te pide un libro, vas a la estantería, lo buscas y lo entregas. Si de pronto recibes 10 clientes a la vez, quizás puedas atenderlos tú solo, pero te moverás más rápido. Sin embargo, ¿qué pasaría si de la noche a la mañana tu biblioteca recibiera a **un millón de personas simultáneamente** pidiendo libros de estanterías que miden kilómetros?

Por más que corras, tus piernas (CPU) y tu memoria (RAM) tienen un límite físico. En este punto, no importa qué tan eficiente seas: necesitas **un equipo de trabajo** que se divida el almacén y las tareas.

En el mundo del análisis de datos, este es el salto entre trabajar con una base de datos local como SQLite en tu laptop y entrar al terreno del **procesamiento distribuido**. En esta unidad, aprenderás cómo las empresas líderes (como Netflix o Uber) logran que miles de computadoras trabajen al unísono como si fueran una sola.

1. El Salto a la Computación Distribuida

Hasta ahora, en este módulo, hemos manipulado datos usando SQL y Python en entornos locales. Esto funciona de maravilla cuando los datos caben en la memoria de tu computadora. Pero cuando hablamos de Terabytes o Petabytes de información (el famoso Big Data que vimos en la unidad anterior), una sola máquina fallará por falta de recursos.

¿Por qué escala el procesamiento?

El procesamiento distribuido es la solución al problema de la **escalabilidad**. Existen dos formas de escalar: 1. **Escalamiento Vertical:** Comprar una computadora más grande (más RAM, mejor procesador). Problema: Es extremadamente caro y tiene un techo físico inevitable. 2. **Escalamiento Horizontal:** Añadir más computadoras comunes conectadas entre sí. Esta es la base de los frameworks modernos: si el trabajo crece, simplemente añadimos otro eslabón a la cadena.

2. La Anatomía de un Sistema Distribuido: Clusters y Nodos

Un sistema distribuido no es una masa amorfa de máquinas; es una orquesta organizada. Para entender cómo funciona, debemos dominar dos términos fundamentales:

El Cluster

Un **Cluster** es un conjunto de computadoras (llamadas nodos) que están conectadas mediante una red de alta velocidad y que trabajan juntas para realizar una tarea. Para el usuario final o el analista, el cluster parece un solo sistema superpotente, pero detrás de escena hay cientos de máquinas colaborando.

Nodos: El Jefe y los Obreros

Dentro de un cluster, las máquinas se dividen roles para mantener el orden. Esta arquitectura se conoce comúnmente como **Master-Worker** (Maestro-Trabajador):

- **Nodo Maestro (Master Node):** Es el cerebro de la operación. No realiza el "trabajo pesado" de procesar datos. Su función es coordinar: decide qué parte del archivo va a qué máquina, monitorea si un trabajador se "enfermó" (falló) y redirige las tareas.
- **Nodos Trabajadores (Worker/Slave Nodes):** Son los procesadores. Reciben instrucciones del Maestro, ejecutan los cálculos sobre los datos que tienen asignados y devuelven el resultado.

Error común de principiante: Pensar que "más nodos significa siempre más velocidad". *La realidad:* Existe algo llamado **overhead de comunicación**. Si tienes una tarea muy pequeña y la repartes entre 100 máquinas, el tiempo que tarda el Maestro en dar las órdenes y recibir las respuestas será mayor que el tiempo que tardaría una sola máquina en hacer el trabajo. El procesamiento distribuido solo brilla cuando la tarea es masiva.

3. Apache Hadoop: El Abuelo del Almacenamiento Masivo

Lanzado a mediados de los 2000, **Hadoop** cambió la historia al permitir que empresas comunes procesaran volúmenes de datos que antes solo Google podía manejar. Aunque hoy existen herramientas más rápidas, Hadoop sigue siendo el ecosistema fundamental.

Hadoop se basa en dos pilares:

1. **HDFS (Hadoop Distributed File System):** Es un sistema de archivos que corta un archivo gigante en "pedazos" (bloques) y los reparte en diferentes nodos del cluster. Además, crea copias de cada pedazo por seguridad (tolerancia a fallos).
2. **MapReduce:** Es el "modelo de pensamiento". * **Map:** El sistema reparte la tarea (por ejemplo, contar palabras en un millón de libros) entre los nodos. * **Reduce:** El sistema recolecta los resultados parciales de cada nodo y los suma para dar la respuesta final.

Caso de uso: Amazon usa conceptos de Hadoop para almacenar el histórico de ventas de años. No necesitan la respuesta en segundos, pero necesitan que ni un solo bit de información se pierda si un disco duro se rompe.

4. Apache Spark: Velocidad en Memoria

Si Hadoop es un tren de carga (potente pero lento porque escribe todo en el disco duro), **Apache Spark** es un coche de carreras. Spark nació para solucionar la lentitud de Hadoop MapReduce.

¿Por qué Spark es el favorito de los Data Scientists?

La gran diferencia es que Spark procesa los datos **en la memoria RAM** siempre que sea posible, en lugar de leer y escribir constantemente en el disco duro. Esto lo hace hasta **100 veces más rápido** que Hadoop para ciertas tareas.

Conceptos que debes conocer de Spark: * **procesamiento en Memoria:** Ideal para algoritmos de Machine Learning que necesitan pasar por los mismos datos una y otra vez (iteración). * **PySpark:** Es la interfaz que permite a los analistas de Python usar Spark con una sintaxis muy parecida a Pandas, pero con el poder de un cluster detrás.

Ejemplo real: Netflix utiliza Spark para entrenar sus modelos de recomendación. Analizan tu historial de clics, lo comparan con el de millones de personas y, gracias a la velocidad de Spark, pueden actualizar tus sugerencias casi diariamente.

5. Apache Kafka: El Sistema Nervioso de los Datos

Hasta ahora, Hadoop y Spark se enfocan en procesar datos que ya están guardados (procesamiento por lotes o *batch*). Pero, ¿qué pasa si los datos nunca dejan de llegar?

Aquí entra **Apache Kafka**. Kafka no es una base de datos ni un motor de cálculo; es una **plataforma de streaming de eventos**.

El modelo Publicador/Suscriptor

Imagina que Kafka es una central de correos gigante: * **Productores:** Aplicaciones que envían datos (ej. un sensor de temperatura, el GPS de un Uber). * **Consumidores:** Aplicaciones que necesitan esos datos (ej. un tablero que muestra el precio de la gasolina en tiempo real).

Kafka actúa como un amortiguador (buffer). Si una aplicación envía 1 millón de mensajes por segundo y el sistema de análisis solo puede procesar 500,000, Kafka guarda el resto de forma segura hasta que el sistema se ponga al día.

Caso de estudio: Uber. Cuando pides un viaje, tu GPS envía señales constantes. Kafka recibe esos miles de millones de señales de GPS y las entrega ordenadamente a los algoritmos de Spark que calculan la tarifa dinámica según la demanda en ese preciso segundo.

6. Comparativa Rápida: ¿Cuándo usar qué?

Como futuro científico de datos, tu trabajo no es saber cómo instalar estas herramientas (eso es para los Ingenieros de Datos), sino saber **cuál pedir** según el problema:

Característica	Hadoop (MapReduce)	Apache Spark	Apache Kafka
Rol Principal	Almacenamiento masivo y procesamiento batch económico.	Procesamiento de alto rendimiento y Machine Learning.	Ingesta de datos en tiempo real y mensajería.
Velocidad	Lento (basado en Disco).	Muy rápido (basado en RAM).	Ultra rápido (Latencia mínima).
Estado de los datos	Datos "en reposo" (históricos).	Datos "en reposo" o micro-lotes.	Datos "en movimiento" (streaming).

7. Errores y Mitos Comunes

1. **"Hadoop está muerto":** Falso. Aunque MapReduce se usa menos, el sistema de archivos de Hadoop (HDFS) sigue siendo el estándar de almacenamiento para muchos lagos de datos (*Data Lakes*).
2. **"Spark reemplaza a las bases de datos":** No. Spark es un motor de procesamiento. Normalmente lee de una base de datos (como una que manejes con SQLAlchemy o DuckDB), procesa los datos y guarda el resultado.
3. **"Cualquier problema de ML requiere Spark":** Para nada. Muchos modelos de Machine Learning de Scikit-Learn funcionan mejor y más rápido en una sola máquina potente con Pandas. Solo pide Spark cuando tus datos superen los 10-20 GB (una regla general flexible).

Síntesis de la Unidad

Hemos recorrido el camino desde el límite de una sola computadora hasta el poder de un ejército de servidores: * Aprendimos que un **Cluster** es un equipo de cómputo y que los **Nodos** se dividen en jefes

(Master) y ejecutores (Worker). * Vimos que **Hadoop** es la base del almacenamiento distribuido. * Descubrimos que **Spark** es la herramienta estrella para analistas por su velocidad de procesamiento en memoria. * Entendimos que **Kafka** permite que los datos fluyan en tiempo real sin saturar nuestros sistemas.

En la siguiente unidad, cerraremos este módulo conectando todos estos conceptos de manipulación y escala en un flujo de trabajo coherente.

PRE-ENTREGA DEL PROYECTO

Actividad evaluable · parte del proyecto integrador

Pre-entrega: Adquisición y Normalización con Pandas

El objetivo de esta actividad es aplicar técnicas de adquisición de datos desde múltiples fuentes (CSV y Excel) para consolidar un dataset limpio y listo para el análisis, utilizando la librería `pandas`.

Cómo resolver esta entrega

Propósito

Simular un escenario real donde los datos de ventas provienen de distintos departamentos en formatos diferentes (un archivo CSV de transacciones y un Excel con el catálogo de productos). Deberás integrar y normalizar esta información.

Tarea

1. **Configuración**: Crea un nuevo repositorio en GitHub para este proyecto.
2. **Adquisición**: * Carga un archivo CSV que represente transacciones (debe incluir: id_producto, cantidad, fecha). * Carga un archivo Excel (.xlsx) que contenga el maestro de productos (id_producto, nombre_producto, precio_unitario).
3. **Transformación**: * Verifica y corrige tipos de datos (asegura que las fechas sean objetos datetime). * Maneja valores nulos (elimina o imputa según el contexto). * Realiza un `merge` entre ambos datasets usando el ID del producto como llave.
4. **Normalización**: Crea una nueva columna que calcule el 'total_venta' (cantidad * precio_unitario).
5. **Exportación**: Genera un archivo final en formato Parquet para optimizar el almacenamiento.

Errores Comunes

- No cerrar el motor de lectura de Excel (`openpyxl` debe estar instalado).
- Intentar hacer un merge con columnas de tipos diferentes (ej. ID como string en uno y entero en otro).
- Olvidar incluir un archivo README explicativo en el repositorio.

Objetivos de aprendizaje

- Cargar datos desde múltiples formatos como CSV y Excel utilizando pandas.
- Implementar técnicas de limpieza y manejo de valores nulos en datasets reales.
- Relacionar conjuntos de datos mediante operaciones de unión y normalización.
- Exportar datos procesados a formatos de almacenamiento eficiente como Parquet.

Criterios de aceptación

- El repositorio contiene el código fuente funcional sin errores de ejecución.
- Se realiza un merge correcto entre al menos dos fuentes de datos de distinto formato.
- El dataset final incluye una columna calculada y no presenta valores nulos en campos críticos.
- El README detalla los pasos para reproducir la carga de datos y las librerías necesarias.
- Se incluye el archivo de salida final en formato Parquet.

Rúbrica de evaluación: Adquisición y normalización de datos con Pandas

Esta rúbrica evalúa un repositorio de GitHub que demuestra la adquisición de datos desde CSV y Excel, su limpieza, integración mediante merge, normalización de variables y exportación final a Parquet. Se valora tanto la corrección técnica como la organización del repositorio, la documentación y la capacidad de presentar un flujo de trabajo reproducible y profesional, acorde a un proyecto real de ciencia de datos en nivel principiante.

Criterio	Descripción	Peso
Adquisición e integración de fuentes de datos	Evalúa si el repositorio carga correctamente el archivo CSV de transacciones y el archivo Excel del maestro de productos, utilizando pandas y openpyxl de forma adecuada para preparar ambos datasets antes de su integración.	27%
Limpieza, tipos de datos y manejo de nulos	Evalúa si el estudiante verifica y corrige tipos de datos, especialmente fechas como datetime, y si maneja valores nulos de manera coherente con el contexto del problema antes de realizar análisis o combinaciones.	18%
Merge y coherencia de claves	Evalúa si el repositorio integra correctamente ambos datasets mediante la llave `id_producto`, cuidando que los tipos de datos sean compatibles y que el resultado preserve la información esperada.	18%
Normalización y cálculo de total_venta	Evalúa si el estudiante crea correctamente la columna `total_venta` como producto de cantidad por precio_unitario y si el resultado es coherente con los datos integrados.	14.000000000000002%
Exportación a Parquet y entregable reproducible	Evalúa si el repositorio genera correctamente el archivo final en formato Parquet y si el entregable está organizado de forma reproducible, incluyendo script o notebook y archivos necesarios para entender y ejecutar el proceso.	14.000000000000002%
Documentación y organización del repositorio	Evalúa la presencia de README explicativo, estructura clara del repositorio, nombres comprensibles y evidencia de una presentación ordenada del proyecto en GitHub.	9%

Total: 100 pts | Aprobación: 70 pts

MÓDULO 4

Introducción a Machine Learning y Data Acquisition I

Introducir conceptos de Machine Learning y procesos de adquisición de datos.

El ciclo de vida del Machine Learning: de la pregunta al modelo

Imagina que trabajas en el equipo de datos de una aplicación de streaming de música, similar a Spotify. Un lunes por la mañana, el director de producto te hace una pregunta: "¿Podemos predecir qué usuarios cancelarán su suscripción el próximo mes para ofrecerles un descuento preventivo?".

En ese momento, tienes dos opciones. La primera es entrar en pánico y empezar a programar algoritmos complejos al azar. La segunda —la que distingue a un profesional— es aplicar el **Ciclo de Vida del Machine Learning**.

El Machine Learning (ML) no se trata solo de "entrenar modelos"; es un proceso iterativo, un mapa que nos guía desde una duda de negocio ambigua hasta una solución tecnológica que genera valor real. En esta unidad, aprenderás a recorrer ese camino, entendiendo que cada etapa es un eslabón crítico: si uno falla, el modelo final será inútil.

1. El ciclo de vida del ML: Un proceso iterativo

A diferencia del desarrollo de software tradicional, donde el flujo suele ser más lineal (escribir código -> probar -> desplegar), el Machine Learning es profundamente experimental. No siempre sabemos de antemano si los datos que tenemos son suficientes o si el algoritmo elegido funcionará.

El ciclo de vida se visualiza como un círculo porque, a menudo, los resultados de la evaluación nos obligan a volver atrás: a recolectar más datos, a limpiar mejor los existentes o a redefinir el problema original. Es un baile constante entre los datos y los algoritmos.

¿Por qué necesitamos un ciclo definido?

Sin una estructura, los proyectos de ciencia de datos suelen fracasar por tres razones comunes: 1. **Falta de alineación:** El modelo es técnicamente brillante pero resuelve un problema que a nadie le interesa o que no tiene impacto económico. 2. **Datos de mala calidad:** Aplicamos algoritmos avanzados a datos con errores, obteniendo predicciones absurdas (el famoso concepto *Garbage In, Garbage Out* o "si entra basura, sale basura"). 3. **Imposibilidad de despliegue:** El modelo funciona en la computadora del científico de datos, pero es imposible de integrar en la infraestructura real de la aplicación.

2. Definición del problema: La piedra angular

Todo gran proyecto de ML comienza con una pregunta, no con un código. Esta fase consiste en traducir una necesidad vaga de la empresa en un requisito técnico concreto.

Transformando problemas de negocio en problemas de ML

Un problema de negocio es: "Estamos perdiendo clientes". Un problema de ML es: "**Clasificar** si un usuario activo cancelará su suscripción en los próximos 30 días basándonos en su historial de escucha y pagos de los últimos 3 meses".

Aquí es donde decides el tipo de tarea que vas a realizar: * **Clasificación:** ¿El resultado es una categoría? (Ej: ¿Es fraude o no?). * **Regresión:** ¿El resultado es un número continuo? (Ej: ¿Cuál será el precio de venta de esta casa?).

Definición de métricas de éxito

¿Cómo sabremos si el modelo es "bueno"? No basta con una intuición. Necesitamos métricas técnicas conectadas con el negocio. Si por ejemplo estamos prediciendo fallos en piezas de aviones, no nos basta con que el modelo acierte "en promedio"; necesitamos que nunca falle al detectar una pieza rota (esto se llama prioridad en el *Recall*).

Ejemplo real (Uber): Si Uber quiere predecir el tiempo de llegada (ETA), una métrica de éxito fundamental es el **Error Medio Absoluto (MAE)**. Si el modelo dice que el auto llega en 5 minutos pero llega en 15, la experiencia del usuario se arruina. La métrica define el norte del proyecto y determina cuándo el modelo está listo para salir a producción.

3. Adquisición y Preparación de Datos: El 80% del trabajo

Es una verdad universal en Data Science: pasarás mucho más tiempo limpiando datos que ajustando modelos.

Adquisición (Data Acquisition)

Como ya manejas SQL y conceptos de arquitectura, sabes que los datos viven en silos. Debes extraerlos de: * Bases de datos productivas (vía `SQLAlchemy` o conectores directos a PostgreSQL/SQLite). * Archivos de logs en servidores (JSON o texto plano). * APIs externas donde resides información del clima o economía.

Limpieza y Preparación: La regla de oro del "Test Set"

Aquí aplicas herramientas como `Pandas`, `Polars` o `DuckDB` para manejar nulos y duplicados. Pero lo más importante en ML es la **División de datos (Data Splitting)**.

Para evaluar un modelo honestamente, debemos dividir nuestro dataset en: 1. **Entrenamiento (Training Set):** Los datos que el modelo "lee" para aprender los patrones. 2. **Prueba (Test Set):** Un pequeño porcentaje de datos que guardamos bajo llave. El modelo **nunca** debe ver estos datos durante su entrenamiento.

Error conceptual común: Evaluar el modelo con los mismos datos que usaste para entrenarlo. Es como darle a un estudiante un examen con exactamente las mismas preguntas que venían respondidas en su libro de texto; el estudiante no aprendió matemáticas, solo memorizó las respuestas. En ML, esto se llama **Overfitting** (Sobreajuste) y es el error más común en analistas principiantes.

4. Selección, Entrenamiento y Evaluación

Una vez que los datos están listos, entramos en la fase de "cocina" algorítmica.

Selección del Modelo y el "Baseline"

Un error de novato es ir directo a las redes neuronales profundas. La buena práctica profesional comienza con un **baseline** o línea base. El baseline es el modelo más sencillo posible (ej. una Regresión Lineal simple o incluso el promedio histórico). *Si tu algoritmo súper complejo no supera significativamente al baseline, quédate con el simple: es más barato de mantener y más fácil de explicar.*

Entrenamiento

Es el proceso donde el algoritmo busca la relación matemática entre las características (features) y el objetivo (target). En `Scikit-Learn`, esto suele ser tan simple como ejecutar un método `.fit()`.

Evaluación: La hora de la verdad

Ahora sacas del cajón el **Test Set** que habías guardado. Le pides al modelo que prediga esos resultados y los comparas con la realidad. Si los resultados cumplen con la métrica de éxito definida en el paso 1, podemos avanzar. Si no, volvemos atrás: ¿Faltan datos? ¿La limpieza fue insuficiente?

5. El Despliegue (Deployment) y Monitoreo

Un modelo que solo vive en tu notebook no sirve para nada. El despliegue es el proceso de integrar el modelo en el flujo de trabajo de la empresa.

- **Producción:** Convertir el modelo en una API que la app de e-commerce o el banco puedan llamar cada milisegundo.
- **Monitoreo y Mantenimiento:** Los modelos se degradan. A diferencia del software, que funciona igual mientras el servidor esté encendido, el ML depende de la realidad. Si entrenaste un modelo de recomendación de ropa de invierno y llega el verano, el modelo fallará. A este cambio en el comportamiento de los datos reales se le llama **Data Drift** (Deriva de datos).

Síntesis y Conexiones

El ciclo de vida del ML no es una línea que termina, sino un círculo que evoluciona.

1. **Definición:** ¿Cuál es la pregunta y cómo la medimos?
2. **Datos:** Extraer, limpiar y, sobre todo, **separar** en entrenamiento y prueba.
3. **Modelado:** Empezar siempre por un baseline y luego iterar hacia la complejidad.
4. **Evaluación:** Ser estrictos con el Test Set.
5. **Producción:** Desplegar y vigilar que el modelo no se vuelva obsoleto.

En las próximas unidades, aprenderás a distinguir entre los diferentes "sabores" de estos modelos (Supervisado vs No Supervisado) y empezarás a ensuciarte las manos con código Python en el entorno de `Scikit-Learn`. Recuerda: el algoritmo es solo una pieza pequeña en el gran rompecabezas del ciclo de vida.

Aprendizaje supervisado y no supervisado: cuándo usar cada uno

En la unidad anterior, exploramos el ciclo de vida del Machine Learning (ML) y entendimos que el punto de partida siempre es una pregunta de negocio o un problema de datos. Sin embargo, una vez que tenemos la pregunta, el siguiente gran desafío es: **¿Cómo vamos a enseñar el patrón a nuestra computadora?**

Imagina que quieres enseñarle a un niño a distinguir entre frutas. Tienes dos caminos: puedes mostrarle una manzana y decirle "esto es una manzana", o puedes dejarlo en una habitación con 100 frutas mezcladas y pedirle que las agrupe por cómo se ven, sin decirle el nombre de ninguna. Esa diferencia fundamental entre "dar la respuesta" y "dejar que se descubra la estructura" es la esencia de lo que aprenderemos hoy.

1. La Piedra Angular: Datos Etiquetados vs. No Etiquetados

Antes de hablar de algoritmos, debemos hablar del combustible: los datos. En el mundo del Machine Learning, la diferencia entre los tipos de aprendizaje radica casi exclusivamente en la estructura del dataset.

Datos Etiquetados (Labeled Data)

Los datos etiquetados son aquellos donde ya conocemos la "respuesta correcta" para cada ejemplo. En términos técnicos, tenemos **características** (features) y una **etiqueta** (target/objetivo).

- **Ejemplo:** Un dataset de correos electrónicos donde cada fila tiene el texto del mensaje () y una marca manual que dice "Spam" o "No Spam" ().
- **Por qué importa:** Si tienes etiquetas, puedes entrenar un modelo para que aprenda la relación entre las características y el resultado. Es como estudiar con un examen que ya tiene las soluciones al final del libro.

Datos No Etiquetados (Unlabeled Data)

Aquí solo tenemos las **características** . No hay una variable que nos diga qué es cada cosa o qué valor debería tener.

- **Ejemplo:** Los datos de navegación de miles de usuarios en un sitio web (páginas vistas, tiempo de estancia), pero nadie ha clasificado a esos usuarios en categorías.
- **Por qué importa:** El objetivo aquí no es predecir algo, sino encontrar una estructura oculta. El modelo analiza las similitudes para decir: "No sé qué son estos usuarios, pero estos 500 se comportan de forma casi idéntica".

2. Aprendizaje Supervisado: El Modelo con Profesor

El **Aprendizaje Supervisado** se llama así porque el científico de datos actúa como un "supervisor" que le entrega al algoritmo un conjunto de datos donde las respuestas ya están dadas. El modelo hace una predicción, el supervisor (a través de una función matemática) le dice qué tan lejos estuvo de la realidad, y el modelo ajusta sus parámetros para fallar menos la próxima vez.

Su objetivo principal es la **predicción**. Se divide en dos grandes ramas según la naturaleza de lo que queremos predecir:

A. Clasificación (Categorías)

Usamos clasificación cuando la etiqueta es **discreta o categórica**. Estamos tratando de responder a preguntas de tipo "¿A qué grupo pertenece?".

- **Escenario Real (Sector Salud): Google Health** utiliza redes neuronales para analizar imágenes de mamografías. El modelo se entrena con miles de imágenes previamente etiquetadas por radiólogos expertos como "Maligno" o "Benigno".
- **Señal clave:** ¿La respuesta es una etiqueta o nombre? Entonces es clasificación.

B. Regresión (Números Continuos)

Usamos regresión cuando la etiqueta es un **valor numérico continuo**. Estamos tratando de responder a preguntas de tipo "¿Cuánto?".

- **Escenario Real (Proptech): Zillow**, la plataforma inmobiliaria, utiliza su modelo *Zestimate* para predecir el precio de mercado de una casa. El modelo analiza metros cuadrados, ubicación y año de construcción, comparándolos con precios de venta reales (etiquetas) para estimar un número exacto en dólares.
- **Señal clave:** ¿La respuesta es un precio, una temperatura o una cantidad? Entonces es regresión.

3. Aprendizaje No Supervisado: El Explorador Autónomo

En el **Aprendizaje No Supervisado**, le damos al algoritmo los datos y le decimos: "Encuentra algo interesante aquí". No hay etiquetas, por lo tanto, no hay "error" que corregir respecto a una verdad absoluta.

Su objetivo principal es el **descubrimiento de estructuras**.

Clustering (Agrupamiento)

Es la técnica más común. Consiste en agrupar objetos de tal forma que los miembros de un mismo grupo (cluster) sean más similares entre sí que con los de otros grupos.

- **Escenario Real (E-commerce): Amazon** utiliza clustering para segmentar a sus millones de usuarios. El algoritmo puede descubrir un grupo que "compra pañales los viernes y también cerveza", sin que ningún humano le haya dicho previamente que ese grupo de 'padres estresados' existía.
- **Uso:** Segmentación de clientes, compresión de imágenes, organización de noticias similares.

Reducción de Dimensionalidad

A veces tenemos demasiados datos (cientos de columnas) y es imposible visualizarlos o procesarlos eficientemente. Estas técnicas simplifican los datos manteniendo la información esencial.

- **Escenario Real (Genética):** Los investigadores usan técnicas como PCA (Análisis de Componentes Principales) para reducir miles de marcadores genéticos a solo 2 o 3 variables que pueden graficarse para ver cómo se agrupan diferentes poblaciones humanas.

4. ¿Cuándo usar cada uno? La Guía de Decisión

Para decidir qué enfoque tomar, hazte estas tres preguntas en orden:

1. **¿Tengo una variable objetivo (etiqueta) en mis datos históricos?** * Si **SÍ**: Ve por el camino del **Aprendizaje Supervisado**. * Si **NO**: Ve por el camino del **Aprendizaje No Supervisado**.
2. **Si elegiste Supervisado: ¿Qué tipo de dato es la etiqueta?** * Si son categorías (Sí/No, Perro/Gato, Riesgo Alto/Bajo): **Clasificación**. * Si es un número (Precio, Stock, Edad, Probabilidad 0-100%): **Regresión**.
3. **Si elegiste No Supervisado: ¿Qué quieres lograr?** * Si quieres agrupar individuos similares: **Clustering**. * Si quieres simplificar un dataset muy complejo: **Reducción de Dimensionalidad**.

5. Errores Comunes de Principiante

- **Error 1: "El aprendizaje no supervisado no necesita datos".** A menudo se piensa que como no hay etiquetas, el modelo "inventa" el conocimiento. Es al revés: el aprendizaje no supervisado suele requerir **muchos más datos** que el supervisado para poder encontrar patrones estadísticamente significativos sin ayuda.
- **Error 2: Confundir Clasificación con Clustering.** Este es el error más frecuente.
- Si tú defines los grupos de antemano (ej. "Quiero separar Spam de No Spam"): **Clasificación**.
- Si dejas que el algoritmo te diga cuántos grupos hay y qué los caracteriza: **Clustering**.
- **Error 3: Pensar que la Regresión es solo para "el futuro".** Muchos estudiantes creen que la regresión solo sirve para predecir el futuro (pronósticos). En realidad, la regresión sirve para estimar cualquier valor numérico desconocido, incluso en el presente (ej. "¿Cuánto debería valer este diamante según sus quilates?").

Resumen de la Unidad

Características	Aprendizaje Supervisado	Aprendizaje No Supervisado
Datos	Etiquetados ()	No etiquetados ()
Objetivo	Predecir resultados para datos nuevos	Descubrir patrones ocultos
Intervención Humana	Alta (para etiquetar datos)	Baja (el modelo explora solo)
Tipos principales	Clasificación y Regresión	Clustering y Reducción de Dimensión

Características	Aprendizaje Supervisado	Aprendizaje No Supervisado
Ejemplo	Filtro de Spam	Segmentación de mercado

En la práctica profesional, a menudo empezarás con un análisis no supervisado (clustering) para entender a tus clientes y, una vez que el negocio defina qué grupos le interesan, pasarás a un modelo supervisado (clasificación) para asignar automáticamente a los nuevos clientes a esos grupos. ¡Todo es parte de un mismo flujo de valor!

Fuentes de datos y lectura de archivos en Python

En el ciclo de vida del Machine Learning, la **adquisición de datos** es el primer paso técnico. Antes de entrenar modelos complejos, debemos ser capaces de "traer" los datos a nuestro entorno de ejecución de forma íntegra y segura.

1. ¿De dónde vienen los datos?

En la industria, los datos no siempre están en una base de datos limpia. A menudo trabajamos con:

- **Archivos planos (CSV/TSV):** Datos tabulares separados por comas o tabulaciones. Son el estándar de intercambio por su simplicidad.
- **Archivos estructurados (JSON/XML):** Ideales para datos jerárquicos o APIs web.
- **Fuentes Binarias (Parquet/Excel):** Optimizados para rendimiento o uso administrativo.

2. Rutas: El mapa hacia tus datos

Un error de principiante muy común es el `FileNotFoundError`. Para evitarlo, debemos entender las rutas:

- **Ruta Absoluta:** La dirección completa desde la raíz del disco (ej: `C:\Usuarios\Proyecto\data.csv`). El problema es que solo funciona en *tu* computadora.
- **Ruta Relativa:** La dirección respecto a donde está tu script (ej: `data/dataset.csv`). Es la mejor práctica para que tu código sea compartible.

Consejo PRO: Usa siempre la librería `pathlib` de Python. Te permite manejar rutas de forma que funcionen igual en Windows, Mac y Linux sin cambiar barras inclinadas manualmente.

3. Técnicas de Lectura

A. CSV con Pandas

Pandas es la herramienta reina. Con `pd.read_csv()`, no solo lees el archivo, sino que lo conviertes en un **DataFrame**, una estructura poderosa para manipular tablas.

CÓDIGO

```
import pandas as pd
df = pd.read_csv('datos.csv', sep=',', encoding='utf-8')
```

B. JSON con `json`

El formato JSON se lee nativamente como un diccionario de Python. Es vital usar el bloque `with` (context manager), que garantiza que el archivo se cierre correctamente incluso si ocurre un error durante la lectura, liberando memoria del sistema.

CÓDIGO

```
import json
with open('config.json', 'r') as f:
 datos = json.load(f)
```

4. Errores Comunes a Evitar

1. **Ignorar el Encoding:** Si tus datos tienen tildes o "ñ" y ves caracteres extraños como `Ã±`, probablemente el archivo esté en `latin-1` o `utf-8`. Especifica `encoding='utf-8'` al leer.
2. **No cerrar el archivo:** Si abres un archivo sin `with`, el archivo queda "bloqueado" por el sistema operativo hasta que el programa termine.
3. **Asumir el separador:** No todos los CSV usan comas. Algunos usan punto y coma `;` o pipes `|`. Verifica siempre las primeras líneas del archivo antes de leerlo.

En este ejercicio, pondremos en práctica la lectura robusta de ambos formatos.

Preparación inicial de datasets: limpieza, tipos y división

En las unidades anteriores, aprendimos cómo se lee la información desde diversas fuentes. Sin embargo, en el mundo real, los datos rara vez llegan listos para ser procesados por un algoritmo de Machine Learning.

La **preparación inicial** es el puente entre los datos crudos y el modelado. Un modelo de ML es tan bueno como los datos con los que se entrena (*Garbage In, Garbage Out*).

¿Por qué debemos limpiar los datos?

1. **Valores Nulos (`NaN`):** La mayoría de los algoritmos de Scikit-Learn no pueden trabajar con valores faltantes y lanzarán un error si intentas entrenarlos así.
2. **Duplicados:** Las filas repetidas pueden darle una importancia artificial a ciertos patrones, haciendo que el modelo se "aprenda de memoria" esos casos (sobreentrenamiento).
3. **Inconsistencias:** Formatos de fecha distintos o errores tipográficos (ej. "Macho" vs "macho") confunden al modelo, tratándolos como categorías diferentes.

El Análisis Exploratorio Inicial (EDA)

Antes de limpiar, debemos observar. En Pandas, usamos:

- `df.info()`: Para ver tipos de datos y cantidad de no-nulos.
- `df.describe()`: Para obtener estadísticas rápidas (media, desviación estándar, etc.).
- `df.isnull().sum()`: Para contar exactamente cuántos datos nos faltan por columna.

La Regla de Oro: División Train-Test

Uno de los conceptos más críticos en ML es la **División de datos**. Nunca debemos evaluar nuestro modelo con los mismos datos que usamos para entrenarlo.

- **Train (Entrenamiento):** El modelo ve estos datos y aprende sus patrones (típicamente 70-80%).
- **Test (Prueba):** Son datos que el modelo JAMÁS ha visto. Funcionan como el examen final para medir su capacidad real de generalización (típicamente 20-30%).

> **Aviso**
>
> **¡Cuidado!.** Existe un error común llamado *Data Leakage* (Fuga de datos). Ocurre cuando información del set de prueba se filtra al de entrenamiento (por ejemplo, si usamos la media de TODO el dataset para llenar nulos antes de dividir). Lo ideal es dividir y luego limpiar.

En esta unidad, practicaremos estas técnicas fundamentales usando `pandas` y `scikit-learn`.

PRE-ENTREGA DEL PROYECTO

Actividad evaluable · parte del proyecto integrador

Pre-entrega: Ciclo de vida de un proyecto de Machine Learning

El objetivo de esta actividad es practicar los fundamentos del ciclo de vida de un proyecto de Machine Learning, centrándote en la fase inicial de adquisición y comprensión de datos. Aprenderás a identificar fuentes de datos adecuadas y a justificar por qué un problema específico puede resolverse mediante técnicas de aprendizaje automático.

Cómo resolver esta entrega

Pasos a seguir

1. **Selección del Problema**: Elige un escenario de negocio o ciencia (ej. predicción de precios de viviendas, detección de fraude, clasificación de especies de flores).
2. **Hipótesis y Objetivo**: Define claramente qué quieres predecir (el target) y qué tipo de aprendizaje estás proponiendo (Supervisado/No supervisado).
3. **Identificación de Datos**: Busca y documenta una fuente de datos real (ej. Kaggle, UCI Machine Learning Repository o una API pública). Debes indicar el enlace directo.
4. **Diccionario de Datos**: Describe al menos 5 variables (features) del dataset y su tipo (numérico, categórico, ordinal).
5. **Formatos**: Justifica si el formato del archivo (CSV, JSON, SQL) es óptimo para el volumen de datos identificado.

Entrega

Debes redactar esta propuesta técnica en un documento PDF o Word (.docx) que no exceda las 2 páginas. Asegúrate de que la conexión entre el problema y los datos sea coherente.

Objetivos de aprendizaje

- Diferenciar entre tipos de aprendizaje automático para proponer una solución técnica adecuada.
- Identificar fuentes de datos confiables y accesibles para proyectos de Machine Learning.
- Definir un diccionario de datos básico que sirva como base para la etapa de ingeniería de variables.

Criterios de aceptación

- El documento identifica claramente el problema a resolver y el tipo de Machine Learning (Supervisado/No supervisado).
- Se incluye un enlace funcional a un dataset real (fuente externa).
- El diccionario de datos contiene al menos 5 variables descritas con su nombre y tipo de dato correspondiente.

- La justificación de la adquisición de datos es coherente con los objetivos del proyecto planteado.
- El archivo entregado está en formato PDF o DOCX exclusivamente.

Rúbrica de evaluación: Propuesta inicial de proyecto de Machine Learning y adquisición de datos

Esta rúbrica evalúa una propuesta técnica introductoria en la que el estudiante selecciona un problema, define un objetivo de aprendizaje automático, identifica una fuente de datos real, construye un diccionario inicial de variables y justifica el formato de almacenamiento más adecuado. La evaluación prioriza la coherencia entre problema y datos, la claridad conceptual y la viabilidad de la propuesta dentro del contexto de iniciación a Machine Learning.

Criterio	Descripción	Peso
Selección del problema y definición del objetivo	Evalúa si el problema elegido es pertinente para Machine Learning, si el target está claramente definido y si la propuesta indica correctamente el tipo de aprendizaje (supervisado o no supervisado).	30%
Identificación y adecuación de la fuente de datos	Evalúa si la fuente de datos es real, accesible, relevante para el problema y documentada con enlace directo, además de si existe una conexión lógica entre el dataset elegido y el objetivo planteado.	25%
Diccionario de datos y clasificación de variables	Evalúa la calidad del diccionario de datos inicial, incluyendo al menos 5 variables, su descripción y su tipo (numérico, categórico u ordinal), así como la precisión y utilidad de la información presentada.	20%
Justificación del formato de archivo y viabilidad técnica	Evalúa si el estudiante justifica de manera razonada si el formato del archivo propuesto (CSV, JSON, SQL u otro) es adecuado para el volumen y la estructura de datos identificados.	15%
Claridad, estructura y presentación del documento	Evalúa la organización general del texto, la redacción técnica, la legibilidad y el cumplimiento de la extensión solicitada (máximo 2 páginas), así como la calidad de la presentación del archivo final.	10%

Total: 100 pts | Aprobación: 70 pts

MÓDULO 5

Data Acquisition II (APIs)

Implementar la obtención de datos mediante APIs externas de forma segura.

Fundamentos de APIs para la Ciencia de Datos

APIs: Los Mensajeros de la Web

Imagina que estás construyendo un modelo de Machine Learning para predecir el precio de las acciones en el mercado financiero. Tienes el algoritmo perfecto, pero te falta lo más importante: **gasolina para tu motor**, es decir, los datos.

¿Cómo los consigues de forma eficiente? No puedes ir a la página de Yahoo Finance y copiar y pegar los números a mano cada cinco minutos. Tampoco puedes entrar directamente en la base de datos de la Bolsa de Nueva York porque, por razones de seguridad obvias, no te darán la contraseña.

Aquí es donde entran las **APIs** (Application Programming Interfaces). Las APIs son las venas y arterias de la internet moderna. Sin ellas, las aplicaciones no podrían hablar entre sí y el flujo de datos que alimenta la ciencia de datos simplemente se detendría.

En esta unidad, aprenderás qué son realmente, por qué son preferibles a otros métodos de obtención de datos y cómo revolucionan la forma en que trabajamos como Data Scientists.

1. ¿Qué es realmente una API? (El Concepto Fundamental)

Las siglas API significan **Application Programming Interface** (Interfaz de Programación de Aplicaciones). Aunque el nombre suena intimidante y técnico, su concepto es muy sencillo: **es un contrato de comunicación entre dos sistemas.**

Para entenderlo sin tecnicismos, usemos la analogía más famosa en el mundo del desarrollo de software: **El Camarero en un Restaurante.**

La Analogía del Restaurante

Imagina que tú eres el **Cliente** (el script de Python que estás escribiendo). Tienes hambre de datos.

1. **La Cocina:** Es el servidor o la base de datos de una empresa (por ejemplo, Twitter o Google). Allí es donde están los ingredientes y se preparan los platos (los datos), pero a ti no se te permite entrar a la cocina por higiene y seguridad.
2. **El Menú:** Es la **Documentación de la API**. Te dice qué platos están disponibles y qué ingredientes traen. No puedes pedir algo que no esté en el menú.
3. **El Camarero (La API):** Es el intermediario. Tú le das tu pedido (la "petición" o *request*), él va a la cocina, le dice al chef lo que necesitas y regresa a tu mesa con el plato servido (la "respuesta" o *response*).

¿Por qué es brillante este sistema?

- **Abstracción:** No necesitas saber cómo funciona la cocina por dentro ni qué tipo de sartenes usan. Solo necesitas saber cómo pedirle al camarero.
- **Seguridad:** El camarero se asegura de que no entres a la cocina a desordenar nada. Solo te entrega lo que estás autorizado a recibir.

En términos de Data Science, una API es el mecanismo que nos permite "tocar la puerta" de un servidor externo y pedir información de manera estructurada y segura.

2. API vs. Web Scraping: El camino profesional

En módulos anteriores, es posible que hayas oído hablar o practicado el *Web Scraping* (extraer datos directamente del código HTML de una página web). Es vital que un Data Scientist entienda cuándo usar cada uno, porque **las APIs son casi siempre la opción superior.**

Característica	Web Scraping	API
Estructura	Desordenada (HTML diseñado para humanos).	Estructurada (JSON/XML diseñado para máquinas).
Estabilidad	Frágil (si la web cambia un botón, tu código se rompe).	Estable (el "contrato" de la API rara vez cambia).
Legalidad	A veces cuestionable o prohibida en términos de uso.	Oficial y permitida por el proveedor.
Velocidad	Lenta (hay que descargar y procesar toda la web).	Rápida (solo descargas los datos específicos).

Un error común es pensar que "si los datos están en la web, puedo scrapear". Sin embargo, las empresas prefieren que uses su API.

Por ejemplo, si intentas obtener datos de Twitter mediante scraping masivo, probablemente bloqueen tu dirección IP. Si usas su API oficial, tienes un canal de comunicación directo y "bendecido" por ellos.

3. Casos de Uso Reales: Datos en tus dedos

Para aterrizar estos conceptos, veamos cómo las empresas y los científicos de datos utilizan APIs reales hoy en día:

A. OpenWeatherMap (El estándar para principiantes)

Imagínate una aplicación de logística que necesita predecir retrasos en entregas. El clima es un factor clave. En lugar de tener estaciones meteorológicas propias, la aplicación consulta la API de **OpenWeatherMap**.

- **Petición:** "Dame el clima actual de Buenos Aires".

- **Respuesta:** Un archivo pequeño con la temperatura, humedad y velocidad del viento.

B. Alpha Vantage y Yahoo Finance

En el mundo financiero, los modelos de Machine Learning necesitan datos históricos de precios de cierre, volumen de transacciones y dividendos. APIs como **Alpha Vantage** permiten a un script de Python descargar 20 años de historia de una acción (como Apple o Tesla) en cuestión de segundos.

C. Google Maps API

¿Has notado que muchas aplicaciones (como Uber o Rappi) usan mapas de Google? No es que Google les haya dado una copia de sus mapas. Esas aplicaciones llaman a la **API de Google Maps** cada vez que necesitan calcular una ruta o mostrar una dirección. Google cobra por cada vez que el "camarero" (API) lleva esa información.

4. El Ciclo de Vida de una Petición (Request-Response)

Para configurar una adquisición de datos exitosa, debemos entender qué sucede técnicamente cuando consultamos una API. Este proceso se llama el ciclo de **Petición y Respuesta**.

1. **El Endpoint (La Dirección):** Es la URL específica a la que llamamos. No es una página web para ver, sino un "punto de entrada" para datos. * *Ejemplo:* `api.ejemplo.com/v1/usuarios`
2. **El Método HTTP:** Es la acción que queremos realizar. * **GET:** "Tráeme datos" (Es el que más usaremos en Data Science). * **POST:** "Guarda estos datos" (Para enviar información al servidor).
3. **Los Parámetros:** Son los detalles de nuestra búsqueda. Si la API es de cine, el parámetro podría ser `pelicula=Inception`.
4. **La Respuesta (Payload):** Si todo sale bien, la API nos devuelve un código de estado (como el famoso **200 OK**) y los datos, usualmente en formato **JSON**.

¿Qué es JSON? (Breve introducción)

JSON (*JavaScript Object Notation*) es el lenguaje universal de las APIs. Aunque su nombre mencione JavaScript, es un formato de texto que prácticamente todos los lenguajes de programación entienden. Se parece mucho a un diccionario de Python:

CÓDIGO

```
{
 "usuario": "JuanData",
 "id": 4502,
 "activo": true,
 "habilidades": ["Python", "SQL", "Machine Learning"]
}
```

Como verás, es muy fácil de leer tanto para humanos como para computadoras.

5. Errores Comunes y Trampas Conceptuales

Incluso los profesionales cometen errores al empezar con APIs. Aquí te presento los más frecuentes para que puedas evitarlos:

Confusión 1: "La API es la Base de Datos"

Error: Pensar que cuando consultamos una API estamos entrando a la base de datos de la empresa. **Realidad:** La API es una **capa intermedia**. Es como una ventanilla de banco. No entras a la bóveda; hablas con el cajero, y el cajero (basado en reglas) decide qué te entrega. Esto protege los datos sensibles de la empresa.

Confusión 2: "Los datos de la API son siempre gratuitos"

Error: Suponer que todas las APIs son abiertas. **Realidad:** La mayoría de las APIs profesionales tienen **límites de velocidad (Rate Limits)**. Por ejemplo, una API gratuita podría dejarte hacer solo 5 peticiones por minuto. Si necesitas más, debes pagar. Como Data Scientist, debes planificar tu "pipeline" de datos teniendo en cuenta estos límites para no bloquear tu proceso de ingesta.

Confusión 3: "La URL es suficiente para obtener los datos"

Error: Intentar acceder a una API privada solo poniendo el link en el navegador. **Realidad:** Muchas APIs requieren **Autenticación**. Imagina que es un pase VIP. Si no incluyes tu "Token" o "API Key" en la petición, el servidor te responderá con un error **401 Unauthorized**.

6. ¿Por qué esto es vital para la Ciencia de Datos?

Podrías pensar: "¿Por qué no me dan simplemente un archivo CSV y ya?".

La respuesta es que el mundo real se mueve demasiado rápido para los archivos estáticos.

- **Datos en tiempo real:** Si analizas sentimientos en redes sociales durante un evento en vivo, necesitas la API de Twitter para obtener tweets *ahora mismo*.
- **Volumen masivo:** No puedes descargar toda la base de datos de clima del mundo. Usas una API para traer solo los datos de las coordenadas que te interesan.
- **Automatización:** El objetivo final de un Data Scientist no es hacer un análisis una sola vez, sino crear sistemas que se alimenten solos. Las APIs permiten que tu modelo de ML se despierte a las 3:00 AM, consulte nuevos datos, se re-entrene y envíe una predicción sin que tú tengas que mover un dedo.

Síntesis de la Unidad

En esta introducción al Módulo 4, hemos establecido los cimientos para convertirnos en expertos en adquisición de datos:

- Entendimos que una **API** es un mediador oficial entre sistemas.
- Aprendimos la analogía del **camarero**, separando al cliente (nuestro código) de la cocina (el servidor).

- Reconocimos que las APIs son más estables y profesionales que el **Web Scraping**.
- Identificamos el formato **JSON** como el estándar de entrega de datos.

En las próximas unidades, pasaremos de la teoría a la práctica: aprenderás a usar la librería `requests` de Python para enviar estas peticiones, cómo manejar las llaves de seguridad y cómo limpiar esos datos JSON para convertirlos en un DataFrame de Pandas listo para ser analizado.

¡Prepárate, porque estás a punto de conectar tus programas de Python con el resto del mundo digital!

Autenticación, tokens y variables de entorno

Imagina que eres un analista de datos en una empresa de e-commerce y has desarrollado un script brillante que extrae tendencias de ventas en tiempo real desde una API externa. Todo funciona de maravilla hasta que un día, al subir tu código a un repositorio compartido (como GitHub), recibes una alerta de seguridad: **has expuesto tus claves de acceso**.

En cuestión de minutos, bots automatizados han robado tus credenciales y están realizando miles de peticiones pagas a tu nombre. Este escenario, aunque parezca una película de terror, es una realidad diaria para quienes no dominan la gestión segura de accesos.

En esta unidad, aprenderás a transformar un script de extracción "casero" en una herramienta profesional, segura y robusta, distinguiendo entre quién accede y qué puede hacer, y protegiendo tus secretos mediante el uso de variables de entorno.

1. Manejo de autenticación y tokens: El "Pase VIP" de los datos

En el mundo de las APIs, no basta con tocar la puerta; hay que demostrar quién eres. Aquí es donde entran en juego dos conceptos que a menudo se confunden, pero que son fundamentalmente distintos.

Autenticación vs. Autorización

Para entenderlo, imagina que vas a un hotel:

1. **Autenticación:** En la recepción, muestras tu documento de identidad. El recepcionista confirma: "Sí, tú eres Juan Pérez". Responde a la pregunta: **¿Quién eres tú?**
2. **Autorización:** El recepcionista te entrega una tarjeta magnética que solo abre la habitación 402, pero no la 505 ni la oficina del gerente. Responde a la pregunta: **¿A qué tienes permiso de acceder?**

En Data Science, la **autenticación** suele ocurrir una vez (te logueas para obtener una clave), mientras que la **autorización** ocurre en cada petición que haces a la API para extraer un dataset específico.

Introducción a los Tokens (API Keys y JWT)

Enviar tu usuario y contraseña en cada petición HTTP es extremadamente inseguro e ineficiente. Por eso, las APIs modernas utilizan **Tokens**. Un token es una cadena de caracteres alfanuméricos que funciona como un "pase digital" temporal o permanente.

El flujo estándar:

1. **Solicitud:** Envías tus credenciales (a través de una web o una petición inicial).
2. **Validación:** El servidor valida quién eres.
3. **Entrega:** El servidor te devuelve un token (ej: `ghp_12345abcde`).
4. **Uso:** En cada extracción de datos posterior, incluyes ese token en los encabezados (headers) de tu petición.

Ejemplo real: API de Mapas o Seguros

Cuando usas la API de Google Maps para obtener coordenadas, no envías tu contraseña de Gmail. Envías una `API KEY`. Si esa llave se ve comprometida, puedes revocarla sin necesidad de cambiar la contraseña de toda tu cuenta de Google.

2. Gestión de variables de entorno con `.env`

Ahora que sabemos que necesitamos tokens, surge la pregunta crítica: **¿Dónde los guardamos?**

Un error de principiante muy común es el **"Hardcoding"**: escribir el token directamente en el código de Python.

CÓDIGO

```
<h1 class="module" data-eyebrow="MÓDULO">ERROR CRÍTICO: Hardcoding</h1>
<hr class="module-underline"><hr class="module-hairline">
api_key = "12345-mi-super-secreto"
<h1 class="module" data-eyebrow="MÓDULO">Si subes esto a Git, el mundo entero tendrá tu secre
<hr class="module-underline"><hr class="module-hairline">
```

La solución: El archivo `.env`

Las variables de entorno son valores definidos fuera del código, en el sistema operativo o en archivos de configuración especiales. El estándar de la industria es utilizar un archivo llamado `.env` en la raíz de tu proyecto.

Anatomía de un archivo `.env`:

CÓDIGO

```
<h1 class="module" data-eyebrow="MÓDULO">.env - Archivo de configuración secreta</h1>
<hr class="module-underline"><hr class="module-hairline">
API_TOKEN_ESTADISTICAS=sk_live_51MvB8xL...
DB_PASSWORD=admin123
DEBUG_MODE=True
```

- **Regla de oro:** No uses espacios alrededor del signo igual (`=`).
- **Convención:** Usa mayúsculas y guiones bajos (`SNAKE_CASE`) para los nombres.

Implementación en Python con `python-dotenv`

Para que Python pueda "leer" este archivo, utilizamos la librería `python-dotenv`. Primero, debemos instalarla (`pip install python-dotenv`), y luego la usamos así:

CÓDIGO

```
import os
from dotenv import load_dotenv

<h1 class="module" data-eyebrow="MÓDULO">1. Cargar las variables del archivo .env al entorno
<hr class="module-underline"><hr class="module-hairline">
load_dotenv()

<h1 class="module" data-eyebrow="MÓDULO">2. Acceder a la variable de forma segura</h1>
<hr class="module-underline"><hr class="module-hairline">
token = os.getenv("API_TOKEN_ESTADISTICAS")

if not token:
 print("Error: No se encontró el token de la API.")
else:
 print("Token cargado correctamente.")
```

¿Por qué es mejor así? Porque tu código Python ahora pide el token al sistema en lugar de tenerlo escrito. Si cambias de token, solo editas el archivo `.env`, no tocas ni una línea de código.

3. Buenas prácticas para secretos y configuraciones

Manejar datos implica una gran responsabilidad ética y técnica. Aquí te presento las mejores prácticas para que tus proyectos de Data Science sigan estándares profesionales:

1. El `.gitignore`: Tu mejor amigo

Para evitar que el archivo `.env` termine en internet, debes crear un archivo llamado `.gitignore` en la carpeta de tu proyecto. Este archivo le dice a Git: "Por favor, ignora estos archivos, no los subas al servidor".

Dentro de `.gitignore`, simplemente escribe:

CÓDIGO

```
.env
__pycache__/
*.pyc
```

2. El archivo `.env.example`: La guía para otros

Si ocultas el `.env`, ¿cómo sabrá otro analista (o tú mismo en el futuro) qué variables necesita el script para funcionar? Crea un archivo llamado `.env.example` con los nombres de las variables pero con valores vacíos o ficticios:

CÓDIGO

```
<h1 class="module" data-eyebrow="MÓDULO">.env.example - Plantilla de configuración</h1>
<hr class="module-underline"><hr class="module-hairline">
API_TOKEN_ESTADISTICAS=tu_token_aqui
DB_PASSWORD=contraseña_ejemplo
```

3. El encabezado `Authorization`

Cuando realices la petición con la librería `requests`, la forma estándar y segura de enviar el token es a través de los headers, usualmente con el prefijo `Bearer` (que significa "portador").

CÓDIGO

```
import requests
import os
from dotenv import load_dotenv

load_dotenv()
api_token = os.getenv("API_TOKEN_ESTADISTICAS")

<h1 class="module" data-eyebrow="MÓDULO">Configuración del encabezado</h1>
<hr class="module-underline"><hr class="module-hairline">
headers = {
 "Authorization": f"Bearer {api_token}",
 "Accept": "application/json"
}

response = requests.get("https://api.ejemplo.com/v1/data", headers=headers)
```

Diferencias entre tipos de Tokens: ¿Qué es JWT?

A medida que avances, te encontrarás con los **JSON Web Tokens (JWT)**. A diferencia de una API Key simple, un JWT lleva información consigo.

- **Header:** Dice qué tipo de token es.

- **Payload:** Contiene datos (como tu ID de usuario o si eres "Admin"). **Ojo:** El payload es legible por cualquiera que tenga el token, por lo que **nunca** debe contener contraseñas.
- **Signature:** Es la firma digital que garantiza que el token no ha sido alterado.

En el flujo de Machine Learning, los JWT son muy comunes cuando consumes modelos de lenguaje (LLMs) o servicios de nube tipo Google Cloud o AWS, donde el acceso es altamente controlado y temporal.

Errores comunes y cómo evitarlos

1. **Olvidar el `.gitignore`:** Es el error número uno. Si subes el `.env` por accidente, aunque lo borres después, quedará en el historial de Git. Tendrás que borrar el historial o, más fácil, invalidar el token inmediatamente.
2. **Confundir local con producción:** A veces, en tu computadora usas `DB_HOST=localhost`, pero en el servidor de la empresa será una dirección IP. Usar variables de entorno permite que el mismo código funcione en ambos lugares simplemente cambiando el archivo `.env`.
3. **Tipos de datos:** Recuerda que `os.getenv()` siempre devuelve un **string**. Si guardaste `PORT=5432` en el `.env`, en Python debes convertirlo: `port = int(os.getenv("PORT"))`.

Síntesis y Conexión

En esta unidad hemos pasado de un manejo de datos inseguro a uno profesional:

- Entendimos que la **autenticación** verifica quién eres y el **token** actúa como tu llave de acceso.
- Aprendimos que los secretos **jamás** van en el código, sino en un archivo `.env`.
- Descubrimos que el `.gitignore` es esencial para no exponer credenciales al compartir nuestro trabajo.

Esta base es fundamental para lo que sigue. En la próxima unidad, aprenderás a **normalizar los datos** que recibes tras autenticarte. De nada sirve tener la llave del cofre si no sabes cómo organizar el tesoro que hay dentro. ¡Prepárate para transformar estructuras JSON complejas en DataFrames listos para el entrenamiento de modelos de ML!

Peticiones HTTP con Requests en Python

En el mundo del Data Science, no siempre recibiremos un archivo `.csv` o `.json` listo para usar. Muchas veces, los datos "viven" en servidores externos y debemos ir a buscarlos. Aquí es donde entra el protocolo **HTTP** (HyperText Transfer Protocol), el lenguaje que utiliza internet para intercambiar información.

Para interactuar con este protocolo en Python, la herramienta estándar de la industria es la librería **Requests**.

1. Fundamentos de HTTP: El diálogo Cliente-Servidor

Imagina que entras a un restaurante (el **Cliente**). Miras el menú y le pides al camarero (el **Servidor**) un plato específico. El camarero va a la cocina y regresa con tu comida o con una explicación de por qué no pudo traértela.

En la web: - **Petición (Request):** Es lo que tu script de Python le envía a una URL. El método más común para Data Science es `GET`, que significa "tráeme estos datos". - **Respuesta (Response):** Es lo que el servidor te devuelve. Contiene los datos (generalmente en formato JSON) y metadata sobre la operación.

Códigos de estado: ¿Qué pasó con mi pedido?

Todo servidor responde con un código numérico que indica el resultado: - **200 (OK):** Todo salió bien. Aquí están tus datos. - **400 (Bad Request):** Tu petición tiene un error (faltan parámetros, sintaxis incorrecta). - **401/403 (Unauthorized):** No tienes permiso para ver esto (falta un token o llave). - **404 (Not Found):** Lo que buscas no existe en esa dirección. - **500 (Internal Server Error):** El problema es del servidor, no tuyo.

2. La librería `Requests`

Aunque Python trae una librería interna (`urllib`), `requests` es la preferida por ser increíblemente intuitiva y legible ("HTTP para humanos").

Instalación

CÓDIGO

```
pip install requests
```

Anatomía de una petición básica

Para obtener datos, simplemente usamos `requests.get()`:

CÓDIGO

```
import requests

response = requests.get("https://api.ejemplo.com/datos")

<h1 class="module" data-eyebrow="MÓDULO">¿Fue exitoso?</h1>
<hr class="module-underline"><hr class="module-hairline">
print(response.status_code) # Ejemplo: 200

<h1 class="module" data-eyebrow="MÓDULO">¿Qué me devolvió?</h1>
<hr class="module-underline"><hr class="module-hairline">
data = response.json() # Convierte el JSON recibido en un diccionario de Python
```

3. Accediendo a la Respuesta

Un error común es intentar leer la respuesta como si fuera una variable mágica. Debemos especificar qué parte de la respuesta queremos:

1. **`response.status_code`**: Indispensable para validar si el flujo debe continuar.
2. **`response.headers`**: Un diccionario con información técnica (fecha, tipo de contenido, etc.).
3. **`response.json()`**: El método más importante para nosotros. Transforma el cuerpo de la respuesta en tipos de datos de Python (listas y diccionarios).

> **Aviso**
>
> **Ojo con `response.text` vs `response.json()`.** `text` te devuelve una cadena gigante de caracteres (string). `json()` procesa esa cadena y la convierte en una estructura con la que puedes trabajar inmediatamente en tu flujo de ML.

Ejemplo del mundo real: Crypto Data

Supongamos que quieres analizar el precio de Bitcoin. Usando la API de CoinGecko, podrías hacer lo siguiente:

CÓDIGO

```
import requests

url = "https://api.coingecko.com/api/v3/simple/price?ids=bitcoin&vs_currencies=usd"
response = requests.get(url)

if response.status_code == 200:
 precio_btc = response.json()['bitcoin']['usd']
 print(f"El precio actual de BTC es ${precio_btc}")
```

Este pequeño bloque de código es el primer paso de cualquier **pipeline de datos** moderno. En los siguientes pasos, aprenderemos a manejar casos donde las APIs requieren llaves secretas o cuando el volumen de datos es mucho mayor.

Resiliencia: manejo de errores, rate limits y reintentos

Resiliencia en el consumo de APIs REST

Imagina que tu aplicación depende de datos en tiempo real de una API externa, pero de repente comienzas a recibir errores o respuestas lentas. ¿Cómo puedes garantizar que tu flujo de datos sea robusto y no se interrumpa? En este módulo, abordaremos cómo diseñar sistemas resilientes que manejen errores, respeten límites de uso (rate limits) y apliquen reintentos inteligentes para mantener la continuidad de tus procesos.

1. ¿Qué es la resiliencia en APIs?

La *resiliencia* se refiere a la capacidad de un sistema para recuperarse de fallos y continuar operando. En el contexto de APIs REST, implica manejar adecuadamente errores, límites de uso y condiciones inesperadas sin perder datos ni interrumpir el flujo.

2. Rate Limits: Identificación y manejo

Las APIs suelen imponer **rate limits** para controlar la cantidad de solicitudes que un cliente puede hacer en un período determinado. Esto protege la infraestructura y asegura un servicio estable para todos.

¿Cómo identificar rate limits?

Las respuestas HTTP incluyen encabezados que indican el estado del límite, por ejemplo:

Header	Descripción
`X-RateLimit-Limit`	Número máximo de solicitudes permitidas
`X-RateLimit-Remaining`	Solicitudes restantes en el período actual
`X-RateLimit-Reset`	Tiempo en segundos hasta que se reinicia el límite

> **Nota**
> No todas las APIs usan los mismos nombres de encabezados; siempre revisa la documentación específica.

Estrategias para respetar rate limits

- **Monitorear los encabezados** para anticipar el agotamiento del límite.
- **Implementar backoff exponencial** para espaciar reintentos cuando se alcanza el límite.
- **Circuit breakers** para pausar temporalmente las solicitudes y evitar bloqueos prolongados.

3. Backoff Exponencial y Reintentos

Cuando una solicitud falla por razones temporales (como un rate limit o error 503), es recomendable reintentar con pausas crecientes para no saturar el servidor.

¿Qué es backoff exponencial?

Es una técnica que aumenta el tiempo de espera entre reintentos de forma exponencial, por ejemplo:

CÓDIGO

```
import time

def backoff(attempt):
 return min(2 ** attempt, 60) # máximo 60 segundos

for attempt in range(5):
 try:
 # llamada a API
 break
 except TemporaryError:
 wait = backoff(attempt)
 print(f'Reintentando en {wait} segundos...')
 time.sleep(wait)
```

Circuit Breaker

Un patrón que detiene temporalmente las solicitudes tras múltiples fallos consecutivos para evitar sobrecarga y permite recuperación gradual.

4. Manejo y clasificación de errores HTTP

Los códigos de estado HTTP indican el resultado de una solicitud:

Código	Tipo de error	Acción recomendada
4xx	Error del cliente	Revisar solicitud, no reintentar
429	Too Many Requests	Aplicar backoff y reintentar
5xx	Error del servidor	Reintentar con backoff

> **Importante**
> No todos los errores son recuperables. Por ejemplo, un 404 indica recurso no encontrado y no debe reintentarse.

5. Registro y monitoreo de fallos

Registrar errores y eventos de reintentos es clave para:

- Diagnosticar problemas
- Ajustar estrategias de reintentos
- Evitar pérdida de datos

Ejemplo básico de logging en Python:

CÓDIGO

```
import logging

logging.basicConfig(level=logging.INFO)

try:
 response = requests.get(url)
 response.raise_for_status()
except requests.exceptions.HTTPError as e:
 logging.error(f'Error HTTP: {e}')
```

6. Buenas prácticas para evitar pérdida de datos

- Validar respuestas antes de procesar
- Guardar estados intermedios para reanudar procesos
- Documentar y manejar tokens y secretos de forma segura

Resumen

- Las APIs imponen límites que deben respetarse para evitar bloqueos.
- El backoff exponencial es una técnica efectiva para manejar reintentos.
- Clasificar errores HTTP permite decidir cuándo reintentar o abortar.
- Registrar eventos y errores es fundamental para la resiliencia.

En la siguiente unidad, aplicaremos estos conceptos para integrar datos JSON en Pandas y preparar flujos de datos robustos.

Contexto

En entornos reales de Ciencia de Datos, los pipelines de adquisición de datos consumen APIs que pueden ser inestables o tener restricciones de uso. Por ejemplo, una API pública puede limitar a 1000 solicitudes por hora y devolver errores 429 si se excede. Sin una estrategia de manejo de errores y rate limits, un proceso automático puede fallar silenciosamente o perder datos críticos.

Este conocimiento es esencial para diseñar flujos de datos que funcionen de forma confiable en producción, minimizando interrupciones y maximizando la calidad de los datos obtenidos. Además, entender cómo interpretar los encabezados de rate limit y aplicar backoff exponencial permite optimizar el uso de recursos y respetar las políticas de los proveedores de datos.

Estructuras JSON: mapas, listas y patrones anidados

Estructuras JSON y Datos Semiestructurados

En el mundo de la Ciencia de Datos, los datos semiestructurados como JSON son omnipresentes, pero su naturaleza anidada puede dificultar su manipulación directa con herramientas tabulares como Pandas.

En esta unidad, exploraremos las estructuras más comunes de JSON — objetos, arreglos y patrones anidados — y aprenderemos a identificar cómo mapear estas estructuras a tablas relacionales.

Este conocimiento es fundamental para preparar datos que luego podrás limpiar, enriquecer y analizar con eficiencia.

JSON (*JavaScript Object Notation*) es un formato ligero para el intercambio de datos, ampliamente usado en APIs y almacenamiento. Su flexibilidad permite representar datos complejos mediante:

- **Objetos**: Colecciones de pares clave-valor, similares a diccionarios en Python.
- **Arreglos (listas)**: Listas ordenadas de elementos, que pueden ser valores simples o estructuras anidadas.

Patrones Comunes en JSON

1. Objetos Simples

CÓDIGO

```
{
 "nombre": "Ana",
 "edad": 30,
 "ciudad": "Madrid"
}
```

Representa un registro plano, fácil de convertir a una fila en una tabla.

2. Arreglos de Objetos

CÓDIGO

```
{
 "empleados": [
 {"nombre": "Ana", "edad": 30},
 {"nombre": "Luis", "edad": 25}
 ]
}
```

Aquí, el arreglo contiene múltiples objetos, que pueden mapearse a múltiples filas.

3. Objetos Anidados

CÓDIGO

```
{
 "cliente": {
 "nombre": "Ana",
 "contacto": {
 "email": "ana@example.com",
 "tel": "123456789"
 }
 }
}
```

Los objetos anidados requieren aplanamiento para análisis tabular.

4. Arreglos Anidados

CÓDIGO

```
{
 "orden": {
 "id": 123,
 "productos": [
 {"producto": "Libro", "cantidad": 2},
 {"producto": "Lápiz", "cantidad": 5}
 ]
 }
}
```

Combinan objetos y listas anidadas, aumentando la complejidad.

Claves Opcionales y Valores Faltantes

En JSON, no todos los objetos deben tener las mismas claves. Por ejemplo:

CÓDIGO

```
{
 "nombre": "Ana",
 "edad": 30
}
```

vs.

CÓDIGO

```
{
 "nombre": "Luis"
}
```

Al convertir a tablas, las claves faltantes se representan como valores nulos (`NaN` en Pandas).

Estrategias para Mapear JSON a Tablas Relacionales

- **Aplanar objetos anidados**: Convertir claves anidadas en columnas con nombres compuestos, por ejemplo, `contacto.email`.
- **Expandir arreglos**: Cada elemento del arreglo se convierte en una fila separada, replicando los datos del nivel superior.
- **Normalizar datos**: Crear múltiples tablas relacionadas para evitar redundancia, similar a bases de datos relacionales.

> **Nota**
> Estas estrategias preparan el terreno para el uso de `pandas.json_normalize`, que veremos en la siguiente unidad para automatizar este proceso.

Conocimiento clave

- JSON es flexible y puede contener estructuras anidadas complejas.
- Identificar patrones es esencial para convertir JSON en tablas analíticas.
- Claves opcionales generan valores faltantes que deben manejarse adecuadamente.

Contexto

En la práctica, los datos JSON provienen de APIs REST, archivos de configuración o bases de datos NoSQL. Por ejemplo, una API de e-commerce puede devolver pedidos con productos anidados en arreglos, y cada producto tiene atributos propios.

Para analizar ventas, es necesario transformar estos datos en tablas donde cada fila represente un producto vendido, con columnas para atributos del pedido y del producto. Esto facilita cálculos agregados, filtrados y visualizaciones.

Además, entender cómo manejar claves opcionales es crucial para evitar errores en análisis y para preparar datos limpios que alimenten modelos predictivos o dashboards.

De JSON a tablas: estrategias y patrones

En este módulo, descubrirás cómo las APIs REST facilitan el acceso a datos estructurados en formato JSON, y cómo reconocer patrones comunes en estos datos para convertirlos eficientemente en DataFrames con Python y Pandas.

Este conocimiento es clave para manejar datos financieros reales y preparar flujos de trabajo robustos de ciencia de datos.

Como ya hemos visto, una **API REST** (Representational State Transfer) es un conjunto de reglas que permite a aplicaciones comunicarse a través de la web. Los datos se exponen como **recursos** accesibles mediante **endpoints** (URLs específicas). Las operaciones sobre estos recursos se realizan usando métodos HTTP.

Métodos y códigos HTTP

- **GET**: Solicita datos de un recurso.
- **POST**: Envía datos para crear un recurso.
- **PUT**: Actualiza un recurso existente.
- **DELETE**: Elimina un recurso.

Los servidores responden con códigos de estado que indican el resultado:

Código	Significado
200	OK: solicitud exitosa
201	Creado: recurso creado
400	Solicitud incorrecta
401	No autorizado
429	Demasiadas solicitudes
500	Error interno del servidor

Estructura JSON y patrones comunes

El formato **JSON** (JavaScript Object Notation) es un estándar para representar datos estructurados. Sus elementos básicos son:

- **Objetos**: pares clave-valor encerrados en `{}`.
- **Arreglos**: listas ordenadas encerradas en `[]`.
- **Tipos primitivos**: cadenas, números, booleanos, nulos.

En datos financieros, es común encontrar:

- **Listas de registros**: un arreglo donde cada elemento es un objeto con campos homogéneos, ideal para convertir directamente en un DataFrame.
- **Objetos anidados**: campos que contienen otros objetos o listas, que requieren normalización para aplanar la estructura.

Ejemplo

CÓDIGO

```
{
"cliente": "Juan Pérez",
"transacciones": [
{"fecha": "2023-01-01", "monto": 100},
{"fecha": "2023-01-02", "monto": 150}
]
}
```

Introducción a las etapas de Data Wrangling

El **Data Wrangling** es el proceso de preparar datos para análisis. Sus etapas iniciales son:

- **Descubrimiento**: entender la estructura y contenido de los datos.
- **Estructuración**: transformar datos en formatos tabulares, como DataFrames.

Estas etapas son esenciales para manejar JSON complejos y convertirlos en tablas limpias y analizables.

Contexto

En el contexto fintech, las APIs REST son la puerta de entrada para obtener datos financieros en tiempo real o históricos.

Por ejemplo, una API puede entregar información de transacciones bancarias, cotizaciones bursátiles o datos de clientes en formato JSON. Sin embargo, estos datos suelen tener estructuras anidadas o listas de registros que no se pueden analizar directamente.

Comprender los patrones comunes en JSON y cómo planificar su transformación es fundamental para construir pipelines de datos eficientes. Este conocimiento permite anticipar desafíos y aplicar técnicas adecuadas para normalizar y estructurar los datos, facilitando su análisis posterior con Pandas.

Buenas prácticas: reproducibilidad, documentación y control de versiones

¿Alguna vez has compartido un notebook que funcionaba perfectamente en tu máquina, pero al abrirlo en otro entorno no se ejecutaba igual? Este es un problema común en Ciencia de Datos que afecta la reproducibilidad y la colaboración.

En esta unidad, abordaremos cómo estructurar y documentar tus notebooks y pipelines para que sean reproducibles y fáciles de mantener, además de cómo gestionar de forma segura los secretos y usar control de versiones con Git.

Estas habilidades son esenciales para llevar tus proyectos de datos a producción ligera y colaborar eficazmente en equipos.

Reproducibilidad y Documentación en Notebooks

La **reproducibilidad** es la capacidad de que un análisis o pipeline pueda ser ejecutado nuevamente con los mismos resultados, independientemente del entorno o momento. En notebooks (Jupyter o Colab), esto implica:

- **Modularizar** el código: dividir en celdas lógicas y reutilizables.
- **Parametrizar** ejecuciones: usar variables o argumentos para controlar comportamientos sin modificar el código base.
- **Celdas deterministas**: evitar código que dependa de estados externos o aleatorios sin control.
- **Exportar resultados**: guardar salidas importantes para referencia o uso posterior.

Modularización y Parametrización

Modularizar notebooks mejora la legibilidad y facilita la depuración. Por ejemplo, separar la carga de datos, limpieza, análisis y visualización en secciones claras.

La parametrización permite ejecutar el mismo notebook con diferentes entradas sin cambiar el código, usando variables definidas al inicio o herramientas como `papermill`.

Documentación Efectiva

Un buen proyecto incluye un archivo `README.md` que explique:

- Objetivo del proyecto
- Requisitos y dependencias
- Instrucciones para ejecutar notebooks o scripts
- Estructura de carpetas y archivos

Además, usar **tags** en notebooks ayuda a identificar secciones importantes o para exportar contenido específico.

Control de Versiones con Git

Git es fundamental para:

- Registrar cambios y evolución del código
- Facilitar colaboración y revisión
- Mantener historial y revertir errores

Buenas prácticas incluyen:

- Commits frecuentes y descriptivos
- Uso de ramas para desarrollo y producción

- Archivo `.gitignore` para excluir archivos sensibles o temporales

Gestión de Secretos y Seguridad

Nunca debes incluir secretos (como contraseñas, tokens o claves API) directamente en el código o notebooks. En su lugar:

- Usa **variables de entorno** para cargar secretos en tiempo de ejecución.
- Configura `.gitignore` para evitar subir archivos con información sensible.
- Considera herramientas de gestión de secretos para entornos más complejos.

> **Nota**
> La seguridad en pipelines es crítica para evitar fugas de datos y accesos no autorizados.

Resumen de conceptos clave:

Concepto	Descripción breve
Modularización	Dividir código en partes lógicas y reutilizables
Parametrización	Controlar ejecución mediante variables externas
Documentación (README, tags)	Explicar y organizar el proyecto para usuarios
Control de versiones (Git)	Gestionar cambios y colaboración
Gestión de secretos	Evitar hardcodear información sensible

Contexto

En entornos profesionales de Ciencia de Datos, los proyectos suelen involucrar múltiples colaboradores y despliegues en diferentes entornos. Sin reproducibilidad y documentación claras, el mantenimiento y escalabilidad se vuelven inviables.

Por ejemplo, un equipo que desarrolla un pipeline para ingestión y transformación de datos debe asegurarse que cualquier miembro pueda ejecutar el pipeline sin errores, entender su funcionamiento y actualizarlo sin romper funcionalidades previas.

Además, la gestión segura de secretos es fundamental para proteger accesos a APIs, bases de datos y servicios externos, evitando riesgos de seguridad y cumplimiento.

El uso de Git permite mantener un historial claro, facilitar revisiones de código y coordinar el trabajo en equipo, aspectos críticos en proyectos reales.

Normalización de JSON para análisis de datos

En el mundo real, los datos rara vez vienen en tablas perfectas. Cuando consumimos información de APIs (como las de Spotify, Twitter o sistemas de clima), recibimos un formato llamado **JSON** (*JavaScript Object Notation*).

Aunque JSON es excelente para transmitir datos, tiene una característica que lo hace difícil de procesar directamente en Machine Learning: la **anidación**. Un registro de un usuario puede contener una lista de compras, y cada compra una lista de productos. Para entrenar un modelo, necesitamos que esos datos sean **planos** (filas y columnas).

El desafío de la estructura jerárquica

Imagina que recibes esto de una API:

CÓDIGO

```
{
 "usuario": "Juan",
 "pedidos": [
 {"id": 101, "total": 50},
 {"id": 102, "total": 30}
 ]
}
```

Si intentas cargarlo directamente en un DataFrame de Pandas, la columna `pedidos` contendría una lista de diccionarios, lo cual es ineficiente para el cálculo estadístico. El proceso de transformar esta "sopa" en una tabla donde cada pedido sea una fila propia se llama **Normalización** o **Flattening**.

Herramienta clave: `pd.json_normalize`

Pandas ofrece una función extremadamente poderosa para esto. Sus parámetros principales son: 1. **`record_path`** : La ruta hacia la lista que quieres convertir en filas individuales. 2. **`meta`** : Los campos de niveles superiores que quieres conservar para dar contexto a esas filas (como el ID del cliente).

Errores comunes al normalizar

- **Datos faltantes**: Si una API devuelve una lista vacía para un usuario, `json_normalize` puede omitirlo o generar nulos. Siempre verifica la integridad después de aplanar.
- **Claves inconsistentes**: A veces el primer registro tiene el campo `precio` y el segundo no. La normalización creará la columna pero la llenará con `NaN` (nulo) donde falte.
- **Confundir tipos**: JSON guarda números como strings en ocasiones. No olvides convertir tipos (`astype`) tras normalizar.

En esta unidad, aprenderás a tomar una respuesta de API compleja y dejarla lista para tu primer modelo de Machine Learning.

Estrategias de reintento y manejo de errores

En el mundo real del Data Science, el flujo de datos rara vez es perfecto. Cuando trabajamos con APIs externas, nos enfrentamos a redes inestables, servidores saturados y límites de velocidad que pueden hacer que nuestro script de ingesta falle en cualquier momento.

Si tu script de Machine Learning depende de una conexión que se cae durante un segundo, no queremos que todo el proceso se detenga. Para construir arquitectura de datos robusta, debemos distinguir qué fallos son "pasajeros" y cuáles requieren nuestra intervención manual.

Esta unidad se enfoca en las estrategias para que tus scripts sean resilientes:

1. **Reconocer el error**: No todos los errores se solucionan reintentando.
2. **Gestionar la espera**: Cómo esperar de forma inteligente sin saturar al servidor (Backoff exponencial).
3. **Establecer límites**: Saber cuándo rendirse para no entrar en bucles infinitos.

Consejo para usar estas tarjetas: Lee la situación o concepto en el frente, intenta decidir la acción correcta (¿reintentar o corregir?), y luego voltea la tarjeta para verificar la lógica técnica detrás de la decisión.

01 ¿Qué es un error transitorio en el contexto de APIs?

Es un error **temporal** que suele resolverse solo tras un breve periodo de tiempo. Ejemplos: una caída momentánea de internet, un servidor sobrecargado (**503**) o alcanzar el límite de peticiones (**429**). En estos casos, la estrategia correcta es **reintentar**.

02 ¿Qué es un error permanente?

Es un error que no se solucionará por sí solo y requiere un cambio en el código o configuración. Ejemplos: una URL mal escrita (**404**), credenciales inválidas (**401**) o datos mal formados. **Nunca se debe reintentar** automáticamente un error permanente.

03 ¿En qué consiste el Backoff Exponencial?

Es una estrategia donde el tiempo de espera entre intentos aumenta progresivamente (ej. 1s, 2s, 4s, 8s). Se usa para dar tiempo al servidor a recuperarse y evitar el fenómeno del "Retry Storm" (tormenta de reintentos) que podría tumbar el servicio definitivamente.

04 ¿Por qué es crítico configurar un Timeout en las peticiones HTTP?

Es el límite de tiempo que nuestro código espera una respuesta antes de "rendirse". Sin un timeout definido, tu script de ingesta podría quedarse bloqueado indefinidamente si el servidor no responde, deteniendo todo tu pipeline de ML.

05 Acción ante un error HTTP 429.

Significa **Too Many Requests**. El servidor te pide que dejes de enviar peticiones por un momento. La acción correcta es esperar (respetando el encabezado `Retry-After` si existe) y luego reintentar con un backoff más agresivo.

06 Acción ante un error HTTP 500.

Significa **Internal Server Error**. Es un error transitorio común. Debes **reintentar** un número limitado de veces (ej. 3 a 5 intentos) antes de marcar la tarea como fallida en tu flujo de datos.

07 Acción ante errores HTTP 401 o 403.

Significa **Unauthorized** o **Forbidden**. Son errores **permanentes**. Reintentar no servirá de nada; debes detener la ejecución y revisar que tus tokens o variables de entorno (.env) sean correctos.

08 ¿Cuál es el peligro de los reintentos ilimitados?

Las reintentos infinitos pueden causar un bucle que consuma recursos innecesarios. Se debe establecer un **máximo de intentos** y, si fallan, mover el registro a una "cola de errores" o generar una alerta para revisión manual.

09 ¿Qué es el Jitter en una estrategia de reintento?

Añadir un pequeño componente aleatorio al tiempo de espera del reintento. Esto evita que miles de instancias de un script que fallaron al mismo tiempo intenten reconectar exactamente en el mismo segundo, distribuyendo la carga del servidor.

10 ¿Qué es un Circuit Breaker (Disyuntor)?

Es un mecanismo que "abre el circuito" y detiene todas las peticiones a un servicio que está fallando constantemente. Esto protege a tu sistema de perder tiempo en peticiones que sabemos que van a fallar hasta que el servicio externo se estabilice.

PRE-ENTREGA DEL PROYECTO

Actividad evaluable · parte del proyecto integrador

Pre-entrega: Extracción y Almacenamiento Seguro de Datos vía API

Cómo resolver esta entrega

Propósito

El objetivo de esta actividad es que implementes un script funcional que interactúe con una API pública para extraer datos y los almacene en un archivo estructurado, aplicando buenas prácticas de seguridad (gestión de variables de entorno) y manejo de errores.

¿Qué debes construir?

Debes crear un repositorio de código que contenga un script en Python (u otro lenguaje de tu elección) capaz de realizar peticiones `GET` a una API (por ejemplo, PokeAPI, OpenWeather o JSONPlaceholder), procesar la respuesta y guardarla.

Pasos Sugeridos

1. **Inicialización**: Crea un repositorio en GitHub. Incluye un archivo `.gitignore` para evitar subir archivos sensibles.
2. **Configuración de Seguridad**: Utiliza un archivo `.env` para almacenar cualquier API Key o credencial necesaria. Usa una librería como `python-dotenv` para cargar estas variables en tu script.
3. **Extracción**: Implementa la lógica utilizando una librería de peticiones (como `requests`). Asegúrate de manejar posibles códigos de error HTTP (404, 500).
4. **Transformación Básica**: Convierte la respuesta JSON en una estructura de datos limpia (por ejemplo, eliminando campos innecesarios).
5. **Persistencia**: Exporta los datos finales a un archivo `data_extracted.csv` o `data_extracted.json` dentro del mismo repositorio.
6. **Documentación**: Redacta un archivo `README.md` explicando cómo instalar las dependencias y ejecutar el script.

Errores Comunes a Evitar

- **Hardcoding**: Nunca escribas tokens o llaves directamente en el código; usa variables de entorno.
- **Falta de Manejo de Excepciones**: Si la API está caída, tu script no debe romperse abruptamente; debe mostrar un mensaje claro.

Objetivos de aprendizaje

- Implementar el consumo de APIs externas mediante peticiones HTTP.
- Aplicar técnicas de seguridad mediante el uso de variables de entorno para proteger credenciales.

- Manipular datos en formato JSON para su transformación y almacenamiento local.
- Construir flujos de adquisición de datos robustos mediante el manejo de errores de conexión.

Criterios de aceptación

- El repositorio contiene un archivo README.md con instrucciones de configuración y ejecución.
- El script realiza exitosamente una petición a una API externa y recupera información.
- Se utiliza un mecanismo de variables de entorno (.env) para gestionar sensibilidades (si aplica) o configuración.
- Los datos obtenidos se guardan correctamente en un archivo estructurado (CSV o JSON) dentro del repositorio.
- El código incluye al menos un bloque de manejo de errores (try/except o validación de status code) para peticiones fallidas.

Rúbrica de evaluación: Extracción y almacenamiento seguro de datos vía API

Rúbrica para evaluar un repositorio de código que implementa la extracción de datos desde una API pública, su transformación básica y su almacenamiento en un archivo estructurado, aplicando buenas prácticas de seguridad, manejo de errores y documentación. Está diseñada para un nivel principiante, priorizando la funcionalidad real del proyecto, la seguridad en el manejo de credenciales y la claridad del repositorio como entregable profesional.

Criterio	Descripción	Peso
Funcionalidad de extracción desde la API	Evalúa si el script realiza correctamente peticiones GET a una API pública, obtiene datos válidos y ejecuta el flujo principal de forma coherente.	30%
Manejo de errores y robustez	Evalúa si el programa contempla errores frecuentes de red o de la API, como códigos HTTP 404/500, y si responde con mensajes claros sin romperse abruptamente.	18%
Seguridad y gestión de variables de entorno	Evalúa el uso correcto de un archivo .env, la carga de credenciales con una librería adecuada y la ausencia de secretos expuestos en el código o en el repositorio.	20%
Transformación y persistencia de datos	Evalúa si la respuesta de la API se transforma de forma básica y útil, y si el resultado se guarda correctamente en un archivo JSON o CSV dentro del repositorio.	20%
Estructura del repositorio y archivo .gitignore	Evalúa la organización general del repositorio como entregable técnico: presencia de archivos esperados, orden, nombres comprensibles y uso adecuado de .gitignore para un proyecto real.	10%
Documentación y reproducibilidad	Evalúa la calidad del README.md como guía para instalar dependencias, ejecutar el script y comprender el propósito del proyecto.	2%

Total: 100 pts | Aprobación: 70 pts

MÓDULO 6

Pipelines y Optimización de Datos

Diseñar pipelines de datos eficientes utilizando librerías de alto rendimiento.

Arquitectura de pipelines de datos y tipos de flujo

Imagina que eres el responsable de datos en una aplicación de entrega de comida a domicilio como Rappi o Uber Eats. Cada segundo, miles de usuarios abren la app, buscan "hamburguesas", agregan productos al carrito y realizan pagos. Al mismo tiempo, los repartidores actualizan su ubicación GPS y los restaurantes confirman pedidos.

Si quisieras entrenar un modelo de Machine Learning que prediga el tiempo de entrega exacto para un usuario en particular, ¿cómo llevarías toda esa información dispersa (apps, GPS, bases de datos de restaurantes) hasta tu modelo de forma ordenada, limpia y constante?

Aquí es donde entra la **Arquitectura de Pipelines de Datos**. Un pipeline no es solo "mover datos"; es la columna vertebral que permite que la Ciencia de Datos pase de ser un experimento en una computadora local a un sistema que genera valor real en producción.

1. Arquitectura de Pipelines de Datos

Un **pipeline de datos** es un sistema de pasos automatizados que transporta datos desde diversas fuentes hacia un destino (un Data Warehouse, un Data Lake o un modelo de ML), transformándolos en el camino para que sean útiles.

En el Módulo 4 aprendiste a conectarte a una API y bajar un JSON. Eso fue una **ingesta unitaria**. Un pipeline es el proceso que toma ese código, lo conecta con otros procesos y asegura que se ejecute todos los días a las 3:00 AM sin errores.

Las 4 Capas de la "Cadena de Producción"

Para entender un pipeline, visualízalo como una fábrica con cuatro secciones claras:

1. **Ingesta (Data Ingestion):** Es la puerta de entrada. Aquí es donde "susurramos" a las fuentes (APIs, archivos CSV, bases de datos como SQLite o logs de servidores) para que nos den su información.
2. **Transformación (Processing):** Los datos rara vez vienen listos. En esta capa limpiamos nulos (visto en Módulo 0), normalizamos estructuras JSON (visto en Módulo 4), filtramos columnas innecesarias y unimos tablas.
3. **Almacenamiento / Carga (Storage/Loading):** Es donde guardamos el producto final. Puede ser un **Data Warehouse** (como BigQuery o Snowflake) si los datos están muy estructurados, o un **Data Lake** si guardamos archivos brutos para exploraciones futuras.
4. **Orquestación:** Es el director de la orquesta. Si el paso de Transformación falla, la Orquestación debe decidir: "¿Vuelvo a intentarlo?", "¿Envío un correo de alerta al Data Scientist?", "¿Detengo todo el proceso?".

> **Importante**
> Un error común es confundir un **Pipeline de Datos** con un **Pipeline de ML**. El pipeline de datos se encarga de que el agua llegue limpia al tanque (calidad y movimiento). El pipeline de ML toma esa agua para fabricar una bebida específica (entrenamiento y predicción del modelo).

2. ETL vs ELT: El Gran Debate Moderno

Históricamente, el estándar fue el **ETL**, pero con la llegada de la nube, el **ELT** ha ganado terreno. La diferencia parece una simple permutación de letras, pero cambia radicalmente quién hace el "trabajo pesado".

ETL (Extract, Transform, Load)

En este modelo, los datos se transforman **antes** de llegar al destino final.

- **Flujo:** Extraes de la fuente -> Los pasas por un servidor de procesamiento (usando Python, Polars o Spark) -> Los cargas limpios en el destino.
- **¿Cuándo usarlo?** Cuando los datos de origen tienen información sensible que no debe llegar al destino (ej. números de tarjetas de crédito que debes anonimizar antes de guardar) o cuando el destino es muy rígido. *

ELT (Extract, Load, Transform)

Aquí, los datos se cargan "crudos" o con una limpieza mínima en el destino, y se transforman **dentro** de la base de datos final.

- **Flujo:** Extraes -> Cargas en el Data Warehouse -> Usas el poder de procesamiento del Data Warehouse (SQL) para transformar.
- **¿Cuándo usarlo?** Es el estándar moderno en Big Data (utilizando herramientas como **DuckDB** o Snowflake). Es mucho más flexible: si te equivocas en una transformación, no tienes que volver a extraer los datos de la fuente, simplemente vuelves a ejecutar tu SQL en el destino.

Característica	ETL	ELT
Poder de cómputo	Se requiere un servidor externo.	Se usa el motor del destino.
Flexibilidad	Baja (si cambias la lógica, re-procesas todo).	Alta (los datos crudos ya están ahí).
Privacidad	Alta (puedes filtrar antes de cargar).	Media (los datos llegan al destino antes de limpiarse).

3. Batch, Streaming e Híbrido

No todos los datos necesitan la misma velocidad. Elegir el tipo de flujo incorrecto puede desperdiciar miles de dólares en servidores o hacer que tu modelo de ML llegue tarde a la acción.

A. Procesamiento por Lotes (Batch)

Es el método tradicional. Los datos se acumulan durante un tiempo y luego se procesan todos juntos en un "lote".

- **Ejemplo:** Un banco que procesa todos los estados de cuenta al final del mes, o un pipeline que actualiza las recomendaciones de productos de Amazon una vez al día.
- **Pros:** Eficiente para grandes volúmenes y fácil de gestionar.
- **Cons:** Los datos siempre tienen un retraso (latencia).

B. Procesamiento en Tiempo Real (Streaming)

Los datos se procesan en el instante en que se generan. No hay esperas.

- **Ejemplo:** Detección de fraudes con tarjetas de crédito. No sirve de nada que el modelo detecte el fraude 24 horas después; debe ocurrir en milisegundos mientras pasas la tarjeta.
- **Tecnologías:** Kafka, AWS Kinesis.
- **Pros:** Información inmediata.
- **Cons:** Mucho más complejo de implementar y costoso de mantener.

C. Arquitectura Híbrida (Lambda / Kappa)

Hoy en día, las grandes empresas usan ambos.

- **Caso de uso (Netflix):** Un flujo de **Streaming** para registrar qué estás viendo en este preciso segundo (para que si cierras la app y la abres en tu tablet, sepas por dónde ibas). Y un flujo **Batch** nocturno que analiza todo tu historial de meses para sugerirte qué ver el siguiente fin de semana.

4. Conceptos de Robustez: Evitando el Caos

Para que tu arquitectura sea profesional (y no solo un script que "corre en mi computadora"), debes considerar tres pilares:

Idempotencia

Un pipeline es **idempotente** si al ejecutarlo dos veces con los mismos datos, el resultado final es el mismo.

- **Mal diseño:** Si ejecutas tu script de ingesta dos veces y terminas con datos duplicados en tu base de datos SQLite.
- **Buen diseño:** El script verifica si el dato ya existe antes de insertarlo (o usa `UPSERT`), asegurando que la base de datos siempre sea consistente sin importar cuántas veces se caiga el internet y reinicies el proceso.

Schema Drift (Cambio de Esquema)

Imagina que la API que consumes (visto en Módulo 4) decide cambiar el nombre del campo `user_id` a `customer_id`. Tu pipeline se romperá inmediatamente.

- **Mejor práctica:** Implementar validaciones de esquema al inicio del pipeline para detener el proceso o alertar antes de que los datos corruptos lleguen a tu modelo de ML.

Manejo de Errores y "Backoff"

Como aprendiste en la unidad anterior sobre APIs, las conexiones fallan. Un pipeline robusto implementa **reintentos automáticos** (Retries) con **Backoff Exponencial** (esperar cada vez un poco más de tiempo antes de volver a intentar), evitando saturar el servidor de origen.

5. Ejemplo de la Industria: Airbnb

Airbnb utiliza una herramienta de orquestación llamada **Apache Airflow** para gestionar miles de pipelines.

1. **Ingesta:** Extraen logs de búsquedas del sitio web (¿Qué ciudades busca la gente?).
2. **Carga (ELT):** Cargan estos gigabytes de datos brutos en un Data Lake en AWS S3.
3. **Transformación:** Utilizan procesos programados para limpiar esos datos, identificar tendencias de precios y preparar los datasets.
4. **ML:** Finalmente, un modelo de Machine Learning toma esos datos procesados para sugerirle a un anfitrión cuánto debería cobrar por su habitación esta noche.

6. Síntesis y Conexión

En esta unidad hemos pasado de ver los datos como "un archivo CSV que abro en Pandas" a verlos como un **flujo continuo y arquitectónico**.

- Aprendimos que un **pipeline** es la automatización del ciclo de vida del dato.
- Vimos que **ELT** suele ser más flexible para proyectos modernos de Science Science que el tradicional **ETL**.
- Diferenciamos que el **Batch** es para análisis profundos y el **Streaming** para decisiones inmediatas.

En las siguientes unidades de este módulo, dejaremos de lado la teoría para meter las manos en la masa. Aprenderemos a usar **Polars** y **DuckDB**, herramientas diseñadas específicamente para que la fase de **Transformación** de estos pipelines sea increíblemente rápida y eficiente, permitiendo procesar millones de filas en segundos en una computadora común.

Pregunta para reflexionar: Si estuvieras diseñando un sistema para monitorear los signos vitales de un paciente en terapia intensiva, ¿elegirías una arquitectura Batch de cada 1 hora o una de Streaming? ¿Por qué?

Procesamiento vectorial para datos de ML

Imagina que tienes una hoja de cálculo con los precios de 1,000 casas. Si quieres aplicar un impuesto del 10% a cada una, podrías ir fila por fila calculando el nuevo valor. Esto es lo que llamamos **procesamiento escalar**: una operación a la vez. Ahora, imagina que pudieras chasquear los dedos y que el 10% se aplicara a todas las 1,000 filas en un solo instante. Eso, a grandes rasgos, es el **procesamiento vectorial**.

En el mundo del Machine Learning (ML), no trabajamos con datos aislados, sino con volúmenes masivos de información. Para que una inteligencia artificial aprenda, necesita realizar millones de cálculos matemáticos por segundo. El procesamiento vectorial es el "superpoder" que permite que estos cálculos ocurran a la velocidad necesaria para que tecnologías como el reconocimiento facial, los traductores automáticos o los coches autónomos sean posibles.

1. ¿Qué es el Procesamiento Vectorial?

El procesamiento vectorial es un paradigma de computación donde una única instrucción se aplica a un conjunto de datos (un vector) en lugar de a un solo número.

De lo Escalar a lo Vectorial

Para entender la diferencia, definamos ambos términos: * **Escalar:** Un solo valor numérico. Ej: `temperatura = 25`. * **Vector:** Una lista ordenada de números. Ej: `temperaturas_semana = [25, 22, 28, 24, 26, 23, 21]`.

En un **procesador escalar** (el CPU tradicional), si quieres sumar dos listas de números, el computador debe: 1. Tomar el primer número de A y el primero de B. 2. Sumarlos. 3. Guardar el resultado. 4. Repetir para el segundo, el tercero, y así sucesivamente.

En un **procesador vectorial** o utilizando técnicas de **vectorización de software** (como las que ofrecen NumPy o Polars), el computador le dice al hardware: "Toma todos estos números y súmalos con todos estos otros en una sola operación".

SIMD: El motor bajo el capó

Técnicamente, esto se conoce como **SIMD** (*Single Instruction*, *Multiple Data* o *Una Instrucción*, *Múltiples Datos*). Es una arquitectura que permite que la CPU o la GPU procesen múltiples elementos de datos simultáneamente con una sola instrucción de control. Esto reduce drásticamente el tiempo de ejecución y el consumo de energía.

2. Vectores y Operaciones Básicas

En Data Science, un vector no es más que una forma de organizar la información para que sea matemáticamente digerible. Cada "característica" de nuestro objeto de estudio se convierte en una dimensión del vector.

Representación de un Objeto como Vector

Supongamos que estamos analizando propiedades inmobiliarias. Cada casa puede representarse como un vector: `Casa_A = [Metros_Cuadrados, Habitaciones, Antigüedad]` `Casa_A = [120, 3, 15]`

Aquí, nuestra "Casa A" es un vector de 3 dimensiones en el espacio de los datos.

Operaciones Fundamentales

Para manipular estos datos en ML, realizamos tres operaciones clave:

A. Suma de Vectores

Si tienes un vector de "Ingresos" y otro de "Bonos", sumarlos vectorialmente te da el "Ingreso Total" de forma inmediata para todos tus usuarios. * **Regla:** Se suma elemento a elemento. Los vectores deben tener la misma longitud.

B. Multiplicación por Escalar

Imagina que quieres convertir todas tus medidas de metros a centímetros. Multiplicas el vector `[1.2, 5.0, 3.4]` por el escalar `100`. El resultado es `[120, 500, 340]`. En ML, esto se usa para **normalizar** o escalar datos (poner todos los números en un rango similar, como de 0 a 1).

C. Producto Punto (Dot Product)

Esta es la operación reina del Machine Learning. Consiste en multiplicar los elementos correspondientes de dos vectores y sumar los resultados. * **Uso real:** Es la base de las Redes Neuronales y de los sistemas de recomendación. Si un vector representa tus "intereses" y otro vector representa las "características de una película", el producto punto nos dice qué tan similares son. Un resultado alto significa: "¡Te va a encantar esta película!".

3. Vectorización de Datos para ML

¿Cómo pasamos de un texto de un tweet o una imagen de un gato a una lista de números? Este proceso se llama **vectorización**.

Vectorización de Texto (NLP)

Las computadoras no entienden el significado de las palabras, solo entienden números. * **Bag of Words (Bolsa de Palabras):** Creamos una lista de todas las palabras posibles y marcamos con un `1` si la palabra está presente y un `0` si no. * **Embeddings:** Es la técnica moderna (usada por ChatGPT o Google). Convierte palabras en vectores densos donde palabras con significados similares terminan "cerca" matemáticamente. Por ejemplo, los vectores de "Rey" y "Reina" estarán muy próximos en el espacio vectorial.

Vectorización de Imágenes (Computer Vision)

Una imagen digital es, en esencia, una matriz de números que representan la intensidad del color de cada píxel. Para procesarla en un pipeline de ML, esta matriz suele "aplanarse" para convertirse en un vector gigante de miles de números que representan cada detalle visual.

4. Aplicaciones en la Industria

¿Dónde se aplica esto en el mundo real?

1. **Google Search:** Cuando buscas algo, Google convierte tu consulta en un vector y busca en su base de datos qué documentos tienen los vectores más similares (usando algo llamado *Vector Search*).
2. **Netflix / Amazon:** Tus gustos son un vector. Cada película es otro vector. El sistema realiza millones de cálculos de "similitud de coseno" (una variante del producto punto) para decidir qué recomendarte hoy.
3. **Tesla Autopilot:** Las cámaras del coche vectorizan la carretera en tiempo real. Los procesadores vectoriales (GPUs) realizan multiplicaciones de matrices masivas para identificar si ese conjunto de píxeles es un peatón o un semáforo en milisegundos.

5. Errores Comunes y Mejores Prácticas

El error de la "falsedad escalar"

Un error muy común de principiante es usar bucles `for` en Python para procesar datos de una lista o un DataFrame de Pandas. * **Mal:** `for i in lista: nuevo_valor = i * 2` (Lento, procesamiento escalar). * **Bien:** `nuevo_vector = vector * 2` (Rápido, procesamiento vectorial usando librerías como NumPy o Polars).

¿Por qué importa? En un conjunto de datos de 10 millones de registros, el bucle `for` podría tardar minutos, mientras que la operación vectorizada tardaría milisegundos.

Confundir Vectores con Listas de Python

Aunque se ven iguales, una lista de Python no está diseñada para el procesamiento vectorial. Si intentas sumar dos listas `[1, 2] + [3, 4]`, Python las une: `[1, 2, 3, 4]`. En cambio, en una librería optimizada para ML (como NumPy o los tipos de datos en Polars), la suma resultaría en `[4, 6]`. Asegúrate siempre de usar las estructuras de datos correctas para el pipeline de ML.

6. Síntesis y Conexión

Hemos aprendido que el procesamiento vectorial es la capacidad de realizar operaciones masivas sobre conjuntos de datos simultáneamente. Los vectores son la "lengua franca" del Machine Learning: todo, desde un precio hasta un sentimiento en un texto, debe convertirse en un vector para ser procesado.

En las siguientes unidades, veremos cómo los asistentes de IA pueden ayudarnos a escribir estos flujos de código optimizados y cómo DuckDB y Polars utilizan estos principios de procesamiento vectorial (y algo llamado "procesamiento por columnas") para manejar gigabytes de datos en segundos. Dominar

la mentalidad vectorial es lo que separa a un analista de datos básico de un Arquitecto de Datos e Ingeniero de ML profesional.

Optimización en memoria con Polars

Al trabajar en Machine Learning, a menudo nos enfrentamos al "muro de la memoria": datasets que son demasiado grandes para ser procesados cómodamente por Pandas, pero no lo suficientemente grandes como para justificar un cluster de Spark. Aquí es donde entra **Polars**.

¿Por qué Polars es más eficiente?

A diferencia de Pandas, que procesa datos de forma "anciosa" (ejecuta cada comando inmediatamente y crea copias en memoria), Polars utiliza dos pilares tecnológicos:

1. **Motor en Rust y Apache Arrow**: Los datos se almacenan en un formato columnar optimizado que permite "Zero-Copy", minimizando el movimiento innecesario de bytes.
2. **Ejecución Diferida (Lazy Evaluation)**: Polars no actúa de inmediato. Primero construye un **Plan de Ejecución**. Si le pides filtrar filas y seleccionar columnas, Polars es lo suficientemente inteligente para leer *solo* esas columnas del disco y aplicar el filtro *mientras* lee, en lugar de cargar todo y filtrar después.

Técnicas Clave de Optimización

- **Selección de columnas (Projection Pushdown)**: Si tu CSV tiene 100 columnas pero solo usas 5 para tu modelo de ML, Polars solo procesará esas 5.
- **Tipos de datos correctos**: Por defecto, Python suele asignar `Int64` (8 bytes) a los números. Si tus datos caben en un `Int32` (4 bytes), ¡puedes reducir el uso de memoria a la mitad instantáneamente!
- **Categorical vs String**: Almacenar la palabra "Electrónicos" 1 millón de veces es ineficiente. El tipo `Categorical` almacena el texto una vez y usa números pequeños como referencia.

Ejemplo de comparación visual

Mientras Pandas carga un bloque masivo y luego lo recorta, Polars dibuja un "mapa" del camino más corto y solo recoge lo necesario.

Enunciado del Ejercicio

Trabajarás con un archivo llamado `datos_ingesta.csv` que simula registros de ventas. Tu misión es transformar este flujo "pesado" en uno optimizado para memoria siguiendo estos pasos:

1. **Lectura Lazy**: No cargarás el archivo a ciegas; crearás un `LazyFrame`.
2. **Selección inteligente**: Solo nos interesan las columnas de identificación, negocio y métricas.
3. **Casting de tipos**: Ajustarás los enteros a 32 bits y los strings repetitivos a categorías.
4. **Filtrado temprano**: Limpiarás los datos (montos mayores a cero) antes de que lleguen a la memoria final.

Dato importante: En este entorno, el archivo `datos_ingesta.csv` ya está disponible para tu código.

Errores comunes:

- Olvidar el método `.collect()`: Sin él, solo tendrás un objeto `LazyFrame` (un plan), no los datos reales.
- No usar `scan_csv`: Si usas `read_csv`, estarás cargando todo en memoria de forma ansiosa, perdiendo las ventajas de la optimización lazy.

Asistentes de IA para escribir y revisar código de datos

Asistentes de IA en el Flujo de Datos

En el mundo moderno del **Data Science**, no se trata de trabajar más duro, sino de trabajar de forma más inteligente. Los asistentes de IA (como GitHub Copilot, ChatGPT o Cursor) se han convertido en compañeros esenciales para los analistas de datos, permitiéndoles acelerar la escritura de código, depurar errores complejos y documentar procesos de manera automática.

Sin embargo, para un profesional de datos, la IA es una **herramienta**, no un reemplazo. Imagina que la IA es un copiloto: puede sugerir la ruta y manejar algunos controles, pero tú eres el capitán que conoce el destino final y debe validar que el avión no se desvíe del curso.

En esta unidad, repasaremos cómo utilizar estos asistentes para:

1. **Generar código base**: Desde consultas SQL complejas hasta scripts de limpieza en Pandas o Polars.
2. **Explicar lógica**: Entender fragmentos de código heredados o funciones matemáticas abstractas.
3. **Optimizar y depurar**: Encontrar ese error de sintaxis o esa ineficiencia en un pipeline que está ralentizando tu proceso.

¿Cómo usar estas tarjetas? Lee la pregunta o concepto en el frente, intenta formular una respuesta detallada en tu mente y luego voltea la tarjeta para verificar. Presta especial atención a la sección de "revisión humana", ya que es donde reside el verdadero valor de un experto en datos.

01 **¿Qué es un asistente de IA para código (AI Coding Assistant)?**

Es un modelo de lenguaje (como GPT-4) integrado en el flujo de trabajo del desarrollador para sugerir líneas de código, completar funciones o explicar lógica técnica en tiempo real. Ejemplos: **GitHub Copilot**, **Cursor** o **Tabnine**.

02 ¿Cuál es el elemento más importante en un prompt para generar código de datos preciso?

Proporcionar **contexto técnico específico**. En lugar de pedir "un join en SQL", es mejor indicar: "Crea una consulta SQLite para unir las tablas 'ventas' y 'productos' usando 'id_prod', filtrando por el año 2023".

03 ¿Qué significa el uso de la IA para la "deconstrucción de código"?

Pedirle a la IA que explique qué hace una línea de código específica. Esto es vital para aprender nuevas librerías (como **Polars**) y asegurar que el código generado no sea una "caja negra" para nosotros.

04 ¿Qué es una "alucinación" en el contexto de asistentes de IA?

Es la tendencia de la IA a generar información falsa o código sintácticamente correcto pero lógicamente erróneo. En datos, esto puede resultar en filtros mal aplicados o cálculos estadísticos que parecen correctos pero usan la fórmula equivocada.

05 ¿Qué es la "Revisión Humana" (Human-in-the-loop)?

Es el proceso obligatorio donde el analista lee, prueba y valida el código sugerido por la IA para detectar errores lógicos, de seguridad o ineficiencias antes de integrarlo al pipeline de producción.

06 ¿Cuáles son los 4 pasos clave para un "Súper Prompt" de manipulación de datos?

1. Define el rol (ej. "Actúa como experto en Pandas"). 2. Describe la estructura de entrada (columnas y tipos de datos). 3. Especifica la salida deseada (el resultado esperado). 4. Menciona restricciones (ej. "no uses bucles for, usa vectorización").

07 ¿Cómo se utiliza el "Natural Language to Code" en herramientas como Copilot?

El uso de **comentarios de código** como prompts. Escribir un comentario descriptivo en Python (ej. `# Función para normalizar precios entre 0 y 1`) permite que el asistente sugiera el código exacto debajo del comentario.

08 ¿Por qué la IA no puede reemplazar el conocimiento del dominio del negocio?

La IA no comprende la semántica del negocio. Puede unir tablas por una columna 'ID' que técnicamente encaja, pero que para el negocio no tienen relación (ej. ID de empleado con ID de producto). La validación lógica depende siempre del humano.

09 ¿Cuáles son los tres pilares al auditar código generado por IA?

1. **Lógica**: ¿El cálculo es correcto? 2. **Seguridad**: ¿Hay exposición de datos sensibles o riesgos de inyección SQL? 3. **Eficiencia**: ¿La solución escala para millones de registros o es lenta?

10 ¿Cómo ayuda la IA en el control de calidad (QA) de un pipeline?

Ideal para **unittesting** (pruebas unitarias). Puedes darle una función a la IA y pedirle: "Genera 5 casos de prueba para esta función, incluyendo valores nulos y casos borde".

PRE-ENTREGA DEL PROYECTO

Actividad evaluable · parte del proyecto integrador

Pre-entrega: Construcción de un Pipeline de Ingesta y Limpieza

El objetivo de esta actividad es diseñar e implementar un pipeline de datos que automatice el proceso de extracción, transformación y carga (ETL) utilizando librerías de alto rendimiento como Pandas o Dask. Este entregable te permitirá aplicar conceptos de modularización y eficiencia en el manejo de volúmenes de datos crecientes.

Cómo resolver esta entrega

Pasos a seguir:

1. **Inicialización**: Crea un repositorio público en GitHub con un archivo `main.py` para la ejecución del pipeline y una carpeta `src/` para los módulos de lógica.
2. **Extracción**: Implementa una función que lea un dataset (puedes usar un archivo CSV público de Kaggle o similar) asegurando el uso correcto de tipos de datos para optimizar la memoria.
3. **Transformación**: Crea un script de limpieza que elimine duplicados, gestione valores nulos de forma justificada y normalice nombres de columnas. Separa esta lógica en una función independiente dentro de tu módulo de transformaciones.
4. **Carga**: Simula la carga de datos procesados exportando el resultado a un archivo Parquet, justificando por qué este formato es más eficiente para Machine Learning.
5. **Documentación**: El archivo `README.md` debe explicar cómo ejecutar el código y describir cada etapa del pipeline desarrollado.

Evita errores comunes:

- No subas el dataset original al repositorio si es muy pesado; utiliza archivos `.gitignore`.
- Asegúrate de que el pipeline no sea un script monolítico; utiliza funciones y modularización.

Objetivos de aprendizaje

- Implementar un pipeline de ETL modular utilizando Python y librerías de procesamiento de datos.
- Optimizar el consumo de memoria mediante el uso de formatos de archivo eficientes como Parquet.
- Documentar la arquitectura y los pasos técnicos de procesamiento de datos en un README profesional.

Criterios de aceptación

- El repositorio debe contener al menos dos archivos .py (uno para ejecución y otro para lógica modular).
- El código debe incluir una etapa de transformación que maneje valores nulos y duplicados.
- El script debe generar un archivo de salida en formato Parquet.
- El archivo README debe incluir instrucciones claras sobre cómo instalar dependencias y ejecutar el pipeline.

Rúbrica de evaluación: Pipeline de ingesta y limpieza con ETL modular

Esta rúbrica evalúa un repositorio público de GitHub que implementa un pipeline de datos modular para extraer, transformar y cargar información usando herramientas como Pandas o Dask. Se valora la estructura real de proyecto, la eficiencia en el manejo de datos, la calidad de la limpieza, la exportación a Parquet y la documentación técnica necesaria para reproducir la solución. Está diseñada para orientar la mejora del trabajo antes de su entrega.

Criterio	Descripción	Peso
Estructura del repositorio y modularización	Evalúa si el proyecto está organizado como un repositorio de código real, con archivos y carpetas esperadas, separación clara de responsabilidades y ausencia de un script monolítico.	25%
Extracción de datos y manejo eficiente de tipos	Evalúa la función de lectura del dataset, la correcta carga del archivo y el uso de tipos de datos adecuados para optimizar memoria y rendimiento.	20%
Limpieza y transformación de datos	Evalúa la calidad de la función de limpieza: eliminación de duplicados, tratamiento justificado de valores nulos y normalización de nombres de columnas, todo ello con lógica separada y coherente.	25%
Carga del resultado y elección de formato	Evalúa si el pipeline exporta correctamente los datos procesados a Parquet y si la elección del formato está técnicamente justificada en relación con eficiencia y uso en Machine Learning.	15%
Documentación técnica y reproducibilidad	Evalúa la claridad del README y la capacidad del repositorio para que otra persona entienda cómo ejecutar el proyecto y qué hace cada etapa del pipeline.	10%
Buenas prácticas de repositorio y manejo de archivos	Evalúa si el repositorio sigue prácticas básicas de trabajo profesional: uso adecuado de `.gitignore`, no subir datasets pesados, y manejo responsable de archivos y artefactos del proyecto.	5%

Total: 100 pts | Aprobación: 70 pts

MÓDULO 7

SQL Básico y Motores Modernos

Utilizar DuckDB como motor de consulta analítica de alto desempeño.

Introducción a SQL para ciencia de datos

¿Sabías que más del 80% de los datos en las empresas están almacenados en bases de datos relacionales? Para un científico de datos, dominar SQL es fundamental para extraer y preparar datos de manera eficiente. En esta unidad, exploraremos las sentencias básicas de SQL que permiten seleccionar, filtrar y agrupar datos, además de entender cómo se clasifican las sentencias SQL en DDL, DML y DQL. Al finalizar, podrás explicar y aplicar consultas básicas que son la base para cualquier análisis de datos.

Este conocimiento es esencial para integrar SQL con Python, facilitando la manipulación de datos en entornos reales de trabajo, y sentará las bases para construir pipelines de datos robustos y reproducibles.

Fundamentos de SQL: Sintaxis y Clasificación

SQL (*Structured Query Language*) es el lenguaje estándar para interactuar con bases de datos relacionales. Se divide en tres categorías principales:

- **DDL (Data Definition Language):** Sentencias que definen la estructura de la base de datos, como `CREATE`, `ALTER` y `DROP`.
- **DML (Data Manipulation Language):** Sentencias que modifican los datos, como `INSERT`, `UPDATE` y `DELETE`.
- **DQL (Data Query Language):** Sentencias que consultan datos, principalmente `SELECT`.

Sentencia SELECT

La sentencia `SELECT` es la base para extraer datos:

CÓDIGO

```
SELECT columna1, columna2
FROM tabla;
```

Permite elegir columnas específicas de una tabla.

Filtrado con WHERE

Para filtrar filas según condiciones:

CÓDIGO

```
SELECT *
FROM tabla
WHERE columna1 = 'valor';
```

Se pueden usar operadores lógicos (`=` , `>` , `<` , `AND` , `OR`).

Agrupamiento con GROUP BY

Para agregar datos y obtener resúmenes:

CÓDIGO

```
SELECT columna1, COUNT(*)
FROM tabla
GROUP BY columna1;
```

Agrupa filas con valores iguales en `columna1` y aplica funciones agregadas como `COUNT()` , `SUM()` , `AVG()` .

> **Nota**
> Estas sentencias forman la base para consultas más complejas y son esenciales para la manipulación de datos en ciencia de datos.

Ejemplo Industrial

En análisis de ventas, podrías usar:

CÓDIGO

```
SELECT region, SUM(ventas) AS total_ventas
FROM ventas_mensuales
WHERE año = 2023
GROUP BY region;
```

Para obtener las ventas totales por región en 2023, facilitando la toma de decisiones estratégicas.

Conceptos Clave de SQL

Alguna vez te has preguntado cómo los científicos de datos y analistas gestionan grandes volúmenes de información estructurada para extraer insights valiosos?

La respuesta está en el dominio de SQL (Structured Query Language), el lenguaje estándar para interactuar con bases de datos relacionales.

Comprenderás también por qué SQL es un lenguaje declarativo y cómo esta característica facilita la expresión de consultas complejas sin preocuparte por los detalles de ejecución. Este conocimiento es fundamental para avanzar en la integración de SQL con Python y Pandas, herramientas clave en el ecosistema de ciencia de datos.

Fundamentos de SQL

SQL es un lenguaje declarativo diseñado para gestionar y consultar datos estructurados almacenados en bases de datos relacionales.

Su naturaleza declarativa significa que el usuario especifica *qué* datos desea obtener o modificar, sin detallar *cómo* el sistema debe realizar esas operaciones.

Sublenguajes de SQL

SQL se compone de varios sublenguajes, cada uno con funciones específicas:

Sublenguaje	Descripción	Ejemplos de Sentencias
DDL (Data Definition Language)	Define y modifica la estructura de la base de datos.	`CREATE`, `ALTER`, `DROP`
DML (Data Manipulation Language)	Manipula los datos dentro de las tablas.	`INSERT`, `UPDATE`, `DELETE`
DQL (Data Query Language)	Consulta y recupera datos.	`SELECT`
TCL (Transaction Control Language)	Controla las transacciones para asegurar integridad.	`COMMIT`, `ROLLBACK`
DCL (Data Control Language)	Gestiona permisos y seguridad.	`GRANT`, `REVOKE`

> **Nota**
> En esta unidad nos enfocaremos en DDL, DML y DQL, que son esenciales para la manipulación básica de datos.

Operaciones Básicas de SQL

A continuación, se presentan las sentencias más comunes y su sintaxis básica:

- **CREATE**: Crea una nueva tabla o estructura.

CÓDIGO

```
CREATE TABLE nombre_tabla (
 columna1 tipo_dato,
 columna2 tipo_dato,
 ...
);
```

- **INSERT**: Inserta nuevos registros en una tabla.

CÓDIGO

```
INSERT INTO nombre_tabla (columna1, columna2) VALUES (valor1, valor2);
```

- **SELECT**: Consulta datos de una o más tablas.

CÓDIGO

```
SELECT columna1, columna2 FROM nombre_tabla WHERE condición;
```

- **UPDATE**: Modifica registros existentes.

CÓDIGO

```
UPDATE nombre_tabla SET columna1 = valor_nuevo WHERE condición;
```

- **DELETE**: Elimina registros.

CÓDIGO

```
DELETE FROM nombre_tabla WHERE condición;
```

Ventajas del Enfoque Declarativo

- **Simplicidad:** El usuario se concentra en el resultado deseado, no en el procedimiento.
- **Optimización automática:** El motor de base de datos decide la mejor forma de ejecutar la consulta.
- **Portabilidad:** SQL es un estándar ampliamente soportado, facilitando la interoperabilidad.

Por ejemplo, al escribir una consulta `SELECT`, no necesitas especificar cómo recorrer índices o tablas, el motor optimiza internamente.

Conexión con conceptos previos

Esta unidad se apoya en la comprensión del proceso de adquisición de datos y los tipos de datos estructurados, semi-estructurados y no estructurados, que vimos anteriormente.

SQL es la herramienta clave para manipular datos estructurados, especialmente en bases relacionales, y será fundamental para integrar consultas dentro de flujos de trabajo en Python y Pandas, que abordaremos en la siguiente unidad.

Aplicación Práctica en Ciencia de Datos

En la práctica, estas sentencias permiten a los científicos y analistas de datos extraer subconjuntos relevantes de grandes bases de datos para análisis exploratorios, modelado y reportes. Por ejemplo, filtrar clientes activos, agrupar transacciones por categoría o seleccionar variables específicas para un modelo predictivo.

Además, entender la diferencia entre DDL, DML y DQL ayuda a mantener la integridad y seguridad de los datos, evitando modificaciones accidentales mientras se realizan consultas.

Este conocimiento es la base para integrar SQL con Python usando librerías como `pandas` y motores embebidos como `sqlite3` o `DuckDB`, que veremos en unidades posteriores para construir pipelines reproducibles y eficientes.

SQL aplicado al análisis exploratorio

Hasta ahora, has recorrido un camino fascinante: desde dominar las estructuras de Python y el manejo de APIs, hasta construir pipelines de datos y optimizar procesos con Polars. Has aprendido a extraer datos de la web y a transformarlos para que estén listos para el Machine Learning.

Sin embargo, en el mundo real de la Ciencia de Datos, los datos no siempre están en un archivo CSV limpio o en una API amigable. La gran mayoría de la información estructurada del planeta reside en **Bases de Datos Relacionales**. Como futuro experto en ML y arquitectura de datos, tu capacidad para "hablar" con estas bases de datos mediante **SQL (Structured Query Language)** es lo que determinará qué tan rápido puedes obtener los *insights* necesarios para alimentar tus modelos.

En esta unidad, no solo repasaremos la sintaxis; aprenderemos a ver SQL como la primera herramienta de **Análisis Exploratorio de Datos (EDA)**. Antes de cargar millones de filas en un DataFrame de Pandas o Polars, SQL nos permite interrogar a la base de datos de forma eficiente, extrayendo exactamente lo que necesitamos.

1. SQL aplicado al análisis de datos: El porqué antes del cómo

Imagina que trabajas como analista de datos en una plataforma de e-commerce global como **Mercado Libre**. Tienes acceso a una base de datos con miles de millones de transacciones. Si intentas descargar toda la tabla de ventas a tu computadora para analizarla con Python, lo más probable es que tu memoria RAM colapse o que la transferencia tarde horas.

Aquí es donde entra SQL como herramienta estratégica. En lugar de traer todos los datos hacia tu código, llevas tu "pregunta" (la consulta SQL) hacia los datos. El motor de la base de datos procesa la información y solo te devuelve el resultado final.

El rol de SQL en el flujo de Machine Learning

En un flujo profesional de ML, SQL cumple funciones críticas:

1. **Ingeniería de Características (Feature Engineering):** Crear nuevas variables directamente en la base de datos (por ejemplo, calcular el promedio de compra de un usuario en los últimos 30 días).
2. **Muestreo:** Extraer una porción representativa de los datos para entrenamiento.
3. **Filtrado de Calidad:** Eliminar registros nulos o inconsistentes antes de que lleguen al modelo.

2. Consultas básicas: SELECT y FROM

Toda interacción con una base de datos comienza con la estructura más fundamental: ¿Qué quiero ver y de dónde lo saco?

La anatomía de una consulta

CÓDIGO

```
SELECT nombre_producto, precio
FROM inventario_ventas;
```

- **SELECT:** Define las columnas (campos) que quieres recuperar.
- **FROM:** Especifica la tabla de origen.

El riesgo del "SELECT *"

Es muy común ver a principiantes usar `SELECT * FROM tabla;` para ver todo. Si bien es útil para dar un vistazo rápido, en **Arquitectura de Datos** y **ML** se considera una mala práctica por dos razones:

1. **Rendimiento:** Transmitir columnas que no vas a usar consume ancho de banda y memoria innecesariamente.
2. **Fragilidad:** Si la estructura de la tabla cambia en el futuro (se añade una columna pesada de tipo imagen o texto largo), tus scripts de Python podrían romperse por falta de memoria.

Mejor práctica: Sé explícito. Si solo necesitas la fecha y el monto para una serie temporal, selecciona solo esas dos columnas.

3. Refinando la búsqueda: WHERE y Operadores

El análisis exploratorio requiere precisión. No queremos "todas las ventas", queremos "las ventas de la categoría 'Electrónica' que superen los $500 USD". Aquí utilizamos la cláusula **WHERE**.

Operadores lógicos y de comparación

Para filtrar datos, utilizamos operadores que ya conoces de Python, pero con una sintaxis SQL:

- **Igualdad y diferencia:** `=` , `!=` (o `<>`)
- **Rangos:** `>` , `<` , `>=` , `<=`
- **Lógica:** `AND` (ambas condiciones), `OR` (alguna condición), `NOT` (lo opuesto).

- **Pertenencia:** `IN ('Categoría A', 'Categoría B')`.

Ejemplo de la vida real: Supongamos que en **Spotify**, un analista busca canciones de un género específico que tengan alta popularidad para un sistema de recomendación:

CÓDIGO

```
SELECT track_name, artist_name, popularity
FROM tracks
WHERE genre = 'Rock'
 AND popularity > 80
 AND release_date >= '2023-01-01';
```

Observa cómo el filtro `WHERE` reduce drásticamente el volumen de datos que llegará a nuestro modelo de ML.

4. Ordenamiento, LIMIT y Agregaciones: De datos a métricas

Una vez que tenemos los datos filtrados, necesitamos darles estructura y obtener resúmenes cuantitativos.

ORDER BY y LIMIT: Los mejores amigos del diagnóstico

Cuando haces EDA, a menudo quieres ver los valores extremos: ¿Quiénes son mis 10 clientes con más quejas? ¿Cuáles son los 5 productos con menos stock?

- **ORDER BY columna [ASC | DESC]:** Ordena los resultados de forma ascendente (por defecto) o descendente.
- **LIMIT n:** Restringe la salida a las primeras `n` filas.

CÓDIGO

```
-- Obtener los 5 productos más caros de la tienda
SELECT nombre, precio
FROM productos
ORDER BY precio DESC
LIMIT 5;
```

Funciones de agregación: El corazón del análisis

Las funciones de agregación transforman múltiples filas en un solo valor. Esto es la base para crear métricas de negocio (KPIs).

Las más importantes son:

- `COUNT()`: Cuenta filas o valores no nulos.
- `SUM()`: Suma valores numéricos.
- `AVG()`: Calcula el promedio.
- `MIN()` / `MAX()`: Encuentra los valores extremos.

Caso de Uso: Análisis de Fraude Imagina que trabajas en un banco (ej. **BBVA** o **Santander**) y quieres saber el monto promedio de las transacciones sospechosas:

CÓDIGO

```
SELECT
 COUNT(*) AS total_transacciones,
 AVG(monto) AS monto_promedio,
 MAX(monto) AS transaccion_mas_alta
FROM transacciones
WHERE etiqueta_fraude = 1;
```

Nota: El comando `AS` se usa para ponerle un "alias" o nombre legible a la columna resultante, lo cual es vital para que al cargar los datos en un DataFrame de Python, las columnas tengan nombres claros.

5. El poder del GROUP BY

Hasta ahora, hemos calculado métricas globales. Pero el análisis real ocurre cuando comparamos grupos. ¿Cómo se comportan las ventas en México comparadas con las de Argentina?

GROUP BY agrupa filas que tienen los mismos valores en columnas específicas para que las funciones de agregación actúen sobre cada grupo.

Ejemplo fundamental:

CÓDIGO

```
SELECT categoria, COUNT(*) AS cantidad_productos, AVG(precio) AS precio_medio
FROM catalogo
GROUP BY categoria;
```

La trampa conceptual: WHERE vs. HAVING

Un error muy común en estudiantes es intentar filtrar el resultado de una agregación usando `WHERE`.

- **WHERE** filtra filas **antes** de que ocurra la agrupación.
- **HAVING** filtra grupos **después** de que se ha calculado la agregación.

Incorrecto: `WHERE SUM(ventas) > 1000` (SQL dará un error porque `WHERE` no conoce la suma hasta que se agrupa). **Correcto:** `GROUP BY tienda HAVING SUM(ventas) > 1000`

6. Errores comunes y mejores prácticas

Para cerrar esta unidad, repasemos los "tropiezos" más frecuentes que vemos en analistas junior:

1. **Olvidar los valores Nulos (NULL):** En SQL, `NULL` no es igual a cero ni a un texto vacío. Si haces `WHERE precio > 100`, los productos con precio `NULL` quedarán fuera. Siempre pregunta: "¿Qué pasa con mis datos faltantes?".

2. **Confundir el orden de las cláusulas:** SQL es muy estricto con el orden. El orden correcto siempre es:
3. `SELECT`
4. `FROM`
5. `WHERE`
6. `GROUP BY`
7. `HAVING`
8. `ORDER BY`
9. `LIMIT`
10. **No usar Alias (AS):** Cuando calculas un promedio como `AVG(salario)`, el resultado suele llamarse simplemente `avg` o algo genérico. Usa nombres descriptivos como `AS salario_promedio_departamento` para facilitar el trabajo posterior en Python.

Resumen de la Unidad

- **SQL** es más que un lenguaje de consulta; es una herramienta de pre-procesamiento masivo para Data Science.
- **SELECT y FROM** son el punto de partida; evita el `SELECT *` para mantener la eficiencia.
- **WHERE** nos permite filtrar los datos antes de traerlos a memoria.
- **ORDER BY y LIMIT** son esenciales para explorar tendencias y valores atípicos (outliers).
- **GROUP BY** junto con funciones de agregación (`SUM`, `AVG`, `COUNT`) nos permite generar métricas por segmento, lo cual es la base de cualquier reporte analítico.

En la siguiente unidad, aprenderemos cómo conectar **DuckDB**, un motor analítico ultra-rápido, con nuestro código de Python para ejecutar estas consultas de forma profesional. Sin embargo, antes de pasar al código, es fundamental que domines la lógica SQL pura.

¿Listo para poner a prueba tus habilidades de consulta? Vamos al ejercicio práctico.

Conectar DuckDB con Python y consultar datos

Hasta ahora has aprendido qué es DuckDB y por qué es tan potente para el análisis de datos masivos gracias a su almacenamiento columnar. Pero en el mundo real del **Machine Learning**, no escribimos SQL en una consola aislada; lo integramos directamente en nuestros flujos de trabajo de **Python**.

Esta integración es lo que permite que DuckDB actúe como el "músculo" que procesa los datos pesados, entregándole a nuestras librerías de ML (como Scikit-Learn) datos ya limpios y resumidos.

1. ¿Por qué conectar DuckDB con Python?

Python es el lenguaje estándar para ciencia de datos, pero a veces **Pandas** sufre cuando los datasets superan los 5GB o 10GB de RAM. Al conectar DuckDB con Python: * **Delegas el procesamiento**: DuckDB procesa los datos usando SQL eficiente (C++) mientras Python solo gestiona el flujo. * **Inter-**

operabilidad: DuckDB puede leer un DataFrame de Pandas como si fuera una tabla SQL y viceversa, ¡sin mover datos de la memoria!

2. Instalación e Importación

DuckDB no tiene dependencias externas complejas, lo que lo hace increíblemente fácil de instalar:

CÓDIGO

```
pip install duckdb
```

En tu script de Python, lo importas de forma estándar:

CÓDIGO

```
import duckdb
```

3. Modos de Conexión

Existen dos formas principales de conectarse a DuckDB:

1. **En Memoria (`:memory:`)**: La base de datos vive solo mientras el script está corriendo. Es perfecta para ETLs temporales o pruebas rápidas.
2. **Persistente (`archivo.db`)**: Los datos se guardan en un archivo en tu disco. Si el archivo no existe, DuckDB lo crea. Si ya existe, se conecta a él manteniendo tus tablas previas.

CÓDIGO

```
<h1 class="module" data-eyebrow="MÓDULO">Conexión en memoria</h1>
<hr class="module-underline"><hr class="module-hairline">
con = duckdb.connect(database=':memory:')

<h1 class="module" data-eyebrow="MÓDULO">Conexión persistente</h1>
<hr class="module-underline"><hr class="module-hairline">
con = duckdb.connect(database='datos_empresa.duckdb')
```

4. Ejecución de Consultas

Una vez establecida la conexión (`con`), tienes dos caminos principales para ejecutar SQL:

- **`con.execute("SELECT...")`**: Sigue el estándar de Python DB-API. Es ideal para comandos que no devuelven datos (como `CREATE` o `INSERT`).
- **`con.sql("SELECT...")`**: Es una función propia de DuckDB que devuelve un objeto de consulta. Es muy potente porque permite encadenar métodos como `.df()` para obtener un DataFrame de Pandas inmediatamente.

Ejemplo rápido de flujo de trabajo

Imagina que tienes miles de registros de sensores y quieres cargarlos:

CÓDIGO

```
import duckdb

<h1 class="module" data-eyebrow="MÓDULO">Conectamos</h1>
<hr class="module-underline"><hr class="module-hairline">
con = duckdb.connect("sensores.db")

<h1 class="module" data-eyebrow="MÓDULO">Creamos tabla</h1>
<hr class="module-underline"><hr class="module-hairline">
con.execute("CREATE TABLE IF NOT EXISTS lecturas (id INTEGER, valor FLOAT)")

<h1 class="module" data-eyebrow="MÓDULO">Insertamos</h1>
<hr class="module-underline"><hr class="module-hairline">
con.execute("INSERT INTO lecturas VALUES (1, 25.5), (2, 28.1)")

<h1 class="module" data-eyebrow="MÓDULO">Consultamos y convertimos a Pandas para ML</h1>
<hr class="module-underline"><hr class="module-hairline">
df = con.sql("SELECT * FROM lecturas WHERE valor > 26").df()
print(df)
```

Errores comunes a evitar

1. **Conexiones múltiples al mismo archivo**: DuckDB (al igual que SQLite) bloquea el archivo para escritura. Si tienes un Jupyter Notebook con la conexión abierta e intentas ejecutar otro script que use el mismo archivo `.db`, obtendrás un error de "Connection Error". Recuerda usar `con.close()` o reiniciar el kernel.
2. **Inyección de SQL**: Nunca uses f-strings de Python para meter variables directamente en tus queries (ej: `f"SELECT * FROM t WHERE id={mi_id}"`). Usa parámetros seguros para evitar brechas de seguridad.

Ejecución eficiente de queries analíticas

Imagina que estás en una biblioteca gigante con diez millones de libros. Si necesitas saber únicamente el título de los libros escritos por Gabriel García Márquez, ¿qué harías? ¿Sacarías los diez millones de libros de los estantes, los pondrías en una mesa y luego leerías sus portadas una por una? Por supuesto que no. Irías directamente al fichero (el índice) de autores, buscarías "García Márquez" y anotarías solo los títulos.

En el mundo del **Machine Learning y la Ciencia de Datos**, las bases de datos son nuestra biblioteca. Cada vez que ejecutamos una consulta (query), estamos pidiendo recursos: tiempo de procesador (CPU), memoria RAM y ancho de banda. Si escribes consultas ineficientes sobre datasets de millones de filas (algo común en ML), no solo tardarás horas en obtener resultados, sino que podrías colapsar el servidor o generar costos altísimos si usas servicios en la nube.

En esta unidad, aprenderás a dejar de "sacar todos los libros" y empezar a consultar como un profesional, utilizando **DuckDB** y principios de optimización analítica.

1. El costo de la curiosidad: SELECT específico vs SELECT *

Como principiante, es tentador usar `SELECT * FROM tabla` para "ver qué hay". En una tabla pequeña de 100 filas, no pasa nada. Pero en Data Science, trabajamos con **motores analíticos columnares** como DuckDB.

¿Por qué evitar el asterisco (*)?

Los motores modernos como DuckDB o BigQuery almacenan los datos por columnas, no por filas. Si una tabla tiene 200 columnas (características o *features* para tu modelo de ML) y tú solo necesitas la `edad` y el `ingreso`, usar `SELECT *` obliga al motor a:

1. Leer los archivos de las 200 columnas desde el disco.
2. Cargar esos datos en la memoria RAM.
3. Transferir esos datos a través de la red o hacia tu notebook de Python.

Ejemplo real: Imagina un dataset de transacciones bancarias de 10GB. Si solo necesitas el `monto` y la `fecha` (que representan quizás 200MB de esos 10GB), al usar `SELECT *` estás desperdiciando el 98% de los recursos.

La regla de oro: "Trae solo lo que vas a usar".

CÓDIGO

```
-- Ineficiente (trae 50 columnas de sensores)
SELECT * FROM sensores_iot WHERE temperatura > 30;

-- Eficiente (solo lo necesario para el entrenamiento del modelo)
SELECT timestamp, temperatura, humedad FROM sensores_iot WHERE temperatura > 30;
```

2. Filtrar antes de procesar: El poder del WHERE

El orden de los factores **sí altera el producto** en SQL. Uno de los errores más comunes es realizar operaciones costosas (como JOINs o agregaciones) sobre todo el dataset y filtrar los resultados al final.

El principio de "Filtrado Temprano"

Debes aplicar el `WHERE` lo más pronto posible. Piensa en ello como un embudo: quieres que la menor cantidad de datos posible pase a las etapas siguientes de la consulta.

Caso de industria: Netflix analiza el comportamiento de visualización. Si quieren saber qué películas de acción vio un usuario en Argentina, primero filtran por `pais = 'AR'` y `genero = 'Accion'`, y *después* calculan estadísticas. Si calcularan las estadísticas de todos los usuarios del mundo y luego filtraran por Argentina, el sistema tardaría demasiado en cargar las sugerencias.

3. Plan de ejecución: El "GPS" de tu consulta

¿Alguna vez te has preguntado cómo decide la base de datos qué hacer primero? Para eso existe el **Plan de Ejecución**.

El motor de la base de datos no lee tu código SQL como una receta paso a paso de arriba hacia abajo. En su lugar, toma tu consulta y crea un "mapa" de cómo obtener los datos de la forma más barata y rápida posible. A este proceso se le llama **Optimización de Consultas**.

¿Cómo ver el plan en DuckDB?

Puedes usar la palabra clave `EXPLAIN` antes de tu consulta:

CÓDIGO

```
EXPLAIN SELECT nombre, salario FROM empleados WHERE departamento = 'Ventas';
```

Esto te devolverá una tabla (o un árbol) que muestra los pasos. Los términos clave que verás son:

- **Seq Scan (Sequential Scan):** Es el equivalente a buscar el libro leyendo toda la biblioteca. Es lento y costoso porque lee cada fila de la tabla.
- **Filter:** Representa tu cláusula `WHERE`.
- **Projection:** Representa las columnas que elegiste en el `SELECT`.

Anticipando errores: Muchos estudiantes creen que `EXPLAIN` ejecuta la consulta. No es así. `EXPLAIN` solo planea. Para ver el plan y además ejecutarlo viendo los tiempos reales, se usa `EXPLAIN ANALYZE`.

4. Escaneo de tablas e índices

Si el **Sequential Scan** es buscar hoja por hoja, el **Index Scan** es usar el índice al final del libro.

¿Qué es un índice?

Es una estructura de datos adicional (oculta para el usuario) que guarda una copia ordenada de una columna específica junto con un puntero a la fila original.

Ejemplo práctico: Si buscas un cliente por su `ID_Fiscal`, y esa columna tiene un índice, la base de datos encontrará el registro en microsegundos, sin importar si la tabla tiene 1 o 100 millones de filas.

El compromiso del índice (Trade-off): Un error frecuente es pensar: "¡Pondré índices en todas las columnas!".

- **Ventaja:** Las lecturas (`SELECT`) son ultra rápidas.
- **Desventaja:** Las escrituras (`INSERT` , `UPDATE` , `DELETE`) se vuelven lentas, porque la base de datos tiene que actualizar el índice cada vez que cambias un dato. Además, los índices ocupan espacio en el disco.

Como Data Scientist, solemos trabajar en entornos de "Lectura intensiva", por lo que los índices en fechas, IDs y categorías de filtrado frecuente son tus mejores aliados.

5. Errores comunes y mejores prácticas

Error 1: Usar funciones en la columna del WHERE

CÓDIGO

```
-- MAL: El motor no puede usar índices en la columna 'fecha'
-- porque tiene que aplicar una función a cada fila primero.
SELECT * FROM ventas WHERE YEAR(fecha) = 2024;

-- BIEN: El motor puede buscar directamente en el rango.
SELECT * FROM ventas WHERE fecha >= '2024-01-01' AND fecha <= '2024-12-31';
```

Error 2: Ordenar (ORDER BY) sin necesidad

Ordenar es una de las tareas más pesadas para una computadora. Si vas a cargar los datos en un DataFrame de Pandas para visualizarlos después, no gastes recursos ordenándolos en SQL a menos que sea estrictamente necesario para un `LIMIT` (ej: los 10 más caros).

Resumen de mejores prácticas:

1. **Limita las columnas:** Usa nombres específicos, nunca `*`.
2. **Filtra temprano:** Usa `WHERE` con columnas indexadas siempre que puedas.
3. **Usa LIMIT para explorar:** Si solo quieres ver una muestra, usa `LIMIT 10`. Esto le dice al motor que se detenga en cuanto encuentre las primeras 10 filas, ahorrando escaneos innecesarios.
4. **Entiende tu motor:** DuckDB es increíblemente rápido para agregaciones (SUM, AVG) porque es columnar, así que aprovecha para hacer los cálculos pesados en SQL antes de pasar los datos a Python.

Conexión con lo que viene

Ahora que sabes cómo pedir los datos de forma eficiente, en la siguiente unidad aprenderemos a aplicar estos conceptos para **manejar archivos de datos masivos (como Parquet y CSV gigantes)** sin que tu computadora se quede sin memoria. La eficiencia no es un lujo, es una necesidad técnica en el flujo de Machine Learning.

Manejar archivos grandes con DuckDB

El Problema de la Memoria RAM

Como analista de datos, seguramente te has enfrentado al temido `MemoryError` en Python. Esto sucede porque librerías tradicionales como **Pandas** están diseñadas para cargar el dataset *completo* en la

memoria RAM antes de permitirte hacer cualquier operación. Si tienes 8GB de RAM e intentas abrir un archivo de 10GB, tu computadora simplemente se detendrá.

DuckDB cambia las reglas del juego mediante una arquitectura de **procesamiento por bloques (streaming)**.

¿Cómo maneja DuckDB archivos grandes?

En lugar de ser una "aspiradora" que succiona todo el contenido del disco a la RAM, DuckDB actúa como un **filtro inteligente**:

1. **Lectura Directa (Zero-Copy):** DuckDB puede ejecutar SQL directamente sobre archivos `.csv` o `.parquet` sin necesidad de "importarlos" a una tabla interna.
2. **Procesamiento en Bloques (Vectorizado):** Lee los datos en pequeños trozos (vectores). Analiza un bloque, extrae lo que necesitas, libera la memoria y pasa al siguiente. Esto le permite procesar datasets de cientos de gigabytes incluso en una laptop estándar.
3. **Paralelización:** Por defecto, DuckDB detecta cuántos núcleos (CPU) tiene tu máquina y los usa todos para leer el archivo simultáneamente.

Formatos: CSV vs Parquet

Aunque DuckDB es increíblemente rápido con CSV, el formato **Parquet** es su mejor aliado: * **CSV:** Es un formato de texto plano. DuckDB debe leer cada línea para encontrar lo que buscas. * **Parquet:** Es un formato **columnar**. Si tu tabla tiene 100 columnas pero tu consulta solo usa 2, DuckDB *físicamente solo lee* esas 2 columnas del disco. Esto reduce drásticamente el uso de memoria y tiempo.

Ejemplo de sintaxis:

CÓDIGO

```
-- Consultar un CSV directamente
SELECT * FROM 'datos_gigantes.csv' WHERE region = 'LATAM';

-- Consultar un Parquet (mucho más rápido)
SELECT * FROM 'datos_gigantes.parquet' WHERE ventas > 1000;
```

Tarea

Completa el script de Python para comparar cómo DuckDB puede filtrar un archivo directamente desde el disco frente al método tradicional de carga total en memoria. observe la diferencia en el tiempo de respuesta y el manejo de recursos.

Errores comunes a evitar:

- **Cargar el CSV con Pandas y luego pasarlo a DuckDB:** ¡Esto anula el beneficio! Si Pandas lo carga primero, ya consumiste toda la RAM. Deja que DuckDB lea el archivo directamente.

- **No usar filtros en la consulta:** Si haces `SELECT * FROM 'archivo.csv'` sin filtros (`WHERE`) ni límites (`LIMIT`), DuckDB terminará intentando mostrar millones de filas en tu consola, lo cual también es lento.

DuckDB frente a Pandas en análisis de datos

DuckDB frente a Pandas: La batalla por la eficiencia

Has llegado al final del módulo sobre **Motores Modernos**. A estas alturas, sabes que no existe una única herramienta "mejor" para todo, sino herramientas "adecuadas" para problemas específicos.

En el mundo del Python para Data Science, **Pandas** ha sido el estándar absoluto durante años. Sin embargo, con el crecimiento del volumen de los datos, han surgido motores como **DuckDB** que desafían la forma en que procesamos información.

¿Por qué comparar DuckDB y Pandas?

Pandas es fabuloso para la manipulación flexible de datos en memoria, pero sufre cuando los datasets superan unos pocos gigabytes o cuando necesitamos realizar operaciones complejas de unión (joins) y agregación. DuckDB, al ser un motor **SQL analítico (OLAP)**, está diseñado desde cero para la velocidad y el uso eficiente del hardware moderno.

01 **¿Cuál de las dos herramientas aprovecha mejor el paralelismo de forma nativa?**

DuckDB. A diferencia de Pandas, que es mayoritariamente mononúcleo (single-threaded), DuckDB utiliza automáticamente **todos los núcleos de tu CPU** para ejecutar consultas SQL, lo que acelera dramáticamente los cálculos en datasets grandes.

02 **¿Qué herramienta tiene un consumo de memoria RAM mucho más elevado al cargar archivos grandes?**

Pandas. Suele requerir entre **5 y 10 veces** el tamaño del archivo original en memoria RAM para operar cómodamente, mientras que DuckDB puede procesar datos que no caben en la RAM mediante el uso eficiente del disco (out-of-core processing).

03 **Si tienes un archivo Parquet de 10GB y solo necesitas 2 columnas, ¿qué herramienta es más rápida para la extracción inicial?**

DuckDB. Gracias a su motor SQL, DuckDB es extremadamente eficiente consultando archivos directamente sin cargarlos por completo en memoria. Puedes hacer un `SELECT` con filtros sobre un archivo de 10GB en segundos.

04 ¿En qué escenario de volumen de datos sigue siendo Pandas la opción más conveniente por su agilidad?

Pandas. Para micro-datasets (ej. 100 filas) y transformaciones muy específicas (como limpiar strings de forma compleja), el overhead de iniciar un motor SQL puede no valer la pena frente a la simplicidad de un DataFrame.

05 ¿Cuál es la mejor opción para realizar un JOIN complejo entre tres tablas de gran tamaño?

DuckDB (SQL). Los motores de bases de datos están optimizados específicamente para algoritmos de unión (Hash Joins). Pandas puede volverse muy lento y agotar la memoria al intentar cruzar tablas con millones de registros.

06 ¿Qué herramienta se prefiere para la fase de visualización y el entrenamiento final de un modelo de ML?

Pandas. Su ecosistema para visualización (Seaborn, Matplotlib) y preparación final de vectores para modelos de Scikit-Learn es mucho más maduro y directo una vez que los datos ya han sido filtrados y agregados.

07 ¿Qué significa que DuckDB sea un motor "Out-of-Core"?

Se refiere a la capacidad de procesar datos que superan la capacidad de la memoria RAM, moviendo bloques de datos desde el disco de forma eficiente. **DuckDB** lo hace de forma nativa; **Pandas** no (suele dar un error de MemoryError).

08 Más que elegir una u otra, ¿cuál es la mejor práctica en un pipeline de datos moderno?

La integración (Sinergia). Un flujo moderno utiliza **DuckDB** para extraer, filtrar y agregar grandes volúmenes de datos mediante SQL, y luego exporta el resultado final (ya reducido y manejable) a un **Pandas DataFrame** para el análisis estadístico detallado.

PRE-ENTREGA DEL PROYECTO

Actividad evaluable · parte del proyecto integrador

Pre-entrega: Análisis de Datos Meteorológicos con DuckDB e SQL

El objetivo de este entregable es consolidar el uso de **DuckDB** como motor analítico de alto rendimiento para procesar datos externos de manera eficiente. Aprenderás a realizar consultas SQL directamente sobre archivos de datos sin necesidad de una base de datos tradicional pesada.

Cómo resolver esta entrega

Propósito

Configurar un entorno analítico ligero y ejecutar consultas SQL complejas para extraer insights de un dataset de serie temporal (clima), demostrando la velocidad y simplicidad de DuckDB.

Pasos sugeridos

1. **Preparación del Entorno**: Crea un nuevo repositorio en GitHub para este proyecto.
2. **Obtención de datos**: Descarga o referencia via URL un dataset en formato CSV o Parquet (puedes usar datasets públicos de clima de NOAA o similares).
3. **Implementación de Scripts**: Escribe un script de Python o un archivo SQL que realice lo siguiente mediante DuckDB: * Carga y registro del archivo como una tabla virtual. * Limpieza básica (manejo de nulos en temperaturas). * Análisis agregados (promedios mensuales, máximos históricos por estación).
4. **Documentación**: Crea un archivo `README.md` que explique cómo ejecutar el análisis y cuáles fueron los tiempos de respuesta o hallazgos principales.

Errores comunes

- Intentar cargar el archivo en una base de datos SQL tradicional (como MySQL) en lugar de consultar directamente el archivo con DuckDB.
- No documentar las dependencias necesarias en un archivo `requirements.txt`.

Objetivos de aprendizaje

- Configurar DuckDB como motor de consulta analítica local.
- Ejecutar consultas SQL sobre fuentes de datos externas (CSV/Parquet) sin carga previa en tablas físicas.
- Aplicar funciones de agregación y filtrado SQL para la extracción de insights de negocio.
- Implementar flujos de trabajo de ingeniería de datos simplificados mediante SQL moderno.

Criterios de aceptación

- El repositorio contiene un script funcional que utiliza la librería de DuckDB.
- El código realiza al menos tres consultas analíticas diferentes (promedios, conteos, filtrados complejos).
- Se incluye un archivo README.md con instrucciones claras para replicar el análisis.
- La consulta de datos se realiza directamente sobre archivos (CSV o Parquet) aprovechando las capacidades de DuckDB.
- No se incluyen archivos binarios de la base de datos, priorizando el enfoque de motor embebido.

Rúbrica de evaluación: Análisis de Datos Meteorológicos con DuckDB e SQL

Rúbrica para evaluar un repositorio de GitHub que implemente un análisis de datos meteorológicos usando DuckDB como motor analítico, incluyendo consulta directa sobre archivos externos, limpieza básica de datos, agregaciones relevantes y documentación clara de ejecución y hallazgos.

Criterio	Descripción	Peso
Uso correcto de DuckDB y consulta directa sobre archivos	Evalúa si el repositorio utiliza DuckDB como motor analítico principal para consultar directamente archivos CSV o Parquet, sin depender de una base de datos tradicional para cargar los datos.	30%
Implementación del análisis y limpieza de datos	Evalúa si el script o archivo SQL realiza limpieza básica de nulos en temperaturas y consultas agregadas útiles, como promedios mensuales y máximos históricos por estación, con resultados coherentes.	25%
Estructura del repositorio y organización de archivos	Evalúa la claridad y orden del repositorio como proyecto real: presencia de archivos esperados, organización adecuada, nombres comprensibles y coherencia entre script, datos de referencia y documentación.	15%
Documentación de ejecución, dependencias y hallazgos	Evalúa la calidad del README.md y si explica cómo ejecutar el análisis, qué dependencias se requieren y cuáles son los hallazgos o tiempos de respuesta observados.	20%
Reproducibilidad y calidad técnica del entregable	Evalúa si el repositorio puede ejecutarse razonablemente por otra persona, incluyendo coherencia técnica general, presencia de requirements.txt cuando aplica, y ausencia de errores estructurales que impidan probar el análisis.	10%

Total: 100 pts | Aprobación: 70 pts

MÓDULO 8

SQL Analítico Avanzado

Dominar consultas SQL complejas para la preparación de features.

Funciones de ventana para analizar cada fila sin perder contexto

Imagina que trabajas como Data Scientist en una plataforma de e-commerce similar a Amazon o Mercado Libre. Tu jefe te hace una pregunta que parece sencilla: *"Para cada pedido realizado hoy, quiero ver el monto de la compra, pero también el promedio de compra de ese usuario en particular para saber si gastó más o menos de lo habitual"*.

Si intentas resolver esto solo con lo que sabes de `GROUP BY`, te enfrentarás a un muro. Si agrupas por usuario para calcular el promedio, perderás el detalle de los pedidos individuales (las filas se "colapsan"). Si no agrupas, tienes los pedidos individuales, pero no tienes el promedio calculado sobre el grupo. Antes, la solución era hacer consultas complejas, guardar resultados en tablas temporales o realizar "Self-Joins" (unir la tabla consigo misma), procesos que son lentos y propensos a errores.

Aquí es donde entran las **Window Functions** (Funciones de Ventana). Son, sin duda, una de las herramientas más potentes del SQL analítico moderno. Te permiten realizar cálculos "sobre" un conjunto de filas (la ventana) mientras mantienen intacta la identidad de cada fila individual.

1. El concepto fundamental: ¿Qué es una "Ventana"?

En SQL tradicional, estamos acostumbrados a dos modos de operación:

1. **Fila a fila:** Operaciones como `precio * 1.16` que afectan a cada registro de forma aislada.
2. **Agregación (GROUP BY):** Operaciones que agrupan muchas filas en una sola (como `SUM` o `AVG`), perdiendo el nivel de detalle original.

Una **Window Function** rompe esta limitación. Imagina que cada fila de tu tabla tiene una "ventana" a través de la cual puede "mirar" a otras filas relacionadas (por ejemplo, otras compras del mismo cliente) para hacer un cálculo, y luego poner el resultado de ese cálculo justo al lado de la información original de la fila.

Por qué importan en Machine Learning

En la preparación de características (*Feature Engineering*), necesitamos contexto. Un modelo de ML no solo necesita saber que un precio es de $100; necesita saber si ese precio es el más alto que ha tenido ese producto en el último mes. Las funciones de ventana nos permiten crear estas variables de contexto directamente en el motor de base de datos de forma extremadamente eficiente.

2. La Cláusula OVER: El motor de la ventana

La palabra clave `OVER` es lo que le dice a SQL: *"No agrupes esta fila; en su lugar, abre una ventana para realizar un cálculo"*.

La sintaxis básica sigue este patrón:

CÓDIGO

```
SELECT
 columna_id,
 monto,
 FUNCION_AGREGACION(monto) OVER() as calculo_ventana
FROM pedidos;
```

Si usas `OVER()` vacío, la "ventana" es **toda la tabla**.

- **Ejemplo:** Si tienes 1,000 pedidos, `SUM(monto) OVER()` pondrá el total de ventas de toda la empresa en una columna nueva, repetida en cada una de las 1,000 filas. Esto es útil para calcular porcentajes (ej. `monto / SUM(monto) OVER() * 100`).

3. PARTITION BY: Definiendo los límites

Rara vez queremos mirar toda la tabla. Lo más común es querer mirar filas que compartan algo en común, como el mismo `cliente_id` o la misma `categoria_producto`.

`PARTITION BY` divide las filas en grupos lógicos. El cálculo de la función de ventana se reinicia con cada nuevo grupo.

Ejemplo concreto: Análisis por Cliente

Supongamos que tenemos la tabla `pedidos`:

pedido_id	cliente_id	monto
1	A	100
2	A	200
3	B	500

Si ejecutamos:

CÓDIGO

```
SELECT
 pedido_id,
 cliente_id,
 monto,
 AVG(monto) OVER(PARTITION BY cliente_id) as promedio_cliente
FROM pedidos;
```

Resultado:

pedido_id	cliente_id	monto	promedio_cliente
1	A	100	150
2	A	200	150
3	B	500	500

Observa cómo las filas 1 y 2 mantienen su `monto` original, pero ahora tienen acceso al promedio de su "partición" (Cliente A). Esto es imposible de lograr con un `GROUP BY` simple sin perder las filas individuales.

4. ORDER BY en Ventanas: El secreto de los acumulados

Cuando añadimos `ORDER BY` dentro de la cláusula `OVER`, la ventana cambia de comportamiento: ya no mira "todo el grupo", sino que mira **desde el inicio del grupo hasta la fila actual**. Esto es lo que permite crear **totales acumulados** o promedios móviles.

Ejemplo: Ventas acumuladas en el tiempo

CÓDIGO

```
SELECT
 fecha,
 monto,
 SUM(monto) OVER(ORDER BY fecha) as ventas_acumuladas
FROM ventas_diarias;
```

Aquí, SQL ordena por fecha y, para cada fila, suma los montos de todas las filas anteriores incluyendo la actual. Es vital para reportes de crecimiento diario o de "quema de presupuesto".

5. Combinando todo: PARTITION BY + ORDER BY

Este es el nivel donde realmente empezamos a construir arquitectura de datos profesional. Podemos agrupar por una categoría y ordenar por tiempo simultáneamente.

Caso de Uso: Ranking de productos por categoría Imagina que quieres saber cuáles son los 3 productos más vendidos *dentro de cada categoría*.

CÓDIGO

```
SELECT
 categoria,
 producto,
 ventas,
 ROW_NUMBER() OVER(PARTITION BY categoria ORDER BY ventas DESC) as ranking_en_categoria
FROM catalogo_productos;
```

Aquí, la ventana se "resetea" cada vez que cambia la categoría, y dentro de cada categoría, numera las filas del 1 al N basándose en quién vendió más.

6. Errores comunes y mejores prácticas

Error 1: Confundir WHERE con el filtrado de ventanas

Un error frecuente para principiantes es intentar filtrar el resultado de una función de ventana en la misma consulta:

- *Incorrecto:* `SELECT ... ROW_NUMBER() OVER(...) as rank WHERE rank = 1`
- *Por qué:* SQL procesa el `WHERE` **antes** que las Window Functions. Para filtrar por un ranking, debes usar una subconsulta o un CTE (Common Table Expression), que aprenderemos más adelante.

Error 2: Malinterpretar el impacto en el rendimiento

Aunque son más eficientes que los Self-Joins, las funciones de ventana requieren memoria para ordenar y particionar los datos. En motores como DuckDB o MySQL 8.0+, están muy optimizadas, pero en tablas de miles de millones de filas, un `PARTITION BY` sobre una columna sin índices puede ser lento.

Mejor Práctica: Claridad ante todo

Si bien puedes poner muchas lógicas dentro de un `OVER`, si la expresión se vuelve demasiado larga, considera usar **Named Windows** (si tu motor SQL lo soporta) para reutilizar la definición de la ventana:

CÓDIGO

```
SELECT
 AVG(monto) OVER w,
 SUM(monto) OVER w
FROM pedidos
WINDOW w AS (PARTITION BY cliente_id ORDER BY fecha);
```

7. Síntesis y aplicaciones en la industria

Las funciones de ventana son la columna vertebral del análisis de datos moderno. En empresas como **Uber**, se usan para comparar el tiempo de llegada actual con el promedio de la zona en la última hora. En **Netflix**, se utilizan para determinar en qué orden un usuario consume contenido de una misma franquicia.

Resumen de lo aprendido:

1. **Contexto:** Las funciones de ventana permiten hacer cálculos sobre grupos sin colapsar las filas.
2. **OVER():** Es la puerta de entrada; define que habrá una ventana.
3. **PARTITION BY:** Define los grupos de comparación (¿A quiénes miro?).
4. **ORDER BY:** Define la secuencia y crea efectos acumulativos (¿En qué orden miro?).

Dominar estas funciones no solo limpia tu código SQL eliminando Joins innecesarios, sino que te otorga la capacidad de extraer variables complejas para tus modelos de Machine Learning de forma directa y elegante. En la siguiente unidad, profundizaremos en funciones específicas de ranking y cómo calcular diferencias entre filas (como el crecimiento mes a mes).

Ranking, acumulados y diferencias entre filas

En el análisis de datos para Machine Learning, muchas veces no nos basta con saber cuánto se vendió en total. Necesitamos saber: ¿Este producto es el más vendido de su categoría? ¿Las ventas están subiendo o bajando respecto a ayer? ¿Cuál es el total acumulado del mes?

Tradicionalmente, en SQL básico (con `GROUP BY`), responder esto requiere subconsultas lentas y complejas. Las **Funciones de Ventana Avanzadas** nos permiten mirar "hacia atrás", "hacia adelante" o "hacia arriba" sin mezclar todas las filas en un solo grupo.

1. Funciones de Ranking: Poniendo orden al caos

Cuando queremos asignar una posición, tenemos tres herramientas principales que se comportan distinto ante los **empates**:

- **`ROW_NUMBER()`**: Asigna un número único y secuencial (1, 2, 3, 4...). Si dos filas son idénticas, a una le dará el 1 y a la otra el 2 arbitrariamente.
- **`RANK()`**: Si hay un empate, asigna el mismo número, pero **salta** posiciones (1, 2, 2, 4...). Es ideal para podios deportivos.
- **`DENSE_RANK()`**: Si hay un empate, asigna el mismo número y **no salta** (1, 2, 2, 3...). Es el más común en Data Science para crear categorías de "Top productos".

2. LAG y LEAD: El viaje en el tiempo

Estas funciones son esenciales para generar **features temporales** (variables que dependen del tiempo) para modelos de ML:

- **`LAG(columna, n)`**: Accede al valor de la fila que está `n` posiciones **atrás**. Es perfecto para calcular el crecimiento diario (Ventas Hoy - Ventas Ayer).
- **`LEAD(columna, n)`**: Accede al valor de la fila que está `n` posiciones **adelante**. Útil para ver qué evento ocurrió justo después de una acción del usuario.

3. Acumulados (Running Totals)

¿Alguna vez has visto un gráfico de líneas que siempre sube? Probablemente es un acumulado. En SQL, si usas una función de agregación como `SUM()` pero incluyes un `ORDER BY` dentro del `OVER`, SQL no suma todo el grupo, sino que suma **desde el inicio hasta la fila actual**.

Ejemplo conceptual:

CÓDIGO

```
SELECT
 fecha,
 monto,
 SUM(monto) OVER (ORDER BY fecha) as acumulado
FROM ventas;
```

Si el lunes vendimos 10 y el martes 5, la fila del martes mostrará un acumulado de 15.

Errores Comunes a evitar

1. **Olvidar el `ORDER BY` en `LAG/LEAD`**: Al motor de SQL no le importa el orden en que ves los datos en pantalla; si no especificas el orden dentro de la ventana, el "anterior" será aleatorio.
2. **Confundir `RANK` con `DENSE_RANK`**: Si necesitas los "Top 10" y tienes muchos empates, `RANK` podría devolverte solo 5 productos (porque el 6to puesto se 'saltó'). Usa `DENSE_RANK` si quieres grupos compactos.

JSON en SQL: almacenar, validar y extraer atributos

En el mundo moderno del Data Science, no todos los datos llegan en tablas ordenadas. Las APIs, los logs de servidores y las bases de datos NoSQL suelen entregar información en formato **JSON** (JavaScript Object Notation).

Como Data Scientist o Data Architect, necesitas saber cómo integrar estos datos semiestructurados en tu flujo de trabajo SQL sin perder la capacidad analítica.

¿Por qué JSON dentro de SQL?

Tradicionalmente, SQL era estrictamente relacional. Sin embargo, en arquitecturas modernas de **Machine Learning**, a menudo nos enfrentamos a dos retos: 1. **Esquema flexible**: Atributos que cambian frecuentemente (ej. metadatos de un sensor). 2. **Ingesta rápida**: Es más barato guardar el JSON "crudo" (Raw) y procesarlo después (enfoque ELT) que transformarlo antes de guardarlo.

Conceptos Clave

1. Validación de Integridad

Antes de extraer datos, debemos asegurarnos de que el texto guardado sea un JSON bien formado. Motores como **DuckDB** o **PostgreSQL** ofrecen funciones como `json_valid()` para evitar errores en tiempo de ejecución.

2. Extracción de Atributos (Escalares vs. Objetos)

Existen dos formas principales de extraer datos: - **Scalares (`->>`)**: Extraen el valor como texto simple (un nombre, una fecha). Ideal para columnas que irán directo al modelo de ML. - **Objetos/Arrays (`->`)**: Extraen una pieza del JSON manteniendo su formato. Útil si quieres seguir navegando en una estructura anidada.

3. Desanidado (Unnesting)

A veces, un campo JSON contiene una lista (array), como los productos comprados en un solo ticket. Para analizarlos individualmente, usamos funciones como `unnest()`, que transforman un array de una sola fila en múltiples filas independientes.

Errores Comunes

- **Confundir tipos de datos**: Extraer un número como texto (`->>`) y olvidar convertirlo a `INTEGER` antes de hacer cálculos.
- **Rutas incorrectas**: Olvidar que las rutas suelen empezar con `$` (raíz del documento) o usar puntos donde no corresponde.
- **Ignorar nulos**: Si una clave no existe en el JSON, SQL devolverá `NULL`. Siempre verifica la completitud de tus features extraídas.

Agregaciones multi-tabla con JOIN y GROUP BY

En el camino hacia la creación de modelos de Machine Learning robustos, la calidad de las "features" (o variables de entrada) lo es todo. Rara vez los datos necesarios para una predicción residen en una sola tabla. Imagina que quieres predecir si un cliente abandonará una suscripción: necesitarás sus datos demográficos (Tabla A), pero sobre todo su comportamiento de compra reciente (Tabla B) y el tipo de productos que consume (Tabla C).

El desafío real no es solo "pegar" estas tablas, sino **resumir** la información de cientos de transacciones en métricas únicas por cliente (como el "gasto total" o "frecuencia de compra") sin cometer errores de duplicación que arruinen tus datos. En esta unidad, aprenderás a dominar el arte de las agregaciones multi-tabla, combinando la potencia de los `JOIN` con la precisión del `GROUP BY` y los filtros avanzados del `HAVING`.

1. El Porqué del JOIN con Agregación

Hasta ahora, has utilizado `GROUP BY` para resumir datos dentro de una misma tabla. Sin embargo, en la arquitectura de datos moderna, la información está normalizada para evitar redundancia.

El problema: Si intentas calcular el gasto promedio de un cliente usando solo la tabla de `Pedidos`, obtendrás un ID de cliente y un número. Pero para que esa feature sea útil en un modelo de ML o un reporte de negocio, necesitas el nombre del cliente, su segmento o su ubicación geográfica, datos que viven en la tabla `Clientes`.

La solución: Unir las tablas antes de agrupar. La secuencia lógica es:

1. **Unir (`JOIN`):** Expandir el horizonte de datos combinando filas relacionadas.
2. **Agrupar (`GROUP BY`):** Colapsar esas filas expandidas basándose en una entidad (ej. el cliente).
3. **Agregar (`SUM`, `AVG`, etc.):** Calcular la métrica sobre el grupo.

2. Los Pilares de la Unión: INNER vs. LEFT JOIN

En el contexto de la ciencia de datos, la elección del tipo de unión impacta directamente en el sesgo de tu dataset.

INNER JOIN: El filtro implícito

El `INNER JOIN` solo devuelve filas donde hay una coincidencia en ambas tablas.

- **Riesgo en ML:** Si haces un `INNER JOIN` entre `Clientes` y `Pedidos`, los clientes que **nunca han comprado** desaparecerán de tu resultado. Si tu objetivo es predecir quién comprará por primera vez, habrás eliminado accidentalmente a tu población objetivo.

LEFT JOIN: La integridad de la población

El `LEFT JOIN` conserva todos los registros de la tabla de la izquierda (ej. todos los clientes), incluso si no tienen registros asociados en la tabla de la derecha (pedidos).

- **Valor en ML:** Los clientes sin compras aparecerán con valores `NULL` en las columnas de pedidos. Esto te permite usar funciones como `COALESCE(SUM(monto), 0)` para asignarles un gasto de $0, manteniendo a todos los sujetos en el estudio.

3. Dominando el GROUP BY en Escenarios Multi-tabla

Cuando unes varias tablas, la estructura del resultado se vuelve más compleja. Es aquí donde muchos analistas cometen el error de la "columna perdida".

La Regla de Oro

Cualquier columna que aparezca en tu `SELECT` y que **no** esté dentro de una función de agregación (como `SUM` o `COUNT`), **debe** estar incluida en la cláusula `GROUP BY`.

Ejemplo de escenario real: Queremos saber cuánto ha gastado cada cliente y ver su nombre.

CÓDIGO

```
SELECT
 c.cliente_id,
 c.nombre,
 SUM(p.total) AS gasto_total
FROM clientes c
LEFT JOIN pedidos p ON c.cliente_id = p.cliente_id
GROUP BY c.cliente_id, c.nombre;
```

Si olvidaras `c.nombre` en el `GROUP BY`, la mayoría de los motores de SQL (excepto algunas configuraciones laxas de MySQL) arrojarían un error. ¿Por qué? Porque SQL no sabe qué nombre elegir si un `cliente_id` tuviera, hipotéticamente, dos nombres distintos. Al incluirlos en el `GROUP BY`, le aseguras a SQL que quieres el resumen por cada combinación única de ID y Nombre.

4. Agregaciones Condicionadas: La Feature Engineering Eficiente

A veces no necesitas el total global, sino el total de una categoría específica. En lugar de hacer múltiples subconsultas, usamos el patrón de **Agregación Condicionada** con la sentencia `CASE WHEN`.

Imagina que quieres crear dos features para un modelo: "Gasto en Electrónica" y "Gasto en Ropa". Puedes hacerlo en una sola pasada:

CÓDIGO

```
SELECT
 c.nombre,
 SUM(CASE WHEN cat.nombre = 'Electrónica' THEN p.monto ELSE 0 END) AS gasto_electronica,
 SUM(CASE WHEN cat.nombre = 'Ropa' THEN p.monto ELSE 0 END) AS gasto_ropa
FROM clientes c
JOIN pedidos p ON c.cliente_id = p.cliente_id
JOIN productos pr ON p.producto_id = pr.producto_id
JOIN categorias cat ON pr.categoria_id = cat.categoria_id
GROUP BY c.nombre;
```

Este enfoque es extremadamente eficiente en motores como **DuckDB** o **BigQuery**, ya que evita leer la tabla múltiples veces.

5. El Filtro Post-Agregación: HAVING

Es muy común confundir `WHERE` con `HAVING`. La diferencia es el **orden de ejecución**:

1. **WHERE:** Filtra filas **antes** de que ocurra la agregación. (Ej. "Solo considerar pedidos del año actual").
2. **HAVING:** Filtra los grupos **después** de que se han calculado las agregaciones. (Ej. "Solo mostrar clientes cuyo gasto total sea mayor a $1000").

Trampa Conceptual: No puedes usar `WHERE gasto_total > 1000` porque la columna `gasto_total` no existe hasta que el `GROUP BY` termina su trabajo.

CÓDIGO

```
SELECT
 c.nombre,
 COUNT(p.pedido_id) AS num_pedidos
FROM clientes c
JOIN pedidos p ON c.cliente_id = p.cliente_id
WHERE p.fecha > '2023-01-01' -- Filtro previo
GROUP BY c.nombre
HAVING COUNT(p.pedido_id) > 5; -- Filtro sobre el resumen
```

6. Errores Comunes y Mejores Prácticas

El error de la "Explosión por Duplicate"

Si unes una tabla de `Clientes` con `Emails` (donde un cliente tiene 3 emails) y luego con `Pedidos`, ¡tus pedidos se triplicarán en la suma!

- **Solución:** Siempre verifica la cardinalidad de tus relaciones. Si una relación es 1:N, asegúrate de que estás agrupando por la clave primaria de la tabla "1" para evitar inflar los números.

El uso de Alias

En consultas multi-tabla, el uso de prefijos (`c.nombre` en lugar de `nombre`) no es opcional por etiqueta, es una necesidad de claridad. Evita la ambigüedad, especialmente cuando ambas tablas tienen columnas llamadas `created_at` o `id`.

Resumen de la Unidad

- **JOIN** expande, **GROUP BY** resume.
- Usa **LEFT JOIN** para evitar el sesgo de exclusión en tus datasets de ML.
- Toda columna no agregada en el `SELECT` debe ir al `GROUP BY`.
- **CASE WHEN** dentro de un `SUM` permite crear múltiples features categóricas en una sola query.
- **HAVING** es para filtrar resultados agregados; **WHERE** es para filtrar datos base.

Dominar estas técnicas te permite transformar datos crudos y dispersos en un "Feature Store" sólido, listo para alimentar modelos predictivos de alta precisión.

Domina los conceptos críticos de SQL Analítico

¡Bienvenido a la sesión de repaso! En el mundo del Machine Learning, la calidad y estructura de tus **features** (características) dependen directamente de tu habilidad para manipular datos en la base de datos.

Este conjunto de tarjetas está diseñado para ayudarte a memorizar y diferenciar herramientas clave: desde cómo crear ránkings y series temporales con **Window Functions**, hasta cómo procesar datos semiestructurados (**JSON**) y optimizar el rendimiento mediante el **Plan de Ejecución**.

01 ¿Qué hace la función `ROW_NUMBER()` y en qué se diferencia de los ránkings?

Asigna un número secuencial único a cada fila. Si hay empates, **no los considera**; simplemente asigna números distintos (ej. 1, 2, 3, 4). Es ideal para paginación o para forzar un ID único temporal.

02 Diferencia entre `RANK()` y `DENSE_RANK()`.

Ambas asignan el mismo número a valores idénticos. La diferencia es que `RANK()` deja un **hueco** en la secuencia después de un empate (ej. 1, 2, 2, 4), mientras que `DENSE_RANK()` no deja huecos (ej. 1, 2, 2, 3).

03 ¿Para qué sirven las funciones `LAG` y `LEAD`?

Permiten acceder a datos de filas distintas a la actual sin hacer JOINs. * **LAG(col):** Accede a la fila **anterior**. * **LEAD(col):** Accede a la fila **siguiente**. Útiles para calcular diferencias de precios o tiempo entre eventos.

04 ¿Cuál es la regla de oro al usar `GROUP BY` con múltiples columnas?

Todas las columnas presentes en el `SELECT` que **no** formen parte de una función de agregación (como SUM o AVG) **deben** estar listadas obligatoriamente en la cláusula `GROUP BY`.

05 ¿Qué es `JSON_VALUE` y cuál es su sintaxis básica?

Es una función para extraer un valor **escalar** (un solo dato como texto o número) de una cadena JSON. * Ejemplo: `JSON_VALUE(columna_json, '$.perfil.nombre')`. Recuerda que para extraer objetos o arrays completos se usa `JSON_QUERY`.

06 ¿Para qué se utiliza `ISJSON()`?

Es una función booleana que devuelve `1` (verdadero) si el texto es un formato JSON correcto y `0` si no lo es. Es vital para filtrar datos corruptos antes de intentar extraer atributos en un pipeline de ML.

07 ¿Qué problema resuelve la función `TRIM()`?

Elimina los espacios en blanco adicionales tanto al **inicio** como al **final** de una cadena de texto. Es fundamental para normalizar categorías que vienen de entradas manuales de usuarios.

08 ¿Cómo funciona la función `COALESCE` para manejar nulos?

Evalúa una lista de valores y devuelve el **primer valor que no sea NULL**. * Ejemplo: `COALESCE(telefono_movil, telefono_fijo, 'Sin contacto')`. Es más flexible que `ISNULL` porque acepta múltiples argumentos.

09 ¿Qué es un Plan de Ejecución y por qué es importante para un Analista de Datos?

Es el "mapa de ruta" generado por el optimizador de la base de datos para ejecutar una consulta. Analizarlo permite ver si el motor está haciendo un **Table Scan** (lento, lee todo) o un **Index Seek** (rápido, busca directo).

10 ¿Qué significa un "Costly Table Scan" en un reporte de optimización?

Indica que se está leyendo la tabla completa fila por fila porque no hay un índice que ayude a encontrar el dato. En tablas grandes, este es el principal culpable de que un modelo de datos tarde horas en procesarse.

PRE-ENTREGA DEL PROYECTO

Actividad evaluable · parte del proyecto integrador

Pre-entrega: Ingeniería de Features con SQL Analítico Avanzado

El objetivo de esta práctica es transformar datos crudos de comportamiento de usuarios en un dataset de entrenamiento (Features) utilizando SQL avanzado. Deberás aplicar funciones de ventana, transformaciones de agregación y filtros complejos sobre un escenario de e-commerce.

Cómo resolver esta entrega

Propósito

Consolidar la capacidad de preparar datos para Machine Learning directamente desde la base de datos, optimizando el rendimiento y asegurando la consistencia de las variables.

Pasos sugeridos

1. **Configuración del Entorno**: Crea un repositorio en GitHub para alojar tus consultas SQL y la documentación del proceso.
2. **Definición de Features**: Implementa una consulta que genere las siguientes variables por usuario: * *Recency*: Días desde la última compra (usando funciones de fecha). * *Frequency*: Cantidad de pedidos únicos. * *Monetary*: Gasto total acumulado. * *Trend*: Delta de gasto entre el último mes y el promedio histórico (usando `OVER` y `PARTITION BY`).
3. **Manejo de Nulos**: Asegúrate de que los usuarios sin actividad reciente tengan valores coherentes (por ejemplo, 0 en lugar de NULL en agregaciones).
4. **Documentación**: En el README del repositorio, explica la lógica analítica detrás de cada feature creada y por qué son útiles para un modelo de predicción de abandono (Churn).

Errores Comunes

- Olvidar el `GROUP BY` cuando se mezclan funciones de agregado con columnas dimensionales.
- No filtrar periodos de tiempo incorrectos que pueden sesgar las métricas de tendencia.

Objetivos de aprendizaje

- Implementar funciones de ventana (Window Functions) para calcular métricas de tendencia temporal.
- Construir un dataset de ingeniería de variables (Feature Engineering) mediante consultas SQL complejas.
- Realizar agregaciones avanzadas para segmentar el comportamiento de usuarios.
- Documentar la lógica de negocio aplicada en las transformaciones de datos para facilitar la reproducibilidad.

Criterios de aceptación

- El repositorio contiene un archivo .sql con la consulta principal funcional.
- La consulta utiliza al menos dos funciones de ventana (ej. RANK, LAG, LEAD o SUM OVER).
- Se incluye el cálculo de al menos tres métricas clásicas de comportamiento (ej. Recency, Frequency, Monetary).
- El README describe claramente el objetivo de cada feature generada.
- El código SQL sigue buenas prácticas de legibilidad (indentación y alias descriptivos).

Rúbrica de evaluación — Ingeniería de Features con SQL Analítico Avanzado

Rúbrica para evaluar un repositorio público de GitHub con consultas SQL orientadas a la preparación de features para Machine Learning en un contexto de e-commerce. Se valora la corrección técnica de las métricas (Recency, Frequency, Monetary y Trend), el uso adecuado de SQL analítico, el manejo de nulos y la calidad de la documentación en README como guía de interpretación y reproducibilidad.

Criterio	Descripción	Peso
Implementación de features SQL (Recency, Frequency, Monetary y Trend)	Evalúa si la consulta construye correctamente las variables solicitadas por usuario, usando funciones de fecha, agregación y funciones de ventana para producir un dataset útil para entrenamiento.	35%
Uso de SQL analítico y estructura de consulta	Evalúa el uso correcto de `GROUP BY`, agregaciones, filtros y funciones de ventana (`OVER`, `PARTITION BY`) para evitar errores lógicos y sesgos en las métricas.	25%
Manejo de nulos y consistencia de datos	Evalúa si el repositorio contempla adecuadamente usuarios sin actividad reciente o valores faltantes, sustituyendo nulos por valores coherentes cuando corresponde y manteniendo la consistencia analítica del dataset.	15%
Documentación técnica en README	Evalúa la claridad y utilidad del README para explicar la lógica de las features, su relación con churn y las decisiones analíticas tomadas en la solución.	15%
Estructura del repositorio y completitud del entregable	Evalúa si el repositorio presenta una organización propia de un proyecto real, con archivos esperados, nombres claros y un entregable completo y accesible.	10%

Total: 100 pts | Aprobación: 70 pts

MÓDULO 9

Automatización y Patrones de Datos

Implementar capas de persistencia y automatización de procesos ETL.

Mapeo ORM con SQLAlchemy 2.0 desde cero

Hasta ahora, has trabajado intensamente con SQL puro y bibliotecas como Pandas y DuckDB para manipular datos. Sin embargo, en el desarrollo de software profesional y en la ingeniería de datos, escribir consultas SQL como cadenas de texto directamente en tu código Python (**"raw SQL"**) puede volverse un caos difícil de mantener y propenso a errores.

Imagina que estás construyendo un sistema de recomendación para una plataforma de e-commerce. Tienes tablas para `Usuarios`, `Productos` y `Compras`. Cada vez que quieras interactuar con un usuario, ¿realmente quieres escribir `SELECT * FROM usuarios WHERE id = 1` y luego mapear manualmente cada columna a un diccionario o una lista? ¿Qué pasa si cambias el nombre de una columna en la base de datos? Tendrías que buscar y reemplazar ese nombre en cientos de archivos de código.

Aquí es donde entra el **ORM (Object-Relational Mapping)**, y específicamente **SQLAlchemy 2.0**, el estándar de la industria para cerrar la brecha entre el mundo de los objetos de Python y el mundo de las tablas relacionales.

1. El Concepto de ORM (Object-Relational Mapping)

El mapeo objeto-relacional es una técnica que nos permite consultar y manipular datos de una base de datos utilizando un paradigma orientado a objetos.

¿Por qué necesitamos un "traductor"?

El problema fundamental es la **desajuste de impedancia**:

- **En Python (Objetos):** Pensamos en términos de clases, atributos, herencia y métodos. Los datos son "objetos" vivos en memoria.
- **en SQL (Relacional):** Los datos son filas en tablas con tipos estrictos, claves primarias y relaciones de clave foránea.

Un ORM actúa como un puente. En lugar de recibir una tupla de datos anónima de una base de datos, el ORM te entrega una **instancia de una clase**.

Concepto en SQL	Concepto en ORM (Python)
Tabla	Clase (Class)
Fila / Registro	Objeto (Instancia de la clase)

Concepto en SQL	Concepto en ORM (Python)
Columna	Atributo de la clase
Relación (Join)	Propiedad de la clase (Atributo de navegación)

¿Por qué SQLAlchemy?

SQLAlchemy no es solo un ORM; es una "caja de herramientas de SQL". La versión **2.0** representa una evolución significativa (lanzada recientemente para modernizar la sintaxis), adoptando los tipos de Python modernos (type hinting) y un estilo de codificación mucho más limpio y seguro.

2. Definición de la Base Declarativa

En SQLAlchemy 2.0, todo comienza con una "clase base". Esta clase sirve como el registro central donde se guarda toda la información de las tablas que definas. Sin esta base, SQLAlchemy no sabría que tus clases de Python están destinadas a ser tablas de base de datos.

Para crear esta base, utilizamos `DeclarativeBase`:

CÓDIGO

```
from sqlalchemy.orm import DeclarativeBase

class Base(DeclarativeBase):
 pass
```

¿Por qué hacemos esto? Esta clase `Base` heredará una propiedad llamada `.metadata`. El objeto `metadata` es como un catálogo que contiene las definiciones de todas las tablas que crearemos a continuación. Cuando le digas a SQLAlchemy "crea todas mis tablas", él consultará este catálogo.

3. Mapeo de Clases y Columnas con SQLAlchemy 2.0

Aquí es donde ocurre la magia. Vamos a definir una clase que represente una entidad de negocio. Supongamos que estamos trabajando en un flujo de ML que analiza el comportamiento de clientes.

Estilo Moderno: `Mapped` y `mapped_column`

En la versión 2.0, SQLAlchemy utiliza anotaciones de tipo de Python para hacer el código más legible y ayudar a los editores de código (como VS Code) a darte sugerencias.

CÓDIGO

```
from typing import Optional
from sqlalchemy import String, Integer, Float
from sqlalchemy.orm import Mapped, mapped_column

class User(Base):
 __tablename__ = "user_account" # Nombre de la tabla en SQL

 # Definición de columnas
 id: Mapped[int] = mapped_column(primary_key=True)
 name: Mapped[str] = mapped_column(String(50))
 fullname: Mapped[Optional[str]] = mapped_column(String(100))
 average_spend: Mapped[float] = mapped_column(Float, default=0.0)

 def __repr__(self) -> str:
 return f"User(id={self.id!r}, name={self.name!r}, fullname={self.fullname!r})"
```

Desglosemos los componentes:

1. **`__tablename__`** : Es obligatorio. Define cómo se llamará la tabla físicamente dentro de SQLite, PostgreSQL o el motor que uses.
2. **`Mapped[int]`** : Esta es una anotación de tipo. Le dice a Python y a SQLAlchemy que este atributo será de tipo entero.
3. **`mapped_column()`** : Es la función que configura los detalles de la columna en la base de datos: * `primary_key=True` : Indica que este campo identifica de forma única a la fila. * `String(50)` : Especifica el tipo de dato SQL (VARCHAR de 50 caracteres). * `Optional[str]` : En combinación con SQLAlchemy, esto permite que la columna sea `NULL` . Si no usas `Optional` , SQLAlchemy asumirá por defecto que la columna es `NOT NULL` .

4. Diferencias entre Tipos de Python y Tipos de SQL

Un error común para principiantes es confundir el tipo de Python con el tipo de la base de datos.

- El tipo de Python (`int` , `str`) ayuda a tu código a saber cómo tratar el dato en memoria.
- El tipo de SQLAlchemy (`Integer` , `String` , `DateTime`) le dice al motor de la base de datos cómo almacenar el dato en el disco.

Python	SQLAlchemy	SQL Generado
`int`	`Integer`	`INTEGER`
`str`	`String` / `Text`	`VARCHAR` / `TEXT`
`float`	`Float`	`FLOAT` / `NUMERIC`
`bool`	`Boolean`	`BOOLEAN` / `SMALLINT`
`datetime`	`DateTime`	`TIMESTAMP`

5. Aplicaciones en el Mundo Real: Flujos de Datos y ML

¿Por qué un Data Scientist querría aprender esto en lugar de solo usar `pd.read_sql()`?

Caso: Ingesta de Datos Automatizada

Imagina un pipeline que recibe datos de una API de clima cada hora.

1. Recibes un JSON.
2. Validas los datos.
3. **ORM en acción:** Creas un objeto `WeatherRecord(temperature=22.5, city="CDMX")`.
4. Lo guardas en la base de datos.

Si usaras SQL puro, tendrías que lidiar con problemas de formateo de strings, riesgos de SQL Injection y transformaciones manuales. Con SQLAlchemy, simplemente trabajas con objetos Python.

Caso: Catálogo de Modelos de ML

Puedes usar SQLAlchemy para crear una tabla que registre cada experimento de Machine Learning que realices:

CÓDIGO

```
class MLModel(Base):
 __tablename__ = "model_registry"
 id: Mapped[int] = mapped_column(primary_key=True)
 model_name: Mapped[str] = mapped_column(String(100))
 accuracy: Mapped[float] = mapped_column(Float)
 parameters: Mapped[str] = mapped_column(String) # JSON como string
 created_at: Mapped[datetime] = mapped_column(insert_default=func.now())
```

Esto te permite buscar modelos con una precisión superior al 90% usando sintaxis de Python pura, integrándose perfectamente en cualquier aplicación web o script de automatización.

6. Errores Comunes y Mejores Prácticas

El error del "Old Style"

Muchos tutoriales en internet usan la versión 1.4 de SQLAlchemy (donde se usaba `Column(Integer, primary_key=True)`). Aunque funciona, en la versión 2.0 es preferible usar el estilo `Mapped` y `mapped_column` para aprovechar el tipado estático y las nuevas optimizaciones de rendimiento.

No olvidar la Clave Primaria

Toda clase mapeada **debe** tener al menos una columna marcada como `primary_key=True`. Si lo olvidas, SQLAlchemy lanzará un error porque no sabrá cómo rastrear de forma única ese objeto en la base de datos.

El nombre de la tabla

Asegúrate de que `__tablename__` sea único en tu base de datos. Es una convención común usar nombres en minúsculas y en plural (ej. `users`, `products`) o con guiones bajos (`sales_orders`).

Síntesis de la Unidad

En esta lección hemos sentado las bases del Módulo 8:

1. Entendimos que el **ORM** es un traductor entre Objetos y Tablas.
2. Aprendimos a definir una **clase Base** que orquestará nuestras entidades.
3. Vimos cómo usar **SQLAlchemy 2.0** para definir tablas mediante clases, usando `Mapped` para los tipos y `mapped_column` para las restricciones.
4. Identificamos que esta técnica es vital para crear pipelines de datos robustos, seguros y fáciles de mantener.

En la siguiente unidad, aprenderemos a conectar este modelo con una base de datos real (como SQLite) y a realizar nuestras primeras operaciones de lectura y escritura de forma profesional.

Patrones seguros para ETL y validación de esquemas

En el mundo del Machine Learning y la analítica de datos, solemos decir que "la calidad de la entrada determina la calidad de la salida". Pero hay un paso previo que es igual de crítico: **la seguridad y la integridad del movimiento de esos datos**. Si tu flujo de datos (ETL) es vulnerable o si permite que datos corruptos "contaminen" tu base de datos analítica, no solo fallarán tus modelos, sino que podrías estar comprometiendo información sensible de clientes o del negocio.

Imagina que estás construyendo una tubería para transportar agua desde un manantial hasta una ciudad. No basta con que el agua llegue; necesitas que la tubería sea resistente a filtraciones (seguridad), que tenga filtros para que no pase arena (validación) y que solo las personas autorizadas puedan abrir las válvulas (mínimo privilegio). En esta unidad, aprenderás a construir esas "tuberías de datos" de forma profesional y segura.

1. Patrones seguros ETL: Diseñando con desconfianza

El patrón de 'Staging Area' asegura que los datos se validen en un entorno aislado antes de ser promovidos a la base de datos de producción.

Un proceso **ETL (Extract, Transform, Load)** es, por definición, un puente. Y los puentes son puntos críticos de seguridad. Cuando movemos datos desde una API externa hacia nuestro almacenamiento local o en la nube, existen riesgos de interceptación o acceso no autorizado.

El concepto de "Datos en Movimiento" vs. "Datos en Reposo"

Para un Data Scientist, es vital distinguir estos dos estados: - **Datos en Reposo (At Rest):** Datos guardados en tu disco duro, en SQLite o en un bucket de S3. Se protegen con cifrado de disco. - **Datos en**

Movimiento (In Transit): Aquí es donde vive el ETL. Son los datos viajando por la red. Se protegen mediante protocolos como **TLS/SSL** (el "HTTPS" que ves en el navegador).

Patrón de Área de Transición (Staging Area)

Un error común de principiante es cargar datos directamente desde la fuente a la tabla final de producción. El patrón seguro recomienda el uso de una **Staging Area**. 1. **Extracción:** Se descargan los datos a un entorno aislado (una tabla temporal o un archivo Parquet cifrado). 2. **Validación y Limpieza:** Se ejecutan los tests de seguridad y esquema. 3. **Carga (Promoción):** Solo si todo es correcto, se mueven a la tabla final.

Este aislamiento evita que un error en el origen bloquee o corrompa tu base de datos principal.

2. Principio de mínimo privilegio en ETL

Otorgar solo los permisos necesarios a los procesos ETL previene accesos no autorizados y minimiza el daño en caso de brechas de seguridad.

Este es uno de los pilares de la seguridad informática aplicado a datos. El **Principio de Mínimo Privilegio (PoLP)** establece que una identidad (ya sea un usuario o un script de Python) debe tener solo los permisos estrictamente necesarios para realizar su función, y nada más.

El error del "Usuario Administrador"

Muchos estudiantes conectan su script de Python a la base de datos usando el usuario `root` o `admin`. * **El riesgo:** Si alguien logra inyectar código malicioso en tu script o si accidentalmente escribes un `DROP TABLE`, el script tendrá permiso para borrar toda la base de datos. * **La solución:** Crear usuarios específicos para el proceso ETL.

Función	Permisos Sugeridos	¿Por qué?
Extractor	`SELECT` (Solo lectura)	Solo necesita leer de la fuente, no cambiar nada.
Transformador	`SELECT` e `INSERT` en tablas temporales	Lee datos crudos y escribe resultados intermedios.
Cargador	`INSERT` y `UPDATE`	Necesita llevar los datos finales al destino.

Ejemplo Industrial: Empresas financieras como JPMorgan utilizan "Service Accounts" (Cuentas de Servicio). El script de ETL no tiene una contraseña que un humano use, sino un token de acceso que solo sirve para insertar datos en una tabla específica durante una ventana de tiempo seleccionada.

3. Validación automática de esquemas

¿Qué pasa si tu código de ML espera que la columna `precio` sea un número decimal, pero la API de origen de pronto empieza a enviar el texto `"Consultar precio"`? El sistema fallará, o peor aún, el modelo de ML entrenará con datos basura.

La **validación de esquema** es el proceso de verificar que los datos cumplen con el "contrato" esperado antes de ser procesados.

¿Qué es un Esquema? (Schema)

Es el plano de construcción de tus datos. Define: 1. **Tipos de datos:** ¿Es entero, flotante, texto o booleano? 2. **Campos obligatorios:** ¿Puede faltar el email del cliente? 3. **Restricciones:** ¿La edad puede ser negativa? ¿El país debe estar en una lista predefinida?

Herramientas de Validación Profesional

En el ecosistema Python, existen dos estándares para automatizar esto:

A. JSON Schema

Si recibes datos de una API (como vimos con `requests`), puedes definir un archivo JSON que describa la estructura válida.

CÓDIGO

```
{
 "type": "object",
 "properties": {
 "id": {"type": "integer"},
 "email": {"type": "string", "format": "email"},
 "score": {"type": "number", "minimum": 0}
 },
 "required": ["id", "email"]
}
```

Si un registro no cumple esto, el valider lo rechaza antes de que toque tu base de datos.

B. Pydantic (El estándar moderno de ML)

Pydantic utiliza clases de Python para validar datos. Es extremadamente rápido y se integra perfectamente con herramientas que ya conoces. * **Cómo funciona:** Defines una clase que representa tu dato. Si al intentar crear un objeto los datos no coinciden, Pydantic lanza una excepción clara.

4. Protección de datos: En tránsito y en reposo

Como analistas de datos, manejamos a menudo **PII (Personally Identifiable Information)**, como nombres, correos o direcciones. Proteger esto no es solo buena práctica, es ley (GDPR o leyes locales de protección de datos).

Técnicas de Protección en el ETL

1. **Data Masking (Enmascaramiento):** Consiste en ocultar partes de los datos sensibles durante el proceso. Por ejemplo, transformar `user_email@gmail.com` en `u***l@gmail.com` antes de guardarlo en la base de datos de entrenamiento.

2. **Hashing:** Transformar un dato (como un ID de usuario) en una cadena de caracteres irreconocible pero única. Esto permite "unir" tablas sin saber quién es la persona real.
3. **Cifrado con Variables de Entorno:** Nunca, bajo ninguna circunstancia, escribas contraseñas o llaves de cifrado directamente en tu código (`hardcoding`). Como aprendiste en el módulo de APIs, usa archivos `.env` .

Un error frecuente: Pensar que, porque la base de datos está en tu computadora, es segura. Si subes ese archivo de base de datos a un repositorio de GitHub público por error, tus datos en reposo ahora son públicos. El cifrado y el uso de `.gitignore` son tus mejores aliados.

5. Errores comunes y mejores prácticas

Error 1: Confiar ciegamente en la fuente de datos

Muchos analistas asumen que si la API funcionó ayer, funcionará igual hoy. Las APIs cambian, las columnas desaparecen y los formatos varían. * **Mejor práctica:** Implementa una validación "Fail-Fast" (fallo rápido). Si el esquema no coincide, detén el proceso inmediatamente en lugar de procesar datos erróneos.

Error 2: El "Select *" en ETL

Usar `SELECT *` en un proceso de extracción es peligroso. Si el administrador de la fuente agrega una columna con datos sensibles (por ejemplo, `contraseña_hash`), tu proceso ETL la extraerá y la llevará a tu área analítica sin que te des cuenta. * **Mejor práctica:** Nombra explícitamente las columnas que necesitas. Esto se llama **"Allow-listing"**.

Error 3: No tener logs de errores (Audit Logging)

Cuando un registro falla la validación, ¿a dónde va? Si simplemente lo ignoras, perderás visibilidad sobre la calidad de tus datos. * **Mejor práctica:** Crea una "tabla de errores" o un archivo de log donde guardes los registros rechazados y el motivo. Esto te permitirá hablar con los ingenieros de la fuente de datos para reportar el problema.

Resumen de la Unidad

- **ETL Seguro:** Siempre usa áreas de transición (staging) y protege los datos en movimiento con TLS.
- **Mínimo Privilegio:** Crea usuarios en tu base de datos (DuckDB, SQLite o PostgreSQL) que solo tengan los permisos necesarios para la tarea actual.
- **Validación de Esquema:** No es opcional. Usa herramientas como Pydantic para asegurar que tus modelos de Machine Learning reciban datos limpios y con el formato correcto.
- **Protección de PII:** Enmascara o anonimiza datos sensibles antes de que lleguen a tu entorno de análisis.

En la siguiente unidad, daremos un paso más hacia la automatización profesional aprendiendo sobre "Hooks", que nos permitirán disparar estas validaciones de forma automática cada vez que intentemos mover datos.

Automatización y Patrones de Datos en Entornos de Ingeniería

Imagina que eres el responsable de datos en una tienda de comercio electrónico que procesa miles de pedidos por hora. Cada vez que alguien compra un producto, deben ocurrir varias cosas: el inventario debe actualizarse, el equipo de logística debe recibir una alerta de envío, el departamento de marketing necesita registrar la preferencia del cliente para futuras campañas y el equipo de finanzas debe generar una factura.

Si tuvieras que hacer esto manualmente por cada pedido, no solo necesitarías un ejército de personas, sino que el margen de error humano sería catastrófico. Un dígito mal puesto en el inventario y estarías vendiendo productos que no existen. Aquí es donde entra la **automatización** y la identificación de **patrones de datos**.

En esta unidad, no solo aprenderemos qué son estos conceptos, sino cómo se convierten en el "sistema nervioso" de cualquier empresa moderna basada en datos. Aprenderás a identificar procesos repetitivos, a entender cómo los datos siguen reglas lógicas y por qué automatizar no es solo "ahorrar tiempo", sino garantizar la integridad de la información.

1. ¿Qué es realmente la Automatización de Datos?

Para un principiante, la automatización a menudo suena como "robots haciendo el trabajo de humanos". En el contexto de la Ciencia de Datos y los procesos ETL (Extracción, Transformación y Carga), la automatización es la **configuración de sistemas que ejecutan tareas repetitivas sin intervención humana directa, basándose en disparadores (triggers) predefinidos.**

El problema de la intervención manual

Antes de la automatización, los científicos de datos pasaban el 80% de su tiempo "limpiando" y "moviendo" datos manualmente (un proceso conocido como *data wrangling*). Esto presentaba tres grandes problemas: 1. **Latencia:** Los datos no estaban listos cuando se necesitaban. 2. **Inconsistencia:** Diferentes personas aplicaban diferentes criterios de limpieza. 3. **Inescalabilidad:** Si el volumen de datos crecía un 100%, el esfuerzo humano también debía crecer proporcionalmente.

La solución: El flujo automatizado

Automatizar significa crear un camino por el cual el dato viaja desde su origen hasta su destino, sufriendo transformaciones en el camino de forma autónoma.

Ejemplo real: Google Classroom Considera cómo un profesor usa herramientas automáticas. Cuando un estudiante envía un cuestionario digital (Google Forms): * **Disparador:** El clic en "Enviar". * **Acción Automatizada:** El sistema compara las respuestas con la clave, calcula la nota y la publica en el tablón (Google Classroom). * **Resultado:** El profesor no "corrigió"; el profesor diseñó el sistema que corrige automáticamente.

2. Patrones de Datos: Las Reglas del Juego

La automatización no puede existir sin **patrones**. Un patrón de datos es una regularidad, una estructura o una regla lógica que se repite en un conjunto de información. Si los datos fueran caóticos y nunca siguieran una forma predecible, no podríamos automatizarlos.

Tipos de Patrones que podemos automatizar

A menudo se piensa que los patrones son solo secuencias numéricas (como 2, 4, 6...), pero en la industria, los patrones son lógicos y estructurales:

1. **Patrones de Texto:** * *Ejemplo:* "Si el cuerpo del correo de soporte contiene la palabra 'Reembolso', clasifícalo como 'Prioridad Alta'". * Aquí, la palabra clave es el patrón que activa una automatización de clasificación.
2. **Patrones Temporales:** * *Ejemplo:* "Cada viernes a las 11:59 PM, genera un respaldo de la base de datos de ventas de la semana". * El patrón es el ciclo de tiempo (recurrencia).
3. **Patrones de Eventos (Event-driven):** * *Ejemplo:* "Si un sensor de temperatura en una fábrica supera los 80°C, envía una alerta al equipo de mantenimiento". * El patrón es el cruce de un umbral específico.

La Analogía de la Cocina Automatizada

Para entender mejor esta relación, imagina una cocina industrial: * **La Automatización** es la cinta transportadora y las máquinas que cortan vegetales sin que un chef las toque. * **Los Patrones de Datos** son las "recetas" o reglas de ruteo. La receta dice: *"Si la orden es una ensalada, envía el plato a la estación de frío; si es un chuletón, envíalo a la parrilla"*.

Sin la receta (el patrón), la cinta transportadora (la automatización) llevaría la carne al refrigerador y la lechuga al fuego, creando un desastre. **Automatizar sin entender el patrón de tus datos es simplemente acelerar el caos.**

3. Aplicaciones del Mundo Real

Veamos cómo grandes empresas utilizan la combinación de automatización y patrones para operar a escala global.

Caso 1: Shopify y el Inventario Inteligente

Shopify es una plataforma que permite a millones de personas tener sus propias tiendas online. Su sistema de automatización de datos funciona bajo patrones de flujo: * **Escenario:** Una tienda vende 50 camisetas. El sistema de Shopify detecta que el inventario ha bajado de 100 a 50. * **El Patrón:** `IF (Stock < Umbral_Seguridad) THEN (Notificar_Proveedor) AND (Cambiar_Estado_Web)`. * **Impacto:** El dueño de la tienda no tiene que estar revisando el almacén cada hora; los datos "hablan" y actúan por él.

Caso 2: Zapier y el "Pegamento" de Internet

Zapier es una herramienta que permite conectar diferentes aplicaciones (como Slack, Gmail y Excel) sin saber programar. Se basa enteramente en **Patrones de Intercambio de Datos**. * **Uso común:** Un equipo de marketing recibe leads (clientes interesados) a través de un formulario de Facebook. * **Automatización:** Zapier detecta el patrón del nuevo lead y automáticamente: 1. Lo agrega a una fila en una hoja de Google Sheets. 2. Envía un mensaje al canal del equipo de ventas en Slack. 3. Envía un correo electrónico de "Bienvenida" al cliente. * **Resultado:** Se ahorran horas de copiar y pegar información entre aplicaciones.

4. Diferenciando Conceptos: No te confundas

Es común que los principiantes mezclen términos. Aquí aclaramos las confusiones más frecuentes:

Error 1: "Automatización es igual a Inteligencia Artificial (IA)"

Realidad: No necesariamente. La mayoría de la automatización industrial es "determinística", es decir, sigue reglas fijas de `Si ocurre A, entonces haz B`. La IA, por el contrario, intenta "aprender" el patrón por sí misma cuando las reglas no son tan claras. * *Automatización básica:* Si el cliente gasta más de $1000, dale 10% de descuento. (Regla fija). * *IA:* Analizar el comportamiento del cliente para predecir si está a punto de dejar de comprar y ofrecerle un incentivo personalizado.

Error 2: "Automatizar reemplaza a los humanos"

Realidad: En ciencia de datos, la automatización se ve como un modelo de **"Devolución de Valor"**. Al automatizar el movimiento tedioso de datos entre carpetas y bases de datos, el profesional de datos queda libre para tareas de alto valor: como interpretar los resultados, encontrar insights estratégicos y tomar decisiones que las máquinas no pueden tomar.

Error 3: "Necesito ser un experto programador para automatizar"

Realidad: Gracias a las herramientas **Low-Code** (herramientas que requieren poco código) como Microsoft Power Automate o Zapier, cualquier persona con una mentalidad lógica puede configurar patrones de datos complejos mediante interfaces visuales.

5. Mejores Prácticas y Errores Comunes

Para que tus procesos de automatización sean exitosos, sigue estos consejos de expertos:

Buena Práctica	Por qué importa
Validar antes de automatizar	Si tus datos de entrada están mal (ej. nombres con números), la automatización repartirá errores por todo el sistema a gran velocidad.
Mantener un registro (Logs)	Siempre guarda un historial de qué se automatizó y cuándo. Si algo falla, necesitas saber en qué paso del patrón ocurrió el error.
Diseñar para el fallo	¿Qué pasa si el servidor de correos se cae? Tu automatización debe tener un plan B o una notificación de error "humano".

El error más común: La "Automatización Frágil"

Ocurre cuando creas una automatización que depende de que los datos sean *exactamente* iguales siempre. * **Ejemplo de error:** Crear una automatización que lee la columna "B" de un Excel. Si alguien añade una columna nueva al principio, la columna "B" se mueve a la "C", y tu proceso automatizado empezará a leer datos equivocados (como fechas en lugar de precios). * **Solución:** Los patrones deben ser robustos. En lugar de decir "Lee la columna B", el patrón debería ser "Busca la columna que tenga el encabezado 'Precio'".

6. Síntesis y Conexión con el Próximo Nivel

Hemos visto que la **automatización** es el motor que mueve los datos sin intervención manual, y los **patrones** son el mapa que ese motor sigue para saber a dónde ir. Sin patrones claros, la automatización es ciega; sin automatización, los patrones son inútiles para la escala masiva que requiere la ciencia de datos actual.

En las unidades anteriores, aprendiste a mapear objetos con SQLAlchemy y a validar esquemas de forma segura. Estos son los cimientos: la validación asegura que el dato tenga el "formato de patrón" correcto antes de que las herramientas de automatización que veremos a continuación (como los motores de sesiones y los hooks) tomen el control.

En las siguientes unidades de este módulo, pasaremos a la acción técnica: veremos cómo los **Motores y Sesiones** permiten que estos patrones ocurran de forma segura dentro de una base de datos corporativa, asegurando que si una parte de la automatización falla, los datos no se corrompan.

Puntos Clave para Recordar: * La automatización libera al humano de tareas repetitivas y reduce el error. * Los patrones de datos son las reglas lógicas (SI esto, ENTONCES aquello) que guían la automatización. * Existen herramientas como Zapier y Shopify que ya aplican estos conceptos a gran escala. * Para automatizar bien, primero se debe asegurar que el dato es de calidad (basura entra, basura sale).

Sesiones, motores y operaciones seguras con ORM

En la unidad anterior, aprendimos cómo trazar un mapa entre nuestras clases de Python y las tablas de SQL (Mapeo ORM). Sin embargo, el mapa por sí solo no "mueve" los datos. Para interactuar realmente con la base de datos, necesitamos entender la infraestructura de comunicación de SQLAlchemy.

El Corazón de la Conexión: El Engine

El **Engine** (Motor) es el objeto central que permite a SQLAlchemy hablar con la base de datos. Es el encargado de: 1. **Conocer el dialecto**: ¿Estamos hablando con SQLite, PostgreSQL o MySQL? Cada uno tiene sus "modismos". 2. **Manejar el Connection Pool**: Gestionar eficientemente la apertura y cierre de conexiones para no saturar el servidor.

Se crea una sola vez para toda la aplicación:

CÓDIGO

```
from sqlalchemy import create_engine
engine = create_engine("sqlite:///datos.db")
```

El Espacio de Trabajo: La Sesión (Session)

Si el *Engine* es el cable que conecta la luz, la **Sesión** es el interruptor que usas para realizar tareas específicas. En el flujo de trabajo de un Data Scientist, la sesión representa una **unidad de trabajo**.

Imaginalo como una "zona de borrador": 1. Añades objetos a la sesión (`session.add()`). 2. Modificas sus atributos. 3. Si todo está correcto, ordenas "guardar todo" con un `session.commit()` . 4. Si algo sale mal, puedes deshacer todo con un `session.rollback()` .

Operaciones Seguras en SQLAlchemy 2.0

Antiguamente, SQLAlchemy usaba un estilo llamado "Query API". En la versión 2.0, se ha unificado el lenguaje para que sea más explícito y seguro mediante el uso de `select()` .

- **Insertar**: Creamos una instancia de nuestra clase y la añadimos.
- **Consultar**: Utilizamos la función `select(Clase).where(Criterio)` .
- **Actualizar**: Buscamos un objeto, cambiamos su valor en Python y hacemos commit. ¡SQLAlchemy detecta el cambio automáticamente!

Errores comunes de principiantes

1. **Olvidar el `commit()`** : Es muy frecuente agregar datos y pensar que ya están guardados. Sin commit, los cambios desaparecen al cerrar la sesión.
2. **No usar el bloque `with`** : Abrir una sesión y no cerrarla puede dejar conexiones "colgadas", bloqueando la base de datos. Usa siempre `with Session(engine) as session:` para asegurar el cierre automático.

Hooks para automatizar validaciones y tareas de datos

Hooks y la Automatización del Flujo de Datos

Como analista o científico de datos, gran parte de tu éxito depende de la **confiabilidad de tus procesos**. Imagina que subes un cambio a tu script de limpieza y, sin darte cuenta, introduces un error que borra la mitad de tus registros. O peor aún, que un sistema externo te envía datos corruptos y tu pipeline de Machine Learning intenta entrenar con ellos.

Aquí es donde entran los **hooks** (ganchos). Un hook es esencialmente un "disparador" que permite ejecutar código adicional de manera automática cuando ocurre un evento específico. Es como poner un detector en una línea de ensamblaje: si una pieza no cumple con el estándar, el hook detiene la línea o la corrige antes de continuar.

---

01 ¿Qué es un hook en el contexto de automatización?

Es un punto de intervención en un flujo de trabajo donde se puede **ejecutar código automáticamente** al ocurrir un evento específico, sin modificar la lógica principal del programa.

02 ¿Para qué sirven los Git Hooks en un proyecto de datos?

Son scripts que se ejecutan localmente en tu computadora ante eventos de Git. El más común en datos es el **pre-commit**, que puede validar la calidad del código o de los datos antes de permitir que se guarden los cambios.

03 ¿Qué diferencia a un Webhook de una API tradicional?

Es una comunicación **HTTP de sistema a sistema**. Un sistema (emisor) envía datos a una URL (receptor) automáticamente cuando sucede algo, como la llegada de un nuevo registro a una API o base de datos externa.

04 ¿Cuál es la función de un hook pre-commit en un pipeline de ML?

Se utiliza para **validar esquemas** y formatos. Si el script de validación detecta que una columna falta o tiene tipos de datos erróneos, el commit falla y protege la integridad del repositorio.

05 ¿Cuál es la diferencia técnica entre un Trigger de SQL y un Hook de software?

Un **trigger** es un objeto de la base de datos que reacciona a cambios en las tablas (INSERT, UPDATE), mientras que un **hook** suele ser un script externo al motor de base de datos que reacciona a eventos de software o procesos.

06 ¿Cómo ayudan los hooks de herramientas como GitHub Actions al Data Science?

Permiten implementar **CI/CD para datos**. Por ejemplo, puedes configurar que cada vez que se haga un `push` con datos nuevos, se ejecute automáticamente un re-entrenamiento del modelo o se actualice un reporte.

07 ¿A qué se refiere el término "automatización dirigida por eventos" (event-driven)?

Se refiere a la automatización **reactiva**: el código no se ejecuta solo porque sí, sino como respuesta directa a una acción previa (un "disparo" o trigger).

08 ¿Cuál es el beneficio principal de automatizar validaciones en un flujo ETL?

Evitar la **degradación de datos** (data drift). Al automatizar pruebas de calidad en cada paso, aseguras que los datos que alimentan al modelo siempre cumplan con los requisitos técnicos definidos.

PRE-ENTREGA DEL PROYECTO

Actividad evaluable · parte del proyecto integrador

Pre-entrega: Persistencia y la integración de scripts ETL

El objetivo de este ejercicio es implementar un flujo de trabajo que automatice la persistencia de datos desde un origen transaccional hacia una base de datos analítica, asegurando que el proceso sea repetible y esté documentado. Esta práctica consolida el conocimiento sobre la capa de persistencia y la integración de scripts ETL (Extract, Transform, Load).

Cómo resolver esta entrega

Qué debes construir

Debes crear un repositorio de código que contenga un script de automatización (Python o SQL) encargado de:

1. Conectar con una fuente de datos (puede ser un dataset local o una base de datos remota).
2. Realizar una transformación básica de limpieza o agregación.
3. Cargar el resultado en una base de datos local (SQLite o MySQL).
4. Un archivo README.md que explique cómo ejecutar el flujo.

Pasos sugeridos

1. **Inicialización**: Crea un repositorio en GitHub con un archivo `.gitignore` adecuado.
2. **Persistencia**: Diseña el esquema de la tabla de destino donde se guardarán los datos transformados.
3. **Automatización**: Escribe el script que realice la extracción y la carga. Asegúrate de tratar errores comunes de conexión.
4. **Documentación**: En el README, describe el patrón de datos utilizado y las instrucciones de configuración del entorno.

Conexión con el contenido

Este entregable aplica directamente los conceptos de 'Capas de Persistencia' y 'Automatización de Procesos ETL' del módulo, permitiéndote pasar de la ejecución manual de consultas a un proceso de datos orquestado por código.

Objetivos de aprendizaje

- Implementar un proceso de persistencia de datos automatizado mediante scripts.
- Diseñar una estructura de tablas adecuada para el almacenamiento de datos procesados.
- Configurar un flujo ETL básico que integre extracción, transformación mínima y carga.
- Documentar procesos técnicos para asegurar la reproducibilidad del pipeline de datos.

Criterios de aceptación

- El repositorio contiene un script funcional que ejecuta la carga de datos sin errores de sintaxis.
- El README incluye una sección de 'Requisitos' y 'Pasos para ejecutar' clara y concisa.
- Se incluye un archivo SQL o una sección en el script que defina la creación de la tabla de destino (DDL).
- El historial de commits muestra un desarrollo incremental y descriptivo del proyecto.
- El proceso de carga implementa una lógica para evitar la duplicidad de datos (por ejemplo, IF NOT EXISTS o limpieza previa).

Rúbrica de evaluación: Automatización ETL con persistencia de datos

Esta rúbrica evalúa un repositorio que implementa un flujo ETL básico y automatizado, con persistencia de datos en una base local, estructura de proyecto clara y documentación suficiente para reproducir la solución. Está diseñada para un nivel beginner y valora tanto la funcionalidad como la organización técnica y la claridad del README.

Criterio	Descripción	Peso
Funcionalidad del flujo ETL	Evalúa si el repositorio contiene un script que conecta con una fuente de datos, extrae información, aplica una transformación básica y carga el resultado en una base de datos local de forma coherente.	35%
Diseño de persistencia y esquema de datos	Evalúa la calidad del diseño de la tabla o esquema de destino, incluyendo si el modelo de datos es apropiado para la información transformada y si la persistencia refleja buenas decisiones básicas de organización.	20%
Automatización, manejo de errores y ejecución reproducible	Evalúa si el script está orientado a la automatización real del proceso, incluyendo organización del código, tratamiento básico de errores comunes y posibilidad de ejecutar el flujo de manera repetible.	25%
Estructura del repositorio y prácticas básicas de proyecto	Evalúa la organización general del repositorio como proyecto de código real: estructura de carpetas, uso de .gitignore, separación de archivos y presentación de artefactos relevantes.	12%
Documentación y guía de ejecución	Evalúa la calidad del README.md como guía para entender el patrón de datos, preparar el entorno y ejecutar el flujo ETL sin depender de explicaciones externas.	8%

Total: 100 pts | Aprobación: 70 pts

MÓDULO 10

ML Lab I: Preprocesamiento y Pipelines

Ejecutar preprocesamiento avanzado, diseñar pipelines de Scikit-Learn y preparar un reporte de limpieza y transformación reutilizando los datos persistidos anteriormente.

EDA profundo: calidad, distribución y relaciones

Imagínate que acabas de comprar una casa antigua para renovarla y convertirla en una residencia inteligente. ¿Empezarías a pintar las paredes y a instalar sensores de última generación sin antes revisar si hay goteras, si los cimientos están firmes o si la instalación eléctrica soporta la carga? Probablemente no. En el mundo de la Ciencia de Datos, el **Análisis Exploratorio de Datos (EDA)** es esa inspección profunda de la estructura.

Muchos principiantes cometen el error de saltar directamente al entrenamiento del modelo de Machine Learning (ML) apenas cargan un dataset. Sin embargo, un modelo es tan bueno como los datos que lo alimentan. Si entrenas un algoritmo con datos que tienen sesgos, errores de captura o valores extremos no analizados, el resultado será un modelo "ciego" o, peor aún, engañoso. En esta unidad, aprenderás a realizar un EDA profundo que te permitirá tomar decisiones informadas sobre cómo preprocesar tus datos para maximizar el rendimiento de tus futuros modelos.

1. El Análisis Exploratorio de Datos (EDA) profundo

El EDA no es un paso único en una lista de tareas; es un **ciclo iterativo** de investigación. Mientras que un EDA básico se limita a ver las primeras filas y contar nulos, un **EDA profundo** busca entender la naturaleza de las variables, cómo se comportan entre sí y qué historias nos están contando los datos.

¿Por qué "profundo"?

El término "profundo" se refiere a una actitud crítica. No nos basta con saber que una columna tiene un promedio de 50. Queremos saber:

- ¿Representa ese promedio a la mayoría de los usuarios o está inflado por unos pocos clientes "VIP"?
- ¿Por qué faltan datos en la columna "Ingresos"? ¿Es un error técnico o es porque los usuarios con salarios bajos prefieren no declarar esa información?
- ¿Existe una relación lógica entre la edad de un cliente y su volumen de compras, o estamos viendo una coincidencia estadística?

El flujo mental del EDA

1. **Observación:** Ver la estructura (filas, columnas, tipos de datos).
2. **Cuestionamiento:** ¿Por qué hay valores cero en una columna de "Presión Arterial"? (Claramente un error de captura).

3. **Visualización:** Convertir tablas en gráficos para detectar patrones que el ojo humano no ve en filas de números.
4. **Hipótesis:** "Parece que los datos nulos solo ocurren en los registros de fines de semana".

2. Estadísticos descriptivos: Más allá del promedio

Para describir un dataset, necesitamos herramientas que nos digan dónde está el "centro" de los datos y qué tan "esparcidos" están.

Medidas de Tendencia Central

- **Media (Promedio):** Es la suma de todos los valores dividida por el total. Es muy útil, pero **peligrosa**. Si tienes 9 personas que ganan $1,000 y una que gana $1,000,000, el promedio será de $100,900. ¿Representa eso la realidad del grupo? No.
- **Mediana:** Es el valor que queda justo en medio cuando ordenas los datos. En el ejemplo anterior, la mediana sería $1,000. Es mucho más "robusta" ante valores extremos.
- **Moda:** El valor que más se repite. Crucial para variables categóricas (como "Color favorito" o "Estado Civil").

Medidas de Dispersión

- **Desviación Estándar:** Nos dice qué tan lejos, en promedio, están los puntos de la media. Una desviación estándar alta significa que los datos están muy dispersos; una baja, que están agrupados cerca del promedio.
- **Rango Intercuartílico (IQR):** Se centra en el 50% central de los datos (la diferencia entre el percentil 75 y el 25). Es la base para detectar outliers de forma estadística.

La forma de la distribución: Asimetría y Curtosis

No todos los datos siguen una "campana de Gauss" perfecta (distribución normal).

- **Asimetría (Skewness):** Si la "cola" de la distribución es muy larga hacia la derecha, decimos que tiene asimetría positiva (típico en salarios).
- **Curtosis:** Indica qué tan "puntiaguda" es la distribución y cuánta densidad hay en las colas.

Ejemplo real: Amazon y el inventario Amazon no puede basar sus pedidos de inventario solo en el "promedio" de ventas diarias. Si el promedio de venta de un libro es 10 ejemplares al día, pero la desviación estándar es de 50, significa que hay días de 0 ventas y días de 200. Ignorar la dispersión causaría que Amazon se quedara sin stock en los días pico.

3. Detección de outliers y anomalías

Un **outlier** (o valor atípico) es una observación que se desvía tanto de las otras que levanta sospechas de que fue generada por un mecanismo diferente.

¿Cómo detectarlos?

1. **Visualmente:** El **Boxplot** (diagrama de caja) es la herramienta reina. Cualquier punto que quede fuera de los "bigotes" de la caja es considerado un candidato a outlier.
2. **Estadísticamente (Regla del IQR):** * Calculamos . * Límite Inferior . * Límite Superior . * Cualquier valor fuera de estos límites es un outlier.
3. **Z-Score:** Indica a cuántas desviaciones estándar se encuentra un punto de la media. Típicamente, un Z-score mayor a 3 o menor a -3 se considera una anomalía.

El dilema del científico de datos: ¿Borro o conservo?

Un error frecuente de principiante es borrar todos los outliers de inmediato. ¡Error! Primero debemos entender el **porqué**:

- **Error de entrada de datos:** Si una persona tiene 250 años de edad, es un error. **Se corrige o se borra.**
- **Evento genuino pero raro:** En la detección de fraude bancario, la transacción de $50,000 de alguien que suele gastar $10 es un outlier, pero es precisamente lo que queremos detectar. **Se conserva.**
- **Variabilidad natural:** En el precio de las casas, una mansión es un outlier respecto a departamentos pequeños, pero no es un error. **Se puede tratar por separado.**

4. Relaciones entre variables: Correlación no es Causalidad

En el preprocesamiento, necesitamos saber si dos variables están relacionadas. Si dos columnas dicen casi lo mismo (ej. "Precio en USD" y "Precio en ARS"), una es redundante y puede confundir al modelo.

Coeficiente de Correlación de Pearson

Mide la relación lineal entre dos variables numéricas entre -1 y 1.

- **1:** Relación positiva perfecta (si uno sube, el otro también).
- **-1:** Relación negativa perfecta (si uno sube, el otro baja).
- **0:** No hay relación lineal.

¡Alerta de error común! Existe una famosa analogía en estadística: "El consumo de helados y los ataques de tiburones están altamente correlacionados". ¿Significa esto que comer helado atrae tiburones? No. Ambos aumentan porque es **verano**. Esto se llama una "variable de confusión". Como científicos de datos, debemos usar el EDA para explorar estas relaciones sin saltar a conclusiones apresuradas de causa y efecto.

5. Aplicaciones en la industria: Casos reales

Caso 1: Institución Financiera (FICO)

Al analizar solicitudes de crédito, el EDA profundo revela que los valores nulos en "Ingresos Mensuales" no son aleatorios. A menudo, las personas con ingresos muy altos o muy bajos omiten este dato. Si un analista simplemente borrara las filas con nulos, estaría eliminando a los clientes más rentables o a los de mayor riesgo, sesgando completamente el modelo de riesgo.

Caso 2: Salud (IBM Watson Health)

En el procesamiento de registros médicos, un EDA detectó que ciertos sensores de ritmo cardíaco daban lecturas de "0" cuando perdían conexión. Un modelo de ML sin preprocesamiento podría interpretar que el paciente ha fallecido, cuando en realidad es un problema de calidad de datos (outlier técnico).

6. Síntesis y Mejores Prácticas

Hacer un EDA profundo significa ser un detective de los datos. Antes de avanzar a la limpieza técnica o a los transformadores de Scikit-Learn (que verás en las próximas unidades), asegúrate de:

- **No confiar solo en `df.describe()`:** Los números pueden ocultar distribuciones bimodales (dos picos) que solo un histograma revelaría.
- **Cuestionar los nulos:** Investiga si el dato falta por azar o por un patrón oculto.
- **Analizar el contexto:** Un valor de $0 es normal en "Descuento", pero es un error en "Precio de Venta".
- **Documentar hallazgos:** El EDA termina con una lista de decisiones: "Imputaremos la mediana en X porque tiene distribución sesgada", "Eliminaremos la columna Y por alta correlación con Z".

Este proceso te dará la confianza necesaria para diseñar **pipelines** robustas, sabiendo exactamente por qué estás aplicando cada transformación.

Detección y tratamiento de valores faltantes

En problemas reales de Ciencia de Datos, **los valores faltantes son la norma, no la excepción**. Pueden surgir por errores de carga, fallas en sistemas, datos opcionales no completados o integraciones incompletas entre fuentes. Antes de cualquier análisis exploratorio o modelado, es fundamental **diagnosticar y tratar correctamente estos valores**, ya que afectan estadísticas descriptivas, visualizaciones y el desempeño de los modelos.

Esta práctica propone implementar manualmente un pipeline básico de **detección, imputación y trazabilidad de valores faltantes**, replicando a bajo nivel lo que hacen herramientas automáticas como pandas, scikit-learn o librerías de profiling.

Simulación del dataset y representación de valores faltantes

El dataset simulado representa un **contexto de negocio realista de e-commerce / ventas**, con variables:

- **Numéricas**:
- `order_qty` (cantidad de unidades)
- `price_unit` (precio unitario)
- **Categóricas**:
- `product_cat` (categoría del producto)

- `country` (país)

Todos los valores se representan inicialmente como **strings**, incluyendo los valores faltantes, que se codifican explícitamente como `"NULL"`. Esto refleja escenarios comunes donde los datos provienen de archivos planos, logs o sistemas legacy sin tipado explícito.

Objetivo conceptual: forzar al estudiante a **inferir tipos de datos** en lugar de asumirlos.

Identificación del tipo de cada columna

Antes de imputar, es necesario decidir **cómo tratar cada columna**, lo que depende de su tipo:

- **Columnas numéricas** imputación por media
- **Columnas categóricas** imputación por moda

En la práctica, el tipo de columna se detecta **a partir de los valores no nulos**:

- Si **todos los valores no** `"NULL"` **pueden convertirse a** `int`, la columna se considera numérica.
- Si **alguno no puede**, se trata como categórica.
- Si la columna está completamente vacía, no hay evidencia del tipo y se maneja como caso especial.

Objetivo conceptual: entender que el tipo de una variable no siempre está explícito y debe inferirse desde los datos observados.

Estrategias de imputación según tipo de dato

Imputación numérica

Para columnas numéricas:

- Se calcula la **media aritmética** de los valores no nulos.
- La media se **redondea hacia abajo** (`floor`) para mantener valores enteros.
- Si la columna está completamente vacía, se imputa con **0**.

Esta estrategia es simple pero muy utilizada como baseline en análisis exploratorio.

Imputación categórica

Para columnas categóricas:

- Se utiliza la **moda**, es decir, el valor más frecuente.
- Se implementa mediante `Counter`.
- Si la columna no tiene valores observados, se imputa con `"NA"`.

Objetivo conceptual: diferenciar claramente entre imputaciones estadísticas (numéricas) y de frecuencia (categóricas).

Creación de variables flag (trazabilidad)

Además de imputar, la práctica exige crear una **columna flag por cada columna original**, que indique:

- `1` el valor original era `"NULL"`
- `0` el valor original estaba presente

Este paso es clave porque:

- Permite **preservar información** sobre la ausencia de datos.
- Es una práctica común en **feature engineering**, ya que la ausencia en sí misma puede ser informativa.
- Facilita auditoría y explicabilidad del preprocesamiento.

Objetivo conceptual: entender que imputar no debe ocultar el hecho de que el dato faltaba.

Construcción del dataset final

El dataset resultante concatena, por fila:

1. **Los valores imputados** (con tipo correcto)
2. **Los flags de ausencia**, en el mismo orden de las columnas originales

El formato final sigue estrictamente la consigna:

CÓDIGO

```
[val_1 ... val_M flag_1 ... flag_M]
```

Esto simula un escenario típico de **preprocesamiento previo a un modelo de Machine Learning**, donde todas las variables deben estar completas y correctamente tipadas.

Relación con herramientas automáticas

Aunque en la práctica se implementa todo manualmente, este ejercicio permite comprender qué hacen internamente herramientas como:

- `SimpleImputer` (scikit-learn)
- Pipelines de preprocesamiento
- Sistemas de validación de calidad de datos
- Librerías de profiling automático

Aprendizaje clave: antes de usar automatismos, es fundamental entender la lógica que hay detrás.

Profiling automático: pandas-profiling y Sweetviz

En problemas reales de Ciencia de Datos, **los valores faltantes son la norma, no la excepción**. Pueden surgir por errores de carga, fallas en sistemas, datos opcionales no completados o integraciones incompletas entre fuentes.

Antes de cualquier análisis exploratorio o modelado, es fundamental **diagnosticar y tratar correctamente estos valores**, ya que afectan estadísticas descriptivas, visualizaciones y el desempeño de los modelos.

Esta práctica propone implementar manualmente un pipeline básico de **detección, imputación y trazabilidad de valores faltantes**, replicando a bajo nivel lo que hacen herramientas automáticas como pandas, scikit-learn o librerías de profiling.

Simulación del dataset y representación de valores faltantes

El dataset simulado representa un **contexto de negocio realista de e-commerce / ventas**, con variables:

- **Numéricas**:
- `order_qty` (cantidad de unidades)
- `price_unit` (precio unitario)
- **Categóricas**:
- `product_cat` (categoría del producto)
- `country` (país)

Todos los valores se representan inicialmente como **strings**, incluyendo los valores faltantes, que se codifican explícitamente como `"NULL"`.

Esto refleja escenarios comunes donde los datos provienen de archivos planos, logs o sistemas legacy sin tipado explícito.

Objetivo conceptual: forzar al estudiante a **inferir tipos de datos** en lugar de asumirlos.

Identificación del tipo de cada columna

Antes de imputar, es necesario decidir **cómo tratar cada columna**, lo que depende de su tipo:

- **Columnas numéricas** imputación por media
- **Columnas categóricas** imputación por moda

En la práctica, el tipo de columna se detecta **a partir de los valores no nulos**:

- Si **todos los valores no** `"NULL"` **pueden convertirse a** `int`, la columna se considera numérica.
- Si **alguno no puede**, se trata como categórica.
- Si la columna está completamente vacía, no hay evidencia del tipo y se maneja como caso especial.

Objetivo conceptual: entender que el tipo de una variable no siempre está explícito y debe inferirse desde los datos observados.

Estrategias de imputación según tipo de dato

Imputación numérica

Para columnas numéricas:

- Se calcula la **media aritmética** de los valores no nulos.
- La media se **redondea hacia abajo** (`floor`) para mantener valores enteros.
- Si la columna está completamente vacía, se imputa con **0**.

Esta estrategia es simple pero muy utilizada como baseline en análisis exploratorio.

Imputación categórica

Para columnas categóricas:

- Se utiliza la **moda**, es decir, el valor más frecuente.
- Se implementa mediante `Counter`.
- Si la columna no tiene valores observados, se imputa con `"NA"`.

Objetivo conceptual: diferenciar claramente entre imputaciones estadísticas (numéricas) y de frecuencia (categóricas).

Creación de variables flag (trazabilidad)

Además de imputar, la práctica exige crear una **columna flag por cada columna original**, que indique:

- `1` el valor original era `"NULL"`
- `0` el valor original estaba presente

Este paso es clave porque:

- Permite **preservar información** sobre la ausencia de datos.
- Es una práctica común en **feature engineering**, ya que la ausencia en sí misma puede ser informativa.
- Facilita auditoría y explicabilidad del preprocesamiento.

Objetivo conceptual: entender que imputar no debe ocultar el hecho de que el dato faltaba.

Construcción del dataset final

El dataset resultante concatena, por fila:

1. **Los valores imputados** (con tipo correcto)
2. **Los flags de ausencia**, en el mismo orden de las columnas originales

El formato final sigue estrictamente la consigna:

CÓDIGO

```
[val_1 ... val_M flag_1 ... flag_M]
```

Esto simula un escenario típico de **preprocesamiento previo a un modelo de Machine Learning**, donde todas las variables deben estar completas y correctamente tipadas.

Relación con herramientas automáticas

Aunque en la práctica se implementa todo manualmente, este ejercicio permite comprender qué hacen internamente herramientas como:

- `SimpleImputer` (scikit-learn)
- Pipelines de preprocesamiento
- Sistemas de validación de calidad de datos
- Librerías de profiling automático

Aprendizaje clave: antes de usar automatismos, es fundamental entender la lógica que hay detrás.

Limpieza de datos y tratamiento de nulos

En el módulo anterior realizamos un **Análisis Exploratorio de Datos (EDA)** para entender la calidad de nuestra información. Identificamos que los datos rara vez vienen perfectos: faltan valores, hay registros repetidos y existen inconsistencias.

Hoy aprenderemos a pasar del diagnóstico a la **acción**. La limpieza de datos no es solo "borrar lo que estorba", sino tomar decisiones estratégicas que preserven la integridad de los modelos de Machine Learning que construiremos después.

1. La anatomía de un "Nulo"

En Python y Data Science, el "vacío" tiene nombres técnicos distintos según la librería: * `None`: El tipo de dato "nulo" nativo de Python. * `NaN` **(Not a Number)**: El estándar de Numpy/Pandas para representar valores faltantes en columnas numéricas. * `NULL`: El término que verás en bases de datos (SQL).

¿Por qué aparecen nulos?

Es vital preguntarse la causa antes de programar la solución: 1. **Error de captura**: Un sensor falló o un usuario olvidó llenar un campo. 2. **No aplicabilidad**: En un dataset de vehículos, la columna "Cilindrada" será nula para autos eléctricos. No es un error, es un dato que no debe existir. 3. **Privacidad**: El usuario decidió no compartir su salario.

2. Estrategias de limpieza

A. Eliminación (Deletion)

Es la opción más radical. Se usa cuando: * La fila tiene demasiados nulos para ser útil. * La columna no aporta información (ej. 90% de nulos). * **Riesgo**: Si eliminas demasiados datos, puedes introducir un sesgo (ej. eliminar a todos los usuarios jóvenes porque no llenaron su perfil).

B. Imputación Estadística

Consiste en llenar los huecos con valores calculados: * **Media**: Útil para datos numéricos con distribución normal. * **Mediana**: La mejor opción cuando hay **outliers** (valores extremos), ya que no se deja arrastrar por ellos. * **Moda**: Ideal para variables categóricas (el valor más frecuente).

C. Imputación por Constante

Llenar con un valor fijo como `"Desconocido"` o `0`. Es útil para mantener la categoría de que el dato "faltaba", lo cual en sí mismo puede ser una señal para el modelo de ML.

3. Scikit-Learn y el `SimpleImputer`

Aunque Pandas permite usar `.fillna()`, en la arquitectura de Machine Learning preferimos usar `SimpleImputer` de Scikit-Learn. ¿Por qué? Porque permite integrar la limpieza dentro de un **Pipeline**, asegurando que las mismas reglas de limpieza aplicadas en el entrenamiento se apliquen igual cuando el modelo reciba datos nuevos en producción.

> **Importante**
> Un error común es calcular la media con todo el dataset antes de separar en Train/Test. Esto causa **Data Leakage** (fuga de información). El "Imputer" de Scikit-Learn aprende la media solo del set de entrenamiento.

En el siguiente ejercicio, aplicarás estas reglas a un dataset de ventas para asegurar que ningún registro incompleto detenga tu flujo de trabajo.

Multivariado: pairplots, heatmaps y correlaciones

¿Por qué explorar múltiples variables a la vez?

Imaginemos que trabajan con un conjunto de datos de clientes donde tienen variables como edad, ingresos, gasto mensual y satisfacción. ¿Cómo pueden descubrir relaciones complejas entre estas variables que no se ven al analizar una o dos a la vez? Aquí es donde el análisis multivariado cobra protagonismo.

En esta unidad, exploraremos dos herramientas visuales fundamentales para entender múltiples variables simultáneamente: **pairplots** y **heatmaps**. Aprenderán a seleccionar y usar estas técnicas para detectar patrones, relaciones y posibles anomalías en sus datos, habilidades clave para cualquier científico de datos que busque insights profundos y reproducibles.

Al finalizar, podrán interpretar matrices de correlación y visualizaciones multivariadas para apoyar hipótesis y preparar análisis más avanzados.

¿Qué es el análisis multivariado?

El **análisis multivariado** consiste en examinar simultáneamente más de dos variables para entender relaciones complejas y patrones en los datos. A diferencia del análisis bivariado, que se enfoca en pares de variables, el multivariado permite una visión más holística.

Pairplots

Un **pairplot** es una matriz de gráficos que muestra la relación entre cada par de variables numéricas en un dataset. Cada celda contiene un gráfico de dispersión para un par de variables, mientras que la diagonal suele mostrar histogramas o densidades de cada variable individual.

- **Ventajas:** Permite detectar correlaciones, agrupamientos y distribuciones de forma rápida.
- **Ejemplo en Python:** usando `seaborn.pairplot`

CÓDIGO

```
import seaborn as sns
sns.pairplot(df)
```

Heatmaps y matrices de correlación

Una **matriz de correlación** cuantifica la relación lineal entre variables numéricas, con valores entre -1 y 1.

- 1 indica correlación positiva perfecta
- -1 indica correlación negativa perfecta
- 0 indica ausencia de correlación lineal

Un **heatmap** es una representación visual de esta matriz, donde los colores indican la fuerza y dirección de la correlación.

- **Ventajas:** Facilita la identificación rápida de relaciones fuertes o débiles.
- **Ejemplo en Python:**

CÓDIGO

```
import seaborn as sns
corr = df.corr()
sns.heatmap(corr, annot=True, cmap='coolwarm')
```

> **Nota**
> La correlación no implica causalidad, pero es un buen punto de partida para hipótesis.

Relación con reducción de dimensionalidad

Aunque en esta unidad solo introducimos el análisis multivariado visual, estas técnicas preparan el terreno para métodos como PCA (Análisis de Componentes Principales), que veremos en unidades posteriores para visualizar estructuras complejas en menos dimensiones.

Contexto

En la práctica diaria de un científico de datos, enfrentarse a datasets con múltiples variables es la norma. Por ejemplo, en análisis de clientes, variables demográficas, comportamentales y transaccionales se combinan para entender segmentos y patrones.

Utilizar pairplots y heatmaps les permitirá:

- Visualizar rápidamente relaciones y distribuciones
- Detectar variables altamente correlacionadas que podrían redundar en modelos
- Identificar patrones que sugieran segmentaciones o anomalías

Estas visualizaciones son especialmente útiles en notebooks reproducibles, donde pueden documentar y compartir insights con equipos multidisciplinarios.

Además, dominar estas técnicas les facilitará la transición a análisis más avanzados, como la reducción de dimensionalidad y la detección de anomalías, que abordaremos en las siguientes unidades.

Dimension reduction (PCA) y visualización

Marco teórico — Reducción dimensional con PCA

1. ¿Por qué reducción dimensional?

En problemas reales de negocio (retail, marketing, finanzas, producto), solemos trabajar con **muchas variables numéricas simultáneamente**: precios, cantidades, descuentos, visitas, conversiones, ratings, etc.

Esto genera varios desafíos:

- Dificultad para **visualizar** los datos (más de 3 dimensiones).
- Variables **correlacionadas** entre sí (redundancia de información).
- Modelos más complejos, inestables o difíciles de interpretar.

La **reducción dimensional** busca resolver estos problemas encontrando una representación más simple de los datos, que conserve la mayor cantidad de información relevante posible.

2. ¿Qué es PCA (Principal Component Analysis)?

PCA es una técnica **no supervisada** que transforma un conjunto de variables originales en un nuevo conjunto de variables llamadas **componentes principales**, que cumplen:

- Son **combinaciones lineales** de las variables originales.
- Son **ortogonales** entre sí (no correlacionadas).
- Están ordenadas por la **cantidad de varianza que explican**:
- PC1 explica la mayor varianza posible.
- PC2 explica la mayor parte de la varianza restante, y así sucesivamente.

PCA **no selecciona variables**, las **recombina**.

3. Justificación del dataset simulado

En la práctica se genera un **dataset multivariado realista de negocio**, con variables como:

- `price_unit`
- `order_qty`
- `discount_pct`
- `visits`
- `conversion_rate`
- `rating`

Para que PCA tenga sentido, no basta con datos aleatorios independientes.

Por eso se introducen **variables latentes**, que representan factores ocultos típicos del negocio, por ejemplo:

- Popularidad del producto
- Demanda
- Satisfacción del cliente

Estas variables latentes generan **correlaciones estructurales** entre las variables observadas. Esto permite que PCA detecte patrones reales y no ruido.

4. Importancia del escalado previo

Antes de aplicar PCA, **es obligatorio estandarizar los datos** cuando:

- Las variables están en **escalas distintas** (precios, porcentajes, conteos, tasas).
- Se usa la **matriz de covarianza**, como en PCA clásico.

La estandarización garantiza que:

- Cada variable tenga media 0 y varianza 1.
- Ninguna variable domine el análisis solo por su escala.

En la práctica se utiliza:

> **Nota**
>
> **StandardScaler.** (media = 0, desviación estándar = 1)

Sin este paso, PCA daría resultados engañosos.

5. Cálculo de los componentes principales

Al aplicar PCA con `n_components=2`, se obtiene:

- **Dos vectores de componentes principales**
- Cada vector tiene un coeficiente por variable original.
- Indican cuánto contribuye cada variable a ese componente.
- **Varianza explicada**
- Proporción de la variabilidad total capturada por cada componente.

Interpretación típica:

- PC1 suele capturar un **factor dominante de negocio** (por ejemplo volumen o demanda).
- PC2 suele capturar un **factor secundario** (por ejemplo eficiencia o calidad).

6. Proyección al espacio 2D

Cada observación original (con M variables) se proyecta a un punto en 2D:

- Coordenada en PC1

- Coordenada en PC2

Esto permite:

- Visualizar patrones y agrupamientos.
- Detectar segmentos naturales.
- Identificar outliers multivariados.
- Analizar similitud entre observaciones.

Dos puntos cercanos en la proyección representan observaciones con perfiles similares en las variables originales.

7. Visualización y segmentación

El scatter plot de la proyección PCA:

- Resume la estructura multivariada en un plano 2D.
- Puede colorearse por alguna variable de negocio (ej. cuartiles de precio).

Esto ayuda a responder preguntas como:

- ¿Existen clusters naturales?
- ¿Ciertas variables explican separaciones claras?
- ¿Hay observaciones extremas?

8. Interpretación de los componentes

Para interpretar PCA correctamente, es clave analizar:

1. **Magnitud de los coeficientes** Variables con valores absolutos grandes influyen más.
2. **Signo del coeficiente** Indica la dirección de la relación.
3. **Varianza explicada acumulada** Mide cuánta información se conserva al reducir dimensiones.

No existe una interpretación “correcta” única:

PCA es una **herramienta exploratoria**, no un modelo causal.

9. Objetivo del laboratorio

Este laboratorio busca que el estudiante:

- Entienda **qué problema resuelve PCA**

Aplique correctamente:

- escalado
- ajuste del modelo
- proyección

Sepa **leer e interpretar**:

- componentes
- varianza explicada
- visualizaciones

Detección visual de anomalías y outliers

Fundamentación teórica de la práctica

Objetivo general de la práctica

El objetivo de esta práctica es **detectar y analizar valores atípicos (outliers)** en un conjunto de datos **multivariado**, combinando:

- **Criterios estadísticos objetivos** (regla del IQR)
- **Análisis visual simplificado** (resumen tipo boxplot)
- **Inspección manual guiada**, simulando una visualización interactiva

Esto refleja un escenario real de negocio, donde la detección de anomalías **no es completamente automática**, sino que requiere **criterio analítico y validación humana**.

Generación de un dataset multivariado realista

¿Por qué no usar datos aleatorios simples?

En problemas reales, las variables:

- **No son independientes**
- Presentan **estructuras latentes** (segmentos, clusters, correlaciones)
- Contienen **ruido y excepciones reales**

Por eso, el dataset simulado se construye con:

a) Factores latentes

Se generan variables latentes (`latent_a` , `latent_b`) que:

- Representan causas no observables (ej. comportamiento de clientes, nivel socioeconómico)
- Introducen **correlación entre features**

Esto evita un dataset artificialmente “limpio”.

b) Generación de múltiples features

Cada variable numérica (`feat1` , `feat2` , …):

- Es una **combinación lineal** de factores latentes
- Incluye **ruido aleatorio**

- Tiene escalas distintas

Esto simula métricas de negocio como:

- Ventas
- Frecuencia de compra
- Duración
- Montos
- Scores internos

c) Inyección controlada de outliers

Se introducen explícitamente ~3% de observaciones anómalas:

- En **dimensiones aleatorias**
- Con **desvíos grandes**

Esto permite:

- Validar si el método estadístico funciona
- Comparar detección automática vs manual

Detección de outliers con la regla del IQR

¿Qué es el IQR?

El **Interquartile Range (IQR)** se define como:

IQR=Q3−Q1IQR = Q3 - Q1IQR=Q3−Q1

Donde:

- `Q1` : percentil 25
- `Q3` : percentil 75

Regla clásica de detección

Un valor se considera outlier si:

x\Q3+1.5·IQRx \< Q1 - 1.5 \cdot IQR \quad \text{o} \quad x > Q3 + 1.5 \cdot IQRx\Q3+1.5·IQR

Ventajas:

- No asume normalidad
- Robusta a valores extremos
- Muy utilizada en exploración inicial de datos

Aplicación multivariada

La regla se aplica **por cada dimensión**:

- Una observación es anómala si:
- Es outlier en **al menos una variable**
- Se registra:
- En qué variables falla
- Cuántas veces aparece

Esto permite entender **la naturaleza del outlier**, no solo su existencia.

Combinación con detección manual

¿Por qué incluir detección manual?

En contextos reales:

- No todo outlier estadístico es un error
- Algunos valores extremos son **eventos válidos de negocio**

Ejemplos:

- Venta extraordinaria
- Cliente premium
- Evento puntual

Por eso:

- Se permite definir `MANUAL_OUTLIERS`
- Se combinan con los detectados automáticamente

Esto refuerza la idea de **criterio analítico**, no solo automatización.

Resumen textual tipo “boxplot multivariado”

¿Por qué un resumen textual?

Dado que el ejercicio **simula una visualización**, se construye un resumen que incluye:

Por cada variable:

- Q1, Q3, IQR
- Límites inferior y superior
- Cantidad de outliers
- Índices detectados

Esto emula lo que se observaría en:

- Boxplots
- Dashboards
- Herramientas BI

Resumen por observación

Para cada punto marcado como anómalo:

- Se listan las dimensiones donde falla
- Se justifica por qué fue detectado

Esto entrena al estudiante a:

- **Explicar decisiones**
- Comunicar resultados técnicos

Tabla para inspección manual

Se genera un CSV con:

- Solo los puntos sospechosos
- Sus valores originales
- Dimensiones donde fueron detectados

Esto simula:

- Un flujo de revisión humana
- Análisis exploratorio posterior
- Validación con expertos de negocio

¿Por qué scatter matrix?

Permite:

- Ver relaciones bivariadas
- Identificar outliers visualmente
- Detectar clusters o patrones anómalos

Las anomalías se destacan visualmente para:

- Confirmar detección
- Encontrar falsos positivos

Formato de salida requerido

El output final cumple **estrictamente** con el enunciado:

1. Número total de outliers detectados (`K`)

2. Lista ordenada de índices (0-based)

Esto refuerza:

- Precisión
- Reproducibilidad
- Evaluación automática

Serialización de resultados

Finalmente:

- Se guarda un JSON con métricas y resultados
- Se convierten tipos NumPy a tipos nativos

Esto simula:

- Persistencia de resultados
- Integración con otros sistemas
- Auditoría de decisiones

Feature engineering y transformadores personalizados

¿Por qué algunos modelos de Machine Learning (ML) parecen "mágicos" mientras que otros, con los mismos datos, fallan estrepitosamente? La respuesta rara vez está en el algoritmo y casi siempre está en los **datos**.

Imagina que estás construyendo un sistema para predecir si una propiedad en Airbnb se alquilará rápido. Tienes la fecha de publicación, pero el modelo no entiende de "martes" o "fines de semana". Si transformas esa fecha en una variable booleana como `es_fin_de_semana`, de repente el modelo "ve" un patrón claro. Ese proceso de convertir datos crudos en señales potentes es el **Feature Engineering**.

En esta unidad, aprenderás a dejar de ver los datasets como tablas estáticas para verlos como materia prima que puedes moldear. Aprenderás a automatizar estas transformaciones creando tus propios "moldes" (transformadores personalizados) compatibles con el estándar de la industria: **Scikit-Learn**.

1. Feature Engineering: El arte de crear señales

El **Feature Engineering** (Ingeniería de Características) es el proceso de usar el conocimiento del dominio (tu entendimiento del negocio) para crear variables que faciliten el aprendizaje del modelo.

¿Por qué es fundamental?

Los algoritmos de ML son, en esencia, motores matemáticos. Si les entregas datos irrelevantes o mal formateados, el resultado será pobre (el famoso principio *Garbage In, Garbage Out*). Una buena característica puede: 1. **Simplificar el modelo:** Un modelo simple con buenas variables suele superar a

uno complejo con variables pobres. 2. **Capturar relaciones no lineales:** Si sabes que el área de una casa tiene una relación cuadrática con el precio, crear la variable `area_al_cuadrado` ayuda al modelo a capturar esa relación sin esfuerzo. 3. **Reducir el ruido:** Al limpiar y transformar, eliminas información que confunde al algoritmo.

Ejemplo conceptual: El caso de una tienda de E-commerce

Supongamos que tienes una tabla de ventas con la columna `hora_transaccion`. - **Dato Crudo:** `2023-10-27 14:30:00` - **Feature Engineering 1:** Extraer la hora (`14`). - **Feature Engineering 2:** Crear una categoría `franja_horaria` (Madrugada, Mañana, Tarde, Noche). - **Feature Engineering 3:** Variable binaria `hora_pico` (1 si es entre 18:00 y 20:00, 0 si no).

Cada uno de estos pasos "mastica" la información para que el modelo no tenga que descubrir estas reglas de negocio por sí solo.

2. Codificación y escalado de variables

Antes de inventar nuevas variables, debemos asegurar que las que ya tenemos hablen el "idioma" de las matemáticas (números).

A. Codificación de Variables Categóricas

Las computadoras no entienden qué es "Rojo" o "Azul". Necesitamos convertirlos a números.

1. One-Hot Encoding (OHE)

Crea una columna nueva por cada categoría. Se usa cuando **no hay un orden intrínseco**. - **Ejemplo:** Columna `Ciudad` (Madrid, Barcelona, Valencia). - **Resultado:** Tres columnas donde solo una contiene un `1` y las demás un `0`. - *Ojo:* No lo uses si tienes miles de categorías (como "Código Postal"), ya que crearías miles de columnas (La "maldición de la dimensionalidad").

2. Ordinal Encoding

Asigna un número entero a cada categoría siguiendo un orden. - **Ejemplo:** Columna `Talla` (S, M, L, XL). - **Resultado:** S=1, M=2, L=3, XL=4. Aquí el número tiene significado: 4 es mayor que 1.

B. Escalado de Variables Numéricas

Muchos algoritmos (como KNN o SVM) calculan distancias entre puntos. Si tienes una variable `Edad` (0-100) y otra `Salario Anual` (0-200,000), el salario dominará completamente el cálculo solo por tener números más grandes.

1. Estandarización (StandardScaler)

Transforma los datos para que tengan una **media de 0 y una desviación estándar de 1**. Es la opción más robusta si tus datos siguen una distribución normal (campana de Gauss). $$z = \frac{x - \mu}{\sigma}$$

2. Normalización (MinMaxScaler)

Escala los datos a un rango fijo, usualmente **[0, 1]**. Es útil cuando necesitas que todos los valores sean positivos o cuando los algoritmos lo requieren explícitamente (como en redes neuronales).

3. Transformadores personalizados en Scikit-Learn

Aquí es donde pasamos de ser analistas que ejecutan scripts a ingenieros que construyen herramientas. Scikit-Learn nos permite crear nuestras propias clases de transformación que se integran perfectamente en su ecosistema.

¿Por qué crear transformadores propios?

Imagina que para tu modelo siempre necesitas: 1. Limpiar espacios en blanco de los strings. 2. Reemplazar valores negativos por cero. 3. Calcular el logaritmo de una columna.

En lugar de repetir este código en cada notebook, encapsulas la lógica en una clase. Esto garantiza que tu preprocesamiento sea **consistente, reusable y fácil de testear**.

La anatomía de un transformador

Para que Scikit-Learn reconozca tu clase como un transformador, debes heredar de dos clases base: `BaseEstimator` y `TransformerMixin`. Tu clase **debe** implementar dos métodos principales:

1. **`fit(X, y=None)`** : Aquí es donde el transformador "aprende". Por ejemplo, si estás escalando, aquí se calcularía la media. Como en muchos transformadores personalizados de feature engineering no necesitamos "aprender" nada (solo aplicar una regla), este método suele simplemente devolver `self`.
2. **`transform(X)`** : Aquí ocurre la magia. Es donde aplicas la lógica de transformación a los datos y devuelves el nuevo DataFrame o array.

Ejemplo de código: El transformador "Calculador de Antigüedad"

Supongamos que trabajas en banca y quieres una variable que sea "Años desde la apertura de cuenta".

CÓDIGO

```
import numpy as np
import pandas as pd
from sklearn.base import BaseEstimator, TransformerMixin
from datetime import datetime

class DateToSeniorityTransformer(BaseEstimator, TransformerMixin):
 def __init__(self, column_name):
 self.column_name = column_name

 def fit(self, X, y=None):
 # No aprendemos nada de los datos, solo devolvemos self
 return self

 def transform(self, X):
 X_copy = X.copy()
 current_year = datetime.now().year
 # Aplicamos la lógica de negocio
 X_copy['years_seniority'] = current_year - pd.to_datetime(X_copy[self.column_name]).dt.year
 # Eliminamos la columna original para no confundir al modelo
 return X_copy.drop(columns=[self.column_name])
```

4. Aplicaciones del mundo real

Caso de Uso: Netflix y el Feature Engineering

Netflix no solo sabe qué películas ves; sabe *cuándo* las ves. Un transformador personalizado en su arquitectura podría tomar la marca de tiempo de reproducción y convertirla en: - `prefijo_horario`: ¿Es un usuario que ve contenido en el almuerzo o antes de dormir? - `es_maraton`: ¿Vio más de 3 episodios seguidos?

Esta ingeniería de variables es lo que permite que su sistema de recomendación sea tan preciso. No usan el timestamp crudo; usan la **señal de comportamiento** derivada de él.

Caso de Uso: Amazon SageMaker Feature Store

En empresas de escala como Amazon, las características (features) se consideran activos valiosos. Utilizan **Feature Stores** (como el de SageMaker) para almacenar estas variables ya calculadas. Un equipo de científicos puede crear una variable compleja (ej. "Gasto promedio del usuario en los últimos 30 días") y otros equipos pueden reusarla usando transformadores estandarizados.

5. Errores comunes y mejores prácticas

El peligro del Data Leakage (Fuga de datos)

Este es el error más común y crítico para un principiante. Ocurre cuando información del conjunto de **prueba (test)** se filtra en el entrenamiento a través del preprocesamiento.

- **El Error:** Calcular la media de *todo* el dataset y luego usarla para escalar.
- **La Consecuencia:** Tu modelo se verá increíblemente preciso en tus pruebas, pero fallará en la vida real porque "conoce" el futuro (la media de los datos que no debería haber visto).
- **La Solución:** Siempre haz `.fit()` **solo** en el conjunto de entrenamiento y luego `.transform()` tanto en entrenamiento como en prueba.

Diferencia crucial: `.fit()` vs. `.transform()`

- `.fit()`: El profesor estudia las respuestas correctas para aprender el patrón (Cálculo de parámetros como media, desviación, o categorías únicas).
- `.transform()`: El profesor aplica lo aprendido a los exámenes de los alumnos (Aplica el escalado o la codificación basada en esos parámetros).
- `.fit_transform()`: Una forma rápida de hacer ambos en un solo paso, pero **solo para el set de entrenamiento**.

6. Síntesis y Conexiones

Hemos pasado de la limpieza básica (nulos y duplicados) al diseño arquitectónico de datos. 1. Entendimos que el **Feature Engineering** es donde inyectamos inteligencia de negocio al dataset. 2. Aprendimos que el **escalado y la codificación** son pasos técnicos obligatorios para que los algoritmos funcionen. 3. Descubrimos cómo encapsular nuestras reglas en **Transformadores Personalizados** para crear código profesional y robusto.

En la siguiente unidad, aprenderás a encadenar todos estos pasos (limpieza, escalado y tus nuevos transformadores) en un **Pipeline de Scikit-Learn**, asegurando que tus datos fluyan de forma automática y segura hacia el modelo.

Diseño de pipelines con Scikit-Learn

En el desarrollo de Machine Learning, el camino desde los datos crudos hasta una predicción no es una línea recta, sino una serie de transformaciones. Tradicionalmente, los analistas aplican limpiezas y transformaciones paso a paso. Sin embargo, este enfoque manual tiene un riesgo crítico: el **Data Leakage** (fuga de datos) y la falta de consistencia entre entrenamiento y producción.

Las **Pipelines** de Scikit-Learn resuelven esto encapsulando todo el flujo en un solo objeto.

¿Por qué Pipelines? La lucha contra el Data Leakage

Imagina que escalas tus datos (StandardScaler) usando el promedio de todo tu dataset *antes* de dividirlo en entrenamiento (train) y prueba (test). Al hacer esto, la información del conjunto de test se

"filtra" en el entrenamiento a través del promedio. El modelo ahora conoce detalles del futuro que no debería saber.

Un Pipeline garantiza que:

1. Las transformaciones se aprendan (`.fit`) **solo** en el conjunto de entrenamiento.
2. Esas mismas reglas se apliquen (`.transform`) al conjunto de prueba o nuevos datos sin re-calcular los parámetros.

Componentes de un Pipeline

Un Pipeline es esencialmente una lista de pasos donde cada paso es una tupla: `('nombre', objeto)` .

- **Transformadores**: Son los pasos intermedios (ej. Imputación, Escalado, Selección de columnas). Deben implementar los métodos `.fit()` y `.transform()` .
- **Estimador final**: Es el modelo (ej. Regresión, Bosque Aleatorio). Debe implementar `.fit()` y `.predict()` .

ColumnTransformer: Procesamiento por tipo de datos

En el mundo real, los datasets son mixtos. No quieres escalar nombres de ciudades ni aplicar OneHotEncoding a la edad. Para esto usamos `ColumnTransformer` , que nos permite aplicar rutas de transformación distintas según la columna, y luego unir los resultados automáticamente.

Ejemplo de Estructura:

1. **Rama Numérica**: Imputar nulos con mediana Escalar.
2. **Rama Categórica**: Imputar nulos con "S/D" OneHotEncoding.
3. **Combinador**: Une ambas ramas en una sola matriz de entrenamiento.
4. **Modelo**: Clasificador final.

PRE-ENTREGA DEL PROYECTO

Actividad evaluable · parte del proyecto integrador

Pre-entrega: Construcción de un Pipeline de Preprocesamiento Robusto

El objetivo de esta práctica es consolidar tus habilidades en la automatización de la limpieza y transformación de datos utilizando **Scikit-Learn Pipelines**. En lugar de realizar transformaciones manuales y dispersas, construirás una arquitectura reutilizable que prepare los datos para un modelo de Machine Learning de manera profesional.

Cómo resolver esta entrega

Propósito

Evitar la fuga de datos (*data leakage*) y asegurar que el preprocesamiento sea consistente tanto en el entrenamiento como en la inferencia, encapsulando toda la lógica en un único objeto de pipeline.

Pasos sugeridos

1. **Carga y Exploración**: Utiliza un conjunto de datos (puedes usar el dataset de `housing` o `iris` de sklearn, o uno propio persistido en módulos anteriores). Identifica columnas numéricas y categóricas.
2. **Definición de Transformers**: Crea un `ColumnTransformer` que realice: * **Numéricas**: Imputación de valores nulos (media/mediana) y escalado (`StandardScaler` o `MinMaxScaler`). * **Categóricas**: Imputación de valores frecuentes y codificación (`OneHotEncoder`).
3. **Ensamblaje del Pipeline**: Integra el preprocesador en un `Pipeline` final que incluya al menos un estimador básico (ej. Regresión Lineal o Logística).
4. **Validación**: Ejecuta un `train_test_split` y entrena el pipeline completo. Genera un reporte de métricas básico.
5. **Persistencia**: Incluye en el repositorio el código necesario para exportar el pipeline entrenado usando la librería `joblib` .

Errores comunes

- Aplicar `fit_transform` a todo el dataset antes del split (esto causa data leakage).
- Olvidar manejar categorías nuevas o desconocidas en el `OneHotEncoder` (usa `handle_unknown='ignore'`).

Entrega

Debes subir el código a un repositorio público siguiendo una estructura limpia de archivos .py o .ipynb.

Objetivos de aprendizaje

- Implementar ColumnTransformers para el manejo simultáneo de datos heterogéneos.
- Diseñar Pipelines de Scikit-Learn que integren pasos de preprocesamiento y modelado.
- Aplicar técnicas de escalado y codificación categórica evitando la fuga de datos.
- Persistir modelos y transformadores utilizando herramientas de serialización como joblib.

Criterios de aceptación

- El repositorio contiene un archivo README.md inicial con la descripción del dataset utilizado.
- El código utiliza la clase `Pipeline` de Scikit-Learn para encadenar al menos dos pasos de transformación.
- Se implementa un `ColumnTransformer` que diferencia explícitamente el tratamiento de variables numéricas y categóricas.
- El código incluye la división de datos (Train/Test) antes de cualquier ajuste de parámetros de preprocesamiento.
- El pipeline final es funcional y permite ejecutar el método `.predict()` sobre nuevos datos sin errores de formato.

Rúbrica de evaluación — ML Lab I: Construcción de un Pipeline de Preprocesamiento Robusto

Rúbrica para evaluar un repositorio público con un notebook o script de Python que implemente un pipeline de preprocesamiento con Scikit-Learn, evitando fuga de datos, incorporando transformaciones para variables numéricas y categóricas, validación básica y persistencia del modelo entrenado. Está diseñada para un nivel principiante con enfoque práctico y formativo.

Criterio	Descripción	Peso
Implementación del pipeline de preprocesamiento y estimador	Evalúa si el entregable construye correctamente un `ColumnTransformer` y lo integra en un `Pipeline` funcional con un estimador básico, aplicando transformaciones apropiadas a variables numéricas y categóricas.	31%
Prevención de fuga de datos y validación del proceso	Evalúa si el estudiante evita `data leakage` aplicando el preprocesamiento dentro del flujo correcto de entrenamiento y prueba, y si realiza una validación básica adecuada con `train_test_split` y métricas simples.	23%
Manejo de datos numéricos y categóricos	Evalúa si el repositorio identifica adecuadamente tipos de columnas y aplica estrategias de limpieza y transformación acordes a cada tipo, incluyendo imputación de nulos y tratamiento de categorías desconocidas.	14.000000000000002%
		9%

Criterio	Descripción	Peso
Persistencia del pipeline y reutilización	Evalúa si el repositorio incluye la exportación del pipeline entrenado con `joblib` y si el artefacto resultante queda preparado para reutilización futura en inferencia o experimentación.	
Estructura del repositorio y documentación	Evalúa la organización general del repositorio, la claridad del README y la calidad de la presentación del trabajo como entregable profesional de código.	14.000000000000002%
Calidad técnica del código y coherencia general	Evalúa la legibilidad, consistencia y corrección general del código, incluyendo uso de buenas prácticas básicas, nombres comprensibles y coherencia entre los pasos del ejercicio.	9%

Total: 100 pts | Aprobación: 70 pts

MÓDULO 11

ML Lab II: Modelado y Métricas

Entrenar, evaluar y validar modelos de machine learning.

Regresión lineal y logística: intuición, variables y predicción

Bienvenidos al Laboratorio de Modelado. Hasta este punto del programa, has aprendido a limpiar datos, realizar extracciones complejas desde APIs y bases de datos, y construir pipelines robustos. Ahora, entramos en el corazón de la ciencia de datos: **el modelado predictivo**.

Imagina que trabajas para una plataforma de *e-commerce*. El equipo de finanzas te pregunta: *"¿Cuánto dinero gastará este cliente el próximo mes?"*. Minutos después, el equipo de marketing te pregunta: *"¿Este cliente cancelará su suscripción el próximo mes?"*.

Aunque ambas preguntas buscan predecir el futuro, requieren herramientas fundamentalmente distintas. La primera pregunta busca un **número** (dinero); la segunda, una **categoría** (se queda o se va). En esta unidad, aprenderás a distinguir entre la **Regresión Lineal** y la **Regresión Logística**, los dos pilares sobre los cuales se construye casi toda la intuición del Machine Learning moderno.

1. El punto de partida: Variables dependientes e independientes

Antes de elegir un modelo, debemos entender con qué piezas estamos jugando. En cualquier problema de aprendizaje supervisado, trabajamos con dos tipos de variables:

- **Variables Independientes (Predictores o):** Son los datos que ya conocemos y que usamos para realizar la predicción. Por ejemplo: la edad del cliente, sus compras anteriores, los minutos que pasa en la app, o su código postal.
- **Variable Dependiente (Objetivo o):** Es lo que queremos predecir. Su valor "depende" de las variables independientes.

El secreto para elegir el modelo correcto reside exclusivamente en la naturaleza de la Variable Dependiente ().

2. Regresión Lineal: Prediciendo el "Cuánto"

La **Regresión Lineal** se utiliza cuando nuestra variable objetivo es **continua**. Un dato continuo es aquel que puede tomar cualquier valor dentro de un rango: el precio de una casa, la temperatura de un motor, o los ingresos de una empresa

Ejemplo en la Industria: Zillow y los precios inmobiliarios

Empresas como **Zillow** o **Redfin** utilizan modelos de regresión para sus algoritmos de valoración automática. Si una casa tiene 3 habitaciones y está en un barrio específico, el modelo multiplica estas características por sus respectivos coeficientes para entregarte un número: "Esta casa vale $350,200 USD".

Un error común es intentar usar regresión lineal para predecir categorías (como "Sí" o "No"). Si lo hiciéramos, el modelo podría predecir un valor de 1.5 o -0.2, lo cual no tiene sentido en una respuesta binaria. Por eso, para clasificar, necesitamos a su "prima", la regresión logística.

3. Regresión Logística: Prediciendo el "Qué"

A pesar de su nombre, la **Regresión Logística** no predice números en una escala infinita, sino que se usa para **Clasificación**. Específicamente, para predecir la probabilidad de que algo pertenezca a una categoría o no (Clasificación Binaria).

¿Por qué no una línea recta?

Si queremos predecir si un correo es Spam (1) o No Spam (0), una línea recta fallaría porque seguiría creciendo hasta el infinito. Necesitamos algo que "se doble" y se mantenga atrapado entre el 0 y el 1.

Aquí entra la **Función Sigmoide**. Esta función toma cualquier número resultante de una ecuación lineal y lo comprime en un rango de **0 a 1**.

- Si el resultado es **0.85**, el modelo nos está diciendo: "Hay un 85% de probabilidad de que sea Spam".
- Normalmente, usamos un umbral (threshold) de 0.5: cualquier valor mayor a 0.5 se clasifica como 1 (Spam), y menor a 0.5 como 0 (No Spam).

Ejemplo en la Industria: Nubank y el Riesgo de Crédito

Las Fintechs como **Nubank** o **Stripe** reciben miles de solicitudes de crédito por segundo. Su variable dependiente es binaria: ¿El cliente pagará su deuda? (1: Sí, 0: No). Utilizan regresión logística para procesar variables como el historial crediticio y los ingresos. Si el modelo devuelve una probabilidad de "pago" del 92%, el crédito se aprueba automáticamente. Si es del 15%, se rechaza.

4. Diferencias clave: ¿Cuál usar?

Para decidir, hazte una sola pregunta: **¿Qué tipo de respuesta necesito?**

Característica	Regresión Lineal	Regresión Logística
Tipo de Variable Objetivo ()	Continua (Números)	Categórica / Binaria (Clases)
Resultado del Modelo	Un valor numérico exacto	Una probabilidad entre 0 y 1
Pregunta que responde	¿Cuánto...? / ¿Cuántos...?	¿Es A o B? / ¿Sucederá?
Ejemplo rápido	Predecir la temperatura de mañana.	Predecir si mañana lloverá o no.

El "Matiz" de las Probabilidades

Es vital entender que la Regresión Logística es, técnicamente, un modelo de regresión que predice una **probabilidad**. Sin embargo, en el día a día del Data Science, la clasificamos como una herramienta de **Clasificación** porque su fin último es ayudarnos a tomar una decisión discreta (Aprobar/Rechazar, Fraude/Legal, Compra/No Compra).

5. Errores comunes y Mejores Prácticas

A medida que comiences a entrenar modelos con `Scikit-Learn`, ten en cuenta lo siguiente:

1. **Confundir el objetivo:** El error más frecuente es aplicar `LinearRegression()` para un problema de clasificación. Python te dará un resultado, pero será estadísticamente inválido. ¡Verifica siempre tu variable objetivo!
2. **No escalar las variables:** Ambos modelos son sensibles a la escala de los datos. Si una variable es "Ingresos" (de 0 a 1,000,000) y otra es "Edad" (de 0 a 100), el modelo podría darle una importancia injusta a los ingresos solo por tener números más grandes. Como aprendiste en el módulo de preprocesamiento, usar `StandardScaler` es una mejor práctica fundamental antes de pasar los datos a estos algoritmos.
3. **Asumir linealidad siempre:** La regresión lineal asume que la relación entre y es una línea recta. Si la relación es curva (por ejemplo, el crecimiento de una bacteria), el modelo tendrá un desempeño pobre.

Síntesis y Conexión

Hemos establecido las bases del modelado predictivo:

- La **Regresión Lineal** es para predecir cantidades continuas trazando líneas rectas.
- La **Regresión Logística** es para predecir categorías binarias transformando esa línea en una curva de probabilidad (Sigmoide).
- La elección depende totalmente de la **Variable Dependiente**.

En la siguiente unidad, profundizaremos en la "caja negra" de estos modelos: ¿Cómo decide el algoritmo qué línea es la mejor? Veremos conceptos como la **función de pérdida** y el **error cuadrático medio**, que son las reglas que usa la computadora para aprender de sus errores.

Cómo aprende un modelo: línea recta, probabilidad y función de pérdida

Imagina que eres un analista en una empresa de bienes raíces. Tu jefe te entrega una lista de cientos de casas vendidas el último año con sus metros cuadrados y sus precios finales. Luego, te hace una pregunta sencilla: "Si mañana entra al mercado una casa de 120 metros cuadrados, ¿a qué precio deberíamos publicarla?".

En este momento, tienes dos opciones. Podrías adivinar basándote en tu intuición, o podrías usar los datos para encontrar un patrón. Si eliges los datos, estás a punto de entrar en el mundo del **Machine Learning**. Pero, ¿cómo hace una computadora para "aprender" ese patrón a partir de una simple lista de números? No es magia; es una combinación de geometría básica y una búsqueda incesante por cometer el menor error posible.

En esta unidad, vamos a desglosar el motor que mueve a los modelos más fundamentales de la ciencia de datos: la regresión lineal y la regresión logística. Entenderás cómo una línea recta se convierte en una predicción y cómo medimos qué tan "equivocado" está nuestro modelo para poder corregirlo.

1. La base de todo: La ecuación de la recta

Para predecir el precio de una casa o la probabilidad de que un cliente abandone una suscripción, empezamos con una herramienta que probablemente viste en la escuela secundaria: **la ecuación de la recta**.

¿Por qué una recta?

En Machine Learning, a menudo asumimos que existe una relación lineal entre nuestras variables. Si los metros cuadrados aumentan, el precio tiende a subir de forma constante. Esa "tendencia" se representa mejor con una línea recta.

La ecuación matemática se ve así:

$y = m \cdot x + b$

O, en la notación más común de Machine Learning (usando letras griegas llamadas "betas"):

$\hat{y} = \beta_0 + \beta_1 \cdot x$

Donde:

- Ejemplo concreto: El precio de las casas

Supongamos que después de analizar los datos, nuestro modelo encuentra estos valores:

- Nuestra ecuación de predicción sería: **Precio = 50,000 + 1,500 * (Metros Cuadrados)**

Si alguien te pregunta por una casa de 100 m², simplemente reemplazas: USD.

2. Intercepto y Coeficiente: El ADN del modelo

A estos dos valores, y , los llamamos **parámetros del modelo**. Aprender en Machine Learning significa encontrar los mejores valores posibles para estos parámetros.

El Intercepto ()

El intercepto representa el "valor base". En nuestro ejemplo de las casas, el intercepto de 50,000 podría interpretarse como el precio del terreno o los costos fijos antes de sumar el valor por metro construido.

- **Importante:** A veces el intercepto no tiene un sentido lógico en el mundo real (por ejemplo, una casa de 0 metros cuadrados no suele costar dinero), pero es matemáticamente necesario para ubicar la línea en la altura correcta del gráfico.

El Coeficiente ()

Este es el valor más importante para el analista de datos porque nos da la **interpretabilidad**.

- Si es positivo (1,500), significa que a más metros, más precio.
- Si fuera negativo, significaría que al aumentar , disminuye (por ejemplo, la relación entre la antigüedad de un auto y su precio).
- Si es cero, el modelo nos está diciendo que esa variable no sirve para predecir .

En la práctica: Empresas como **Zillow** utilizan modelos que, aunque tienen miles de variables más, funcionan bajo este mismo principio de asignar un peso (coeficiente) a cada característica (ubicación, baños, año de construcción) para llegar a un precio estimado.

3. ¿Cómo sabe el modelo si está aprendiendo bien?

Imagina que lanzas una línea al azar sobre un gráfico de puntos. Probablemente, la mayoría de los puntos estén lejos de tu línea. Esos puntos lejos representan **errores**.

Para que un modelo mejore, necesita una forma de medir qué tan lejos está de la realidad. Aquí es donde entran las **Funciones de Pérdida (Loss Functions)**. Piensa en ellas como un "puntaje de dolor": cuanto más alto es el número, más equivocado está el modelo y más debe cambiar.

A. Para predicciones numéricas: MSE (Mean Squared Error)

En problemas de regresión (predecir números como el precio), usamos el **Error Cuadrático Medio (MSE)**.

¿Cómo funciona?

1. Calculamos la diferencia entre el valor real y el predicho ().
2. Elevamos esa diferencia al cuadrado. Esto hace dos cosas: elimina los signos negativos (para que los errores no se cancelen entre sí) y penaliza mucho más a los errores grandes que a los pequeños.
3. Calculamos el promedio de todos esos cuadrados.

$MSE = \frac{1}{n} \sum (y_{real} - y_{predicho})^2$

¿Por qué al cuadrado? Imagina que el modelo se equivoca por 2 dólares en una casa y por 100 en otra. Al elevar al cuadrado, el error de 2 se vuelve 4, pero el de 100 se vuelve 10,000. El MSE le grita al modelo: "¡Ese error de 100 es inaceptable, corrígelo de inmediato!".

B. Para probabilidades: Log Loss (Entropía Cruzada)

¿Qué pasa cuando no predecimos un precio, sino una probabilidad (0 a 1)? Por ejemplo, la probabilidad de que una transacción bancaria sea fraude. Aquí, una línea recta no funciona bien porque podría predecir valores como -5 o 150%, lo cual no tiene sentido en probabilidades.

Para clasificar, usamos la **Regresión Logística**, que utiliza una curva en forma de "S" (llamada sigmoide). Y para medir el error de esa curva, usamos **Log Loss**.

La lógica del Log Loss: Esta función penaliza la **falsedad con confianza**.

- Si el modelo predice un 99% de probabilidad de que una transacción es fraude y resulta que **SÍ** era fraude, el Log Loss es casi cero (buen trabajo).
- Pero si el modelo predice un 99% de probabilidad de fraude y resulta que **NO** era fraude (era una compra legítima del cliente), el Log Loss será altísimo.

El modelo es "castigado" fuertemente por estar muy seguro de algo que terminó siendo falso. Esto es vital para empresas como **Mastercard** o **Stripe**, donde un "falso positivo" (bloquear a un cliente real) tiene un costo de reputación muy alto.

4. El proceso de aprendizaje: Minimizando la pérdida

Ya tenemos la ecuación (la recta) y tenemos la métrica (el error). Ahora, ¿cómo se ajustan los coeficientes automáticamente?

A través de un proceso llamado **Optimización**. El algoritmo más famoso es el **Descenso del Gradiente**.

1. El modelo empieza con coeficientes al azar ().
2. Calcula la pérdida (MSE o Log Loss).
3. Calcula en qué dirección debe mover los coeficientes para que la pérdida baje (como alguien bajando una montaña en la niebla, sintiendo la inclinación del suelo con los pies).
4. Ajusta un poco los coeficientes y repite el proceso miles de veces hasta llegar al fondo del valle, donde el error es el mínimo posible.

5. Errores comunes y mejores prácticas

Confundir Parámetros con Hiperparámetros

Un error frecuente al empezar es pensar que nosotros elegimos el valor de los coeficientes (). No es así. Nosotros elegimos el modelo (Lineal) y quizás algunos ajustes externos (como la velocidad de aprendizaje), pero **los coeficientes los encuentra el algoritmo** analizando los datos.

Escalar los datos

Imagina que intentas predecir la salud de una persona usando su "Edad" (de 0 a 100) y su "Salario" (de 0 a 1,000,000). Si no escalas o normalizas estos datos, el modelo le dará muchísima más importancia al salario simplemente porque los números son más grandes, sesgando los coeficientes. Siempre es buena práctica llevar todas las variables a una escala similar (por ejemplo, de 0 a 1) antes de entrenar estos modelos.

Datos Anómalos (Outliers) en MSE

Debido a que el MSE eleva el error al cuadrado, un solo dato extremadamente mal cargado (una casa de 10 dólares o una de 100 millones en un barrio de clase media) puede "tironear" de la línea recta y arruinar todo el modelo. El análisis previo de outliers (visto en módulos anteriores) es crucial aquí.

Síntesis y Conexión

Hemos aprendido que un modelo de Machine Learning es, en esencia, una estructura matemática que busca ajustarse a la realidad.

- La **ecuación de la recta** nos da la estructura básica ().
- Los **coeficientes** nos dicen la importancia y dirección de cada variable.
- La **función de pérdida** (MSE para números, Log Loss para probabilidades) nos dice qué tan lejos estamos de la verdad.
- Y la **optimización** es el proceso de "perillear" esos coeficientes hasta que el error sea mínimo.

En la siguiente unidad, daremos un paso más allá. ¿Qué pasa cuando una sola línea no es suficiente? Veremos cómo los **Ensembles** combinan muchas de estas ideas para crear modelos mucho más poderosos y resistentes al error. Pero recuerda: incluso el algoritmo más complejo de IA moderna tiene en su ADN estos principios de pesos, predicciones y funciones de pérdida.

Ensembles para clasificación: bagging, boosting y stacking

¿Alguna vez has escuchado el dicho "dos cabezas piensan mejor que una"? En la vida cotidiana, cuando tenemos que tomar una decisión importante —como comprar un coche o elegir una carrera universitaria— no solemos confiar en la opinión de una sola persona. Consultamos a varios amigos, expertos y familiares para llegar a una conclusión más robusta.

En el Machine Learning, ocurre exactamente lo mismo. Un solo modelo (un "experto solitario") puede equivocarse debido a sus propios sesgos o limitaciones. Aquí es donde entran los **Ensembles** (o ensamblajes): una estrategia poderosa que combina múltiples modelos para obtener una predicción final mucho más precisa y confiable que la de cualquier modelo individual por separado.

En esta unidad, exploraremos las tres arquitecturas de ensamble más importantes de la industria: **Bagging**, **Boosting** y **Stacking**. Aprenderás cómo funcionan "bajo el capó" y, lo más importante, cómo decidir cuál de ellas usar para equilibrar la eterna balanza del Machine Learning: el **sesgo** y la **varianza**.

1. El corazón del problema: Sesgo vs. Varianza

Antes de sumergirnos en los tipos de ensembles, debemos entender por qué los necesitamos. Todo modelo de clasificación se enfrenta a dos "enemigos":

1. **Sesgo (Bias):** Ocurre cuando el modelo es demasiado simple para capturar los patrones de los datos (subajuste o *underfitting*). Por ejemplo, intentar separar datos complejos con una simple línea recta.
2. **Varianza (Variance):** Ocurre cuando el modelo es demasiado complejo y se vuelve muy sensible a pequeñas variaciones en los datos de entrenamiento (sobreajuste o *overfitting*). El modelo "memoriza" el ruido en lugar de aprender el patrón.

¿Qué hace un ensemble? Utiliza la fuerza del grupo para atacar uno de estos dos problemas (o ambos).

2. Bagging: La fuerza del paralelismo

Bagging es el acrónimo de *Bootstrap Aggregating*. Su filosofía es simple: entrenar múltiples modelos (generalmente de la misma familia) de forma independiente y **paralela**, y luego promediar sus resultados.

¿Cómo funciona el Bagging?

Imagina que tienes un dataset de 1000 filas. En lugar de entrenar un solo árbol de decisión gigante con todas las filas, el Bagging hace lo siguiente: 1. **Bootstrapping:** Crea varios subconjuntos de datos seleccionando filas al azar **con reemplazo** (una misma fila puede aparecer varias veces en un subconjunto). 2. **Entrenamiento:** Entrena un modelo (generalmente un árbol de decisión) para cada subconjunto. Todos se entrenan al mismo tiempo. 3. **Agregación:** Para clasificar una nueva instancia, cada árbol emite su "voto". La clase con más votos es la ganadora (votación mayoritaria).

El ejemplo rey: Random Forest

El **Random Forest** (Bosque Aleatorio) es la implementación más famosa de Bagging. Además de usar subconjuntos de datos, el Random Forest añade una capa extra de aleatoriedad: en cada nodo de los árboles, solo considera un subconjunto aleatorio de **características** (columnas).

- **¿Por qué importa?** Esto asegura que los árboles sean muy distintos entre sí. Si todos los árboles fueran similares, cometerían los mismos errores. Al ser diversos, sus errores se cancelan entre sí al promediar.

¿Cuándo usar Bagging?

El objetivo principal del Bagging es **reducir la varianza**. Si tienes un modelo potente pero muy inestable (como un Árbol de Decisión profundo), el Bagging lo "suaviza" y lo hace más robusto.

> **Ejemplo**
>
> **Caso de uso real. Netflix** utiliza variaciones de Random Forest para sus sistemas de recomendación. Al combinar miles de árboles, evitan que el sistema se vuelva demasiado específico para un solo usuario y logran generalizar mejor los gustos globales.

3. Boosting: El arte de aprender de los errores

A diferencia del Bagging, donde los modelos son independientes, en **Boosting** los modelos se entrenan de forma **secuencial**. Cada nuevo modelo intenta corregir específicamente los errores cometidos por el modelo anterior.

¿Cómo funciona el Boosting?

1. Se entrena un primer modelo (generalmente muy simple, llamado "weak learner" o aprendiz débil, como un árbol de solo un nivel).
2. Se identifican los datos que el modelo clasificó **mal**.
3. Se le da **más importancia (peso)** a esos errores para el entrenamiento del siguiente modelo.
4. El proceso se repite: cada modelo se enfoca en "lo que el anterior no pudo resolver".

Algoritmos estrella: AdaBoost y XGBoost

- **AdaBoost (Adaptive Boosting):** Fue uno de los pioneros. Ajusta los pesos de las filas en cada iteración.
- **Gradient Boosting (y su versión optimizada XGBoost):** En lugar de ajustar pesos, este algoritmo intenta predecir el "residuo" (el error) del modelo anterior usando el descenso de gradiente.

¿Cuándo usar Boosting?

El objetivo principal del Boosting es **reducir el sesgo**. Convierte una serie de modelos mediocres en un modelo predictivo extremadamente preciso. Sin embargo, hay que tener cuidado: si te pasas de iteraciones, el Boosting puede empezar a memorizar el ruido y aumentar la varianza (overfitting).

> **Ejemplo**
>
> **Caso de uso real. Uber** utiliza **XGBoost** para optimizar sus tiempos de llegada (ETA) y precios dinámicos. Es el estándar de oro en competiciones de ciencia de datos por su increíble precisión en datos tabulares complejos.

4. Stacking: El panel de expertos

Stacking (apilamiento) lleva el concepto de diversidad a otro nivel. Mientras que Bagging y Boosting suelen usar el mismo tipo de modelo (árboles), el Stacking combina modelos de **diferentes familias** (por ejemplo, una Regresión Logística, un SVM y un KNN).

¿Cómo funciona el Stacking?

1. **Nivel 0 (Base Models):** Entrenas varios modelos diferentes (por ejemplo, un Random Forest, una Regresión Logística y un clasificador K-Nearest Neighbors).
2. **Nivel 1 (Meta-Model):** En lugar de simplemente votar, las predicciones de los modelos de Nivel 0 se convierten en las "entradas" (features) para un nuevo modelo final, llamado **Meta-Modelo**.
3. El Meta-Modelo aprende qué experto es más confiable para qué tipo de casos. Por ejemplo, puede aprender que "cuando el Random Forest y el KNN no están de acuerdo, la Regresión Logística suele tener la razón".

¿Cuándo usar Stacking?

Se usa cuando buscas la máxima precisión posible y no te importa la complejidad computacional. Es una técnica común para ganar competiciones en **Kaggle**.

> **Ejemplo**
>
> **Caso de uso real.** En el sector de **Ciberseguridad**, el Stacking se utiliza para detectar malware. Se combinan modelos que analizan el tráfico de red, otros que analizan el código del archivo y otros que observan el comportamiento del sistema, permitiendo que un meta-modelo tome la decisión final basada en todas estas señales heterogéneas.

5. Cuadro Comparativo: ¿Cuál elegir?

Característica	Bagging	Boosting	Stacking
Entrenamiento	Paralelo (Independiente)	Secuencial (Dependiente)	Paralelo + Meta-modelo
Objetivo Principal	Reducir Varianza (evitar overfitting)	Reducir Sesgo (mejorar precisión)	Mejorar predicción combinando fortalezas
Modelos base	Usualmente homogéneos (ej. Árboles)	Usualmente homogéneos (ej. Árboles)	Heterogéneos (ej. RF + LogReg + SVM)
Riesgo	Generalmente seguro contra overfitting	Alto riesgo de overfitting si no se controla	Complejo de implementar y mantener
Ejemplo clásico	Random Forest	XGBoost / AdaBoost	"Blending" de modelos en Kaggle

6. Errores comunes y mejores prácticas

1. **Pensar que el Ensemble siempre es mejor:** Si tienes pocos datos, un ensemble sofisticado como Boosting puede sobreajustar (overfit) rápidamente. A veces, una Regresión Logística simple es más robusta.
2. **Falta de diversidad:** En Bagging, si todos tus modelos son casi iguales, el ensemble no aportará nada. La aleatoriedad (en filas o columnas) es la clave.
3. **Ignorar el costo computacional:** Entrenar 1000 modelos en un ensemble consume mucho más tiempo, memoria y energía que uno solo. En aplicaciones de tiempo real (como detectar un obstáculo en un coche autónomo), la velocidad del modelo es tan importante como su precisión.
4. **No limpiar los datos antes de Boosting:** A diferencia de Random Forest (Bagging), que es muy resistente a valores atípicos (*outliers*), los algoritmos de Boosting pueden obsesionarse con los outliers al intentar "corregir el error", arruinando el modelo.

Síntesis: De la intuición a la práctica

Hemos pasado de entender que un modelo puede ser limitado a ver cómo la colaboración entre ellos resuelve problemas complejos. - Si tu modelo es errático y cambia mucho con distintos datos: **Bagging**. - Si tu modelo es demasiado simple y no llega al nivel de precisión deseado: **Boosting**. - Si tienes varios modelos buenos y quieres que un "juez" decida cómo combinarlos: **Stacking**.

En la siguiente unidad, aprenderemos a medir exactamente *qué tan bien* lo están haciendo estos ensambles utilizando métricas como la **Matriz de Confusión** y el **F1-Score**.

Métricas para clasificación: matriz de confusión, F1 y AUC-ROC

Imagina que trabajas en el equipo de datos de un banco y has desarrollado un modelo de Machine Learning para detectar transacciones fraudulentas. Tu modelo presume con orgullo una **exactitud (accuracy) del 99.9%**. Suena perfecto, ¿verdad?

Sin embargo, al revisar los datos reales, descubres que el 0.1% de las transacciones son efectivamente fraudes. Si tu modelo simplemente decidiera marcar *todas* las transacciones como "legítimas" sin analizarlas, ¡aún así tendría un 99.9% de exactitud! Pero el banco seguiría perdiendo millones porque no detectó ni un solo fraude.

Este escenario ilustra por qué la exactitud es engañosa y por qué, como profesionales de los datos, necesitamos herramientas más precisas para evaluar el éxito de un modelo. En esta unidad, aprenderás a mirar "bajo el capó" de las predicciones utilizando la **Matriz de Confusión**, el **F1-Score** y la curva **AUC-ROC**.

1. La Matriz de Confusión: El Mapa de la Verdad

La **Matriz de Confusión** es la base fundamental de todas las métricas de clasificación. No es más que una tabla de 2x2 (para problemas binarios) que cruza la realidad con las predicciones del modelo.

Los cuatro cuadrantes de la matriz

Para entenderla, supongamos un modelo que predice si un correo es **Spam (Positivo)** o **No Spam (Negativo)**:

1. **Verdaderos Positivos (TP - True Positives):** El modelo dijo "es Spam" y efectivamente era Spam. ¡Acierto!
2. **Verdaderos Negativos (TN - True Negatives):** El modelo dijo "No es Spam" y el correo era legítimo. ¡Acierto!
3. **Falsos Positivos (FP - False Positives):** El modelo dijo "es Spam", pero era un correo importante de tu jefe. Este es el famoso **Error Tipo I**.
4. **Falsos Negativos (FN - False Negatives):** El modelo dijo "No es Spam", pero terminó siendo un correo de phishing. Este es el **Error Tipo II**.

¿Por qué es importante esta distinción?

Porque en el mundo real, los errores no tienen el mismo costo. En una prueba de cáncer, un **Falso Negativo** (decir que alguien está sano cuando está enfermo) es catastrófico. En cambio, en un filtro de correo, un **Falso Positivo** (mandar un correo importante a la papelera) es más molesto que dejar pasar un poco de spam.

2. Precisión y Recall: ¿Calidad o Cantidad?

A partir de la matriz de confusión, derivamos dos métricas que suelen estar en conflicto:

Precisión (Precision)

Responde a la pregunta: **De todos los que predije como positivos, ¿cuántos eran realmente positivos?** $$Precision = \frac{TP}{TP + FP}$$ * **Cuándo importa:** Cuando el costo de un Falso Positivo es muy alto (ej. bloquear una tarjeta de crédito legítima). Queremos estar muy seguros cuando decimos que algo es positivo.

Exhaustividad o Sensibilidad (Recall)

Responde a la pregunta: **De todos los casos que eran realmente positivos, ¿cuántos logré atrapar?** $$Recall = \frac{TP}{TP + FN}$$ * **Cuándo importa:** Cuando el costo de un Falso Negativo es crítico (ej. diagnóstico de enfermedades o detección de fallos en motores de aviones). Queremos atrapar a todos los "sospechosos", aunque nos equivoquemos un poco.

3. El F1-Score: El Balance Necesario

A menudo, mejorar la precisión hace que el recall baje, y viceversa. Es como una manta corta: si te tapas los pies, te descubres la cabeza. El **F1-Score** es una métrica que combina ambas en un solo número.

La Media Armónica

El F1-Score no usa un promedio normal (media aritmética), sino una **media armónica**: $$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

¿Por qué no usar el promedio común? Imagine un modelo con un Recall de 1.0 (atrapa todo) pero una Precisión de 0.01 (falla casi siempre). El promedio normal daría 0.50, lo cual parece "aceptable". La media armónica, en cambio, dará un valor cercano a 0 si cualquiera de las dos métricas es baja. El F1-Score nos obliga a ser buenos en ambos frentes.

4. Curva ROC y AUC: Evaluando la Capacidad de Separación

Hasta ahora, hemos asumido que el modelo decide si algo es positivo o negativo usando un umbral fijo (usualmente 0.5 de probabilidad). Pero, ¿qué pasa si cambiamos ese umbral?

La Curva ROC (Receiver Operating Characteristic)

Es una gráfica que muestra el rendimiento del modelo en **todos** los umbrales posibles. Relaciona: * **Eje Y:** Tasa de Verdaderos Positivos (Recall). * **Eje X:** Tasa de Falsos Positivos (qué tan rápido nos equivocamos al intentar atrapar positivos).

AUC (Area Under the Curve)

El AUC es el área bajo esa curva. Es un valor entre 0 y 1: * **AUC = 0.5:** El modelo es tan útil como lanzar una moneda al aire. No sabe distinguir entre clases. * **0.7 - 0.8:** Es un modelo aceptable/bueno. * **0.9 o**

más: Es un modelo excelente. * **AUC = 1.0:** El modelo es perfecto; separa perfectamente los casos positivos de los negativos.

La gran ventaja del AUC: A diferencia del F1-Score, el AUC evalúa al modelo independientemente del umbral que elijas. Te dice qué tan capaz es el modelo de poner a los "positivos" por encima de los "negativos" en términos de probabilidad.

5. Aplicaciones en la Industria

Veamos cómo eligen métricas las empresas líderes:

1. **Detección de Fraude (Stripe/Mastercard):** Priorizan la **Precisión** dentro de un **F1-Score** equilibrado. Si bloqueas demasiadas compras legítimas (Falsos Positivos), los usuarios dejarán de usar tu plataforma por frustración. El AUC se usa para comparar si un nuevo modelo de Deep Learning es superior al anterior sin importar el umbral de bloqueo.
2. **Diagnóstico por Imágenes (Salud):** En la detección de neumonía mediante rayos X, se prioriza el **Recall**. Es preferible hacerle una prueba adicional a alguien sano (Falso Positivo) que enviar a casa a alguien enfermo que necesita tratamiento urgente (Falso Negativo).
3. **Motores de Recomendación (Netflix):** Aquí importa que lo que te recomienden te guste (**Precisión**). Si no te muestran *todas* las películas que te podrían gustar (bajo Recall), no importa tanto, siempre que las 5 que te muestran sean excelentes.

6. Errores Comunes y Mejores Prácticas

- **Error: Optimizar solo Accuracy en datasets desbalanceados.**
- *Por qué:* Como vimos al inicio, puedes tener un 99% de exactitud y ser un modelo inútil. Siempre revisa la **Matriz de Confusión**.
- **Error: Ignorar el contexto del negocio.**
- *Solución:* Pregunta siempre: "¿Qué duele más: un Falso Positivo o un Falso Negativo?". Esa respuesta definirá si optimizas Precision o Recall.
- **Mejor Práctica: Usar `classification_report` de Scikit-Learn.** Esta función te devuelve Precision, Recall y F1 para cada clase por separado, permitiéndote ver si el modelo es malo solo con la clase minoritaria.

Resumen de la Unidad

- La **Matriz de Confusión** desglosa aciertos y errores en cuatro cuadrantes (TP, TN, FP, FN).
- La **Precisión** penaliza los Falsos Positivos; el **Recall** penaliza los Falsos Negativos.
- El **F1-Score** es el equilibrio ideal cuando necesitamos un balance entre precisión y recall.
- El **AUC-ROC** mide la capacidad global del modelo para distinguir entre clases, sin importar el umbral de decisión.

Elegir la métrica correcta es, quizás, la decisión más importante de un Data Scientist, ya que determina qué "comportamiento" premiarás en tu algoritmo.

Validación cruzada: estimar desempeño con múltiples particiones

Imagina que estás entrenando para una maratón. Si solo entrenas en una pista plana y soleada, ¿cómo sabrás si estás listo para una carrera con colinas, lluvia o viento? Si el día de la maratón las condiciones cambian, tu entrenamiento previo no servirá de mucho porque se "especializó" demasiado en un solo escenario.

En Machine Learning sucede lo mismo. Hasta ahora, hemos aprendido a dividir nuestros datos en dos grupos: **Entrenamiento (Train)** y **Prueba (Test)**. Sin embargo, confiar en una sola división aleatoria es como entrenar solo en esa pista plana: podrías tener suerte y que los datos del Test sean muy fáciles, o mala suerte y que sean atípicos, dándote una falsa sensación de éxito o fracaso.

La **Validación Cruzada (Cross-Validation)** es la solución profesional a este problema. Es el proceso de "correr la maratón" múltiples veces en diferentes condiciones para asegurarte de que tu modelo es verdaderamente robusto y no solo "tuvo suerte" con una partición de datos.

1. El problema de la partición única: ¿Por qué no basta con Train/Test?

Cuando usamos una división simple (por ejemplo, 80% para entrenar y 20% para probar), estamos tomando una fotografía estática de nuestros datos. El rendimiento que obtenemos en ese 20% depende Enteramente de qué filas cayeron allí por azar.

Existen dos riesgos principales: 1. **Variabilidad del Azar:** Si los datos son pocos, un par de valores atípicos en el conjunto de prueba pueden desplomar tu métrica de precisión, aunque tu modelo sea bueno. 2. **Sobreajuste al Test (Data Leakage indirecto):** Si ajustas tu modelo muchas veces para que rinda bien en **ese** conjunto de prueba específico, eventualmente estarás "enseñándole" los secretos de ese 20% de datos, perdiendo la capacidad de generalizar ante datos nuevos del mundo real.

La validación cruzada resuelve esto haciendo que **cada dato del dataset tenga la oportunidad de ser parte del entrenamiento y parte de la validación**.

2. K-Fold Cross-Validation: El estándar de la industria

La técnica más utilizada se llama **K-Fold**. El concepto es sencillo: dividimos el dataset en "K" partes iguales (llamadas *folds* o pliegues).

¿Cómo funciona el proceso?

Si elegimos un **K = 5** (un valor estándar muy común), el proceso sigue estos pasos:

1. **División:** El dataset se divide en 5 partes iguales.
2. **Iteración 1:** Se entrena el modelo con las partes 2, 3, 4 y 5. Se valida con la parte 1.
3. **Iteración 2:** Se entrena con 1, 3, 4 y 5. Se valida con la parte 2.

4. **Repetición:** Esto se repite hasta que cada una de las 5 partes haya sido usada como conjunto de validación exactamente una vez.
5. **Promedio:** Al final, tendrás 5 métricas de desempeño (por ejemplo, 5 valores de F1-Score). Calculas el **promedio** y la **desviación estándar** para conocer el rendimiento real.

¿Por qué importa el promedio y la desviación?

Si obtienes un F1-Score promedio de 0.85, pero en un fold fue de 0.99 y en otro de 0.40, tu modelo es **inestable**. La validación cruzada te avisa que tu modelo depende demasiado de cómo se reparten los datos. Un modelo ideal tendría un promedio alto y una desviación estándar baja (resultados similares en todos los folds).

3. Stratified K-Fold: Manejando el desbalance de clases

Imagina que trabajas en un banco detectando fraudes con tarjeta de crédito. En tu base de datos, el 99% de las transacciones son legítimas y solo el 1% son fraude.

Si usas un K-Fold normal, existe el riesgo de que en uno de tus *folds* de validación **no caiga ningún caso de fraude**. El modelo no tendrá nada que validar y la métrica de esa iteración será inútil.

Aquí es donde entra el **Stratified K-Fold (Validación Cruzada Estratificada)**.

El concepto de Estratificación

Esta variante asegura que **cada fold mantenga la misma proporción de clases que el dataset original**. Si tu dataset tiene un 1% de fraude, cada uno de tus 5 folds tendrá exactamente un 1% de casos de fraude.

Regla de oro: Siempre que estés trabajando en un problema de **clasificación**, especialmente si las clases están desbalanceadas, utiliza `StratifiedKFold` en lugar de `KFold` simple.

4. Leave-One-Out (LOO): El caso extremo

Existe una variante llamada **Leave-One-Out Cross-Validation (LOOCV)** donde el valor de *K* es igual al número total de filas en tu dataset (*n*).

Si tienes 100 datos: * Entrenas con 99 datos y validas con 1 solo dato. * Repites esto 100 veces.

¿Cuándo usarlo? Solo cuando tienes **muy pocos datos** (por ejemplo, un estudio médico con solo 20 pacientes). **¿Por qué no usarlo siempre?** Es computacionalmente carísimo. Si tienes un millón de filas, tendrías que entrenar el modelo un millón de veces. Para la mayoría de los casos en Data Science moderno, K=5 o K=10 es más que suficiente.

5. Aplicaciones y Casos del Mundo Real

Las grandes empresas tecnológicas no lanzan modelos basados en una sola partición de datos:

- **Sistemas de Recomendación (Netflix/Spotify):** Utilizan validación cruzada para asegurarse de que sus algoritmos de recomendación funcionan igual de bien para usuarios nuevos que para usuarios antiguos, evitando sesgos por la fecha de registro.
- **Detección de Fraude (Bancos como HSBC o JPMorgan):** Utilizan obligatoriamente **Stratified K-Fold**. Dado que el fraude es un "evento raro", necesitan asegurar que el modelo sea evaluado contra esos casos críticos en múltiples escenarios para evitar que un modelo "perezoso" simplemente aprenda a decir que "nada es fraude" para obtener un 99% de precisión.
- **Diagnóstico Médico:** En el desarrollo de modelos para detectar enfermedades a partir de radiografías, donde los datos suelen ser costosos y escasos, la validación cruzada permite extraer la máxima información posible de cada imagen disponible sin desperdiciar datos en una sola partición de test.

6. Errores comunes y mejores prácticas

Error 1: "La Validación Cruzada entrena mi modelo final"

Realidad: La CV es una herramienta para **evaluar**. Una vez que has usado CV para decidir qué algoritmo o qué hiperparámetros funcionan mejor, debes re-entrenar tu modelo final usando **todo** el conjunto de entrenamiento disponible antes de pasarlo a producción.

Error 2: Olvidar el conjunto de "Prueba Final" (Hold-out set)

Muchos analistas cometen el error de usar validación cruzada y creer que ya no necesitan un conjunto de test independiente. **La práctica correcta:** 1. Separa un 20% de datos y guárdalos en un "cofre bajo llave" (Test Set). 2. Usa el 80% restante para hacer Validación Cruzada (Train/Validation). 3. Solo cuando estés totalmente seguro de tu modelo, ábrelo y pruébalo una única vez contra ese 20% inicial. Esto garantiza que no te has "auto-engañado" durante el proceso de optimización.

Error 3: No usar estratificación en clasificación

Como mencionamos, si no estratificas, tus métricas de F1-Score o AUC-ROC variarán salvajemente entre folds simplemente porque algunos folds tendrán más ejemplos de la clase positiva que otros.

Síntesis y Conexiones

Hemos pasado de evaluar modelos con una simple división a usar un sistema de rotación múltiple que nos da una visión honesta del rendimiento.

- **K-Fold:** Divide el dataset en K partes y rota la validación.
- **Stratified:** Asegura que la proporción de "clases" sea constante en cada parte.
- **Promedio y Desviación:** Son tus nuevos mejores amigos para medir la estabilidad.

En la siguiente unidad, daremos el paso final: ¿cómo usamos esta validación cruzada para encontrar automáticamente los mejores "ajustes" (hiperparámetros) de nuestro modelo? La validación cruzada será el motor que impulse esa búsqueda inteligente.

Ajuste de hiperparámetros con GridSearchCV

Como científicos de datos, rara vez nos conformamos con el primer modelo que entrenamos. Imagina que estás afinando un instrumento musical o buscando la temperatura exacta para que un pastel suba perfectamente: eso es el **Ajuste de Hiperparámetros**.

1. Parámetros vs. Hiperparámetros

Es fundamental distinguir estos dos conceptos:

- **Parámetros:** Son los valores que el modelo aprende automáticamente de los datos durante el entrenamiento (como los coeficientes en una regresión). Tú no los tocas.
- **Hiperparámetros:** Son "perillas" que tú configuras *antes* de que el modelo empiece a aprender. Por ejemplo, ¿cuántos árboles debe tener mi bosque? ¿Qué tan profundo puede ser cada árbol?

2. El problema del ajuste manual

Podrías intentar cambiar los valores uno por uno manualmente:

1. Pruebo con 10 árboles... Accuracy 85%.
2. Pruebo con 20 árboles... Accuracy 87%.
3. Pruebo con 20 árboles y profundidad 5...

Este proceso es lento, tedioso y muy propenso a errores. Además, ¿cómo sabemos que ese 87% no es solo "buena suerte" con una partición de datos específica?

3. GridSearchCV: La búsqueda exhaustiva

`GridSearchCV` (de Scikit-Learn) es nuestra herramienta de automatización. Funciona bajo dos principios:

1. **Grid (Rejilla):** Define una lista de valores para cada hiperparámetro. El sistema probará **todas las combinaciones posibles**.
2. **CV (Cross-Validation):** Para cada combinación, no solo entrena una vez, sino que realiza una validación cruzada (visto en la unidad anterior). Esto asegura que los mejores parámetros lo sean por su capacidad de generalizar, no por azar.

4. Atributos clave tras la búsqueda

Una vez ejecutado el método `.fit()`, el objeto resultante nos ofrece información valiosa:

- `best_params_`: El diccionario con la combinación ganadora.
- `best_score_`: El promedio de la métrica (accuracy, F1, etc.) obtenida por la mejor combinación durante la validación cruzada.
- `best_estimator_`: El modelo final, ya entrenado con los mejores parámetros sobre todo el conjunto de entrenamiento, listo para predecir.

Ejemplo Rápido de Concepto

Si defines: `param_grid = {'n_estimators': [10, 50], 'max_depth': [2, 5]}` GridSearchCV probará 4 modelos (10-2, 10-5, 50-2, 50-5), multiplicados por el número de carpetas (folds) de tu validación cruzada. ¡Es potente pero consume cómputo!

Tarjetas de repaso: decisiones clave en modelado y evaluación

¡Has llegado al final del laboratorio de modelado! Esta unidad está diseñada para consolidar los conceptos más importantes que hemos cubierto: desde la elección de algoritmos de regresión hasta la optimización fina con `GridSearchCV`.

En el mundo real del Machine Learning, no basta con saber escribir el código; la parte más difícil es **tomar la decisión correcta**:

- ¿Debo usar una regresión lineal o logística para este problema?
- ¿Es preferible un modelo robusto como Random Forest o uno secuencial como XGBoost?
- ¿Qué métrica importa más si estoy detectando fraude bancario?

Estas tarjetas te ayudarán a fijar esas definiciones y criterios de decisión en tu memoria.

01 ¿Cuál es la diferencia principal entre la Regresión Lineal y la Regresión Logística?

La **Regresión Lineal** predice valores numéricos continuos (como el precio de una casa), mientras que la **Regresión Logística** predice la probabilidad de que algo pertenezca a una categoría (como si un correo es Spam o No).

02 ¿Qué es la función Sigmoide y para qué sirve?

Es una curva que comprime cualquier valor numérico a un rango entre **0 y 1**. Se usa en la regresión logística para convertir el resultado de una ecuación lineal en una **probabilidad**.

03 ¿Cómo funciona un Random Forest?

Es un tipo de Ensemble basado en **Bagging** que entrena múltiples árboles de decisión en paralelo. Cada árbol usa una submuestra aleatoria de datos y características, reduciendo la varianza y evitando el sobreajuste.

04 ¿Cuál es la lógica principal detrás de las técnicas de Boosting?

El **Boosting** entrena modelos de forma **secuencial**, donde cada nuevo modelo intenta corregir los errores cometidos por el anterior. Ejemplos comunes son XGBoost y AdaBoost.

05 ¿Qué es el F1-Score y cuándo es preferible usarlo sobre la Exactitud (Accuracy)?

Es la media armónica entre **Precision** y **Recall**. Se utiliza cuando tenemos datasets desbalanceados y necesitamos un equilibrio entre evitar falsos positivos y no perder los casos positivos reales.

06 ¿Qué mide la métrica AUC-ROC?

El **AUC (Área bajo la curva)** mide la capacidad del modelo para distinguir entre clases. Un valor de 1.0 es un modelo perfecto, mientras que 0.5 indica que el modelo no es mejor que el azar.

07 ¿En qué consiste el K-Fold Cross-Validation?

Dividir el dataset en **K grupos** (folds). El modelo se entrena K veces, usando cada vez un grupo distinto para validar y los demás para entrenar. Esto nos da una estimación del desempeño más robusta y fiable.

08 ¿Para qué sirve el Stratified K-Fold?

Es una variante de K-Fold que asegura que cada grupo mantenga la **misma proporción de clases** que el dataset original. Es fundamental cuando trabajamos con clases muy minoritarias (ej. detección de enfermedades raras).

09 ¿Cuál es la diferencia entre un parámetro y un hiperparámetro?

Los **parámetros** son aprendidos por el modelo durante el entrenamiento (ej. coeficientes), mientras que los **hiperparámetros** son configuraciones que nosotros definimos antes de empezar (ej. profundidad máxima de un árbol).

10 ¿Qué hace la herramienta GridSearchCV de Scikit-Learn?

Es una técnica de búsqueda exhaustiva que prueba **todas las combinaciones posibles** de una lista de hiperparámetros definida por el usuario, usando validación cruzada para encontrar la mejor de ellas.

11 En detección de fraude, ¿qué error suele ser más costoso para el negocio?

Un error de **Falso Negativo (FN)**, ya que significaría dejar pasar una transacción fraudulenta sin detectarla, resultando en pérdida de dinero. Maximizar el **Recall** suele ser prioridad aquí.

12 ¿Qué es el Overfitting o sobreajuste?

Significa que el modelo ha "memorizado" tanto los datos de entrenamiento (incluyendo el ruido) que pierde la capacidad de generalizar a datos nuevos. Se detecta cuando el error en entrenamiento es muy bajo pero en test es muy alto.

PRE-ENTREGA DEL PROYECTO

Actividad evaluable · parte del proyecto integrador

Pre-entrega: Modelado y Evaluación de Clasificación

El objetivo de esta práctica es entrenar un modelo de clasificación binaria y evaluar su desempeño utilizando las métricas vistas en clase. Trabajarás con un dataset clásico para predecir si un cliente abandonará un servicio (Churn).

Cómo resolver esta entrega

Pasos a seguir:

1. **Preparación del Entorno**: Crea un repositorio en GitHub y clona el proyecto en tu máquina local. Asegúrate de incluir un archivo `.gitignore` para Python.
2. **Análisis y Modelado**: Crea un Notebook de Jupyter (`modelado_metricas.ipynb`) donde realices la carga de datos, la división de conjuntos de entrenamiento y prueba (80/20), y entrenes un modelo de **Logistic Regression** o **Random Forest**.
3. **Evaluación de Métricas**: Calcula las siguientes métricas para tu modelo utilizando el set de prueba: * Accuracy * Precision * Recall * F1-Score * Matriz de Confusión
4. **Interpretación**: En una celda de Markdown dentro del mismo notebook, justifica qué métrica es la más importante para el problema de Churn (pérdida de clientes) y explica por qué basándote en los costos de los falsos negativos.
5. **Documentación**: Configura un archivo `README.md` que explique brevemente cómo ejecutar el notebook y un resumen de los resultados obtenidos.
6. **Publicación**: Sube tus cambios al repositorio y asegúrate de que sea público para su revisión.

Objetivos de aprendizaje

- Implementar un flujo de entrenamiento y validación de modelos de Machine Learning.
- Calcular e interpretar métricas de clasificación como Precision, Recall y F1-Score.
- Utilizar matrices de confusión para diagnosticar errores del modelo.
- Justificar la selección de una métrica de evaluación según el contexto de negocio.

Criterios de aceptación

- El repositorio contiene un Notebook ejecutable sin errores con la división de datos (Train/Test).
- Se incluyen al menos 4 métricas de evaluación diferentes (Accuracy, Precision, Recall, F1).
- La matriz de confusión se visualiza claramente mediante un mapa de calor o tabla.
- El README incluye una breve descripción del problema y las instrucciones de ejecución.

- Existe una conclusión escrita que vincula las métricas obtenidas con el impacto en el negocio (Churn).

Rúbrica de evaluación: Modelado y métricas para clasificación binaria (Churn)

Rúbrica para evaluar un repositorio público de GitHub con un notebook de Jupyter, entrenamiento de un modelo de clasificación binaria, cálculo e interpretación de métricas y documentación básica. El foco está en la correcta implementación del flujo de machine learning, la calidad de la evaluación y la claridad de la presentación del trabajo en un proyecto reproducible.

Criterio	Descripción	Peso
Estructura del repositorio y entrega reproducible	Evalúa que el repositorio esté correctamente organizado, sea público y contenga los archivos mínimos esperados para reproducir la práctica: notebook, README, `.gitignore` y una estructura clara de proyecto.	15%
Carga de datos, división del conjunto y preparación del experimento	Evalúa si el notebook carga correctamente los datos, realiza la división entrenamiento/prueba 80/20 y establece un flujo de experimentación adecuado para clasificación binaria.	20%
Entrenamiento del modelo de clasificación	Evalúa si se entrena correctamente un modelo de Logistic Regression o Random Forest, con una implementación coherente con el problema de clasificación binaria.	25%
Cálculo e interpretación de métricas de evaluación	Evalúa el cálculo correcto de Accuracy, Precision, Recall, F1-Score y Matriz de Confusión en el conjunto de prueba, así como su presentación de forma comprensible.	25%
Interpretación del problema de churn y justificación de la métrica principal	Evalúa la celda de Markdown donde se argumenta cuál métrica es más importante para churn y por qué, considerando especialmente el costo de los falsos negativos.	10%
Documentación en README y comunicación del proyecto	Evalúa que el README explique cómo ejecutar el notebook y resuma los resultados obtenidos de forma breve, útil y clara para un tercero.	5%

Total: 100 pts | Aprobación: 70 pts

MÓDULO 12

Visualización de Datos

Inicia tu camino en el análisis de datos aprendiendo a transformar información compleja en narrativas claras y accionables. Este módulo introduce la estructura del storytelling aplicada a datos, los componentes esenciales de visualizaciones efectivas y los principios básicos de usabilidad y percepción visual.

Fundamentos y Estrategias de Visualización de Datos

Visualización de Datos: El Arte de Hacer Visible lo Invisible

Imagina que entras en un almacén gigantesco, del tamaño de varios estadios de fútbol. El lugar está repleto de estantes que llegan hasta el techo, y cada estante está lleno de cajas sin etiquetas. Dentro de esas cajas hay facturas, registros de temperatura, nombres de clientes y fechas de entrega. Tienes exactamente cinco minutos para decirle al dueño del almacén si el negocio está ganando dinero o si está a punto de quebrar.

¿Cómo lo harías? Leer cada papel es imposible. Abrir cada caja tomaría años. Este es el problema fundamental de la era moderna: **tenemos océanos de datos, pero nos ahogamos en la búsqueda de conocimiento.**

Aquí es donde entra la **Visualización de Datos**. No se trata simplemente de hacer "dibujos bonitos" o gráficas coloridas para una presentación de PowerPoint. Es la disciplina de traducir números abstractos a un lenguaje que el cerebro humano está diseñado para entender instantáneamente: el lenguaje visual.

En esta unidad, consolidaremos todo lo aprendido en el módulo, explorando por qué la visualización es la herramienta más poderosa del científico de datos, cómo elegir la representación adecuada para cada historia y cómo evitar las trampas que convierten un buen dato en una mentira visual.

1. ¿Por qué visualizamos? El poder de la percepción humana

Para entender por qué visualizamos, primero debemos entender una limitación biológica. El cerebro humano no evolucionó para procesar hojas de cálculo de Excel con un millón de filas. Evolucionó para detectar patrones en la naturaleza: el movimiento de un depredador en la hierba, la madurez de una fruta por su color o la posición del sol para orientarse.

La superioridad del canal visual

Cuando miramos una tabla de números, nuestra "memoria de trabajo" (una especie de RAM mental de muy corto plazo) tiene que hacer un esfuerzo enorme para comparar el valor de la fila 10 con el de la fila 500. Sin embargo, cuando vemos dos barras de distinto tamaño, el cerebro procesa esa

diferencia de forma **pre-atentiva**. Esto significa que antes de que seas consciente de que estás analizando un gráfico, tu cerebro ya sabe qué barra es mayor.

El ejemplo del Cuarteto de Anscombe: Este es uno de los ejemplos más famosos en la estadística. Imagina cuatro conjuntos de datos distintos. Si calculas sus estadísticas básicas (el promedio, la varianza), ¡todos son idénticos! Sin embargo, al graficarlos, descubres que uno es una línea recta, otro es una curva perfecta, otro tiene un valor atípico (outlier) y el último es un grupo vertical. Sin la visualización, habrías asumido que los cuatro grupos de datos se comportaban igual. La visualización revela la **verdadera estructura** que los números ocultan.

2. El Proceso de Transformación: De "Inventario" a "Menú"

Para enseñar visualización a principiantes, a menudo usamos la **Analogía del Menú de Restaurante**:

- **Los Datos Crudos** son como el inventario de un supermercado mayorista. Hay miles de productos, están en cajas, no hay un orden aparente para el consumidor final. Es abrumador y nadie puede "comer" de un inventario.
- **La Visualización de Datos** es el menú del restaurante. El chef (tú, el analista) ha seleccionado los ingredientes más importantes, los ha organizado por categorías (entradas, platos fuertes) y les ha dado un contexto (precios, descripciones).

El objetivo de la visualización no es mostrar *todo* lo que hay en la base de datos, sino **curar y dar contexto** para que el usuario pueda tomar una decisión (pedir un plato o, en el caso de la empresa, aprobar un presupuesto).

Los pasos del flujo de trabajo:

1. **Recolectar:** Obtener los números brutos (ej. recibos de ventas).
2. **Limpiar:** Eliminar errores o duplicados que puedan ensuciar la gráfica.
3. **Elegir:** Seleccionar el gráfico que mejor cuente la historia.
4. **Mapear:** Convertir los números en atributos visuales (ej. que $1,000 equivalgan a 100 píxeles de altura).
5. **Comunicar:** Añadir títulos, leyendas y anotaciones que guíen al lector.

3. La Decisión Crucial: ¿Qué gráfico utilizar?

Un error frecuente es pensar que "cualquier gráfico sirve para cualquier dato". Esto es falso. La elección del gráfico depende enteramente de la **pregunta** que intentes responder.

A. Comparación de categorías (¿Quién vende más?)

Cuando tienes grupos distintos (nombres de países, categorías de productos), el **Gráfico de Barras** es el rey indiscutible.

- *Por qué:* El ojo humano es extremadamente preciso midiendo la longitud desde una base común. Podemos distinguir fácilmente si una barra es un 5% más alta que otra.
- *Ejemplo real:* Comparar las ventas de iPhone vs. Android en el último trimestre.

B. Evolución en el tiempo (¿Cómo vamos?)

Para mostrar tendencias, cambios o ciclos a lo largo de días, meses o años, el **Gráfico de Líneas** es esencial.

- *Por qué:* Las líneas conectan los puntos creando una sensación de continuidad y movimiento. Nos permiten ver rápidamente si algo está subiendo, bajando o manteniéndose estable.
- *Ejemplo real:* El valor de una acción en la bolsa de valores durante los últimos 10 años.

C. Partes de un todo (¿Cómo se divide el presupuesto?)

Aquí es donde aparecen los famosos **Gráficos de Pastel (Pie Charts)**.

- *Advertencia:* Los estudiantes principiantes suelen abusar de ellos. El ojo humano es malo comparando ángulos y áreas.
- *Mejor práctica:* Úsalos solo si tienes 2 o 3 categorías máximo (ej. "Votos a favor vs. Votos en contra"). Si tienes 15 categorías, un gráfico de pastel se vuelve ilegible. En su lugar, usa un gráfico de barras horizontales ordenadas de mayor a menor.

D. Relación entre variables (¿A más publicidad, más ventas?)

El **Diagrama de Dispersión (Scatter Plot)** coloca puntos en un plano X-Y.

- *Por qué:* Es la herramienta perfecta para detectar correlaciones. Si los puntos forman una nube que sube hacia la derecha, sabemos que ambas variables crecen juntas.
- *Ejemplo real:* Ver la relación entre las horas de estudio de un estudiante y su calificación final.

4. Casos de Éxito en la Industria: Visualización en Acción

La visualización no es solo teoría; es el motor detrás de las decisiones más importantes en empresas y gobiernos.

Caso 1: El Tablero de Control de la Pandemia (Johns Hopkins)

Durante la crisis del COVID-19, la Universidad Johns Hopkins creó un mapa interactivo que se convirtió en la fuente principal de información para gobiernos de todo el mundo.

- **El valor:** Convirtió millones de filas de reportes hospitalarios en círculos rojos sobre un mapa mundial. Los tomadores de decisiones podían ver los "puntos calientes" (hotspots) al instante y enviar recursos (oxígeno, médicos) a las zonas con círculos más grandes.
- **Lección:** Una buena visualización puede salvar vidas al permitir una respuesta rápida ante emergencias.

Caso 2: Netflix y la Estrategia de Contenidos

¿Alguna vez te has preguntado por qué Netflix parece saber exactamente qué serie recomendarte? Netflix utiliza herramientas como **Tableau** para visualizar el comportamiento de millones de usuarios.

- **La aplicación:** Visualizan en qué minuto exacto la gente deja de ver un episodio o qué géneros son más populares en ciertos países. Si los datos visualizados muestran que los usuarios que ven

"Drama" también suelen ver "Documentales de Crímenes", Netflix decide producir una serie que combine ambos.

- **Lección:** La visualización dirige la estrategia de inversión de miles de millones de dólares.

5. El Lado Oscuro: Errores Comunes y Ética Visual

Como dice la frase célebre: "Es fácil mentir con estadísticas, pero es mucho más fácil mentir con un gráfico". Como futuro profesional de los datos, debes ser capaz de detectar y evitar estas trampas.

Error 1: El "Chart Junk" (Ruido Visual)

Muchos principiantes intentan decorar sus gráficos con efectos 3D, sombras, fondos con imágenes o demasiados colores. Esto se llama *chart junk*.

- **El problema:** El exceso de decoración distrae del dato. Si un gráfico 3D hace que una barra parezca más alta de lo que realmente es, estás engañando al lector.
- **Mejor práctica:** Menos es más. Si un elemento (como una línea de cuadrícula o un color brillante) no ayuda a entender el dato, elimínalo.

Error 2: Ejes Truncados (La mentira de la escala)

Imagina un gráfico de barras donde la escala no empieza en cero, sino en 90. Una pequeña diferencia entre 92 y 95 parecerá un abismo gigantesco visualmente.

- **Por qué es un problema:** Es una táctica común en política o marketing para exagerar logros pequeños.
- **La regla de oro:** En los gráficos de barras, el eje de valor **siempre debe empezar en cero**.

Error 3: El Color como Decoración, no como Significado

Usar rojo para "ganancias" y verde para "pérdidas" confunde al cerebro, porque culturalmente asociamos el rojo con peligro o pérdida.

- **Mejor práctica:** El color debe tener una función lógica. Usa colores contrastantes para categorías diferentes y gradaciones de un mismo color para mostrar intensidad (ej. un azul claro para poco calor y un azul oscuro para mucho calor).

6. Síntesis y Conexiones Finales

Hemos recorrido un camino que va desde la biología de nuestra percepción hasta las decisiones estratégicas de gigantes como Netflix. La visualización de datos es el puente final entre el análisis técnico y la acción humana.

Puntos clave para recordar:

1. **Visualizamos para encontrar patrones** que las tablas ocultan.
2. **El gráfico depende de la pregunta:** Barras para comparar, líneas para tendencias, dispersión para relaciones.

3. **La claridad supera a la estética:** Un gráfico simple y honesto es mucho más valioso que uno complejo y confuso.
4. **La ética es fundamental:** No manipules las escalas ni abuses de la decoración para "vender" una conclusión que los datos no respaldan.

Al dominar la visualización, dejas de ser alguien que "maneja datos" para convertirte en alguien que **cuenta historias con evidencia**. En el próximo módulo, daremos el siguiente paso: aprender cómo esos datos que hoy visualizamos pueden usarse para entrenar modelos de Machine Learning que predigan el futuro.

Componentes de una visualización efectiva

¿Alguna vez te has encontrado frente a un gráfico o mapa que, a pesar de ser visualmente atractivo, no logra transmitir claramente el mensaje que esperabas?

En el análisis de datos, no basta con mostrar información; es fundamental que la visualización comunique un insight claro y accionable.

En esta unidad, descubrirás cuáles son los componentes esenciales que hacen que una visualización sea efectiva y cómo seleccionar los datos e insights clave para construir una narrativa visual que impacte a tu audiencia.

Este conocimiento es crucial para transformar análisis complejos en historias comprensibles y útiles, especialmente en proyectos de ciencia de datos y análisis geoespacial donde la claridad y la precisión son vitales.

Componentes de una visualización efectiva

Para que una visualización cumpla su propósito, debe integrar varios elementos que trabajan en conjunto para comunicar un mensaje claro y relevante.

A continuación, presentamos los componentes clave:

1. Datos relevantes

Los datos son la base de cualquier visualización. Sin embargo, no todos los datos disponibles son útiles para el mensaje que queremos transmitir. **Seleccionar datos relevantes** implica identificar aquellos que aportan valor y permiten responder a la pregunta o hipótesis planteada.

> **Ejemplo**
> En un análisis geoespacial para identificar zonas de riesgo de inundación, los datos sobre elevación, precipitación y uso del suelo son más relevantes que, por ejemplo, datos demográficos generales.

2. Mensaje o insight central

Toda visualización debe tener un **mensaje claro** que el espectador pueda entender rápidamente. Este mensaje es el insight que queremos destacar, la conclusión o hallazgo que justifica la visualización.

> **Ejemplo**
> "Las zonas con mayor densidad poblacional presentan mayor vulnerabilidad ante inundaciones".

3. Forma visual

La forma visual se refiere al tipo de gráfico, mapa o representación que mejor comunica el mensaje. La elección debe considerar:

- La naturaleza de los datos (categóricos, numéricos, espaciales)
- La audiencia y su nivel de conocimiento
- La claridad y simplicidad

> **Ejemplo**
> Un mapa coroplético es ideal para mostrar variaciones geográficas de una variable, mientras que un gráfico de barras puede ser mejor para comparar categorías.

4. Anotaciones y llamadas a la acción

Las **anotaciones** (etiquetas, flechas, textos explicativos) guían al espectador hacia los puntos clave, evitando ambigüedades. Las **llamadas a la acción** invitan a tomar decisiones o profundizar en el análisis.

> **Ejemplo**
> Una flecha señalando una zona crítica en un mapa con la etiqueta "Área de alto riesgo" y un texto que sugiera "Priorizar intervenciones aquí".

Resumen visual de componentes

Componente	Descripción breve	Ejemplo práctico
Datos relevantes	Selección de datos que aportan valor al mensaje	Datos de elevación y precipitación
Mensaje central	Insight o conclusión que se quiere comunicar	"Zonas vulnerables a inundación"
Forma visual	Tipo de gráfico o mapa adecuado	Mapa coroplético
Anotaciones y CTA	Elementos que guían y motivan acción	Etiquetas y flechas en el mapa

Contexto

En el contexto de proyectos de ciencia de datos y análisis geoespacial, dominar estos componentes es fundamental para que tus visualizaciones no solo sean estéticamente agradables, sino también

funcionales y persuasivas. Por ejemplo, al presentar un dashboard para un Product Owner, la correcta selección de datos y la claridad del mensaje facilitan la toma de decisiones estratégicas.

Además, la forma visual debe adaptarse a la audiencia: un equipo técnico puede requerir visualizaciones detalladas, mientras que un público ejecutivo valorará la simplicidad y claridad. Las anotaciones y llamadas a la acción son herramientas poderosas para dirigir la atención y fomentar respuestas concretas.

Este enfoque integral prepara el terreno para construir narrativas visuales que conectan con la audiencia y potencian el impacto de tus análisis.

Percepción visual y diseño (Gestalt, Tufte)

¿Alguna vez te has preguntado por qué ciertos gráficos o mapas capturan tu atención y comunican un mensaje claro, mientras que otros resultan confusos o poco atractivos? La respuesta está en cómo nuestro cerebro procesa y organiza la información visual.

En esta unidad, descubrirás los principios fundamentales de la percepción visual humana, especialmente los principios Gestalt, y cómo aplicarlos para diseñar visualizaciones claras y efectivas. Además, aprenderás a seleccionar paletas de colores accesibles y a usar el contraste y las anotaciones de forma estratégica para mejorar la comprensión y usabilidad de tus gráficos y mapas.

Estos conocimientos son esenciales para transformar análisis complejos en narrativas visuales que impacten y guíen la toma de decisiones en proyectos de ciencia de datos y análisis geoespacial.

La **percepción visual** es el proceso mediante el cual nuestro cerebro interpreta y organiza la información que recibe a través de los ojos. Entender cómo funciona este proceso es clave para diseñar visualizaciones que comuniquen eficazmente.

Principios Gestalt Clave

Los principios Gestalt explican cómo agrupamos elementos visuales para dar sentido a una imagen completa. Los más relevantes para el diseño de visualizaciones son:

- **Proximidad:** Elementos cercanos tienden a percibirse como un grupo.
- **Similitud:** Elementos con características similares (color, forma, tamaño) se agrupan mentalmente.
- **Figura-Fondo:** Nuestro cerebro distingue un objeto principal (figura) del fondo que lo rodea.

> **Ejemplo**
> En un mapa coroplético, usar colores similares para áreas relacionadas ayuda a que el usuario identifique patrones rápidamente.

Diseño Accesible: Contraste, Paletas y Anotaciones

Para que una visualización sea efectiva y accesible, es fundamental considerar:

- **Contraste:** Asegura que los elementos importantes se destaquen y sean legibles, especialmente para personas con dificultades visuales.
- **Paletas de colores accesibles:** Utilizar combinaciones que sean distinguibles para personas con daltonismo o baja visión. Herramientas como ColorBrewer son útiles para seleccionar paletas adecuadas.
- **Anotaciones claras:** Etiquetas, leyendas y notas deben ser precisas y ubicadas estratégicamente para guiar la interpretación sin saturar la visualización.

Referencias a Edward Tufte

Edward Tufte, pionero en visualización de datos, enfatiza la importancia de la **claridad, precisión y eficiencia** en el diseño gráfico. Sus principios incluyen:

- Minimizar el "ruido" visual que distrae del mensaje principal.
- Maximizar la "densidad de datos" para mostrar la mayor cantidad de información relevante sin perder legibilidad.
- Usar anotaciones y gráficos que apoyen la narrativa, no que la compliquen.

> **Nota**
>
> **Cita de Tufte.** "Good design is clear thinking made visible."

Con estos fundamentos, estarás listo para aplicar técnicas que mejoren la percepción y comprensión de tus visualizaciones geoespaciales.

Contexto

En el contexto de la ciencia de datos y análisis geoespacial, aplicar los principios de percepción visual y diseño accesible es crucial para comunicar insights complejos de manera efectiva. Por ejemplo, al crear un mapa interactivo que muestre la distribución de un fenómeno ambiental, usar la proximidad y similitud ayuda a que los usuarios identifiquen regiones con características similares rápidamente.

Además, seleccionar paletas de colores accesibles garantiza que la información sea comprensible para una audiencia diversa, incluyendo personas con discapacidades visuales. El contraste adecuado y las anotaciones claras facilitan la interpretación de datos en dashboards y reportes, mejorando la usabilidad y la toma de decisiones.

Este conocimiento complementa los conceptos previos sobre componentes visuales y usabilidad, preparando el terreno para la creación de visualizaciones interactivas y narrativas espaciales en los módulos siguientes.

El Arte y la Ciencia de la Visualización de Datos: Consolidación Final

Imagina que eres un explorador encargado de cartografiar un territorio desconocido. Ante ti tienes un bosque denso, ríos serpenteantes y montañas escarpadas. Si intentaras describir este lugar a alguien usando solo una lista de miles de coordenadas GPS (latitud y longitud), la persona se sentiría abrumada. No sabría dónde están los peligros ni dónde los recursos. Sin embargo, si dibujas un **mapa**, transformas esos datos crudos en una herramienta de navegación.

En la Ciencia de Datos, nos enfrentamos a ese mismo "territorio" caótico: millones de filas en una base de datos. La **visualización de datos** es el mapa que construimos para que nosotros y otros podamos navegar por la complejidad, descubrir rutas ocultas y, finalmente, tomar decisiones informadas.

En esta unidad de cierre, consolidaremos todo lo aprendido en el módulo, explorando por qué la visualización no es un paso "decorativo", sino el puente crítico que conecta el análisis estadístico con el impacto en el mundo real.

1. ¿Por qué Visualizamos? La "Traducción" de Datos

A menudo escuchamos que "una imagen vale más que mil palabras", pero en Ciencia de Datos, **una visualización vale más que un millón de celdas en un Excel**.

El Problema de la Carga Cognitiva

El cerebro humano no está diseñado para procesar y retener largas listas de números. Cuando miramos una tabla, nuestro cerebro realiza un esfuerzo consciente y lento para comparar valores (un proceso llamado *atención serial*). En cambio, el sistema visual humano es increíblemente rápido para identificar patrones, formas y colores (procesamiento *pre-atentivo*).

Ejemplo Concreto: El Cuarteto de Anscombe En 1973, el estadístico Francis Anscombe demostró por qué graficar los datos es vital. Creó cuatro conjuntos de datos que tenían las mismas propiedades estadísticas (media, varianza y correlación idénticas). Si solo miraras las estadísticas, pensarías que los cuatro grupos de datos son iguales. Sin embargo, al visualizarlos, descubres que uno es una línea recta, otro es una curva perfecta, el tercero tiene un valor atípico (outlier) y el cuarto es casi vertical. Sin la visualización, habrías tomado decisiones basadas en una similitud estadística que no existía en la realidad.

2. El Ciclo de la Visualización: Del Ingrediente al Plato

Para entender la visualización de manera estructural, podemos usar el proceso de un chef profesional:

1. **Recolección de Datos (Los Ingredientes):** Obtener los números brutos desde sensores, encuestas o registros de ventas.
2. **Limpieza de Datos (Lavar y Cortar):** Eliminar errores, valores nulos o información irrelevante. Un error común de principiante es intentar visualizar datos "sucios", lo que resulta en gráficos confusos.

3. **Elección del Gráfico (Elegir el Tipo de Plato):** ¿Necesitamos mostrar una tendencia en el tiempo? ¿Una comparación entre categorías? ¿Una relación entre dos variables? (Ver sección de "Errores de Elección").
4. **Codificación Visual (La Cocción):** Aquí aplicamos el color, el tamaño y la posición para resaltar lo importante.
5. **Interpretación (La Degustación):** El momento en que el usuario final mira el gráfico y extrae una conclusión accionable.

3. Casos de Éxito: Visualización en el Mundo Real

La visualización de datos ha cambiado el rumbo de empresas y la salud pública global:

A. El Tablero de COVID-19 de Johns Hopkins University

Durante la reciente pandemia global, este tablero se convirtió en el estándar mundial. Utilizó mapas de calor (heatmaps) y gráficos de líneas simples para que tanto gobernantes como ciudadanos pudieran entender la velocidad del contagio. Fue la prueba máxima de que una visualización clara puede salvar vidas al informar políticas de salud pública en tiempo real.

B. Google Data Studio y la Democratización de Datos

Google permite que cualquier pequeño comercio visualice su tráfico web. Antes, solo expertos en SQL podían entender qué pasaba en un sitio. Hoy, gracias a las herramientas de visualización, un dueño de una pastelería puede ver un gráfico de barras y entender que la mayoría de sus clientes llegan a las 3:00 PM los jueves, ajustando su producción de inmediato.

C. Netflix y el Comportamiento del Usuario

Netflix no solo sabe qué ves; visualiza *en qué momento* dejas de ver una serie. Sus equipos de análisis utilizan visualizaciones complejas para detectar patrones de abandono. Si un gráfico muestra una caída masiva en el minuto 20 del primer capítulo, saben que hay un problema narrativo o técnico en ese punto exacto, permitiéndoles mejorar su contenido futuro.

4. Errores Comunes y Cómo Evitarlos

Incluso con las mejores herramientas, es fácil crear "mentiras visuales" o gráficos confusos. Aquí identificamos las trampas más frecuentes:

Error 1: "El Arcoíris Confuso"

Muchos estudiantes piensan que usar más colores hace que el gráfico sea más atractivo. * **El Problema:** El exceso de colores satura la vista y distrae. El cerebro intenta asignar un significado a cada color cuando, a veces, no lo tiene. * **La Regla:** El color debe ser funcional. Usa el color solo para destacar algo importante o para agrupar categorías relacionadas. Si tienes más de 5-7 categorías, considera usar tonos de un mismo color o replantear el gráfico.

Error 2: El Eje "Truncado" (Eje Y que no empieza en cero)

Este es un error que raya en la manipulación. Si comparas dos barras y haces que el eje Y comience en 50 en lugar de 0, la diferencia entre una barra de 55 y una de 60 parecerá enorme, cuando en realidad es pequeña. * **La Regla:** En gráficos de barras, el eje Y **siempre** debe comenzar en 0 para mantener la integridad de la proporción visual.

Error 3: El Uso Indiscriminado de Gráficos de Pay (Pie Charts)

A los principiantes les encantan los círculos, pero el ojo humano es muy malo calculando ángulos y áreas comparativas. * **El Problema:** Si tienes 10 categorías en un gráfico de pay, las "rebanadas" pequeñas se vuelven imposibles de distinguir. * **La Regla:** Usa gráficos de pay solo si tienes menos de 5 categorías y la suma de todas es exactamente el 100%. Para todo lo demás, un **gráfico de barras** siempre será más legible.

5. El Modelo Mental: La Jerarquía de la Información

Para diseñar como un experto, visualiza tu gráfico como una **pirámide de importancia**:

1. **Nivel Superior (El Título/Mensaje):** Debe decir cuál es la conclusión. En lugar de un título como "Ventas 2023", usa "Las ventas crecieron un 20% en el segundo trimestre". Ayuda a tu lector a saber qué mirar.
2. **Nivel Medio (El Cuerpo del Gráfico):** La parte visual (barras, líneas, puntos). Debe estar libre de basura visual ("chartjunk" según Edward Tufte), como cuadrículas pesadas o fondos de colores.
3. **Nivel Base (Las Referencias):** Ejes, etiquetas de datos y leyendas. Son esenciales para la precisión, pero no deben opacar al gráfico principal.

6. Síntesis y Conclusión del Módulo

A lo largo de este módulo, hemos pasado de entender qué es un dato a cómo narrar una historia con ellos. Hemos visto: - Las **Leyes de Gestalt** que explican cómo agrupamos objetos visualmente. - Los principios de **Tufte** sobre la integridad de los datos. - Y hoy, cómo la visualización actúa como el traductor universal entre la técnica y la estrategia.

Recuerda: Tu objetivo como científico de datos no es solo encontrar la verdad en los números, sino presentarla de una manera tan clara que sea imposible de ignorar. Una buena visualización no solo muestra datos; **genera una conversación y motiva una acción**.

Al cerrar este módulo, pregúntate siempre ante cada gráfico que crees: *¿Qué pregunta responde este visual?* Si la respuesta no es obvia en menos de 5 segundos, simplifica. La claridad es el máximo grado de sofisticación en la ciencia de datos.

PRE-ENTREGA DEL PROYECTO

Actividad evaluable · parte del proyecto integrador

Pre-entrega: Dashboard de Análisis de Ventas Retail

El objetivo de esta actividad es aplicar los principios de visualización de datos y storytelling para transformar un conjunto de datos crudos en una narrativa visual clara y accionable que facilite la toma de decisiones.

Cómo resolver esta entrega

Qué construir

Deberás crear un dashboard interactivo utilizando una herramienta de Business Intelligence (Tableau, Power BI, Looker Studio o similar) basado en un dataset de ventas retail. El dashboard debe responder a una pregunta de negocio específica (por ejemplo: "¿Qué categorías de productos están impulsando el crecimiento en el último trimestre?").

Pasos sugeridos

1. **Definición del usuario**: Identifica quién verá este dashboard (ej. Gerente de Ventas) y qué problema necesita resolver.
2. **Selección de visualizaciones**: Elige al menos 3 tipos de gráficos diferentes que mejor representen los datos (ej. Serie temporal para tendencias, Barras para comparaciones de categorías, Mapas para distribución geográfica).
3. **Aplicación de jerarquía visual**: Utiliza el tamaño, el color y la posición para guiar el ojo del usuario hacia los datos más importantes.
4. **Storytelling**: Añade títulos descriptivos y breves anotaciones que expliquen el "porqué" de las variaciones detectadas en los datos.
5. **Publicación**: Genera un enlace público o compartido para que el tablero sea accesible.

Errores comunes

- **Ruido visual**: Evita usar demasiados colores o elementos decorativos que no aporten información.
- **Gráficos inadecuados**: No uses gráficos circulares (pie charts) para comparar más de 3 categorías.
- **Falta de contexto**: Asegúrate de que todos los ejes tengan etiquetas y los valores tengan formato de moneda o porcentaje según corresponda.

Objetivos de aprendizaje

- Diseñar visualizaciones de datos efectivas basadas en principios de percepción visual.
- Aplicar técnicas de storytelling para comunicar hallazgos de datos de forma clara.
- Implementar jerarquía visual y usabilidad en un tablero interactivo.

- Seleccionar el tipo de gráfico adecuado según la naturaleza de las variables analizadas.

Criterios de aceptación

- El enlace entregado es accesible y el dashboard carga correctamente.
- Incluye al menos tres tipos de gráficos funcionalmente distintos y relevantes para el contexto.
- El dashboard presenta una estructura lógica de lectura (de lo general a lo específico).
- Los títulos de las visualizaciones son descriptivos y no solo indican el nombre de las variables.
- Se utiliza una paleta de colores consistente que facilita la interpretación de los datos sin saturar al usuario.

Rúbrica de evaluación: Dashboard de Análisis de Ventas Retail

Rúbrica para evaluar un dashboard interactivo de análisis de ventas retail en una herramienta de Business Intelligence. La evaluación considera la claridad de la pregunta de negocio, la selección y coordinación de visualizaciones, la jerarquía visual, el storytelling y la calidad técnica de la entrega publicable. Está diseñada para guiar la creación de un tablero útil para la toma de decisiones.

Criterio	Descripción	Peso
Resolución de la pregunta de negocio y enfoque del dashboard	Evalúa si el tablero responde con claridad a una pregunta de negocio específica, si define un usuario objetivo y si mantiene un enfoque coherente con ese problema. Este criterio verifica que el dashboard no sea solo una colección de gráficos, sino una herramienta orientada a decisiones.	25%
Selección y variedad de visualizaciones	Evalúa si el estudiante elige al menos tres tipos de gráficos diferentes y si cada uno es adecuado para el tipo de dato y el mensaje que se quiere comunicar. Se valora que la selección apoye comparaciones, tendencias y distribución de forma efectiva.	20%
Jerarquía visual y diseño de la interfaz	Evalúa el uso del tamaño, el color, la posición y el espacio para dirigir la atención hacia la información más importante. También considera la limpieza visual, la organización general y la ausencia de ruido innecesario.	20%
Storytelling y contexto analítico	Evalúa si el dashboard cuenta una historia comprensible con títulos descriptivos, anotaciones breves y contexto suficiente para interpretar las variaciones observadas. Se valora que el tablero explique no solo qué ocurre, sino también por qué es relevante.	15%
Etiquetado, formato y precisión de presentación	Evalúa la claridad técnica de la presentación: etiquetas de ejes, títulos, formato de moneda o porcentaje, consistencia visual y legibilidad de los valores. Este criterio asegura que la información pueda interpretarse sin ambigüedades.	10%
Accesibilidad y entrega publicable	Evalúa si el dashboard fue publicado correctamente, si el enlace es accesible y si la entrega permite revisar el tablero sin barreras técnicas. Este criterio se centra en la completitud funcional del entregable.	10%

Total: 100 pts | Aprobación: 70 pts

MÓDULO 13

Storytelling

En este módulo intermedio, aprenderás a comunicar hallazgos de datos espaciales mediante técnicas de storytelling y visualización avanzada.

La importancia del Storytelling en Ciencia de Datos

El *storytelling* en ciencia de datos es el arte de usar una narrativa estructurada junto con visualizaciones para comunicar hallazgos de datos de forma clara, memorable y que motive a la acción.

No se trata solo de mostrar gráficos, sino de construir una historia que conecte los datos con el contexto del problema y las decisiones que se deben tomar.

Elementos clave del storytelling:

- **Narrativa:** Estructura lógica que guía al receptor a través de los datos.
- **Visualizaciones:** Gráficos y mapas que facilitan la comprensión.
- **Contexto:** Explicación del porqué y para qué de los datos.

> **Ejemplo**
> En un proyecto de análisis de movilidad urbana, contar cómo cambian los patrones de tráfico durante eventos especiales usando mapas y gráficos para apoyar recomendaciones de gestión vial.

Análisis Espacial

El análisis espacial comprende técnicas para estudiar la ubicación, distribución y relaciones entre entidades geográficas.

Es fundamental para entender fenómenos que dependen del espacio, como la distribución de clientes, recursos naturales o eventos.

Conceptos básicos:

- **Entidades espaciales:** Puntos, líneas y polígonos que representan objetos geográficos.
- **Relaciones espaciales:** Proximidad, contigüidad, superposición.
- **Medidas espaciales:** Distancias, densidades, patrones de agrupamiento.

> **Ejemplo**
> Analizar la concentración de tiendas en una ciudad para identificar áreas con alta competencia o potencial de expansión.

Conexión entre Storytelling y Análisis Espacial:

Combinar narrativa y análisis espacial permite comunicar insights geográficos de forma efectiva, facilitando la toma de decisiones basadas en la ubicación y distribución de datos.

Contexto

En la industria tecnológica, los proyectos de ciencia de datos a menudo requieren integrar datos espaciales para enriquecer el análisis y la comunicación de resultados. Por ejemplo, empresas de logística, urbanismo o marketing digital utilizan análisis espacial para optimizar rutas, planificar campañas o entender comportamientos geográficos.

Este módulo se enfoca en aplicar técnicas de storytelling y análisis espacial para transformar datos en historias visuales que apoyen decisiones estratégicas.

El flujo de trabajo comienza con la extracción y preparación de datos mediante consultas SQL, continúa con el análisis y visualización en Python usando librerías especializadas, y culmina en la creación de notebooks narrativos que integran código, visualizaciones y texto explicativo.

Al dominar estos conceptos, estarás preparado para presentar resultados de modelos predictivos y análisis espaciales de manera clara y persuasiva, una habilidad muy valorada en proyectos de datos actuales.

Storytelling en Ciencia de Datos

El *storytelling* en ciencia de datos es el arte de comunicar hallazgos a través de una narrativa estructurada que combina datos, visualizaciones y contexto para generar comprensión y acción.

No se trata solo de mostrar gráficos o estadísticas, sino de construir una historia que conecte emocionalmente con la audiencia, facilite la recordación y genere confianza.

Beneficios Clave del Storytelling

- **Conexión emocional:** Las historias bien contadas generan empatía y motivan a la audiencia a involucrarse con los datos.
- **Recordabilidad:** Una narrativa clara y visualizaciones efectivas hacen que los insights sean más fáciles de recordar.
- **Confianza:** Presentar datos en un contexto lógico y transparente fortalece la credibilidad.
- **Engagement:** Mantiene la atención y facilita la comprensión de información compleja.

Componentes de una Narrativa de Datos Efectiva

1. **Contexto:** Presentar el problema o pregunta que se busca responder.
2. **Hipótesis:** Formular suposiciones o expectativas basadas en el conocimiento previo.
3. **Análisis:** Mostrar el proceso de exploración y modelado de datos.
4. **Visualizaciones:** Utilizar gráficos claros y relevantes para apoyar los hallazgos.

5. **Conclusiones:** Resumir insights clave y recomendaciones.
6. **Llamado a la acción:** Indicar pasos siguientes o decisiones basadas en los datos.

Introducción al Notebook de Data Storytelling: Concepto y Uso

Un *notebook de data storytelling* es un documento reproducible, comúnmente un Jupyter notebook, que integra código, visualizaciones y texto narrativo para contar una historia de datos completa y transparente.

Ventajas del Notebook Narrativo

- **Reproducibilidad:** Permite que otros verifiquen y reproduzcan el análisis.
- **Integración:** Combina análisis exploratorio, visualizaciones y narrativa en un solo lugar.
- **Claridad:** Facilita la comunicación estructurada y secuencial de hallazgos.

Estructura Típica de un Notebook Narrativo

Sección	Contenido
Introducción	Contexto, objetivos y preguntas de investigación
Preparación Datos	Limpieza y transformación de datos
Análisis	Exploración, modelado y visualizaciones
Resultados	Interpretación de hallazgos y visualizaciones clave
Conclusiones	Resumen de insights y recomendaciones

> **Nota**
> La estructura puede adaptarse según el proyecto, pero debe mantener claridad y coherencia para facilitar la comprensión.

Ejemplo de Uso en la Industria

En proyectos de análisis espacial, como los que abordamos en este módulo, un notebook narrativo puede integrar consultas SQL para preparar datos geoespaciales, análisis con GeoPandas y visualizaciones interactivas con Plotly, todo acompañado de explicaciones que guían al lector a través del proceso y resultados.

Contexto

En el contexto tecnológico actual, donde los proyectos de ciencia de datos generan grandes volúmenes de información, la capacidad de comunicar resultados de manera efectiva es crucial para la toma de decisiones.

El storytelling aplicado a datos espaciales permite que equipos multidisciplinarios comprendan patrones y tendencias geográficas que impactan negocios, urbanismo o medio ambiente.

El uso de notebooks narrativos facilita la colaboración y transparencia, ya que documenta el flujo completo desde la hipótesis hasta la conclusión, integrando código y visualizaciones interactivas. Esto es especialmente valioso en entornos donde se combinan datos de diversas fuentes, como bases SQL y datos geoespaciales, permitiendo un análisis reproducible y accesible para distintos perfiles profesionales.

Esta unidad prepara el terreno para que, en unidades posteriores, puedas aplicar técnicas avanzadas de visualización y usabilidad para crear dashboards y presentaciones que potencien aún más el impacto de tus análisis.

Principios de Usabilidad y Visualización de Datos

Jakob Nielsen, experto en usabilidad, definió 10 heurísticas que sirven como guía para diseñar interfaces que sean fáciles de usar y entender.

Aunque originalmente pensadas para interfaces de software, estos principios son perfectamente aplicables a la visualización de datos y dashboards.

1. Visibilidad del estado del sistema

El usuario debe siempre saber qué está pasando mediante retroalimentación clara y oportuna.

Ejemplo: Un dashboard que muestra un indicador de carga o actualización de datos evita confusión.

2. Relación entre el sistema y el mundo real

Usar lenguaje, símbolos y metáforas familiares para el usuario.

Ejemplo: Mapas con iconos intuitivos y etiquetas claras.

3. Control y libertad del usuario

Permitir deshacer acciones o volver atrás fácilmente.

Ejemplo: Filtros que se pueden resetear con un clic.

4. Consistencia y estándares

Mantener uniformidad en colores, tipografías y comportamientos.

Ejemplo: Usar el mismo color para representar una categoría en todas las visualizaciones.

5. Prevención de errores

Diseñar para evitar que el usuario cometa errores.

Ejemplo: Validar entradas en formularios de filtros para evitar valores inválidos.

6. Reconocimiento en lugar de recuerdo

Minimizar la carga de memoria mostrando opciones y elementos visibles.

Ejemplo: Menús desplegables con opciones claras en lugar de pedir que el usuario recuerde códigos.

7. Flexibilidad y eficiencia de uso

Permitir atajos para usuarios avanzados sin complicar la experiencia básica.

Ejemplo: Atajos de teclado para cambiar vistas o filtros.

8. Estética y diseño minimalista

Evitar información irrelevante o redundante que distraiga.

Ejemplo: Gráficos limpios sin exceso de líneas o colores.

9. Ayudar a los usuarios a reconocer, diagnosticar y recuperarse de errores

Mensajes claros y soluciones sugeridas.

Ejemplo: Alertas que indican por qué no se cargaron datos y cómo solucionarlo.

10. Ayuda y documentación

Proveer soporte accesible para usuarios que lo necesiten.

Ejemplo: Tooltip explicativos o sección de ayuda integrada.

> **Nota**
> Estos principios no solo mejoran la experiencia del usuario, sino que también aumentan la efectividad de la comunicación de datos, facilitando la toma de decisiones basada en insights claros y confiables.

Contexto

En proyectos de ciencia de datos, la visualización es la puerta de entrada para que stakeholders comprendan y actúen sobre los resultados.

Aplicar los principios de usabilidad de Nielsen garantiza que los dashboards y gráficos no solo sean estéticamente agradables, sino también funcionales y accesibles.

Por ejemplo, en un proyecto de análisis espacial con GeoPandas y Plotly, un dashboard que sigue estos principios permitirá a los usuarios explorar datos geográficos sin frustraciones, entender tendencias temporales con animaciones claras y tomar decisiones informadas rápidamente.

Además, la integración de consultas SQL para preparar datos debe reflejarse en visualizaciones que comuniquen claramente qué datos se muestran y cómo se pueden interactuar con ellos. Este enfoque

prepara el terreno para unidades posteriores donde se profundizará en storytelling y visualizaciones interactivas avanzadas.

Análisis Espacial en Ciencia de Datos

Modelos Vectorial y Ráster en Análisis Espacial

En el análisis espacial, los datos geográficos se representan principalmente mediante dos modelos: **vectorial** y **ráster**. Cada uno tiene características, ventajas y aplicaciones específicas.

Modelo Vectorial

El modelo vectorial representa objetos geográficos mediante **puntos**, **líneas** y **polígonos**:

- **Puntos**: Ubicaciones específicas, como estaciones meteorológicas o paradas de autobús.
- **Líneas**: Elementos lineales, como carreteras o ríos.
- **Polígonos**: Áreas definidas, como lagos, zonas urbanas o parcelas de terreno.

Este modelo es ideal para representar datos con límites definidos y es eficiente para análisis que requieren precisión geométrica, como calcular distancias o áreas.

> **Ejemplo**
> Un mapa de rutas de transporte público usa líneas para representar calles y puntos para paradas.

Modelo Ráster

El modelo ráster representa el espacio geográfico como una **rejilla de celdas** (pixeles), donde cada celda contiene un valor que representa una característica, como elevación, temperatura o uso del suelo.

- Cada celda tiene un tamaño fijo (resolución), y el conjunto forma una matriz.
- Es útil para datos continuos y análisis que involucran superficies, como mapas de calor o imágenes satelitales.

> **Ejemplo**
> Un mapa de temperatura ambiental donde cada celda indica la temperatura promedio en esa área.

Comparación y Selección del Modelo Adecuado

Característica	Modelo Vectorial	Modelo Ráster
Representación	Objetos discretos (puntos, líneas, polígonos)	Superficies continuas (rejilla de celdas)
	Alta	Limitada por resolución

Característica	Modelo Vectorial	Modelo Ráster
Precisión geométrica		
Uso típico	Redes, límites administrativos, infraestructuras	Imágenes satelitales, variables ambientales
Análisis común	Distancias, intersecciones, áreas	Análisis de superficie, interpolación

Sistemas de Referencia Espaciales (CRS)

Los **Sistemas de Referencia de Coordenadas (CRS)** definen cómo se representan las coordenadas geográficas en la superficie terrestre. Son esenciales para asegurar que los datos espaciales de diferentes fuentes se alineen correctamente.

Tipos de CRS

- **Geográficas (latitud-longitud):** Basadas en un sistema esférico o elipsoidal, representan posiciones en grados.
- **Proyectadas:** Transforman la superficie curva de la Tierra a un plano mediante proyecciones cartográficas, usando unidades métricas (metros).

Importancia del CRS en el Análisis

- Diferentes datasets pueden usar distintos CRS, lo que puede causar desalineación si no se transforman adecuadamente.
- La elección del CRS afecta la precisión de mediciones espaciales (distancias, áreas).

> **Ejemplo**
> Un dataset en WGS84 (latitud-longitud) debe transformarse a una proyección local para análisis de distancias precisas.

Transformaciones de CRS

La transformación entre CRS es un proceso común y necesario:

- Se utiliza para alinear datos de distintas fuentes.
- Puede introducir errores si no se realiza correctamente.

> **Ejemplo**
> **Ejemplo práctico.** Convertir datos de coordenadas geográficas a UTM para análisis de proximidad.

Conocimiento clave: Entender los modelos vectorial y ráster y manejar correctamente los CRS es fundamental para realizar análisis espaciales precisos y confiables en ciencia de datos.

Contexto

En proyectos de ciencia de datos con componente geoespacial, la correcta representación y referencia de los datos es la base para cualquier análisis posterior, desde la visualización hasta la construcción de modelos predictivos.

Por ejemplo, al analizar la distribución de clientes para una empresa de logística, elegir un modelo vectorial permitirá identificar ubicaciones exactas y rutas óptimas, mientras que un modelo ráster puede ser útil para evaluar variables ambientales que afectan la operación.

Además, al integrar datos de distintas fuentes, como bases de datos SQL con información geográfica y archivos shapefile, es imprescindible que todos compartan un sistema de referencia común para evitar errores en la interpretación espacial. Por ello, dominar las transformaciones de CRS y comprender su impacto es una habilidad práctica que mejora la calidad y utilidad de los resultados en proyectos reales.

Esta unidad prepara el terreno para aplicar estas representaciones y referencias en Python, usando librerías como GeoPandas y herramientas de visualización, que se abordarán en unidades posteriores.

Mapas y Visualizaciones en Python

GeoPandas: Manejo de Datos Geoespaciales en Python

GeoPandas es una extensión de la biblioteca pandas que facilita la manipulación y análisis de datos geoespaciales en Python. Permite trabajar con objetos geométricos como puntos, líneas y polígonos, y realizar operaciones espaciales como uniones, intersecciones y reproyecciones.

Características clave:

- Lectura y escritura de formatos geoespaciales comunes: shapefiles, GeoJSON, entre otros.
- Soporte para sistemas de referencia de coordenadas (CRS) y reproyección.
- Operaciones espaciales vectoriales integradas.

> Por ejemplo, puedes cargar un shapefile de límites administrativos y combinarlo con datos tabulares para crear mapas temáticos.

CÓDIGO

```
import geopandas as gpd
<h1 class="module" data-eyebrow="MÓDULO">Cargar un archivo shapefile</h1>
<hr class="module-underline"><hr class="module-hairline">
mapa = gpd.read_file('limites.shp')
print(mapa.head())
```

Plotly: Visualización Interactiva

Plotly es una biblioteca que permite crear gráficos y mapas interactivos para la web, facilitando la exploración dinámica de datos y el storytelling.

Ventajas:

- Interactividad: zoom, hover, selección.
- Integración con Jupyter notebooks y aplicaciones web.
- Soporte para mapas coropléticos, scatter geoespaciales, y más.

CÓDIGO

```
import plotly.express as px
fig = px.choropleth(mapa, geojson=mapa.geometry, locations=mapa.index, color='poblacion')
fig.show()
```

Combinar GeoPandas y Plotly permite crear visualizaciones estáticas y dinámicas que enriquecen el análisis espacial. Primero, se preparan y manipulan los datos con GeoPandas, luego se visualizan con Plotly para obtener mapas interactivos.

Integración de SQL para Preparación de Datos

El uso de consultas SQL es fundamental para extraer, filtrar y transformar datos antes de visualizarlos. Por ejemplo, puedes usar SQL para seleccionar solo las regiones con ciertos criterios y luego cargar esos datos en Python para su visualización.

CÓDIGO

```
SELECT region, poblacion, geom
FROM datos_espaciales
WHERE poblacion > 100000
```

Esta integración asegura que las visualizaciones sean precisas y relevantes para el análisis.

Conocimientos previos requeridos: comprensión básica de modelos vectoriales y raster, sistemas de referencia de coordenadas (CRS), y fundamentos de storytelling en datos.

Contexto

En proyectos de ciencia de datos con componentes espaciales, la visualización es clave para comunicar hallazgos y apoyar la toma de decisiones.

Por ejemplo, una empresa de logística puede usar mapas interactivos para identificar zonas con alta demanda y optimizar rutas de entrega. En este contexto, GeoPandas permite manejar y preparar los datos geoespaciales, mientras que Plotly facilita la creación de mapas interactivos que los stakeholders pueden explorar fácilmente.

Además, la integración con SQL es común en entornos empresariales donde los datos se almacenan en bases relacionales. Saber cómo extraer y preparar estos datos mediante consultas SQL antes de visualizarlos en Python es una habilidad práctica y muy valorada.

Esta unidad te prepara para aplicar estas técnicas en casos reales, conectando con el flujo de trabajo de análisis espacial y storytelling que hemos venido desarrollando en el módulo.

Storytelling: La Consolidación del Mensaje

Consolidación del Storytelling: El Arte de Comunicar Datos

Imagina que acabas de terminar un análisis espacial complejo. Has procesado gigabytes de información, has mapeado la expansión urbana de una ciudad utilizando Python y has identificado patrones de desigualdad que nadie había notado antes. Te presentas ante el comité de planificación urbana con tus gráficos técnicos y tablas llenas de métricas de correlación espacial. Sin embargo, a los diez minutos, notas que la mitad de la audiencia está mirando su teléfono y la otra mitad tiene una expresión de confusión.

¿Qué falló? No fue la precisión de tus datos, ni la calidad de tu mapa. El problema es que presentaste **datos**, pero no contaste una **historia**.

En esta unidad final del módulo de Storytelling, vamos a consolidar todo lo aprendido sobre la comunicación de hallazgos. No vamos a ver nuevas líneas de código, sino que vamos a aprender a estructurar el pensamiento para que esos datos se conviertan en motores de cambio.

1. El Concepto de Storytelling: Más allá de "Contar Cuentos"

A menudo, en el mundo técnico, se comete el error de pensar que el storytelling es simplemente "adornar" los datos o convertirlos en una ficción. Nada más lejos de la realidad. Expertos como Chris Anderson (CEO de TED) definen el storytelling como el acto de **estructurar una experiencia** para el oyente.

En la Ciencia de Datos, el storytelling es el puente entre el análisis técnico y la toma de decisiones. Es la técnica que nos permite transformar una visualización estática en un arco narrativo que guía a la audiencia desde la confusión inicial hasta una conclusión clara y procesable.

El porqué del Storytelling

Los seres humanos no estamos cableados para recordar listas de números; estamos evolutivamente diseñados para recordar narrativas. Las historias activan áreas del cerebro relacionadas con la empatía y la memoria visual, mientras que los datos puros solo activan las áreas de procesamiento del lenguaje y lógica.

2. La Estructura Narrativa: El Mapa de la Historia

Para que una historia de datos funcione, debe seguir una estructura clara. Una de las más efectivas para principiantes es la **"Montaña de la Historia"** (o Arco Narrativo), adaptada al contexto de los datos:

A. La Configuración (El Mundo Normal)

Aquí es donde estableces el contexto. ¿De qué datos estamos hablando? ¿Cuál es el escenario base?

- *Ejemplo:* "Durante los últimos 10 años, nuestra ciudad ha crecido de manera constante, manteniendo un equilibrio entre zonas verdes y construcción."

B. El Incidente Incitador (El Problema o "La Grieta")

Es el momento en que los datos revelan que algo no está bien. Es la chispa que justifica el análisis.

- *Ejemplo:* "Sin embargo, al cruzar los datos de permisos de construcción con las imágenes satelitales de los últimos 24 meses, detectamos una anomalía: la pérdida del 30% del cinturón forestal del norte."

C. El Ascenso (El Análisis)

Aquí muestras el viaje. No es una lista de todos los modelos que probaste, sino de los hitos importantes que descubriste. Es la fase donde el "héroe" (el analista) enfrenta desafíos en los datos.

- *Ejemplo:* "Al investigar más a fondo, descubrimos que este patrón no era aleatorio; estaba concentrado en zonas de alta especulación inmobiliaria que eludían las normativas actuales."

D. El Clímax (El Hallazgo Principal)

Es el punto de mayor tensión y claridad. Es el gráfico principal, el mapa de calor que lo explica todo.

- *Ejemplo:* "Aquí vemos el impacto real: si no cambiamos la política hoy, en 2030 habremos perdido la principal fuente de refrigeración natural de la ciudad."

E. La Resolución (La Acción)

Toda historia de datos en ciencia de datos debe terminar con un **"Call to Action"** (Llamado a la acción). ¿Qué debemos hacer ahora?

- *Ejemplo:* "Proponemos la creación de un sistema de monitoreo espacial en tiempo real para bloquear permisos automáticos en zonas críticas."

3. Analogías Maestras para Entender la Comunicación

Para dominar el storytelling, los educadores expertos sugieren dos modelos mentales clave:

El "Tráiler de Película"

No intentes mostrar toda la "película" (todo tu cuaderno de Python con 50 celdas de código). Tu presentación debe ser el tráiler: los momentos más impactantes, los giros de trama más importantes y

el gancho que deje a los inversionistas o jefes queriendo saber más. Si necesitan los detalles técnicos, los entregas en un anexo, pero la presentación debe ser ágil.

La Teoría del Iceberg (de Ernest Hemingway)

En una historia de datos, lo que la audiencia ve (la visualización final y la conclusión) es solo el 10% del trabajo (la punta del iceberg). El otro 90% es tu limpieza de datos, tu análisis espacial y tu validación estadística. El error del principiante es intentar sacar todo el iceberg a la superficie, lo cual termina hundiendo la atención del público. **Muestra solo lo que sostiene la historia.**

4. Errores Comunes y Cómo Evitarlos

Incluso los científicos de datos con mucha experiencia caen en trampas de comunicación. Aquí te presentamos las más frecuentes para que puedas evitarlas:

Error 1: Confundir "Detalle" con "Calidad"

Muchos principiantes piensan que si dan más detalles (físicos o técnicos), la historia es mejor.

- **La trampa:** "Usamos una máquina con 64GB de RAM y un algoritmo Random Forest con 500 estimadores y una profundidad de 10..."
- **La corrección:** Enfócate en **detalles emocionales o de impacto**. "Este algoritmo nos permitió identificar focos de pobreza que habían sido invisibles para los censos tradicionales durante una década".

Error 2: Hacer que el héroe sea "Perfecto"

En una historia, el héroe suele ser el equipo de analistas o la tecnología. Si presentas que "todo fue fácil y los datos estaban perfectos", la audiencia desconfía.

- **La corrección:** Menciona brevemente los desafíos. "Al principio, los datos satelitales estaban muy nublados y parecía que el análisis sería imposible, pero mediante una técnica de filtrado temporal, logramos ver a través de la interferencia". Esto genera credibilidad y tensión narrativa.

Error 3: Empezar por el principio cronológico

No empieces con "Me desperté, abrí el dataset, cargué las librerías...".

- **La corrección:** Empieza por el **Hook (El Gancho)**. Empieza con el problema urgente o con un dato sorprendente que rompa el status quo.

5. Casos de Estudio Reales: Storytelling en Acción

Caso 1: TED Talks y el Arco Narrativo

Chris Anderson explica que las charlas más exitosas de TED no son las que presentan más datos, sino las que llevan al espectador por un "valle de dudas" antes de llegar a la solución. Muchos científicos de datos de **Spotify** o **Netflix** usan esta estructura para explicar cómo sus algoritmos de recomendación realmente "entienden" los gustos humanos, personificando al algoritmo como un curador que aprende de tus errores.

Caso 2: El Proyecto del Kennedy Center

En este centro de artes, enseñan a los estudiantes a usar un **Storyboard Grid**. Antes de tocar una sola herramienta digital, los comunicadores deben dibujar en 6 u 8 cuadros la secuencia de su historia.

- *Lección para ti:* Antes de crear tu Dashboard o presentación, dibuja a mano los 6 puntos clave de tu narrativa. Si no puedes contar la historia con 6 dibujos simples, tu análisis es demasiado complejo para ser comunicado con éxito.

6. Terminología Clave de Storytelling

Para hablar el lenguaje de los comunicadores de datos, debes conocer estos términos:

Término	Definición para Scientistas de Datos
The Hook (El Gancho)	La primera oración o imagen que captura la atención. "Cada día, nuestra empresa pierde $5,000 por errores de logística que los datos ya están señalando".
Inciting Incident	El evento que rompe la normalidad. "Un cambio en el clima el mes pasado reveló lo vulnerables que son nuestras rutas del sur".
Data Personification	Darle rasgos humanos a los datos para generar empatía. En lugar de "Usuario 452", hablar de "El cliente que busca su primera casa".
The Ah-ha! Moment	El momento donde la visualización de datos revela la solución de forma evidente.

Síntesis y Reflexión Final

Has llegado al final del módulo de Storytelling dentro del programa de Machine Learning. A lo largo de estas unidades, has visto desde la importancia teórica hasta la creación de mapas complejos en Python. Pero recuerda: **las herramientas cambian, las historias permanecen.**

Dominar el storytelling te diferencia del resto de los analistas técnicos. Te convierte en un líder capaz de influir en la estrategia de una organización. Los datos son el "qué", pero el storytelling es el "para qué".

Puntos clave para recordar:

1. **Simplifica sin perder rigor:** No abrumes con tecnicismos en la narrativa principal.
2. **Sigue un arco:** Inicio (Contexto) -> Nudo (Conflicto de datos) -> Desenlace (Solución/Acción).
3. **Visualiza para narrar:** Usa tus mapas espaciales no solo para mostrar ubicaciones, sino para mostrar movimientos, cambios y desigualdades.
4. **Enfócate en la acción:** Si tu historia no termina con una recomendación clara, es solo ruido.

PRE-ENTREGA DEL PROYECTO

Actividad evaluable · parte del proyecto integrador

Pre-entrega: Transformando datos complejos en una narrativa para la toma de decisiones

El storytelling es la habilidad de transformar datos complejos en una narrativa convincente que impulse la toma de decisiones. En esta actividad, actuarás como un Data Storyteller para comunicar hallazgos sobre un conjunto de datos espaciales.

Cómo resolver esta entrega

Propósito

Diseñar una narrativa visual que no solo muestre datos geográficos, sino que explique el "por qué" y el "qué significa" para una audiencia de negocios, utilizando una estructura narrativa clara (Inicio/Conflicto, Desarrollo/Hallazgos y Conclusión/Acción).

Pasos sugeridos

1. **Selección del Insight**: Elige un patrón espacial relevante (ej. concentración de ventas en una zona, brechas de infraestructura, o patrones de movilidad).
2. **Estructura Narrativa**: Divide tu presentación en tres actos: el contexto del problema, el análisis visual del hallazgo y la recomendación.
3. **Diseño Visual**: Crea visualizaciones que sigan los principios de Gestalt y minimicen la carga cognitiva. El uso del color debe ser estratégico para resaltar el insight espacial.
4. **Anotaciones**: No dejes que el gráfico hable solo. Incluye anotaciones que guíen el ojo del espectador hacia lo más importante.

Errores comunes

- Llenar las diapositivas con tablas de datos en lugar de conclusiones visuales.
- No definir una recomendación clara al final.
- Usar demasiados colores que compitan por la atención del usuario en el mapa.

Objetivos de aprendizaje

- Aplicar técnicas de storytelling para comunicar hallazgos técnicos a audiencias no técnicas.
- Diseñar visualizaciones de datos espaciales que sigan principios de jerarquía visual.
- Construir una narrativa coherente con inicio, nudo y desenlace basada en evidencia de datos.

Criterios de aceptación

- La presentación incluye una estructura narrativa clara con un título orientado a la acción o insight.

- Se incluye al menos una visualización de datos espaciales (mapa) con anotaciones explicativas.
- El diseño evita el desorden visual (clutter) y utiliza el color de forma estratégica para destacar datos clave.
- La conclusión incluye una recomendación específica basada exclusivamente en los datos presentados.

Rúbrica de evaluación: Storytelling con datos espaciales

Evalúa una presentación de 5 a 8 diapositivas que comunique un hallazgo espacial mediante una narrativa clara, visualizaciones efectivas y una recomendación accionable para una audiencia de negocios. La rúbrica prioriza la capacidad de seleccionar un insight relevante, estructurarlo en tres actos (contexto, hallazgo y acción) y presentarlo con claridad visual y anotaciones que guíen la interpretación.

Criterio	Descripción	Peso
Selección y relevancia del insight espacial	Valora si el estudiante elige un patrón espacial claro, significativo y pertinente para el objetivo de negocio, y si explica por qué ese hallazgo merece ser contado.	30%
Estructura narrativa en tres actos	Evalúa si la presentación organiza la historia en contexto del problema, desarrollo de hallazgos y conclusión con acción, manteniendo una secuencia lógica y fácil de seguir.	25%
Calidad de las visualizaciones y uso del diseño	Mide si las visualizaciones son adecuadas para el análisis espacial, si reducen la carga cognitiva y si el uso de color, jerarquía visual y principios de Gestalt ayudan a comprender el mensaje.	20%
Anotaciones y guía de lectura	Evalúa si las anotaciones, etiquetas o llamadas visuales orientan la mirada del espectador hacia los puntos clave y ayudan a interpretar el mapa o gráfico sin depender de explicaciones externas.	15%
Conclusión y recomendación accionable	Valora si la presentación termina con una conclusión clara, conectada con el hallazgo, y una recomendación concreta, realista y útil para la toma de decisiones.	10%

Total: 100 pts | Aprobación: 70 pts

MÓDULO 14

Datos Espaciales y Visualizaciones Animadas

En este módulo, profundizarás en los conceptos esenciales del análisis espacial, aprendiendo a distinguir entre datos vectoriales y ráster para seleccionar el modelo adecuado según el problema.

Modelos de datos espaciales: Vector vs Ráster

En el análisis geoespacial, los datos se representan principalmente mediante dos modelos: **vectorial** y **ráster**. Cada uno tiene características, ventajas y limitaciones que los hacen adecuados para diferentes tipos de análisis.

Datos Vectoriales

El modelo vectorial representa la realidad mediante **elementos geométricos discretos**:

- **Puntos**: Representan ubicaciones específicas sin dimensión, como un árbol, una estación meteorológica o un punto de interés.
- **Líneas**: Conectan puntos para representar elementos lineales como carreteras, ríos o rutas.
- **Polígonos**: Áreas cerradas que representan superficies como lagos, zonas urbanas o parcelas.

Características clave:

- Precisión geométrica alta.
- Adecuado para representar objetos con límites definidos.
- Permite almacenar atributos detallados asociados a cada entidad.

Ejemplo de uso:

- Mapear la ubicación de tiendas (puntos).
- Analizar redes de transporte (líneas).
- Delimitar zonas administrativas (polígonos).

Datos Ráster

El modelo ráster representa la realidad como una **matriz regular de celdas o píxeles**, donde cada celda tiene un valor que representa una característica del área cubierta.

- Cada píxel tiene una ubicación espacial y un valor asociado (por ejemplo, temperatura, elevación, uso del suelo).
- La resolución espacial depende del tamaño de cada píxel.

Características clave:

- Ideal para representar fenómenos continuos o variables espaciales.
- Fácil de realizar operaciones matemáticas y análisis de superficie.

- Puede requerir gran capacidad de almacenamiento para alta resolución.

Ejemplo de uso:

- Mapas de elevación digital (DEM).
- Imágenes satelitales y fotografías aéreas.
- Mapas de cobertura vegetal o temperatura.

Comparación y Consideraciones

Aspecto	Vector	Ráster
Representación	Objetos discretos (puntos, líneas, polígonos)	Superficies continuas (grillas)
Precisión espacial	Alta, definida por coordenadas exactas	Depende de la resolución del píxel
Almacenamiento	Eficiente para datos con pocos objetos	Puede ser grande para alta resolución
Análisis típico	Redes, proximidad, topología	Análisis de superficie, interpolación

---

> **Nota**
> La elección entre vector y ráster depende de la naturaleza del fenómeno a analizar y la pregunta de investigación.

Contexto

En la práctica profesional de ciencia de datos y análisis geoespacial, seleccionar el modelo de datos adecuado es fundamental para garantizar la calidad y relevancia de los resultados. Por ejemplo, si trabajas con ubicaciones específicas como puntos de venta o estaciones meteorológicas, el modelo vectorial es el más adecuado por su precisión y capacidad para manejar atributos detallados.

Por otro lado, si tu análisis involucra variables continuas como temperatura, elevación o cobertura vegetal, el modelo ráster te permitirá trabajar con datos en forma de grillas que representan estas variaciones espaciales de manera eficiente.

Además, entender estas diferencias te ayudará a preparar correctamente tus datos para su manipulación con herramientas como GeoPandas, que maneja principalmente datos vectoriales, o librerías especializadas en ráster como Rasterio (que se abordarán en módulos posteriores).

Este conocimiento también es clave para evitar errores comunes, como superponer capas con modelos incompatibles sin la debida transformación, lo que puede llevar a interpretaciones erróneas en tus análisis y visualizaciones.

Datos Espaciales y Visualizaciones Animadas

Imagina que estás observando un mapa de una ciudad densamente poblada como Madrid o la Ciudad de México. Ves puntos que representan estaciones de metro. Es una imagen estática, útil pero limitada. Ahora, imagina que puedes presionar un botón de "Play" y ver cómo, a lo largo de las horas, esos puntos cambian de tamaño según el flujo de pasajeros: crecen exponencialmente a las 8:00 AM y se reducen al mediodía. De pronto, el mapa deja de ser un dibujo y se convierte en un **organismo vivo**.

Esa es la diferencia fundamental entre tener simplemente datos y dominar los **Datos Espaciales y las Visualizaciones Animadas**. En esta unidad, aprenderemos que el análisis geoespacial no se trata solo de saber *dónde* ocurren las cosas, sino de entender *cómo evolucionan* en el espacio y el tiempo.

1. ¿Qué son realmente los Datos Espaciales?

Muchos principiantes cometen el error de pensar que un dato espacial es simplemente una foto de satélite o un mapa colorido. Sin embargo, la realidad es mucho más profunda.

El Dato oculto tras la imagen

Un dato espacial es cualquier información que identifica la ubicación geográfica y las características de elementos naturales o construidos en la Tierra. Pero lo que lo hace "especial" para la Ciencia de Datos no es la imagen, sino la **base de datos atributiva** que tiene detrás.

Cuando ves un punto en un mapa digital representando un hospital, el sistema no solo sabe que está en la latitud `40.4167` y longitud `-3.7033`. Detrás hay una fila en una tabla que indica: * **Nombre:** Hospital Central. * **Capacidad:** 500 camas. * **Estado:** Operativo. * **Última actualización:** 2024-05-10.

El concepto clave: El Mapa es la Visualización; la Tabla es el Dato. Sin la tabla, el mapa es solo un dibujo. Con la tabla, el mapa es una herramienta de análisis.

2. La Metáfora de la "Torta de Capas" (Layer Cake)

Para entender cómo se estructuran los datos espaciales, los expertos en GIS (*Geographic Information Systems*) utilizan la analogía de la **torta de capas**. Nada en un sistema de mapas moderno es un bloque sólido; todo es un conjunto de capas transparentes superpuestas.

¿Por qué capas?

Imagina que estás diseñando una nueva ruta de logística para una empresa de entregas. No quieres ver todo al mismo tiempo porque te confundirías. El sistema te permite "apilar" información:

1. **Capa Base (El bizcocho):** Es el mapa base. El relieve, las montañas y los ríos. Es el contexto físico.
2. **Capa de Infraestructura (El relleno):** Aquí aparecen las carreteras, puentes y calles.
3. **Capa Administrativa (La cobertura de chocolate):** Las fronteras de los municipios o códigos postales.
4. **Capa Temática (Los adornos):** Es la información que nos interesa hoy. Por ejemplo, la ubicación de nuestros clientes o la densidad de tráfico en tiempo real.

Aplicación Real: La **IDEE (Infraestructura de Datos Espaciales de España)** funciona exactamente así. Permite al usuario seleccionar capas de incendios forestales, demografía o catastro de forma independiente. Si quieres analizar el riesgo de incendios, apagas la capa de "centros comerciales" y enciendes la de "bosques" y "temperatura". Esta modularidad es lo que permite que el análisis espacial sea flexible.

3. Visualizaciones Animadas: Del "Dónde" al "Cómo ocurre"

Si los datos espaciales nos dicen "dónde", la **animación** nos explica "qué pasó después". En el mundo del Machine Learning y la Ciencia de Datos, la animación no es un adorno estético; es una herramienta analítica de primer orden.

La animación como herramienta de análisis

Un error frecuente es pensar que la animación es solo para que una presentación se vea "bonita" ante un cliente. Pero piensa en este escenario: * **Mapa Estático:** Muestra que hubo 100 focos de calor en un bosque el lunes y 500 el martes. * **Mapa Animado:** Muestra el *recorrido* de las llamas segundo a segundo.

La animación te permite identificar el **patrón de propagación**. Te permite ver que el fuego se movió hacia el norte a 5 km/h debido al viento. Sin la animación, esa "velocidad" y esa "dirección" son invisibles; son solo puntos en dos momentos distintos.

El caso de Google Earth Engine

Uno de los ejemplos más potentes de visualización animada en la industria es **Google Earth Engine**. Esta plataforma procesa décadas de imágenes satelitales. Gracias a sus herramientas de animación, los científicos han podido mostrar el retroceso de los glaciares en el Ártico no como dos fotos (1980 vs. 2024), sino como un flujo continuo. Ver el hielo "derretirse" en una transición suave de 10 segundos comunica el impacto del cambio climático de una manera que 50 tablas de datos nunca podrían.

4. Diferencias Críticas: Estático vs. Dinámico vs. Animado

Para un científico de datos, es vital no confundir estos términos:

Tipo de Visualización	Descripción	Ejemplo de Uso
Estática	No cambia. Es una captura de pantalla.	Un mapa en un informe PDF anual.
Dinámica / Interactiva	El usuario puede interactuar (hacer zoom, filtrar).	Google Maps cuando buscas un restaurante.
Animada	El sistema muestra cambios automáticamente a través del tiempo.	Un mapa de vientos moviéndose o la expansión de una epidemia.

Anticipando una confusión común: "Si puedo hacer zoom en un mapa, ¿es animado?". **No.** Eso es interactivo. Un mapa es *animado* cuando los datos cambian frente a tus ojos sin que tú tengas que tocar nada, usualmente siguiendo una línea de tiempo (como el avance del COVID-19 por países).

5. Casos de Estudio: Los Datos Espaciales en Acción

Caso A: UNIDE (San Juan, Argentina) - Gestión Pública

En la provincia de San Juan, Argentina, se utiliza la **UNIDE (Unidad de Datos Territoriales)** para centralizar información. ¿Cómo usan la animación aquí? Al cruzar los datos de salud con los de transporte, pueden animar el flujo de personas hacia los hospitales. Si detectan que a las 10:00 AM hay una saturación en las rutas que llevan al hospital principal, el gobierno puede tomar decisiones sobre el ordenamiento territorial y sincronización de semáforos basándose en el **movimiento** observado, no solo en la ubicación de los edificios.

Caso B: Logística y "Last Mile" (La última milla)

Empresas como Amazon o DHL no solo ven dónde están sus furgonetas. Utilizan visualizaciones animadas internas para monitorear el flujo de entregas. Si una zona de la ciudad se "tiñe de rojo" (animación basada en velocidad) debido a una manifestación, el sistema de Machine Learning predice el retraso y reencamina el resto de las entregas en tiempo real.

6. Errores Comunes y Mejores Prácticas

A medida que comiences a trabajar con estos datos, ten cuidado con estas "trampas":

Error 1: Ignorar los Metadatos

El problema: Descargar un conjunto de puntos de "clima" y empezar a graficarlos sin saber cuándo se tomaron. **La solución:** Un dato espacial sin **metadatos** (información sobre el dato: quién lo midió, con qué precisión, en qué fecha) es peligroso. Siempre verifica la fuente y la vigencia. Si visualizas datos de tráfico de 2020 para tomar decisiones en la actualidad, tus conclusiones serán erróneas.

Error 2: Exceso de Animación ("Efecto Fuegos Artificiales")

El problema: Animar todo solo porque "se puede". Demasiado movimiento distrae al cerebro y no permite ver los datos. **La solución:** Usa la animación solo cuando el **tiempo** sea la variable más importante. Si quieres comparar las ventas de dos tiendas, un gráfico de barras es mejor. Si quieres ver cómo las ventas de una tienda migraron de una sucursal física a la tienda online durante el año, usa un mapa animado.

Error 3: Confundir una Imagen con un Raster

Como vimos en la unidad anterior, el modelo Raster es una cuadrícula de píxeles. Un error común es pensar que una foto aérea de baja resolución es suficiente para un análisis de precisión (como medir el área de un edificio). **La regla de oro:** Usa vectores para precisión (límites, calles) y rásteres para continuos (temperatura, elevación del terreno).

7. Síntesis y Conexiones

Hemos recorrido el camino desde entender el dato detrás del mapa hasta la potencia de la animación para revelar tendencias ocultas.

Resumen de puntos clave: 1. **Datos Espaciales = Ubicación + Atributos.** No son solo dibujos; son bases de datos con coordenadas. 2. **Estructura de Capas:** La información se organiza de forma modular (torta de capas), permitiendo análisis complejos sin saturación visual. 3. **Animación como Análisis:** Animar permite capturar la velocidad, la dirección y la tasa de cambio de los fenómenos, algo que los mapas estáticos no pueden hacer. 4. **Uso Responsable:** La animación debe servir para explicar el paso del tiempo o el movimiento, no para decorar.

En la siguiente unidad, daremos un paso técnico fundamental: entenderemos los **Sistemas de Referencia y Proyecciones**. Porque, aunque ya sabemos *qué* son los datos y cómo *animarlos*, necesitamos asegurarnos de que el mundo "redondo" se traduzca correctamente a nuestra pantalla "plana" sin deformarse. ¿Sabías que el mapa que ves en Google Maps miente sobre el tamaño real de los países? Eso es lo que descubriremos a continuación.

Sistemas de referencia y proyecciones (WGS84, proyectadas)

Un **Sistema de Referencia Espacial (CRS)** es un marco que permite ubicar puntos en la superficie terrestre mediante coordenadas. Los CRS definen cómo se representan las posiciones geográficas en un sistema de coordenadas, ya sea en grados (latitud, longitud) o en unidades planas (metros, pies).

Tipos de CRS

1. CRS Geográficas

- Basados en un sistema de coordenadas angulares: latitud y longitud.
- Ejemplo más común: **WGS84** (World Geodetic System 1984), utilizado por GPS.
- Representan posiciones sobre una esfera o elipsoide.

2. CRS Proyectadas

- Transforman las coordenadas geográficas en un sistema plano mediante una **proyección cartográfica**.
- Usan unidades lineales (metros, pies).
- Ejemplos: UTM (Universal Transverse Mercator), Lambert Conformal Conic.

> **Nota**
>
> Las proyecciones cartográficas introducen distorsiones en áreas, distancias o ángulos, por lo que la elección del CRS proyectado depende del análisis y la región de interés.

Transformaciones entre CRS

Para combinar o analizar capas geoespaciales, es fundamental que todas estén en el mismo CRS. La transformación o reproyección es el proceso de convertir datos de un CRS a otro.

- En Python, bibliotecas como **GeoPandas** y **PyProj** facilitan estas transformaciones.
- La función `to_crs()` de GeoPandas permite reproyectar un GeoDataFrame a un CRS deseado.

Errores comunes al reproyectar

- **No reproyectar capas antes de combinarlas:** puede causar desplazamientos visibles.
- **Confundir CRS geográfico con proyectado:** intentar usar coordenadas geográficas como si fueran proyectadas puede generar errores en cálculos de distancia o área.
- **No verificar el CRS original de los datos:** asumir un CRS incorrecto lleva a reproyecciones erróneas.

Buenas prácticas

- Siempre verificar el CRS original con `gdf.crs`.
- Elegir un CRS proyectado adecuado para la zona de estudio.
- Reproyectar todas las capas al mismo CRS antes de análisis o visualización.
- Documentar las transformaciones realizadas para reproducibilidad.

Ejemplo en Python con GeoPandas

CÓDIGO

```
import geopandas as gpd

<h1 class="module" data-eyebrow="MÓDULO">Cargar capa con CRS geográfico (WGS84)</h1>
<hr class="module-underline"><hr class="module-hairline">
gdf = gpd.read_file('datos.shp')
print(gdf.crs) # Ejemplo: EPSG:4326 (WGS84)

<h1 class="module" data-eyebrow="MÓDULO">Reproyectar a UTM zona 33N (EPSG:32633)</h1>
<hr class="module-underline"><hr class="module-hairline">
gdf_utm = gdf.to_crs(epsg=32633)
print(gdf_utm.crs)
```

Este proceso asegura que las coordenadas estén en metros, facilitando cálculos precisos de distancia y área.

Contexto

En la práctica profesional, trabajar con datos geoespaciales implica integrar múltiples fuentes que pueden venir en distintos CRS. Por ejemplo, un analista puede recibir datos de sensores GPS (en WGS84) y capas administrativas oficiales en proyecciones locales. Sin una correcta transformación, la superposición y análisis espacial serán erróneos, afectando la toma de decisiones.

Además, muchas visualizaciones interactivas y dashboards requieren que los datos estén en un CRS compatible con la herramienta (por ejemplo, Web Mercator EPSG:3857 para mapas web). Por ello, dominar las transformaciones CRS es clave para garantizar la calidad y utilidad de los productos geoespaciales.

Esta unidad te prepara para manejar estas situaciones con confianza, evitando errores comunes y aplicando buenas prácticas que asegurarán la integridad espacial de tus proyectos.

Accesibilidad, estética y comunicación ejecutiva

Imagina que has desarrollado un análisis espacial complejo que revela patrones críticos para la toma de decisiones en tu organización.

Sin embargo, si tu audiencia no puede interpretar fácilmente tus visualizaciones o si la presentación no está adaptada a sus necesidades, el impacto de tu trabajo se pierde.

¿Cómo asegurar que tus insights sean accesibles, claros y persuasivos para diferentes públicos?

En esta unidad, reforzaremos cómo aplicar principios de accesibilidad y comunicación ejecutiva para transformar tus análisis en narrativas visuales efectivas, utilizando paletas de colores adecuadas, anotaciones claras y formatos de presentación orientados a la acción.

Paletas accesibles: selección y aplicación

La accesibilidad en visualización de datos implica que la información sea comprensible para personas con diferentes capacidades visuales, incluyendo daltonismo y baja visión.

Para ello, es fundamental elegir **paletas de colores accesibles** que:

- Mantengan contraste suficiente entre colores
- Eviten combinaciones problemáticas (ej. rojo-verde)
- Sean interpretables en escala de grises

Ejemplos de paletas accesibles:

Paleta	Características	Uso recomendado
ColorBrewer (Set2)	Colores diferenciados y contrastantes	Mapas coropléticos, categorías
Viridis	Gradiente perceptualmente uniforme	Mapas de calor, variables continuas

Herramientas como ColorBrewer y librerías Python (`matplotlib` , `seaborn`) facilitan la selección.

Anotaciones efectivas: claridad y concisión

Las anotaciones en visualizaciones deben guiar al usuario sin saturar la gráfica. Para lograrlo:

- Usa texto breve y directo
- Emplea tipografía legible y tamaño adecuado
- Destaca puntos clave con colores o símbolos
- Evita jerga técnica innecesaria

> **Nota**
> Las anotaciones deben responder a la pregunta "¿Qué debe entender el usuario aquí?" y no solo describir lo obvio.

Preparación de resúmenes ejecutivos y slides para audiencias decisoras

Un **resumen ejecutivo** sintetiza los hallazgos clave y recomendaciones en 1–2 páginas, con lenguaje claro y visualizaciones limpias. Las **slides** deben:

- Priorizar mensajes clave en cada diapositiva
- Usar gráficos simples y paletas accesibles
- Incluir títulos descriptivos y anotaciones
- Seguir principios de diseño (espacio en blanco, alineación, consistencia)

Principios de usabilidad como las **Leyes de Nielsen** y la **Teoría Gestalt** apoyan la efectividad visual y cognitiva.

> **Ejemplo**
> Un slide con un mapa coroplético que resalta zonas críticas con anotaciones breves y un título claro.

Conexión con conceptos previos

Esta unidad refuerza los fundamentos de percepción visual y storytelling aprendidos anteriormente, aplicándolos a la accesibilidad y comunicación ejecutiva para asegurar que tus entregables sean inclusivos y persuasivos.

Contexto

En el contexto de proyectos geoespaciales, la accesibilidad y comunicación ejecutiva son clave para maximizar el impacto de tus análisis.

Por ejemplo, un dashboard interactivo con mapas debe ser usable por personas con diferentes capacidades visuales y debe presentar la información de forma que facilite decisiones rápidas y fundamentadas.

Además, en entornos corporativos o gubernamentales, los resúmenes ejecutivos y presentaciones son herramientas esenciales para comunicar resultados a stakeholders no técnicos. Aplicar paletas accesibles y anotaciones claras garantiza que el mensaje no se pierda por barreras visuales o cognitivas.

Esta unidad te prepara para diseñar entregables que cumplan con estos estándares, facilitando la adopción y acción basada en tus análisis espaciales.

PRE-ENTREGA DEL PROYECTO

Actividad evaluable · parte del proyecto integrador

Pre-entrega: Análisis Espacial Comparativo

El objetivo de esta actividad es que demuestres tu capacidad para distinguir y justificar el uso de modelos de datos espaciales (Vectorial y Ráster) en contextos de Ciencia de Datos aplicados.

Cómo resolver esta entrega

Propósito

Consolidar la comprensión teórica sobre la estructura de los datos espaciales y su aplicación práctica, asegurando que puedas seleccionar la representación más eficiente según la naturaleza del problema geográfico.

Tarea

Debes redactar un informe técnico en formato PDF que resuelva los siguientes puntos:

1. **Definición de Conceptos:** Explica con tus palabras la diferencia fundamental entre el modelo vectorial (puntos, líneas, polígonos) y el modelo ráster (celdas/píxeles).
2. **Casos de Uso:** Propón un escenario real de ciencia de datos donde sea obligatorio usar datos **Vectoriales** (ej. redes de transporte) y otro donde sea ideal usar **Ráster** (ej. modelos de elevación o temperatura).
3. **Selección de Formatos:** Identifica al menos dos extensiones de archivo comunes para cada modelo (.shp, .geojson, .tif, etc.) y explica en qué software o librería de Python se suelen manipular.
4. **Matriz de Decisión:** Crea una tabla comparativa breve resaltando Ventajas y Desventajas de cada formato en términos de precisión y almacenamiento.

Recomendación

Evita definiciones genéricas de diccionario; enfoca tu respuesta en cómo estos formatos afectan el procesamiento en un pipeline de Machine Learning (por ejemplo, la dificultad de tabular píxeles frente a la facilidad de unir tablas de atributos en vectores).

Objetivos de aprendizaje

- Diferenciar las estructuras de datos vectoriales y ráster en el contexto de análisis espacial.
- Categorizar problemas del mundo real según el modelo de datos espaciales más eficiente.
- Identificar formatos de archivo e interoperabilidad de herramientas para datos geográficos.

Criterios de aceptación

- El informe incluye una explicación clara de las diferencias técnicas entre píxeles (ráster) y geometría basada en coordenadas (vectorial).
- Se describen al menos dos casos de uso diferenciados y coherentes con la naturaleza del dato.
- Se listan correctamente al menos dos formatos de archivo para cada tipo de dato espacial.
- La matriz de decisión compara al menos tres variables (precisión, peso de archivo, facilidad de procesamiento).
- El documento se entrega exclusivamente en formato PDF siguiendo una estructura profesional.

Rúbrica de Evaluación: Análisis Espacial Comparativo (Vectorial vs. Ráster)

Rúbrica para evaluar un informe técnico en PDF sobre la comparación entre datos espaciales vectoriales y ráster, con énfasis en definición conceptual, casos de uso en ciencia de datos, selección de formatos y construcción de una matriz de निर्णय/decisión orientada al procesamiento en pipelines de Machine Learning.

Criterio	Descripción	Peso
Comprensión conceptual de los modelos vectorial y ráster	Evalúa si el informe explica con precisión la diferencia fundamental entre ambos modelos, usando sus propias palabras y conectándolo con su estructura y uso en análisis espacial.	30%
Aplicación a casos de uso reales en ciencia de datos	Evalúa la propuesta de un escenario real donde el uso de datos vectoriales sea necesario y otro donde el uso de datos ráster sea ideal, justificando la elección según la naturaleza del problema.	20%
Selección e interpretación de formatos y herramientas	Evalúa si identifica al menos dos extensiones comunes para cada modelo y si explica adecuadamente en qué software o librerías de Python se manipulan habitualmente.	20%
Matriz comparativa de ventajas y desventajas	Evalúa la tabla comparativa sobre ventajas y desventajas de vector y ráster en términos de precisión y almacenamiento, así como la claridad de la comparación.	15%
Enfoque técnico orientado a pipeline de Machine Learning	Evalúa si el informe conecta la teoría de formatos espaciales con su impacto práctico en un flujo de Machine Learning, tal como pide la recomendación del ejercicio.	10%
Calidad técnica y presentación del informe PDF	Evalúa la claridad, organización, redacción técnica y presentación general del archivo entregado como informe PDF.	5%

Total: 100 pts | Aprobación: 70 pts

MÓDULO 15

Visualización de variables categóricas

Este módulo introduce conceptos clave como forma, asimetría, curtosis y técnicas prácticas para detectar outliers usando Python y Pandas. Aprenderás a calcular y visualizar estas métricas, preparando una base sólida para avanzar hacia análisis bivariados y correlaciones.

Introducción a la distribución y medidas resumen

Imagina que tienes un conjunto de datos con las ventas diarias de una tienda durante un mes. ¿Cómo podrías describir rápidamente el comportamiento de esas ventas? ¿Cuál es el valor típico, qué tan dispersos están los datos y si hay días con ventas inusualmente altas o bajas? Estas preguntas son fundamentales en el análisis de datos y se responden a través del estudio de la **distribución univariada** y sus medidas resumen.

En esta unidad, exploraremos cómo describir una variable numérica mediante medidas de tendencia central y dispersión, y cómo visualizar su distribución para detectar patrones y posibles valores atípicos (*outliers*). Estos conceptos son la base para cualquier análisis estadístico y para preparar datos en proyectos de ciencia de datos y machine learning.

Al finalizar, podrás explicar qué representa una distribución univariada y diferenciar entre las principales medidas resumen, entendiendo sus usos y limitaciones en contextos reales de análisis de datos.

Distribución univariada

La **distribución univariada** describe cómo se distribuyen los valores de una sola variable numérica. Nos interesa conocer el patrón de frecuencias, la forma general, la simetría y la presencia de colas o valores extremos.

- **Forma**: Puede ser simétrica, sesgada a la derecha (cola larga hacia valores altos) o a la izquierda (cola larga hacia valores bajos).
- **Simetría**: Indica si la distribución es balanceada alrededor de un punto central.
- **Colas**: Extremos de la distribución donde pueden aparecer valores atípicos.

> **Ejemplo**
>
> Las alturas de personas adultas suelen tener una distribución aproximadamente simétrica, mientras que los ingresos suelen estar sesgados a la derecha.

Medidas de tendencia central

Estas medidas resumen el "centro" o valor típico de la distribución:

- **Media** (`mean`): Promedio aritmético de todos los valores.
- *Ventaja:* Utiliza toda la información.
- *Desventaja:* Sensible a valores extremos.
- **Mediana** (`median`): Valor que divide la distribución en dos partes iguales.
- *Ventaja:* Robusta frente a outliers.
- *Desventaja:* No considera la magnitud de todos los valores.
- **Moda** (`mode`): Valor que aparece con mayor frecuencia.
- *Uso:* Útil para variables categóricas o para detectar valores frecuentes.

> **Ejemplo**
> En un conjunto de salarios, la media puede ser alta por unos pocos salarios muy altos, mientras que la mediana refleja mejor el salario típico.

Medidas de dispersión

Indican cuánto varían los datos alrededor de la tendencia central:

- **Varianza** (`variance`): Promedio de los cuadrados de las diferencias respecto a la media.
- **Desviación estándar** (`std`): Raíz cuadrada de la varianza, en las mismas unidades que los datos.
- **Rango intercuartílico (IQR)**: Diferencia entre el tercer y primer cuartil (Q3 - Q1), mide la dispersión del 50% central.

> **Nota**
> La desviación estándar es sensible a outliers, mientras que el IQR es más robusto.

Visualización univariada

Para entender mejor la distribución, usamos gráficos:

- **Histograma**: Muestra la frecuencia de valores en intervalos (bins).
- **Boxplot (diagrama de caja)**: Resume mediana, cuartiles y posibles outliers.
- **Densidad**: Estimación suave de la distribución de probabilidad.

Estos gráficos permiten detectar asimetrías, concentración de datos y valores atípicos.

Limitaciones y sensibilidad a outliers

- La **media** y la **desviación estándar** pueden ser distorsionadas por valores extremos.
- La **mediana** y el **IQR** son más resistentes a outliers.

Por eso, es importante combinar medidas numéricas con visualizaciones para una interpretación completa.

Contexto

En la práctica del análisis de datos, describir correctamente una variable es el primer paso para entender el comportamiento del fenómeno estudiado. Por ejemplo, en análisis de ventas, conocer la distribución de ingresos diarios ayuda a identificar patrones estacionales o días atípicos que pueden requerir atención especial.

Además, estas medidas resumen son esenciales para preparar datos antes de aplicar modelos predictivos. Por ejemplo, detectar outliers puede evitar que modelos de machine learning se vean sesgados o que las transformaciones de variables sean inapropiadas.

En entornos industriales, como finanzas o salud, interpretar correctamente estas medidas puede influir en decisiones críticas, desde la asignación de recursos hasta la detección de fraudes o anomalías.

Esta unidad sienta las bases para análisis más complejos, como el análisis bivariado y la selección de variables, que veremos en unidades posteriores.

Resumen y visualización de variables categóricas

¿Alguna vez te has preguntado cómo resumir y visualizar datos que no son números, sino categorías? Por ejemplo, ¿cómo analizar la distribución de géneros en un conjunto de datos o la frecuencia de diferentes tipos de productos vendidos?

En este módulo, aprenderás a describir y representar visualmente variables categóricas, una habilidad fundamental para cualquier analista o científico de datos.

Al finalizar esta unidad, serás capaz de interpretar tablas de frecuencia y proporciones, y elegir gráficos adecuados para variables categóricas, facilitando la comunicación clara y efectiva de tus hallazgos. Esta base te preparará para análisis más complejos y para integrar estas técnicas en pipelines reproducibles de análisis y modelado.

Resumen de variables categóricas: frecuencias y proporciones

Las variables categóricas representan datos agrupados en categorías o clases, como "color de producto", "tipo de cliente" o "región geográfica".

Para describirlas, usamos principalmente:

- **Tablas de frecuencia**: cuentan cuántas observaciones hay en cada categoría.
- **Proporciones o porcentajes**: expresan la frecuencia relativa de cada categoría respecto al total.

Por ejemplo, si tenemos una variable `color` con categorías `Rojo`, `Azul` y `Verde`, una tabla de frecuencia podría ser:

Color	Frecuencia	Proporción
Rojo	50	0.50
Azul	30	0.30
Verde	20	0.20

Estas tablas permiten identificar rápidamente la distribución y detectar categorías dominantes o minoritarias.

Visualización con barplots y countplots

Para representar visualmente variables categóricas, los gráficos de barras son la opción más común y efectiva. En Python, con librerías como Matplotlib y Seaborn, podemos usar:

- `barplot`: muestra la altura de barras según valores agregados (por ejemplo, promedio o suma) para cada categoría.
- `countplot`: muestra la frecuencia de cada categoría directamente, ideal para variables categóricas.

Ejemplo con Seaborn:

CÓDIGO

```
import seaborn as sns
import matplotlib.pyplot as plt

<h1 class="module" data-eyebrow="MÓDULO">Supongamos un DataFrame df con una columna &#x27;cat
<hr class="module-underline"><hr class="module-hairline">
sns.countplot(x='categoria', data=df)
plt.title('Frecuencia de categorías')
plt.show()
```

Este gráfico facilita la comparación visual de la distribución de categorías.

Consideraciones con categorías raras y agrupación

En conjuntos de datos reales, algunas categorías pueden tener muy pocas observaciones (categorías raras), lo que puede dificultar el análisis y visualización.

Para manejar esto:

- **Agrupar categorías raras** en una categoría "Otros" para simplificar la interpretación.
- Evaluar si las categorías raras aportan valor o si deben ser excluidas o combinadas.

Esta práctica mejora la claridad de los gráficos y evita que categorías con muy poca representación distorsionen la interpretación.

> **Nota**
> Siempre documenta y justifica cualquier agrupación o exclusión para mantener la reproducibilidad y transparencia del análisis.

Aplicación práctica en análisis de datos

En la industria, la correcta interpretación y visualización de variables categóricas es clave para:

- Identificar segmentos de clientes predominantes o emergentes.
- Detectar desequilibrios en datos que pueden afectar modelos predictivos.
- Comunicar hallazgos a equipos no técnicos mediante gráficos claros.

Por ejemplo, un analista de marketing puede usar tablas de frecuencia y countplots para entender la distribución de respuestas en una encuesta segmentada por región o tipo de producto, facilitando decisiones estratégicas.

Además, estas técnicas preparan el terreno para análisis bivariados y multivariados, donde se comparan categorías entre sí o con variables numéricas, tema que abordaremos en unidades posteriores.

Detección de outliers: teoría y criterios

Detección de Outliers: Fundamentos y Métodos

¿Alguna vez te has preguntado cómo detectar datos que no encajan en un patrón esperado? Estos datos, conocidos como *outliers* o valores atípicos, pueden distorsionar análisis y modelos predictivos si no se manejan adecuadamente.

En esta unidad, aprenderás dos métodos estadísticos fundamentales para identificar outliers: el **rango intercuartílico (IQR)** y el **Z-score**.

Comprenderás no solo cómo calcularlos, sino también cuándo y por qué usarlos, considerando las características de tus datos, como la distribución y el tamaño de la muestra. Este conocimiento es clave para asegurar que tus análisis sean robustos y tus modelos confiables, preparando el camino para técnicas más avanzadas de análisis multivariado y modelado.

Los *outliers* son observaciones que se desvían significativamente del patrón general de un conjunto de datos. Su detección es crucial para evitar sesgos en análisis estadísticos y modelos de Machine Learning.

Método del Rango Intercuartílico (IQR)

El **IQR** es una medida de dispersión que abarca el rango entre el primer cuartil (Q1, 25%) y el tercer cuartil (Q3, 75%) de los datos:

IQR=Q3−Q 1_ IQR_= *Q* 3− *Q* 1

Se considera que un valor es un outlier si está por debajo de:

Q1−1.5×IQ R_ Q_ 1−1.5×*IQR*

o por encima de:

Q3+1.5×IQ R_ Q_ 3+1.5×*IQR*

Ventajas del IQR

- No asume distribución normal; es robusto frente a distribuciones sesgadas.
- Fácil de calcular e interpretar.

Limitaciones del IQR

- Sensible a tamaño de muestra pequeño.
- No detecta outliers en colas extremas si la distribución es muy dispersa.

Método del Z-score

El **Z-score** mide cuántas desviaciones estándar está un dato respecto a la media:

Z=X−μσ *Z*=*σX*−*μ*

Donde:

- X_ X_: valor observado
- μ*μ*: media de la muestra
- σ*σ*: desviación estándar

Un valor con ∣Z∣>3∣*Z*∣>3 suele considerarse outlier.

Ventajas del Z-score

- Útil para datos con distribución aproximadamente normal.
- Permite detectar outliers en ambos extremos.

Limitaciones del Z-score

- No es robusto ante distribuciones no normales o con sesgo.
- La media y desviación estándar pueden estar influenciadas por outliers, afectando el cálculo.

> **Nota**
>
> Existen métodos basados en modelos (como clustering o modelos probabilísticos) para detección de outliers, pero en esta unidad nos enfocamos en métodos estadísticos clásicos para establecer bases sólidas.

Contexto

En la práctica del análisis de datos, la detección de outliers es un paso esencial antes de construir modelos predictivos o realizar inferencias estadísticas. Por ejemplo, en análisis financiero, un outlier puede representar un error de registro o un evento excepcional que debe ser tratado de forma distinta. En calidad de manufactura, detectar valores atípicos puede indicar fallas en el proceso.

Al aplicar el método IQR, es común usarlo en variables con distribuciones desconocidas o sesgadas, ya que no depende de la media ni desviación estándar. Por otro lado, el Z-score es preferible cuando los datos se aproximan a una distribución normal, como en mediciones físicas o biológicas.

Además, el tamaño de la muestra influye: con muestras pequeñas, los límites del IQR pueden ser menos confiables, y el Z-score puede ser inestable si la estimación de la desviación estándar es imprecisa.

Por ello, la elección del método debe considerar la naturaleza de los datos y el contexto del análisis, siempre complementando con visualizaciones como boxplots o histogramas para validar hallazgos.

Transformaciones univariadas para visualización

Transformaciones y escalado para análisis univariado

¿Alguna vez te has encontrado con una variable numérica cuya distribución está tan sesgada que dificulta su análisis o visualización?

En análisis de datos, las transformaciones univariadas son herramientas esenciales para corregir asimetrías, estabilizar varianzas y facilitar interpretaciones más claras. En esta unidad, exploraremos transformaciones comunes como la logarítmica y la Box-Cox, entendiendo cuándo y cómo aplicarlas para mejorar la calidad de tus visualizaciones y análisis estadísticos.

Además, veremos cómo estas transformaciones impactan en histogramas y tests de normalidad, habilidades clave para cualquier analista o científico de datos que busca preparar sus datos para análisis más avanzados o modelado predictivo.

Las transformaciones univariadas son técnicas matemáticas aplicadas a una variable para modificar su distribución, con el objetivo de corregir problemas como la **asimetría** (skewness), **heterocedasticidad** (varianza no constante) o para preparar los datos para análisis posteriores.

MÓDULO

Transformación Logarítmica

La transformación logarítmica es una de las más comunes y se aplica principalmente a variables con distribución sesgada positivamente (cola larga a la derecha).

- **Definición:** Para una variable (X), la transformación es (Y = \log(X)), usualmente con base 10 o natural.
- **Requisitos:** Los valores de (X) deben ser positivos; en caso contrario, se puede aplicar un desplazamiento (X + c) con (c > 0).

Ejemplo en la industria: En análisis financiero, los ingresos o ventas suelen tener distribuciones sesgadas; aplicar logaritmos ayuda a visualizar y modelar mejor estos datos.

Transformación Box-Cox

La transformación Box-Cox es una familia de transformaciones parametrizadas que busca encontrar la mejor transformación para normalizar una variable.

- **Definición:** Para (\lambda \neq 0),Y(lambda)=fracXlambda−1lambd a_ Y_(*lambda*)=*fracXlambda*−1_ lambd a_ Para (\lambda = 0), se define como (Y(0) = \log(X)).
- **Ventaja:** Encuentra automáticamente el parámetro (\lambda) que maximiza la normalidad.
- **Requisito:** (X) debe ser positivo.

Aplicación práctica: En análisis de calidad, para variables con distribución no normal, Box-Cox ayuda a estabilizar la varianza y mejorar la validez de tests estadísticos.

Efectos en histogramas y tests de normalidad

Las transformaciones modifican la forma de la distribución:

- **Histogramas:** Se observa una reducción del sesgo y una forma más simétrica.
- **Tests de normalidad:** Como Shapiro-Wilk o Kolmogorov-Smirnov, suelen mostrar mejores resultados (valores p mayores) tras la transformación.

> **Nota**
> No todas las transformaciones garantizan normalidad perfecta, pero pueden mejorar significativamente la interpretabilidad y análisis.

Escalado y estandarización (breve introducción)

Aunque no es el foco principal, es importante mencionar que el escalado (como la estandarización) transforma variables para que tengan media cero y desviación estándar uno, facilitando comparaciones y modelado.

Conoce más: Para profundizar en Box-Cox, consulta la documentación de `scipy.stats.boxcox` y ejemplos prácticos en Python.

Contexto

En la práctica del análisis de datos, las transformaciones univariadas son pasos previos fundamentales antes de realizar análisis estadísticos o construir modelos predictivos.

Por ejemplo, al analizar ventas mensuales de una empresa, una distribución sesgada puede ocultar patrones importantes o violar supuestos de normalidad necesarios para ciertos tests.

Al aplicar transformaciones como logarítmica o Box-Cox, el analista puede:

- Visualizar histogramas más simétricos y fáciles de interpretar.
- Mejorar la validez de tests de normalidad y homocedasticidad.
- Preparar variables para técnicas de modelado que asumen normalidad o varianza constante.

En Python, estas transformaciones se implementan fácilmente con `numpy` y `scipy.stats`, integrándose en pipelines reproducibles.

Esta unidad prepara el terreno para el análisis bivariado y la selección de variables en módulos posteriores, donde la calidad de las transformaciones impactará directamente en la robustez de los modelos de Machine Learning.

Fundamentos de tests: hipótesis, p-valor y supuestos

Fundamentos de las pruebas estadísticas

¿Alguna vez te has preguntado cómo los científicos y analistas deciden si una diferencia observada en sus datos es real o simplemente producto del azar? En el análisis de datos, esta pregunta se responde mediante pruebas estadísticas, que nos permiten tomar decisiones informadas basadas en evidencia cuantitativa.

En esta unidad, exploraremos los fundamentos que sustentan estas pruebas: desde la formulación de hipótesis hasta la interpretación del p-valor y la importancia de los supuestos que deben cumplirse para que los resultados sean válidos.

Además, conoceremos las pruebas paramétricas más comunes —el t-test y ANOVA— que se utilizan para comparar medias entre grupos cuando se cumplen ciertos supuestos, y la prueba chi-cuadrado, fundamental para analizar relaciones entre variables categóricas.

Al finalizar, estarás preparado para entender cuándo y cómo aplicar estas pruebas en tus análisis, diferenciando claramente entre métodos paramétricos y no paramétricos, y sentando las bases para un análisis estadístico riguroso y reproducible.

1. Estructura de una prueba estadística

Una prueba estadística es un procedimiento formal para evaluar una afirmación (hipótesis) sobre una población a partir de datos muestrales. Se plantean dos hipótesis:

- **Hipótesis nula (`H_0`)**: representa la afirmación de que no hay efecto o diferencia.
- **Hipótesis alternativa (`H_a`)**: representa la afirmación contraria, que sí existe un efecto o diferencia.

El objetivo es determinar si hay suficiente evidencia para rechazar `H_0` a favor de `H_a`.

2. Errores en la toma de decisiones

- **Error tipo I (α)**: rechazar `H_0` cuando es verdadera (falso positivo).
- **Error tipo II (β)**: no rechazar `H_0` cuando es falsa (falso negativo).

El nivel de significancia `α` es la probabilidad máxima tolerada de cometer un error tipo I, comúnmente 0.05.

3. P-valor

El **p-valor** es la probabilidad, bajo la hipótesis nula, de obtener un resultado igual o más extremo que el observado. Un p-valor bajo indica evidencia en contra de `H_0`.

> **Importante**
> El p-valor no es la probabilidad de que `H_0` sea verdadera.

4. Potencia estadística

La potencia es la probabilidad de detectar un efecto real (rechazar `H_0` cuando es falsa), es decir, `1 - β`. Una potencia alta reduce el riesgo de error tipo II.

5. Supuestos en pruebas estadísticas

Para que los resultados sean válidos, ciertas condiciones deben cumplirse, por ejemplo:

- Normalidad de los datos
- Homogeneidad de varianzas
- Independencia de observaciones

El incumplimiento puede invalidar la prueba o requerir métodos alternativos.

6. T-test

El **t-test** compara las medias de dos grupos para determinar si son significativamente diferentes. Se usa cuando:

- La variable dependiente es numérica y aproximadamente normal

- Las varianzas son homogéneas

Ejemplo: comparar la media de ventas entre dos regiones.

7. ANOVA (Análisis de Varianza)

El **ANOVA** extiende el t-test para comparar medias entre tres o más grupos. Evalúa si al menos un grupo difiere significativamente.

Ejemplo: comparar la satisfacción del cliente entre diferentes sucursales.

8. Chi-cuadrado para variables categóricas

La prueba **chi-cuadrado de independencia** evalúa si dos variables categóricas están asociadas o son independientes.

Ejemplo: analizar si la preferencia de producto depende del género.

Se basa en tablas de contingencia y compara frecuencias observadas con esperadas bajo independencia.

Contexto

En el análisis de datos y ciencia de datos, las pruebas estadísticas son herramientas esenciales para validar hipótesis y tomar decisiones basadas en evidencia. Por ejemplo, un analista puede querer saber si una campaña de marketing tuvo un impacto real en las ventas (t-test), o si la preferencia por un producto varía según la región (chi-cuadrado).

Comprender la estructura de las pruebas, los errores posibles y los supuestos garantiza que los resultados sean confiables y aplicables. Además, saber cuándo usar pruebas paramétricas como t-test y ANOVA, o pruebas no paramétricas, es clave para un análisis riguroso.

Este conocimiento es fundamental para preparar datos y seleccionar variables en pipelines de Machine Learning, asegurando que las decisiones de modelado se basen en análisis estadísticos sólidos y reproducibles.

Pruebas no paramétricas y alternativas

¿Por qué usar pruebas no paramétricas en análisis de datos?

Imagina que estás analizando datos de ventas de diferentes regiones, pero tus variables no siguen una distribución normal o tienes muestras pequeñas.

¿Cómo comparar grupos o medir asociaciones sin asumir normalidad? Aquí es donde las **pruebas no paramétricas** se vuelven esenciales. En esta unidad, exploraremos alternativas robustas como Mann-Whitney y Kruskal-Wallis, además de reforzar las medidas de correlación no paramétrica Spearman y Kendall, herramientas clave para relaciones monotónicas.

Al finalizar, podrás identificar cuándo aplicar estas pruebas y cómo interpretarlas en contextos reales de análisis de datos, fortaleciendo tu capacidad para tomar decisiones informadas y reproducibles en tus proyectos de ciencia de datos.

Medidas de correlación no paramétrica: Spearman y Kendall (Refuerzo)

Las correlaciones de **Spearman** y **Kendall** son medidas basadas en rangos que evalúan la asociación entre variables cuando la relación no es necesariamente lineal, sino monotónica.

- Correlación de Spearman (ρ): Calcula la correlación entre los rangos de dos variables. Es menos sensible a valores extremos y no requiere normalidad.
- Correlación de Kendall (τ): Mide la concordancia entre pares de observaciones, evaluando la probabilidad de que las variables cambien en la misma dirección.

> **Ejemplo**
> En análisis de satisfacción del cliente, donde las respuestas son ordinales, Spearman y Kendall permiten medir la relación entre satisfacción y tiempo de uso sin asumir distribuciones específicas.

Pruebas no paramétricas para comparación de grupos

Prueba de Mann-Whitney (U de Mann-Whitney)

- **Propósito:** Comparar dos grupos independientes cuando no se cumple la normalidad.
- **Fundamento:** Evalúa si una muestra tiende a tener valores mayores que la otra, usando rangos combinados.

> **Ejemplo**
> Comparar ingresos entre dos regiones con distribuciones sesgadas.

Prueba de Kruskal-Wallis

- **Propósito:** Extensión de Mann-Whitney para más de dos grupos independientes.
- **Fundamento:** Compara medianas de múltiples grupos usando rangos.

> **Ejemplo**
> Evaluar diferencias en satisfacción entre tres departamentos.

¿Cuándo usar pruebas no paramétricas?

- Datos no normales o con outliers significativos.
- Variables ordinales o de rango.
- Tamaños de muestra pequeños.
- Cuando las pruebas paramétricas no cumplen sus supuestos.

> **Nota**
>
> Aunque menos potentes que las paramétricas, las no paramétricas son más robustas y flexibles en escenarios reales de análisis de datos.

Contexto

En la práctica del análisis de datos, es común enfrentar variables que no cumplen con los supuestos de normalidad o homogeneidad de varianzas necesarios para pruebas paramétricas como el t-test o ANOVA. Por ejemplo, al analizar encuestas de satisfacción o datos financieros con distribuciones sesgadas, las pruebas no paramétricas ofrecen una alternativa confiable.

Además, las medidas de correlación no paramétrica Spearman y Kendall permiten detectar relaciones monotónicas que no son lineales, ampliando el espectro de análisis exploratorio.

Este conocimiento es fundamental para preparar datos y seleccionar variables en pipelines de Machine Learning, asegurando que las decisiones basadas en estadística sean sólidas y reproducibles.

Teoría de correlación y asociación

¿Qué nos dice la correlación sobre nuestras variables?

Imagina que trabajas con un conjunto de datos financieros y quieres entender cómo se relacionan dos indicadores clave, como el ingreso y el gasto. ¿Cómo cuantificas esa relación? Aquí es donde entran las medidas de correlación y asociación, herramientas fundamentales para cualquier analista o científico de datos.

En esta unidad, exploraremos las principales técnicas para medir la relación entre variables numéricas: el coeficiente de Pearson, y las correlaciones no paramétricas de Spearman y Kendall. Aprenderás no solo a calcularlas, sino a interpretar sus resultados en contextos reales, entendiendo sus supuestos y limitaciones.

Este conocimiento es esencial para avanzar en análisis más complejos, como la detección de multicolinealidad y la selección de variables para modelos predictivos. Prepárate para dominar estas herramientas clave que potenciarán tu análisis exploratorio y la calidad de tus modelos.

Correlación y asociación entre variables

La **correlación** es una medida estadística que cuantifica la relación entre dos variables. En términos simples, nos indica si y cómo dos variables cambian juntas. Existen diferentes tipos de correlación, que capturan distintos tipos de relaciones:

- **Correlación lineal:** mide la fuerza y dirección de una relación lineal entre dos variables numéricas.

- **Correlación monotónica:** mide si una variable tiende a aumentar o disminuir cuando la otra lo hace, sin requerir que la relación sea lineal.

> **Nota**
> La correlación no implica causalidad; dos variables pueden estar correlacionadas sin que una cause a la otra.

Coeficiente de correlación de Pearson

El coeficiente de Pearson, denotado como `r`, mide la **relación lineal** entre dos variables numéricas continuas. Su valor oscila entre -1 y 1:

- `r = 1`: correlación lineal positiva perfecta
- `r = -1`: correlación lineal negativa perfecta
- `r = 0`: no hay correlación lineal

Supuestos clave para usar Pearson:

- Las variables deben ser numéricas y continuas.
- La relación entre variables debe ser lineal.
- Las variables deben seguir una distribución aproximadamente normal.
- Homocedasticidad: la variabilidad de una variable es constante a lo largo de la otra.

Ejemplo práctico:

En análisis financiero, Pearson puede medir la relación lineal entre el ingreso mensual y el gasto mensual de clientes para detectar patrones de consumo.

Correlaciones no paramétricas: Spearman y Kendall

Cuando los supuestos de Pearson no se cumplen, especialmente la linealidad o normalidad, recurrimos a correlaciones basadas en rangos:

- **Spearman (`ρ`):** mide la correlación monotónica usando rangos de los datos. Es menos sensible a valores extremos y no requiere normalidad.
- **Kendall (`τ`):** también mide la asociación monotónica, basado en concordancia y discordancia de pares de observaciones. Es robusto y útil con muestras pequeñas.

Ambos coeficientes varían entre -1 y 1, interpretándose similar a Pearson pero para relaciones monotónicas, no necesariamente lineales.

Ejemplo práctico:

En estudios de satisfacción del cliente, donde las variables son ordinales o no lineales, Spearman y Kendall permiten evaluar asociaciones sin asumir distribuciones específicas.

Limitaciones y consideraciones

- **Correlación no implica causalidad:** Una alta correlación no garantiza que una variable cause cambios en la otra.
- **Relaciones no lineales complejas:** Correlaciones clásicas pueden no detectar relaciones no lineales o con patrones complejos.
- **Outliers:** Pueden afectar significativamente los coeficientes, especialmente Pearson.

> **Nota**
>
> **Reflexión.** Siempre complementa el análisis de correlación con visualizaciones y conocimiento del dominio para interpretar resultados correctamente.

Aplicaciones prácticas en análisis de datos

En la industria, estas medidas de correlación son herramientas esenciales para:

- **Análisis exploratorio:** Identificar relaciones relevantes entre variables antes de modelar.
- **Selección de variables:** Detectar variables redundantes o altamente correlacionadas que pueden afectar modelos predictivos.
- **Validación de supuestos:** Confirmar linealidad o monotonicidad para elegir técnicas estadísticas o de machine learning adecuadas.

Por ejemplo, un analista de datos en finanzas puede usar Pearson para evaluar la relación entre indicadores económicos y rendimiento bursátil, mientras que Spearman puede ser útil para variables ordinales como clasificaciones de riesgo.

Este entendimiento prepara el terreno para el siguiente módulo, donde abordaremos cómo estas correlaciones se relacionan con la multicolinealidad y pruebas estadísticas avanzadas, fundamentales para construir modelos robustos y confiables.

PRE-ENTREGA DEL PROYECTO

Actividad evaluable · parte del proyecto integrador

Pre-entrega: Análisis Visual de Distribuciones Categóricas y Outliers

En esta actividad, aplicarás técnicas de ingeniería de datos para caracterizar variables en un dataset real. Tu objetivo es identificar la forma de las distribuciones, calcular estadísticos de tendencia central y dispersión, y detectar valores atípicos (outliers) mediante visualizaciones y métricas.

Cómo resolver esta entrega

Pasos sugeridos:

1. **Preparación del entorno**: Crea un repositorio en GitHub e inicializa un Notebook de Jupyter (`.ipynb`).
2. **Carga y Limpieza**: Importa un dataset de tu elección (o usa el dataset de 'Viviendas' o 'Propinas' de Seaborn). Asegúrate de que contenga variables categóricas y numéricas.
3. **Análisis de Distribución**: * Genera gráficos de barras para las variables categóricas principales. * Calcula y describe la **asimetría** (skewness) y la **curtosis** de las variables numéricas clave para entender su forma.
4. **Detección de Outliers**: * Utiliza diagramas de caja (Boxplots) y diagramas de violín (Violin Plots) para visualizar la distribución y la presencia de valores extremos. * Implementa la técnica del Rango Intercuartílico (IQR) para filtrar y listar aquellos registros considerados outliers.
5. **Documentación**: En el README del repositorio, explica brevemente qué decisiones tomarías con los outliers encontrados (¿eliminarlos?, ¿transformarlos?) basándote en el análisis visual.

Errores comunes:

- Confundir la asimetría positiva (cola a la derecha) con la negativa.
- No documentar el origen de los datos en el repositorio.

Objetivos de aprendizaje

- Calcular e interpretar métricas de asimetría y curtosis en un conjunto de datos.
- Implementar visualizaciones efectivas para identificar valores atípicos mediante Boxplots y Violin Plots.
- Aplicar el método del Rango Intercuartílico (IQR) para la detección estadística de outliers.
- Justificar decisiones de preprocesamiento de datos basadas en la forma de la distribución.

Criterios de aceptación

- El repositorio contiene un archivo `.ipynb` ejecutable y un `README.md` descriptivo.

- Se incluyen al menos tres visualizaciones (Barplot, Boxplot y Violin Plot) generadas con Python (Pandas/Seaborn/Matplotlib).
- Se calculan explícitamente los valores de asimetría (Skewness) y curtosis para al menos una variable numérica.
- Se identifican y segmentan los outliers utilizando la técnica de IQR documentada en el código.
- El código sigue buenas prácticas de nombrado de variables y comentarios explicativos.

Rúbrica de evaluación — Análisis visual de distribuciones categóricas y outliers

Rúbrica para evaluar un repositorio de GitHub con un Notebook de Jupyter y documentación asociada, donde se analizan variables categóricas y numéricas mediante visualizaciones, estadísticos de forma y detección de outliers con IQR. La evaluación considera la calidad técnica del análisis, la interpretación de los resultados, la organización del repositorio y la documentación de decisiones.

Criterio	Descripción	Peso
Preparación, estructura del repositorio y reproducibilidad	Evalúa que el entregable esté organizado como un repositorio real, con Notebook ejecutable, archivos claramente identificados y una estructura que permita comprender y reproducir el análisis.	20%
Carga, limpieza y selección del dataset	Evalúa la importación correcta de un dataset adecuado, la presencia de variables categóricas y numéricas, y el tratamiento básico necesario para poder realizar el análisis solicitado.	15%
Visualización de variables categóricas y numéricas	Evalúa la selección y construcción de gráficos de barras, boxplots y violin plots para explorar distribuciones, detectar patrones y apoyar la interpretación del análisis.	25%
Cálculo e interpretación de asimetría y curtosis	Evalúa la correcta obtención de skewness y kurtosis en variables numéricas, así como la interpretación de su significado en términos de forma de la distribución.	15%
Detección de outliers con IQR	Evalúa la aplicación correcta del rango intercuartílico para identificar valores atípicos, la claridad al listar los registros detectados y la coherencia entre la detección numérica y la evidencia visual.	20%
Documentación, interpretación y decisiones sobre outliers	Evalúa la calidad del README y de las explicaciones del notebook respecto a los hallazgos, la decisión sobre eliminar o transformar outliers, y la comunicación de las conclusiones del análisis.	5%

Total: 100 pts | Aprobación: 70 pts

MÓDULO 16

Teoría de correlación y asociación

En esta unidad, exploraremos las principales técnicas para medir la relación entre variables numéricas: el coeficiente de Pearson, y las correlaciones no paramétricas de Spearman y Kendall. Aprenderás no solo a calcularlas, sino a interpretar sus resultados en contextos reales, entendiendo sus supuestos y limitaciones.

Teoría de correlación y asociación

¿Qué nos dice la correlación sobre nuestras variables?

Imagina que trabajas con un conjunto de datos financieros y quieres entender cómo se relacionan dos indicadores clave, como el ingreso y el gasto. ¿Cómo cuantificas esa relación? Aquí es donde entran las medidas de correlación y asociación, herramientas fundamentales para cualquier analista o científico de datos.

En esta unidad, exploraremos las principales técnicas para medir la relación entre variables numéricas: el coeficiente de Pearson, y las correlaciones no paramétricas de Spearman y Kendall. Aprenderás no solo a calcularlas, sino a interpretar sus resultados en contextos reales, entendiendo sus supuestos y limitaciones.

Este conocimiento es esencial para avanzar en análisis más complejos, como la detección de multicolinealidad y la selección de variables para modelos predictivos. Prepárate para dominar estas herramientas clave que potenciarán tu análisis exploratorio y la calidad de tus modelos.

Correlación y asociación entre variables

La **correlación** es una medida estadística que cuantifica la relación entre dos variables. En términos simples, nos indica si y cómo dos variables cambian juntas. Existen diferentes tipos de correlación, que capturan distintos tipos de relaciones:

- **Correlación lineal:** mide la fuerza y dirección de una relación lineal entre dos variables numéricas.
- **Correlación monotónica:** mide si una variable tiende a aumentar o disminuir cuando la otra lo hace, sin requerir que la relación sea lineal.

> **Nota**
> La correlación no implica causalidad; dos variables pueden estar correlacionadas sin que una cause a la otra.

Coeficiente de correlación de Pearson

El coeficiente de Pearson, denotado como `r`, mide la **relación lineal** entre dos variables numéricas continuas. Su valor oscila entre -1 y 1:

- `r = 1`: correlación lineal positiva perfecta
- `r = -1`: correlación lineal negativa perfecta
- `r = 0`: no hay correlación lineal

Supuestos clave para usar Pearson:

- Las variables deben ser numéricas y continuas.
- La relación entre variables debe ser lineal.
- Las variables deben seguir una distribución aproximadamente normal.
- Homocedasticidad: la variabilidad de una variable es constante a lo largo de la otra.

Ejemplo práctico:

En análisis financiero, Pearson puede medir la relación lineal entre el ingreso mensual y el gasto mensual de clientes para detectar patrones de consumo.

Correlaciones no paramétricas: Spearman y Kendall

Cuando los supuestos de Pearson no se cumplen, especialmente la linealidad o normalidad, recurrimos a correlaciones basadas en rangos:

- **Spearman (`ρ`):** mide la correlación monotónica usando rangos de los datos. Es menos sensible a valores extremos y no requiere normalidad.
- **Kendall (`τ`):** también mide la asociación monotónica, basado en concordancia y discordancia de pares de observaciones. Es robusto y útil con muestras pequeñas.

Ambos coeficientes varían entre -1 y 1, interpretándose similar a Pearson pero para relaciones monotónicas, no necesariamente lineales.

Ejemplo práctico:

En estudios de satisfacción del cliente, donde las variables son ordinales o no lineales, Spearman y Kendall permiten evaluar asociaciones sin asumir distribuciones específicas.

Limitaciones y consideraciones

- **Correlación no implica causalidad:** Una alta correlación no garantiza que una variable cause cambios en la otra.

- **Relaciones no lineales complejas:** Correlaciones clásicas pueden no detectar relaciones no lineales o con patrones complejos.
- **Outliers:** Pueden afectar significativamente los coeficientes, especialmente Pearson.

> **Nota**
>
> **Reflexión.** Siempre complementa el análisis de correlación con visualizaciones y conocimiento del dominio para interpretar resultados correctamente.

Aplicaciones prácticas en análisis de datos

En la industria, estas medidas de correlación son herramientas esenciales para:

- **Análisis exploratorio:** Identificar relaciones relevantes entre variables antes de modelar.
- **Selección de variables:** Detectar variables redundantes o altamente correlacionadas que pueden afectar modelos predictivos.
- **Validación de supuestos:** Confirmar linealidad o monotonicidad para elegir técnicas estadísticas o de machine learning adecuadas.

Por ejemplo, un analista de datos en finanzas puede usar Pearson para evaluar la relación entre indicadores económicos y rendimiento bursátil, mientras que Spearman puede ser útil para variables ordinales como clasificaciones de riesgo.

Este entendimiento prepara el terreno para el siguiente módulo, donde abordaremos cómo estas correlaciones se relacionan con la multicolinealidad y pruebas estadísticas avanzadas, fundamentales para construir modelos robustos y confiables.

Introducción a multicolinealidad y sus efectos

¿Por qué la multicolinealidad importa en modelos predictivos?

Imagina que estás construyendo un modelo de regresión para predecir ventas basándote en múltiples variables, como publicidad en diferentes medios, precios y promociones. ¿Qué sucede si algunas de estas variables están altamente correlacionadas entre sí? Esta situación, conocida como **multicolinealidad**, puede complicar la interpretación de tu modelo y afectar su estabilidad.

En esta unidad, exploraremos qué es la multicolinealidad, cómo detectarla y por qué es crucial para asegurar que tus modelos sean confiables y sus resultados interpretables. Al finalizar, podrás identificar señales de alerta y entenderás la métrica básica para cuantificarla: el Factor de Inflación de la Varianza (VIF).

¿Qué es la multicolinealidad?

La **multicolinealidad** ocurre cuando dos o más variables predictoras en un modelo de regresión están altamente correlacionadas, es decir, una puede predecirse a partir de las otras con un alto grado de precisión. Esto genera problemas porque:

- Dificulta estimar el efecto individual de cada variable sobre la variable objetivo.
- Incrementa la varianza de los coeficientes estimados, haciendo que sean inestables y sensibles a pequeñas variaciones en los datos.

Impacto en modelos de regresión

Cuando existe multicolinealidad:

- Los coeficientes pueden cambiar de signo o magnitud inesperadamente.
- Los intervalos de confianza se amplían, reduciendo la significancia estadística.
- El modelo puede tener buen ajuste global pero pobre capacidad para interpretar variables individuales.

> **Nota**
> La multicolinealidad no afecta la capacidad predictiva del modelo per se, pero sí su interpretabilidad y confiabilidad en inferencias.

Señales de alerta

- Coeficientes con signos contrarios a lo esperado.
- Cambios drásticos en coeficientes al agregar o quitar variables.
- Altas correlaciones entre variables predictoras (por ejemplo, coeficiente de correlación > 0.8).

Métrica para cuantificar: Factor de Inflación de la Varianza (VIF)

El **VIF** mide cuánto se incrementa la varianza de un coeficiente debido a la multicolinealidad. Se calcula para cada variable predictora como:

VIFi=11−Ri 2_ VIFi_=1−*R i* 21

Donde Ri 2_ R i_ 2es el coeficiente de determinación al regredir la variable i_ i_ contra todas las demás variables predictoras.

- Un VIF cercano a 1 indica baja multicolinealidad.
- Valores mayores a 5 o 10 suelen considerarse indicativos de multicolinealidad problemática.

Ejemplo conceptual

Supongamos que tienes dos variables: `Publicidad_TV` y `Publicidad_Radio` que están altamente correlacionadas porque ambas dependen del presupuesto total de marketing. Al incluir ambas en el modelo, los coeficientes pueden volverse inestables y difíciles de interpretar.

Con este conocimiento, estás listo para identificar y cuantificar multicolinealidad en tus datos, un paso clave antes de avanzar hacia la selección y transformación de variables para modelado.

Aplicación práctica y relevancia en análisis de datos

En la práctica, detectar multicolinealidad es fundamental para construir modelos de regresión robustos y confiables, especialmente en contextos de análisis de datos y ciencia de datos donde las variables predictoras pueden estar relacionadas.

Por ejemplo, en análisis de ventas, variables como `Precio`, `Descuento` y `Promoción` pueden estar correlacionadas. Ignorar la multicolinealidad puede llevar a conclusiones erróneas sobre qué factores realmente impactan las ventas.

Además, la multicolinealidad afecta la selección de variables y las transformaciones que aplicarás en etapas posteriores del pipeline de Machine Learning, impactando la reproducibilidad y estabilidad del modelo.

Este conocimiento conecta directamente con los próximos temas del módulo, donde aprenderás a calcular VIF en Python, detectar multicolinealidad en datasets reales y aplicar estrategias para mitigarla, como la selección o transformación de variables.

PRE-ENTREGA DEL PROYECTO

Actividad evaluable · parte del proyecto integrador

Pre-entrega: Análisis de Correlación, comportamiento del Consumidor

El propósito de este entregable es aplicar las técnicas de correlación de Pearson, Spearman y Kendall para identificar relaciones entre variables en un conjunto de datos transaccional simulado. Deberás determinar qué coeficiente es el más adecuado según la distribución y naturaleza de los datos.

Cómo resolver esta entrega

Pasos sugeridos:

1. **Preparación del Entorno**: Crea un Jupyter Notebook (.ipynb) donde importarás las librerías de análisis de datos (como Pandas y Scipy/Numpy).
2. **Análisis de Supuestos**: Antes de calcular, realiza un breve análisis visual (histogramas o tests de normalidad) para decidir si una variable sigue una distribución normal. Esto es clave para elegir entre Pearson (paramétrico) o Spearman/Kendall (no paramétricos).
3. **Cálculo de Coeficientes**: Calcula e interpreta la matriz de correlación utilizando los tres métodos estudiados para comparar cómo varían los resultados ante valores atípicos (outliers) o relaciones no lineales.
4. **Documentación de Hallazgos**: Dentro del mismo notebook, utiliza celdas de Markdown para explicar por qué una correlación fuerte no implica causalidad en el contexto del negocio y justifica cuál de los tres métodos ofrece la visión más precisa de los datos analizados.

Errores comunes:

- Calcular Pearson sobre variables con distribuciones muy sesgadas o con muchos valores atípicos sin justificarlo.
- Confundir una correlación alta con una relación de causa-efecto.
- No incluir la interpretación cualitativa de los coeficientes obtenidos (ej. ¿qué significa un 0.85 en términos de negocio?).

Objetivos de aprendizaje

- Identificar la pertinencia de los coeficientes de Pearson, Spearman y Kendall según la distribución de los datos.
- Calcular matrices de correlación para evaluar la fuerza y dirección de las relaciones entre variables numéricas.
- Interpretar los resultados de correlación evitando la falacia de causalidad.

- Justificar la elección de un método estadístico basado en los supuestos de normalidad y presencia de outliers.

Criterios de aceptación

- El archivo entregado debe ser un .ipynb ejecutable con las celdas de código y resultados visibles.
- Se deben calcular explícitamente los tres coeficientes (Pearson, Spearman y Kendall) para al menos un par de variables.
- El notebook debe incluir una sección de conclusiones donde se explique cuál método es el más robusto para el dataset dado.
- Se debe incluir una visualización (como un heatmap o scatter plot) que apoye la interpretación de la correlación.

Rúbrica de evaluación: Análisis de correlación del comportamiento del consumidor

Esta rúbrica evalúa un notebook de Jupyter en el que se analizan relaciones entre variables transaccionales mediante los coeficientes de Pearson, Spearman y Kendall. Se valoran la preparación del análisis, la elección del método más adecuado según los supuestos de los datos, la interpretación estadística y de negocio, y la claridad de la documentación en Markdown. La rúbrica está diseñada para guiar al estudiante en la construcción de un análisis correcto, justificando decisiones y evitando errores comunes como usar Pearson sin considerar distribución, confundir correlación con causalidad o limitarse a reportar números sin interpretación.

Criterio	Descripción	Peso
Preparación del entorno y estructura del notebook	Evalúa si el archivo .ipynb está correctamente organizado, si importa las librerías necesarias y si presenta una estructura clara que permita seguir el análisis de principio a fin.	13.00000000000002%
Análisis de supuestos y justificación del coeficiente	Evalúa si el estudiante revisa la distribución de las variables o evidencia de normalidad/sesgo y usa esa información para justificar cuándo Pearson es apropiado y cuándo conviene Spearman o Kendall.	24%
Cálculo correcto de correlaciones y comparación entre métodos	Evalúa si el notebook calcula adecuadamente los coeficientes de Pearson, Spearman y Kendall, presenta resultados comparables y reconoce cómo pueden variar ante outliers o relaciones no lineales.	28.999999999999996%
Interpretación estadística y sentido de negocio	Evalúa si el estudiante interpreta los coeficientes de manera cualitativa y contextualizada, explicando qué significa una correlación alta, moderada o baja en términos del comportamiento del consumidor.	19%
		10%

Criterio	Descripción	Peso
Discusión sobre causalidad y limitaciones	Evalúa si el notebook explica correctamente que correlación no implica causalidad y si reconoce límites del análisis correlacional en el contexto del negocio.	
Comunicación, claridad y uso de Markdown	Evalúa la calidad de la redacción, el uso de celdas Markdown para explicar el proceso, y la claridad general del notebook como documento analítico.	5%

Total: 100 pts | Aprobación: 70 pts

MÓDULO 17

Proyecto Final: Pipeline Completo de Ciencia de Datos

Consolidar el aprendizaje entregando un proyecto completo de ciencia de datos que cubra el flujo extremo-a-extremo.

Descripción del entregable final

El entregable final consiste en un **Repositorio de GitHub** (o equivalente) que contiene el código fuente completo del proyecto integrador. Este repositorio no es solo una carpeta con archivos; es el "gemelo digital" de tu trabajo como científico de datos.

Debe ser un producto **autoexplicativo**. Cualquier persona (un reclutador, un colega o tu tutor) debería poder clonar el repositorio, seguir las instrucciones del `README.md` y ejecutar el pipeline completo para ver los resultados. La integración debe ser tal que la salida del script de ingesta alimente al pipeline, este guarde los datos en la base de datos, y el código de ML consuma esos datos para generar la predicción final.

Errores comunes a evitar al cerrar el entregable final

- **Rutas absolutas:** Usar `C:\Users\MiUsuario\...` en lugar de rutas relativas. Esto romperá el código en cualquier otra computadora.
- **Falta de "limpieza final":** Dejar celdas de notebooks vacías, errores en comentarios o código "muerto" que no se usa.
- **No verificar el README:** No probar si las instrucciones que escribiste realmente funcionan desde cero.
- **Hardcoding de credenciales:** Dejar el token de la API escrito directamente en el código.

Cierre del programa

Este proyecto final es el testimonio del camino que has recorrido. Empezaste manejando estructuras básicas de Python y hoy eres capaz de diseñar una arquitectura de datos moderna, procesar volúmenes importantes de información con herramientas de alto rendimiento y entrenar modelos de inteligencia artificial validados estadísticamente.

Este no es el fin, sino tu base para el siguiente paso: ya sea un rol profesional como Data Scientist o la especialización en ingeniería de datos o aprendizaje profundo. ¡Es hora de consolidar tu trabajo y mostrar de lo que eres capaz!

ENTREGA FINAL DEL PROYECTO

Actividad evaluable · proyecto integrador final

Entrega final: repositorio consolidado del pipeline de ciencia de datos

Cómo resolver esta entrega

Objetivo

Consolidar el "Proyecto Integrador" en un repositorio profesional que demuestre el dominio de las herramientas vistas en el programa (Python 3.12, DuckDB, Polars, SQLAlchemy, Scikit-Learn).

Instrucciones Paso a Paso

1. **Preparación del Repositorio:** * Crea un nuevo repositorio en tu plataforma preferida. * Define una estructura de carpetas clara (ej: `src/` para código, `docs/` para documentación, `config/` para variables de entorno).

2. **Integración del Pipeline de Datos:** * Incluye el script de **Ingesta** (Módulo 4/5) que consume la API elegida. Asegúrate de que use `.env` para las llaves. * Integra el motor de procesamiento con **Polars y DuckDB** (Módulo 6/7/8). Los datos ingestados deben fluir hacia este pipeline para ser normalizados. * Asegura la persistencia usando **SQLAlchemy 2.0** (Módulo 9) para guardar los datos procesados en SQLite o una base de datos local.

3. **Finalización del ML Lab:** * Incorpora el código del **Pipeline de Scikit-Learn** desarrollado en las pre-entregas anteriores (Módulo 10/11). * El modelo debe leer los datos directamente de la base de datos creada en el paso anterior. * Asegúrate de incluir la optimización de hiperparámetros (`GridSearchCV`) y la exportación de métricas finales.

4. **Documentación Profesional (README.md):** * **Título y Descripción:** ¿Qué problema resuelve tu proyecto? * **Instalación:** Cómo crear el entorno virtual y ejecutar `pip install -r requirements.txt` . * **Uso:** Cómo ejecutar cada parte del flujo (ej: `python ingesta.py` -> `python train.py`). * **Resultados:** Breve tabla o resumen de las métricas obtenidas por el modelo.

5. **Validación Final:** * Borra tu base de datos local y vuelve a ejecutar todo el flujo desde cero siguiendo tus propias instrucciones de README. Si todo funciona sin errores, estás listo para entregar.

Criterios de Evaluación

- **Completitud:** ¿Están presentes todas las etapas (Ingesta, ETL, DB, ML)?
- **Calidad Técnica:** ¿Se manejan errores? ¿El código es legible?
- **Portabilidad:** ¿Incluye `requirements.txt` y `.env.example` ?

- **Métricas de ML:** ¿El modelo está bien evaluado y no tiene overfitting evidente?

Objetivos de aprendizaje

- Consolidar todas las fases de un proyecto de Data Science en un repositorio de código coherente y funcional.
- Integrar scripts de ingesta (Requests), procesamiento (Polars/DuckDB) y almacenamiento (SQLAlchemy) en un flujo continuo.
- Documentar técnica y funcionalmente un proyecto de Machine Learning para garantizar su reproducibilidad por terceros.
- Evaluar de forma integral el desempeño de un pipeline de datos y un modelo predictivo final.

Rúbrica de evaluación — Entrega final: repositorio consolidado del pipeline de ciencia de datos

Esta rúbrica evalúa la entrega final de un repositorio profesional que integre, de forma coherente y ejecutable, el flujo extremo a extremo de un proyecto de ciencia de datos: ingesta desde API, transformación con Polars y DuckDB, persistencia con SQLAlchemy, entrenamiento y evaluación de un modelo con Scikit-Learn, y documentación clara para reproducir el proyecto desde cero. El foco está en la consolidación final del trabajo previo, la calidad técnica del repositorio y la capacidad de otra persona para ejecutar y comprender el pipeline sin depender de explicaciones externas.

Criterio	Descripción	Peso
Integración completa del pipeline extremo a extremo	Evalúa si el repositorio reúne todas las etapas solicitadas y si estas se conectan correctamente: ingesta desde API, procesamiento/normalización, persistencia en base de datos y consumo de datos por el modelo de ML. Se valora especialmente que el flujo sea coherente, que los componentes se relacionen entre sí y que el proyecto represente una solución integrada, no solo archivos aislados.	35%
Calidad técnica, estructura y robustez del código	Evalúa la claridad, mantenibilidad y confiabilidad técnica del repositorio. Se considera la organización de carpetas, la legibilidad del código, el uso de nombres comprensibles, el manejo de errores, la separación entre lógica de negocio y ejecución, y la presencia de decisiones técnicas que faciliten el mantenimiento del proyecto.	22%
Persistencia, portabilidad y configuración del proyecto	Evalúa si el proyecto puede ejecutarse en otro entorno con la menor fricción posible. Incluye la presencia y uso correcto de requirements.txt, .env.example, la gestión segura de credenciales, y la capacidad del repositorio para ser clonado y ejecutado siguiendo instrucciones razonables sin depender de configuraciones ocultas o locales.	18%
Documentación profesional y re-	Evalúa la calidad del README.md y la capacidad del repositorio para guiar a una persona externa en su instalación, ejecución y comprensión. Se considera si la documentación explica el problema, la instalación, el uso paso a paso, los resultados y la validación final del flujo.	15%

Criterio	Descripción	Peso
producibilidad		
Evaluación del modelo y uso adecuado de métricas	Evalúa la calidad de la validación del modelo de Scikit-Learn: uso de GridSearchCV, reporte de métricas finales, interpretación básica del desempeño y evidencia de que el modelo fue evaluado de forma seria. Se valora que las métricas estén alineadas con el problema y que no haya señales evidentes de sobreajuste sin explicación.	10%

Total: 100 pts | Aprobación: 70 pts

Material complementario

Palabras clave

- **Pipeline de datos:** Es un conjunto de procesos automatizados que mueven datos desde una fuente original hacia un destino donde serán analizados. Funciona como una tubería donde los datos se recolectan, limpian y transforman automáticamente; por ejemplo, extraer ventas diarias de una aplicación para guardarlas en una base de datos centralizada.
- **SQL (Structured Query Language):** Es el lenguaje estándar utilizado para comunicarse con bases de datos relacionales y gestionar la información almacenada en ellas. Permite realizar acciones como buscar registros específicos, sumar ventas totales o modificar la estructura de las tablas, funcionando como un puente para conversar con tus datos.
- **API (Application Programming Interface):** Es un mecanismo que permite que dos programas distintos intercambien información de manera segura y estandarizada. Por ejemplo, cuando una aplicación web obtiene el pronóstico del tiempo de un servidor meteorológico, utiliza una API para solicitar esos datos sin necesidad de acceder a la base de datos interna de la otra empresa.
- **Machine Learning (Aprendizaje Automático):** Es una rama de la inteligencia artificial que entrena a las computadoras para identificar patrones en los datos y realizar predicciones o decisiones sin haber sido programadas explícitamente para cada caso. Un ejemplo común es el filtro de spam en tu correo electrónico, que aprende a distinguir mensajes no deseados basándose en miles de ejemplos previos.
- **DuckDB:** Es una base de datos diseñada específicamente para realizar análisis rápidos de grandes volúmenes de información en computadoras personales. A diferencia de bases de datos tradicionales, DuckDB está optimizada para procesar consultas complejas de manera instantánea, lo cual es ideal para científicos de datos que trabajan localmente con archivos pesados.
- **Feature Engineering (Ingeniería de Características):** Es el proceso de seleccionar, modificar o crear nuevas variables a partir de los datos crudos para ayudar a que los modelos de Machine Learning funcionen mejor. Por ejemplo, si tienes una fecha de nacimiento, transformarla en una variable de 'edad' suele ser más útil para que el modelo identifique patrones de comportamiento.

- **ETL (Extract, Transform, Load):** Es un patrón fundamental de trabajo en análisis de datos que consiste en extraer información de diversas fuentes, transformarla para que sea útil y cargarla en un destino final. Es lo que ocurre cuando descargas datos de una API, eliminas valores incompletos y los guardas en un formato limpio dentro de tu base de datos lista para entrenar modelos.
- **Normalización:** Es el proceso de organizar los datos en una base de datos para reducir la repetición innecesaria de información y evitar errores al realizar cambios. Un ejemplo es separar la información de 'los clientes' de la información de 'los pedidos' en tablas distintas para que, si un cliente cambia su dirección, solo necesites actualizar un único registro en lugar de miles de filas de pedidos.
- **Algoritmo de clasificación:** Es un tipo de modelo supervisado de Machine Learning que se utiliza para asignar las observaciones de entrada a una categoría predefinida. Por ejemplo, un modelo de clasificación puede analizar transacciones bancarias para etiquetarlas automáticamente como 'legítimas' o 'sospechosas de fraude'.
- **Validación Cruzada (Cross-Validation):** Es una técnica estadística para evaluar qué tan bien funcionará tu modelo de datos con información nueva que nunca ha visto. En lugar de evaluar el modelo una sola vez, divide los datos en múltiples subconjuntos para entrenar y probar el modelo varias veces, garantizando así que los resultados sean confiables y no fruto de la casualidad.

Análisis profundo

¿Por qué este tema importa?

Muchas personas empiezan en Data Science pensando que el trabajo empieza cuando se entrena un modelo. En realidad, el trabajo empieza mucho antes: cuando los datos están mal organizados, duplicados o sin relaciones claras. Un mal diseño de base de datos no solo ralentiza consultas; también genera errores silenciosos, métricas engañosas y decisiones incorrectas.

En otras palabras: si el esquema está mal, todo lo que construyas encima puede parecer correcto y aun así estar equivocado.

La idea central

Una base de datos relacional existe para representar el mundo de forma estructurada. No guarda “cosas sueltas”, sino entidades relacionadas entre sí. Por ejemplo, en una tienda online no conviene poner toda la información de clientes, pedidos, productos y pagos en una sola tabla gigante. Eso produce redundancia y problemas de actualización.

La mente principiante suele pensar: “una tabla con todo es más fácil”. Y al principio sí lo parece. Pero esa simplicidad se rompe en cuanto aparece la escala:

- un cliente puede hacer muchos pedidos,
- un pedido puede incluir muchos productos,
- un producto puede aparecer en muchos pedidos,
- y cada pago tiene su propio ciclo de vida.

Ahí es donde el modelado relacional aporta orden.

Cómo pensar el modelado paso a paso

1. Identificar entidades

Primero se pregunta: ¿qué cosas existen en el negocio?

Ejemplo en un e-commerce: - `clientes` - `pedidos` - `productos` - `pagos` - `envíos`

2. Definir atributos

Luego, cada entidad necesita atributos relevantes.

- `clientes`: nombre, email, fecha_alta
- `pedidos`: fecha, estado, total
- `productos`: nombre, categoría, precio

3. Encontrar relaciones

Las relaciones dicen cómo se conectan las entidades.

- un cliente **tiene muchos** pedidos
- un pedido **contiene muchos** productos
- un pedido **tiene un** pago principal

4. Elegir claves primarias y foráneas

La clave primaria identifica una fila de forma única. La clave foránea conecta tablas.

Esto parece un detalle técnico, pero es el mecanismo que permite responder preguntas complejas sin duplicar datos.

Una analogía útil

Imagina una biblioteca. No pondrías todos los libros, autores, préstamos y lectores en una sola ficha. Tendrías:

- una tabla de libros,
- otra de autores,
- otra de socios,
- otra de préstamos.

Cada ficha tiene su función. El sistema funciona porque las relaciones están bien definidas.

Normalización: menos duplicación, menos caos

La normalización busca evitar repetir la misma información muchas veces.

Si guardas el nombre del cliente en cada pedido, cuando cambie su nombre tendrás que actualizar miles de filas. Si olvidas una, tu análisis quedará inconsistente.

La normalización básica ayuda a: - reducir redundancia, - evitar anomalías de actualización, - proteger la integridad de los datos.

Pero ojo: normalizar no significa fragmentar todo sin criterio. En analítica a veces se desnormaliza deliberadamente para acelerar consultas. El punto no es obedecer reglas ciegamente, sino entender el equilibrio entre consistencia y rendimiento.

Qué hacen las empresas en la práctica

- **Amazon** y otros gigantes del comercio usan modelos relacionales y distribuidos para mantener integridad en pedidos, inventario y pagos.
- **Stripe** y plataformas financieras dependen de esquemas muy bien definidos porque la trazabilidad es crítica.
- Equipos de analítica en empresas como **Spotify** o **Airbnb** suelen modelar capas de datos para separar datos crudos, curados y analíticos.

En todos estos casos, el modelado no es un ejercicio académico: es una defensa contra errores costosos.

Errores comunes de estudiantes

"Si funciona en SQLite, ya está listo"

SQLite es excelente para aprender y prototipar, pero eso no significa que el esquema esté bien pensado para producción.

"Una tabla grande evita joins y por eso es mejor"

Evitar joins puede parecer más simple, pero a menudo significa esconder un diseño pobre. Los joins son una herramienta, no un problema.

"Normalización y rendimiento siempre chocan"

No necesariamente. El error es pensar en extremos. Un buen diseño primero prioriza integridad y luego se adapta al caso de uso analítico.

Síntesis

Entender arquitectura relacional, modelado y normalización es aprender a pensar como diseñador de sistemas de datos, no solo como consumidor de tablas. Este tema conecta todo el programa: desde SQL y SQLite hasta ETL, DuckDB, SQLAlchemy y la preparación de datos para ML. Si dominas esta base, tus pipelines serán más confiables y tus modelos partirán de datos mucho más sólidos.

¿Por qué este tema importa?

En ciencia de datos solemos preocuparnos por el resultado final: dashboards, features, métricas y modelos. Pero si una operación falla a mitad de camino y deja los datos a medias, todo el análisis posterior puede quedar contaminado.

Las transacciones existen para evitar precisamente eso. Sin ellas, una carga de datos puede quedar incompleta, una actualización puede duplicar registros o una eliminación puede dejar referencias rotas. El problema no siempre se nota de inmediato; muchas veces aparece semanas después, cuando un modelo aprende con datos inconsistentes.

La idea central

Una transacción agrupa varias operaciones como una sola unidad lógica. O se ejecutan todas, o no se aplica ninguna.

Esto es esencial en casos como: - registrar una venta y descontar inventario, - insertar un pedido y sus ítems, - actualizar estados de una pipeline de ingesta, - mover datos entre tablas sin perder trazabilidad.

ACID explicado de forma intuitiva

Atomicidad

Todo o nada. Si una parte falla, se revierte todo.

Consistencia

La base de datos debe seguir reglas válidas antes y después de la transacción.

Aislamiento

Varias transacciones simultáneas no deben pisarse de forma peligrosa.

Durabilidad

Si la transacción se confirma, los cambios sobreviven a fallos posteriores.

Analogía práctica

Piensa en una transferencia bancaria. Si tu cuenta se debita pero la del destinatario nunca se acredita, el sistema es incorrecto. La transacción asegura que ambas operaciones ocurran como una unidad segura.

Ese mismo principio se aplica en pipelines de datos. Por ejemplo:

1. lees datos desde una API,
2. los limpias,
3. los insertas en una tabla,

4. marcas el lote como procesado.

Si falla el paso 3, no deberías marcar el lote como procesado. Si lo haces, la próxima corrida no intentará recuperarlo y perderás datos.

Qué pasa cuando se entiende mal

Muchos estudiantes creen que "transacción" significa solo hacer `commit` al final. Pero el verdadero valor está en diseñar el proceso para que los errores sean recuperables y no rompan la consistencia.

También confunden "base de datos segura" con "base de datos infalible". ACID no elimina los errores humanos ni las malas decisiones de diseño: solo ofrece una garantía formal sobre el comportamiento de las operaciones.

Ejemplos reales

- En **banca** y **fintech**, ACID no es opcional.
- En plataformas de comercio como **Mercado Libre** o **Shopify**, las transacciones protegen inventario y pagos.
- En sistemas de datos empresariales, herramientas como **PostgreSQL**, **SQLite** y ciertos flujos sobre **SQLAlchemy** usan transacciones para evitar cargas incompletas.

Relación con Data Science

Aunque a veces se piense que esto es "tema de backend", en realidad afecta directamente a Data Science:

- una tabla con duplicados por cargas fallidas altera tasas de conversión,
- un join roto por inconsistencias altera el entrenamiento,
- una feature calculada sobre datos parcialmente cargados produce fugas o sesgos.

Errores comunes

"Si el script terminó sin error, los datos están bien"

No siempre. Puede haber errores lógicos que no levantan excepción.

"Las transacciones son solo para bancos"

No. Son esenciales para cualquier proceso donde una secuencia de pasos deba mantenerse coherente.

"Commit temprano hace el proceso más seguro"

En realidad, puede hacer el proceso más frágil si confirmas cambios antes de validar todo el lote.

Síntesis

Las transacciones y el modelo ACID son el cinturón de seguridad de los sistemas de datos. En este programa, conectan bases de datos, ETL, automatización y persistencia confiable. Entenderlas te ayuda a construir pipelines que no solo funcionan, sino que también resisten fallos del mundo real.

¿Por qué este tema importa?

Un pipeline de datos es mucho más que un script que “lee, transforma y guarda”. En la práctica, es un sistema vivo que debe resistir fallas de red, cambios de esquema, datos incompletos, archivos enormes y resultados inesperados.

Cuando un pipeline está mal diseñado, el problema no es solo técnico: puede bloquear informes, romper entrenamientos de ML o introducir sesgos difíciles de detectar.

La idea central

Un pipeline moderno suele tener etapas como:

1. adquisición,
2. validación,
3. limpieza,
4. transformación,
5. persistencia,
6. monitoreo,
7. recuperación ante errores.

Cada etapa debe poder responder dos preguntas: - ¿qué espera recibir? - ¿qué hace si algo sale mal?

Eso lo diferencia de un script improvisado.

Una analogía útil: la línea de producción

Imagina una fábrica de alimentos. No basta con poner ingredientes en una máquina y esperar un producto final. Hay controles de calidad en cada punto:

- si la materia prima está dañada, se rechaza,
- si una máquina falla, la línea se detiene,
- si el envase no cumple normas, no se despacha.

Un pipeline de datos funciona igual. Cada etapa debería tener validaciones claras para impedir que datos corruptos lleguen al modelo.

Diseño por capas

Capa de ingestión

Aquí entran APIs, archivos CSV, bases de datos o streams. El objetivo no es “procesar” todavía, sino capturar de forma confiable.

Capa de staging

Sirve para guardar datos crudos o semiprocesados. Si el dato falla más adelante, puedes volver a este punto.

Capa de transformación

Aquí limpias tipos, nulos, categorías, outliers y unificas estructuras.

Capa analítica o de features

En esta capa se construyen variables listas para exploración o modelado.

Qué hace robusto a un pipeline

- validaciones de esquema,
- logs claros,
- manejo explícito de errores,
- reintentos con control,
- idempotencia,
- pruebas sobre datos de ejemplo,
- separación entre lógica de negocio y lógica técnica.

El concepto de idempotencia

Este punto es crucial y suele causar confusión. Un proceso es idempotente si repetirlo no produce efectos inesperados.

Ejemplo: si vuelves a correr una carga diaria, no deberías duplicar registros. En vez de insertar sin control, podrías hacer upsert, deduplicación o reemplazo por partición.

Esto es muy importante en producción, donde los procesos pueden reintentarse automáticamente.

Ejemplos reales

- **Airbnb** y **Uber** operan con ecosistemas donde ingesta, validación y orquestación deben ser muy confiables.
- Empresas de analítica usan herramientas como **Airflow**, **Dagster** o **Prefect** para coordinar tareas.
- En entornos locales y de aprendizaje, **DuckDB** y **Polars** permiten diseñar pipelines potentes sin necesidad de infraestructura pesada.

Errores comunes

“Pipeline” es sinónimo de notebook

Un notebook es una herramienta de exploración. Un pipeline es una secuencia estructurada y validable de tareas.

“Si el output se ve bien, el pipeline está bien”

No. Puede haber errores silenciosos en tipos, duplicados o joins.

“Validar datos ralentiza demasiado”

Validar temprano evita corregir desastres después. En la práctica, ahorrarás tiempo.

Síntesis

Pensar en pipelines es pensar en confiabilidad, no solo en transformación. Este tema conecta APIs, SQL, Polars, DuckDB, automatización y ML porque todos dependen de una premisa: los datos deben llegar correctos, repetibles y auditables. Sin esta base, la ciencia de datos pierde solidez.

¿Por qué este tema importa?

Durante años, muchas tareas de análisis en Python se resolvían con Pandas por costumbre. Pero cuando los datos crecen o las transformaciones se vuelven complejas, el costo de usar herramientas no optimizadas se vuelve evidente: más memoria, más tiempo y más frustración.

DuckDB y Polars representan una evolución importante para analítica moderna: permiten trabajar con grandes volúmenes de datos de forma rápida, expresiva y muy cercana a las necesidades reales de un analista o data scientist.

La idea central

- **DuckDB** es un motor SQL analítico embebido, muy eficiente para consultar archivos grandes y ejecutar joins, agregaciones y filtros sin montar una base de datos externa.
- **Polars** es una librería de manipulación de datos en memoria, diseñada para velocidad, paralelismo y menor consumo.

Juntos forman una combinación poderosa: SQL para consultar y organizar, Polars para transformar y preparar features.

Analogía: cocina industrial vs cocina doméstica

Pandas es excelente para muchas tareas, como una cocina doméstica bien equipada. Pero si debes preparar cientos de platos con rapidez y consistencia, necesitas una cocina industrial con mejores flujos.

DuckDB y Polars ofrecen precisamente eso: menos fricción y mayor rendimiento.

Por qué DuckDB cambia la forma de pensar

Muchos estudiantes asumen que SQL solo sirve dentro de una base de datos tradicional. DuckDB rompe esa idea: puedes hacer consultas analíticas directamente sobre CSV, Parquet y otras fuentes locales.

Eso significa que puedes: - explorar archivos grandes sin cargarlos completamente a memoria, - hacer joins complejos, - aplicar agregaciones avanzadas, - integrar SQL con Python de forma muy natural.

Por qué Polars importa tanto

Polars introduce una forma distinta de trabajar:

- operaciones más declarativas,
- ejecución más eficiente,
- soporte excelente para lazy execution,
- mejor rendimiento en transformaciones complejas.

Esto resulta especialmente útil cuando las transformaciones se encadenan y no quieres ejecutar cálculos innecesarios.

Qué suele confundirse

"Pandas está obsoleto"

No. Pandas sigue siendo muy útil y dominante en muchos entornos. La clave es saber cuándo se queda corto.

"DuckDB reemplaza completamente a SQL o a las bases de datos"

No. Es un motor analítico local muy potente, no un sustituto universal de todos los sistemas de almacenamiento.

"Polars solo sirve para velocidad"

La velocidad es importante, pero también lo es la claridad conceptual y la posibilidad de estructurar mejor el flujo.

Ejemplos de uso en la práctica

- Analistas que trabajan con archivos de eventos exportados desde plataformas como **BigQuery**, **Snowflake** o logs de producto.
- Equipos que hacen prototipos locales antes de pasar a infraestructura mayor.
- Proyectos de ciencia de datos donde se necesita validar rápidamente features sobre datasets medianos o grandes.

Cómo se conectan entre sí

Una estrategia muy común es: 1. usar DuckDB para explorar y filtrar datos crudos, 2. pasar resultados a Polars para transformaciones intensivas, 3. guardar salidas en formatos eficientes como Parquet, 4. llevar los datos ya limpios al entrenamiento de modelos.

Síntesis

DuckDB y Polars no son solo "herramientas nuevas"; son una forma distinta de pensar la analítica moderna: más cercana a la performance, al trabajo reproducible y a la exploración ágil. En este programa, son el puente entre SQL, procesamiento eficiente y preparación de datos para ML.

¿Por qué este tema importa?

Un modelo de Machine Learning no aprende magia: aprende patrones construidos a partir de datos. Y muchas de las mejores señales no están explícitas en una tabla; hay que derivarlas.

Ahí entra SQL analítico avanzado. Su verdadero valor no es solo "consultar", sino transformar datos históricos en variables informativas: tendencias, rankings, acumulados, ventanas temporales, cambios entre eventos y métricas por contexto.

Si este tema se entiende mal, los modelos reciben features pobres, inconsistentes o incluso inválidas.

La idea central

El SQL tradicional responde preguntas como: - ¿cuántos registros hay? - ¿cuál es el total por categoría?

El SQL analítico responde preguntas más ricas: - ¿cómo cambia una métrica a lo largo del tiempo? - ¿cómo se compara una fila con las anteriores? - ¿cuál es el ranking relativo de un usuario dentro de su grupo? - ¿qué pasó dentro de una ventana de tiempo?

Eso lo vuelve fundamental para preparar variables para ML.

Funciones de ventana: ver cada fila sin perder contexto

Una ventana permite calcular algo sobre un subconjunto relacionado de filas sin colapsarlas en una sola salida.

Por ejemplo, en vez de preguntar "ventas totales por cliente", puedes preguntar: - la venta acumulada hasta hoy, - la diferencia entre la compra actual y la anterior, - el ranking del cliente dentro de su segmento.

Eso es poder analítico real.

Analogía: seguir la historia, no solo la foto

Un `GROUP BY` te da una foto grupal. Las ventanas te dejan ver la película sin perder personajes.

Esto importa porque muchos problemas de negocio son temporales: - churn, - fraude, - propensión de compra, - retención, - hábitos de uso.

Ejemplo conceptual

Supón una tabla de eventos de usuarios con fecha y monto.

Con SQL analítico puedes crear features como: - número de compras en los últimos 30 días, - monto acumulado histórico, - días desde la última compra, - diferencia entre el evento actual y el anterior, - ranking de compra por usuario.

Estas variables suelen ser más predictivas que los datos crudos.

Cómo se usan en la industria

- Equipos de producto en empresas como **Netflix**, **Spotify** o **Uber** crean features temporales para comportamiento de usuarios.

- En fraude financiero, el contexto temporal es crucial.
- En retail, las ventanas permiten construir señales de frecuencia, recencia y valor monetario.

Qué confunden los principiantes

“Con joins y group by basta”

No siempre. Muchas señales requieren contexto fila a fila.

“Más features siempre mejor”

No. Features mal construidas pueden introducir fuga de información o ruido.

“SQL solo sirve para extraer datos”

En analítica moderna, SQL también es una fábrica de features.

Riesgo importante: data leakage

Un error muy serio consiste en construir una feature usando información del futuro. Por ejemplo, usar una ventana que mira datos posteriores a la fecha de predicción.

Esto hace que el modelo parezca excelente en entrenamiento y falle en producción.

Síntesis

El SQL analítico avanzado transforma bases de datos en motores de ingeniería de variables. En este programa, es uno de los puentes más importantes entre arquitectura de datos y Machine Learning, porque permite extraer señales temporales, contextuales y relacionales que realmente mejoran un modelo.

¿Por qué este tema importa?

Muchos proyectos de Machine Learning fallan no porque el algoritmo sea malo, sino porque el preprocesamiento está mal hecho. Normalizar antes de separar datos, imputar usando información de todo el dataset o transformar train y test de forma distinta puede producir métricas infladas y modelos poco confiables.

Scikit-Learn no solo ofrece modelos; ofrece una forma disciplinada de construir flujos de aprendizaje reproducibles.

La idea central

Un pipeline de Scikit-Learn encadena pasos de transformación y modelado de forma ordenada:

1. limpieza,
2. imputación,
3. codificación,
4. escalado,

5. selección de variables,
6. entrenamiento.

La gran ventaja es que el pipeline aplica exactamente las mismas transformaciones en entrenamiento y evaluación.

Una analogía útil

Piensa en una receta de cocina profesional. No mezclas ingredientes al azar y luego intentas corregir el sabor al final. Sigues un orden: cortar, medir, cocinar, emplatar.

En ML, el orden importa tanto como el ingrediente.

Qué problema resuelve de verdad

Los pipelines resuelven tres problemas clave:

- **reproducibilidad**: el mismo proceso se puede repetir,
- **consistencia**: train, validation y test reciben la misma lógica,
- **seguridad metodológica**: evitan fuga de información.

Fuga de información: el enemigo silencioso

Un ejemplo clásico: escalas todo el dataset antes de dividirlo.

¿Por qué es un problema? Porque el escalador "ve" valores del test y eso contamina el entrenamiento. El modelo se evalúa con ventaja injusta.

Lo correcto es: 1. dividir los datos, 2. ajustar transformaciones solo con train, 3. aplicar esas transformaciones a test.

Transformadores personalizados

A veces las transformaciones no están predefinidas. Ahí entran transformadores personalizados.

Sirven para: - aplicar reglas de negocio, - crear features específicas, - limpiar valores anómalos, - estandarizar formatos complejos.

Esto es muy valioso porque en ciencia de datos real no todo se resuelve con un `StandardScaler` o un `OneHotEncoder`.

Qué hacen las empresas

- Equipos de ML en compañías como **LinkedIn**, **Airbnb** o **Spotify** usan pipelines para asegurar que sus modelos se entrenen y sirvan con lógica estable.
- En entornos productivos, las transformaciones suelen versionarse y validarse como parte del ciclo de vida del modelo.

Errores comunes

“Puedo hacer todo en un notebook y luego copiar el código”

Eso suele generar inconsistencias entre entrenamiento y producción.

“Preprocesar es una etapa secundaria”

No. En muchos problemas, el preprocesamiento determina gran parte del rendimiento.

“Los pipelines son solo para principiantes”

Al contrario: son una práctica profesional esencial.

Síntesis

Scikit-Learn enseña una forma correcta de pensar el ML: datos transformados de manera coherente, sin atajos peligrosos. En este programa, pipelines y preprocesamiento son el puente entre el análisis exploratorio y el modelado serio, y son claves para trabajar con estándares modernos y reproducibles.

¿Por qué este tema importa?

Entrenar un modelo no significa que el modelo sea bueno. Muchas veces un modelo parece excelente solo porque fue evaluado de forma ingenua. La evaluación rigurosa es lo que separa una demostración académica de una solución confiable.

Si este tema se malinterpreta, puedes tomar decisiones empresariales equivocadas basadas en métricas infladas.

La idea central

La pregunta no es solo “¿qué tan bien predice el modelo en mi dataset?”, sino: - ¿funciona en datos no vistos? - ¿es estable entre distintas particiones? - ¿mejoró por una razón real o por azar?

Ahí entran la validación cruzada y el ajuste de hiperparámetros.

Validación cruzada: mirar el rendimiento desde varios ángulos

En vez de hacer una sola división train/test, se hacen múltiples particiones. El modelo se entrena y evalúa varias veces, y luego se promedia el resultado.

Esto reduce la dependencia de una sola partición “afortunada” o “desafortunada”.

Analogía: estudiar para un examen con varios simulacros

Si solo haces un examen de práctica, no sabes si tu resultado fue casualidad. Si haces varios, ves patrones más estables sobre tu preparación.

La validación cruzada hace algo parecido con el modelo.

Por qué importan las métricas

No todas las métricas responden la misma pregunta.

- **Accuracy** puede engañar en clases desbalanceadas.
- **F1** equilibra precisión y recall.
- **AUC-ROC** mide capacidad de ranking entre clases.
- **Matriz de confusión** revela errores concretos.

Elegir una métrica equivocada puede hacer que optimices lo incorrecto.

Hiperparámetros vs parámetros

Este punto confunde mucho.

- Los **parámetros** se aprenden del dato.
- Los **hiperparámetros** se eligen antes o durante el entrenamiento.

Ejemplos: - profundidad de un árbol, - número de estimadores en un ensemble, - regularización, - tasa de aprendizaje.

GridSearchCV: búsqueda ordenada, no adivinanza

Grid Search prueba combinaciones de hiperparámetros y usa validación cruzada para estimar cuál generaliza mejor.

No garantiza encontrar la configuración perfecta, pero sí una búsqueda sistemática mucho más sólida que “probar a ojo”.

Ejemplos de industria

- En **fintech**, una pequeña mejora de recall en fraude puede ahorrar millones.
- En **marketing**, optimizar F1 o AUC puede mejorar campañas de segmentación.
- En sistemas de recomendación o scoring, una evaluación mal diseñada puede llevar a despliegues peligrosos.

Errores comunes

“Si el accuracy es alto, el modelo es bueno”

No siempre. En problemas desbalanceados puede ser totalmente engañoso.

“La validación cruzada reemplaza pensar”

No. Si hay fuga de información o mala ingeniería de variables, CV solo promedia un error mal diseñado.

“Grid Search siempre mejora el modelo”

No. Puede ser costoso, lento y no necesariamente producir mejoras reales.

Síntesis

Validar bien un modelo es aprender a desconfiar de resultados fáciles. Este tema conecta con todo el programa porque depende de la calidad de datos, del preprocesamiento, del diseño del problema y de la elección de métricas. Un buen científico de datos no solo entrena modelos: sabe cuándo merece creerles.

Preguntas frecuentes

P: ¿Por qué aprender DuckDB y Polars si ya domino Pandas?

R: Muchos estudiantes asumen que Pandas es suficiente porque es la herramienta más conocida, pero en entornos de producción profesional, el procesamiento de datos masivos puede volcar la memoria RAM de tu sistema. Pandas no fue diseñado originalmente para el paralelismo multithreaded que hoy requerimos. Polars, por otro lado, utiliza procesamiento vectorial y ejecución perezosa (lazy evaluation) que optimiza el uso de CPU, mientras que DuckDB funciona como una base de datos analítica 'in-process' increíblemente rápida. Imagina que Pandas es una maleta pequeña que debe organizar todo el equipaje antes de cerrar; Polars y DuckDB son sistemas de logística automatizada que manejan el flujo de cajas sin necesidad de tener todo el inventario apilado en un solo lugar al mismo tiempo. Aprender estas herramientas es vital para escalar tus modelos más allá de datasets de prueba.

P: ¿Es obligatorio ser un experto en SQL para desarrollar pipelines de Machine Learning?

R: No necesitas ser un Administrador de Bases de Datos (DBA), pero la realidad de la industria es que el 80% del tiempo de un Data Scientist se consume en preparar los datos. Si no sabes realizar joins complejos, usar window functions para crear variables temporales o manejar el esquema de tus tablas, terminarás haciendo 'data wrangling' deficiente con Python, lo cual es mucho más lento y difícil de auditar. SQL es el lenguaje universal para la extracción de features. Considera a SQL como la fundación de tu casa: puedes intentar construir sobre suelo inestable, pero eventualmente la estructura (el modelo de ML) colapsará. Dominar la manipulación de datos en la base permite que tu código de Python sea más limpio y esté enfocado solo en el entrenamiento del modelo, no en arreglar inconsistencias de origen.

P: ¿Cómo puedo practicar la ingesta de APIs si no tengo un servidor propio?

R: Es un error común pensar que necesitas infraestructura compleja para practicar; el mundo está lleno de APIs públicas gratuitas, como las de clima, tipos de cambio o datasets gubernamentales, que son perfectas para simular un ambiente real. La clave no es solo 'descargar el JSON', sino integrar el manejo de variables de entorno (.env) para no subir tus credenciales a GitHub y construir una lógica de reintento ante fallos de conexión. Piensa en esto como en un entrenamiento de bomberos: no necesitas incendiar tu casa para aprender a usar la manguera. Aprender a manejar la inestabilidad de las peticiones HTTP y la normalización de estructuras anidadas es el estándar que separa a un principiante de un ingeniero de datos capaz de trabajar en entornos de producción con datos en vivo.

P: ¿Qué diferencia real existe entre un script para análisis exploratorio y un pipeline de ML profesional?

R: La diferencia fundamental es la reproducibilidad. Un script de análisis suele ser una serie de celdas en un notebook donde el orden de ejecución importa y los pasos de limpieza están dispersos. Un pipeline profesional, especialmente usando Scikit-Learn, encapsula la limpieza, la transformación y el modelado en objetos reutilizables. Si cambias el dataset de entrada, el pipeline debe seguir funcionando sin intervención manual. La analogía es la diferencia entre cocinar paso a paso mientras inventas la receta (notebook) y una cadena de ensamblaje automatizada en una fábrica (pipeline). Al diseñar pipelines, aseguras que los mismos tratamientos aplicados en el entrenamiento se apliquen exactamente igual cuando el modelo reciba datos nuevos ('inference'); de lo contrario, tu modelo tendrá un desempeño catastrófico en el mundo real.

P: ¿Debo aprender ORM (SQLAlchemy) o escribir consultas SQL puras?

R: En la industria, el debate no es 'uno u otro' sino 'cuándo usar cada uno'. Las consultas SQL puras (o con DuckDB) son insuperables para el análisis exploratorio y la ingeniería de features debido a su velocidad y visibilidad directa sobre los datos. Sin embargo, SQLAlchemy es esencial cuando construyes aplicaciones que persisten datos de forma estructurada y segura. Muchos estudiantes creen que el ORM reemplaza al SQL, pero ignoran que el ORM es una capa de abstracción para gestionar la integridad y las transacciones. Aprender ambos te permite usar SQL para el 'arte' del análisis y el ORM para la 'ingeniería' de la persistencia. Es como saber usar un martillo manual para detalles precisos y una pistola neumática para la construcción a gran escala: ambas herramientas pertenecen a tu cinturón de habilidades.

P: ¿Cómo garantizo que mis modelos de Machine Learning no estén sesgados o aprendiendo ruido?

R: La validación es la etapa más crítica y a menudo la más subestimada por quienes se centran solo en el código del modelo. En lugar de solo mirar el 'Accuracy', aprender a utilizar el F1-Score, la matriz de confusión y, sobre todo, la Validación Cruzada (Cross-Validation), es lo que separa a un modelo robusto de uno que solo tiene buena suerte con los datos de entrenamiento. El sesgo ocurre cuando confiamos en una sola partición de datos. Imagina una escuela que solo evalúa a sus alumnos con las preguntas que dieron en clase: el estudiante memoriza las respuestas, no aprende el concepto. La validación cruzada es como tomar un examen sorpresa de temas relacionados; si el modelo aprueba consistentemente en diferentes subconjuntos, entonces realmente ha aprendido patrones generalizables y no está simplemente memorizando el ruido de tu dataset específico.

Conclusiones

La Convergencia entre Ingeniería de Datos y ML: Del Pipeline al Modelo

Has integrado dos mundos que tradicionalmente operan en silos: la infraestructura de datos y la ciencia predictiva. Ahora comprendes que un modelo de Machine Learning es tan robusto como el pipeline que lo alimenta. A partir de hoy, no solo entrenas algoritmos, sino que diseñas sistemas de ingesta automatizada mediante APIs, normalización JSON y persistencia en bases de datos relacionales, garantizando que el dato llegue limpio y estructurado a tus modelos. Esta transición de 'analista

que hace scripts' a 'arquitecto de soluciones de ML' te permite reducir drásticamente el tiempo de puesta en producción y mejorar la calidad de las predicciones en entornos corporativos donde la reproducibilidad es la regla de oro.

El Nuevo Ecosistema: Python, SQL y el Poder de la Memoria

Has dominado una arquitectura moderna donde herramientas como DuckDB y Polars reemplazan procesos lentos y manuales. El principio que debes retener es el de 'eficiencia computacional': sabes cuándo procesar en memoria mediante estructuras vectoriales y cuándo delegar la carga analítica a un motor SQL de alto desempeño. Ya no te detendrás ante archivos masivos o consultas complejas. Esta capacidad de optimizar flujos de trabajo de datos no solo te hace más productivo, sino que te posiciona como un perfil capaz de escalar proyectos de datos, un activo indispensable para empresas que manejan volúmenes crecientes de información analítica.

El Arte de la Ingeniería de Características mediante SQL Avanzado

A través del aprendizaje de Window Functions, joins complejos y la manipulación de JSON dentro de SQL, has descubierto que la preparación de datos para ML es, en esencia, un trabajo de ingeniería relacional cuidadosa. Ahora posees la capacidad de transformar datos crudos en 'features' ricas en contexto sin necesidad de salir del entorno SQL. En el mundo real, esto significa que serás capaz de construir datasets de entrenamiento mucho más precisos y representativos, evitando sesgos que surgen por una mala agregación. Has pasado de consultar bases de datos a diseñar capas de datos inteligentes que sustentan la lógica de tus modelos.

IA como Copiloto: Hacia un Desarrollo de Datos Asistido

No solo aprendiste a programar; aprendiste a colaborar con asistentes de IA para escalar tu capacidad técnica. Has adoptado el patrón de 'generación asistida y validación humana', asegurando que el código SQL y Python que produces sea eficiente, seguro y auditable. Este cambio de mentalidad es fundamental para tu crecimiento profesional: la IA ahora actúa como una extensión de tu capacidad de resolución de problemas, permitiéndote prototipar más rápido y automatizar tareas repetitivas de validación. Tu valor ahora reside menos en la sintaxis y más en la arquitectura, la estrategia del modelo y la integridad del sistema completo.

Mentalidad de Producto en Ciencia de Datos

El proyecto final es la culminación de tu transformación: has pasado de ejecutar scripts aislados a entregar un producto de datos end-to-end. Has aprendido a valorar la automatización (ETL), la persistencia (ORM/SQLAlchemy) y la validación (Cross-Validation/GridSearch). Este enfoque de 'pipeline completo' te permite visualizar el ciclo de vida del dato desde su origen en una API hasta su impacto en la toma de decisiones. Profesionalmente, esta mentalidad te permite liderar o participar en proyectos donde la calidad, la trazabilidad y el rendimiento no son accesorios, sino pilares de tu arquitectura técnica, permitiéndote entregar resultados tangibles y validados con rigor científico.
